**CS 6290 — Georgia Tech OMSCS**

# High Performance Computer Architecture

**Lecture Notes**

kinginu

Fall 2026

# Contents


**1 Introduction** 1
- 1.1 What is computer architecture? 1
- 1.2 Technology trends 1
- 1.3 Speed, power and cost 2
- 1.4 Power consumption 2
- 1.5 Fabrication cost 3

**2 Metrics and Evaluation** 4
- 2.1 Performance 4
- 2.2 Comparing performance 4
- 2.3 Measuring performance 4
- 2.4 Summarizing performance 5
- 2.5 The iron law of performance 5
- 2.6 Amdahl's law 6

**3 Pipelining** 7
- 3.1 The idea of pipelining 7
- 3.2 Pipeline CPI 8
- 3.3 Stalls and flushes 8
- 3.4 Control dependences 8
- 3.5 Data dependences 8
- 3.6 Dependences and hazards 9
- 3.7 Handling hazards 9
- 3.8 How many stages? 10

**4 Branches** 11
- 4.1 Branches in a pipeline 11
- 4.2 Prediction accuracy and performance 12
- 4.3 Predict not taken 12
- 4.4 The need for better prediction 13
- 4.5 The branch target buffer 13
- 4.6 Direction prediction 14
- 4.7 The two-bit predictor 15
- 4.8 History-based predictors 16
- 4.9 Shared counters: PShare and GShare 17
- 4.10 Tournament and hierarchical predictors 18
- 4.11 The return address stack 19

**5 Predication** 21
- 5.1 Prediction and predication 21
- 5.2 If-conversion 22
- 5.3 Conditional move instructions 22
- 5.4 Performance of if-conversion 23
- 5.5 Benefits and costs of conditional moves 24
- 5.6 Full predication 24

**6 ILP** 27
- 6.1 All instructions in the same cycle 27
- 6.2 Dependences in same-cycle execution 27
- 6.3 Removing false dependences 28
- 6.4 Register renaming 28
- 6.5 Dependences after renaming 29
- 6.6 Instruction-level parallelism 30
- 6.7 Structural and control dependences 30
- 6.8 ILP and IPC 31

**7 Instruction Scheduling** 33
- 7.1 Improving IPC 33
- 7.2 Tomasulo's algorithm 33

7.3 Organization . . . 34
7.4 Issue . . . 35
7.5 Dispatch . . . 35
7.6 Write result . . . 36
7.7 The steps in one cycle . . . 37
7.8 Load and store instructions . . . 38
7.9 A complete example . . . 38
7.10 Timing without tracing the structures . . . 40

**8 ReOrder Buffer** **42**
8.1 Exceptions in out-of-order execution . . . 42
8.2 Branch mispredictions . . . 43
8.3 Correct out-of-order execution . . . 43
8.4 The reorder buffer . . . 43
8.5 Branch misprediction recovery . . . 45
8.6 Exceptions with a reorder buffer . . . 46
8.7 RAT updates on commit . . . 46
8.8 A reorder buffer example . . . 47
8.9 Timing with a reorder buffer . . . 48
8.10 Unified reservation stations . . . 49
8.11 Superscalar processors . . . 49
8.12 Terminology . . . 49
8.13 What is out of order . . . 50

**9 Memory Ordering** **51**
9.1 Dependences through memory . . . 51
9.2 When memory is written . . . 51
9.3 The load-store queue . . . 52
9.4 Out-of-order and in-order load-store execution . . . 52
9.5 Store-to-load forwarding . . . 54
9.6 A load-store queue example . . . 54
9.7 The LSQ, the ROB and the reservation stations . . . 55

**10 Compiler ILP** **57**
10.1 How the compiler can raise IPC . . . 57
10.2 Tree height reduction . . . 57
10.3 Instruction scheduling . . . 58
10.4 Scheduling and if-conversion . . . 60
10.5 Loop unrolling . . . 61
10.6 Function call inlining . . . 63
10.7 Other compiler techniques . . . 64

**11 VLIW** **65**
11.1 Superscalar and VLIW processors . . . 65
11.2 Advantages and disadvantages . . . 66
11.3 VLIW instructions . . . 67
11.4 VLIW processors in practice . . . 68

**12 Cache Review** **69**
12.1 The locality principle . . . 69
12.2 Caches . . . 69
12.3 Cache performance . . . 70
12.4 Blocks and lines . . . 71
12.5 Block number, tag and valid bit . . . 71
12.6 Where a block can be placed . . . 72
12.7 Direct-mapped caches . . . 72
12.8 Set-associative caches . . . 73
12.9 Fully associative caches . . . 74
12.10 Cache replacement . . . 75
12.11 Write policy . . . 75
12.12 Putting it together . . . 76

**13 Virtual Memory** 78
13.1 Why virtual memory . . . 78
13.2 Physical memory and virtual address spaces . . . 78
13.3 Pages and page tables . . . 79
13.4 Page faults . . . 80
13.5 Address translation . . . 80
13.6 The size of a flat page table . . . 81
13.7 Multi-level page tables . . . 81
13.8 Choosing the page size . . . 83
13.9 Memory access time with translation . . . 83
13.10 The translation lookaside buffer . . . 84
13.11 TLB organization and performance . . . 85

**14 Advanced Caches** 86
14.1 Improving cache performance . . . 86
14.2 Pipelined caches . . . 86
14.3 Address translation and the hit time . . . 87
14.4 Aliasing . . . 88
14.5 Way prediction . . . 90
14.6 Replacement policy and hit time . . . 90
14.7 Causes of misses . . . 91
14.8 Larger cache blocks . . . 92
14.9 Prefetching . . . 92
14.10 Loop interchange . . . 93
14.11 Overlapping misses . . . 94
14.12 Cache hierarchies . . . 95
14.13 Local and global miss rates . . . 96
14.14 The inclusion property . . . 97

**15 Memory** 98
15.1 Memory technologies . . . 98
15.2 The SRAM cell . . . 98
15.3 The DRAM cell . . . 99
15.4 Memory chip organization . . . 100
15.5 Refresh . . . 101
15.6 Address multiplexing . . . 101
15.7 Fast page mode . . . 102
15.8 Connecting DRAM to the processor . . . 103

**16 Storage** 105
16.1 Storage . . . 105
16.2 Magnetic disks . . . 105
16.3 Disk access time . . . 106
16.4 Trends for magnetic disks . . . 107
16.5 Optical disks . . . 107
16.6 Magnetic tape . . . 107
16.7 Solid-state disks . . . 108
16.8 Hybrid magnetic–flash disks . . . 108
16.9 Connecting I/O devices . . . 109

**17 Fault Tolerance** 111
17.1 Dependability . . . 111
17.2 Faults, errors and failures . . . 111
17.3 Reliability and availability . . . 112
17.4 Kinds of faults . . . 113
17.5 Improving reliability and availability . . . 113
17.6 Fault-tolerance techniques . . . 113
17.7 Codes for memory and storage . . . 114
17.8 Redundant arrays of independent disks . . . 115
17.9 RAID 0: striping . . . 115
17.10 RAID 1: mirroring . . . 116
17.11 RAID 4: block-interleaved parity . . . 117

17.12 RAID 5: distributed parity . . . 118
17.13 RAID 6: two check blocks . . . 118

**18 Multi-Processing** **120**
18.1 Flynn's taxonomy . . . 120
18.2 Why multiprocessors . . . 120
18.3 Multiprocessors need parallel programs . . . 121
18.4 Centralized shared memory . . . 121
18.5 Distributed shared memory . . . 122
18.6 Message passing and shared memory . . . 123
18.7 Shared-memory hardware . . . 124
18.8 Multithreading performance . . . 125
18.9 Hardware changes for SMT . . . 126
18.10 SMT, the data cache and the TLB . . . 126
18.11 SMT and cache performance . . . 127

**19 Cache Coherence** **128**
19.1 The coherence problem . . . 128
19.2 Coherence . . . 128
19.3 Achieving coherence . . . 129
19.4 Write-update coherence . . . 129
19.5 Write-invalidate coherence . . . 130
19.6 The MSI protocol . . . 131
19.7 The MOSI protocol . . . 132
19.8 The exclusive state . . . 133
19.9 Directory-based coherence . . . 134
19.10 Cache misses and coherence . . . 135

**20 Synchronization** **137**
20.1 The need for synchronization . . . 137
20.2 Locks . . . 138
20.3 Implementing a lock . . . 138
20.4 Atomic read/write instructions . . . 139
20.5 Load-linked/store-conditional . . . 140
20.6 Locks and performance . . . 141
20.7 Barrier synchronization . . . 142

**21 Memory Consistency** **145**
21.1 Coherence and consistency . . . 145
21.2 Why consistency matters . . . 145
21.3 Sequential consistency . . . 147
21.4 Implementing sequential consistency . . . 148
21.5 Relaxed consistency . . . 149
21.6 Data races and consistency . . . 150
21.7 Consistency models . . . 151

**22 Many Cores** **153**
22.1 Coherence traffic . . . 153
22.2 Network on chip . . . 153
22.3 Off-chip traffic and the distributed LLC . . . 154
22.4 The on-chip directory . . . 155
22.5 Sharing the power budget . . . 156
22.6 Turbo mode . . . 157
22.7 SMT, cores and chips . . . 157
22.8 Challenges and solutions . . . 158

**Bibliography** **159**

**Index** **161**

# Introduction

| | |
|---|---|
| **Lecture** | Lesson 1, videos 1–19 |
| **Textbook** | H&P [1] §1.1–1.6 |
| **Keywords** | computer architecture, Moore's law, memory wall, dynamic power, static power, fabrication cost, yield |

This lesson defines what computer architecture is, explains why it is a discipline in its own right, and introduces the technology trends and the three quantities—performance, power and cost—that constrain every design decision in the rest of the course.

## 1.1 What is computer architecture?

The discipline of designing a computer that is well suited to its purpose, using the fabrication technology that is available, is called *computer architecture*. Two points in this definition deserve emphasis.

- *Well suited to its purpose.* A mobile phone, a laptop and a data-center server are all computers, but they optimize for different things: battery life, size and weight in one case; raw performance and throughput per dollar in another. Architecture is therefore about *trade-offs*, not about a single best design.
- *Available technology.* The architect does not design transistors, but must know what the current process can deliver—how many transistors fit on a chip, how fast they switch, how much power they dissipate—because those numbers bound what is achievable.

**Source** Lesson 1, videos 2–5; H&P §1.1–1.3.

The scale involved: 2300 transistors on the Intel 4004 (1971), 80 billion on NVIDIA's H100 (2022). Vendor data.

### 1.1.1 Why do we need it?

Computer architecture serves two goals.

1. **Improve performance**, in the broad sense: execution speed, battery life, size, weight, energy efficiency.
2. **Improve abilities**: make the computer able to do things it could not do before, e.g. 3D graphics, hardware support for debugging, security features.

A further reason is timing. A processor that is designed today will be fabricated several years from now, so the architect must *anticipate* where the technology will be at that time rather than design for today's parameters.

In short: "improve performance" and "improve abilities".

**Tip** Whenever a design decision in later lessons looks over-engineered, ask which technology trend it was anticipating.

## 1.2 Technology trends

### 1.2.1 Moore's law

*Moore's law* is the empirical observation that the number of transistors that can be placed on a chip of a given area doubles roughly every 18 to 24 months. It is a trend, not a law of physics, but it held for several decades and shaped the expectations of the whole industry. It has the following consequences, all with the same 18–24-month doubling period:

- processor *speed* doubles (more transistors allow more parallelism and smaller, faster gates);
- the *energy per operation* halves;
- *memory capacity* doubles.

**Source** [2]; Lesson 1, video 6; H&P §1.4.

**Caution** Moore said one year in 1965 and two years in 1975; the popular "18 months" is David House's figure, not his.

**Example 1.1.** If speed doubles every two years, a factor-of-$2^5 = 32$ improvement is expected over a decade. Conversely, a design team that spends four years producing a processor only 2× faster than the one it replaces has fallen behind: the trend predicts $2^2 = 4\times$ over the same period.

### 1.2.2 The memory wall

Not every component follows the same curve. While processor performance doubled every 18–24 months, memory *latency* improved only by about 1.1× over the same interval. The result is a gap between how fast the processor can consume data and how fast memory can supply it

**Source** [3]; Lesson 1, video 8; H&P §2.1.

that widens every generation (Figure 1.1). This gap is called the *memory wall* and is the reason why caches (Lessons 12–14) are so important.

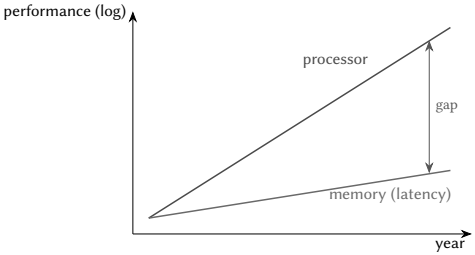



**Figure 1.1:** Processor performance and memory latency improve at different rates; the difference is the memory wall.

> **Key point**
> Memory capacity keeps up with Moore's law; memory *latency* does not. Most of the memory-system design in this book exists to hide that latency.

On Skylake an L1 hit costs 4 cycles and an L2 hit 12; a DRAM access takes 60–90 ns, i.e. 180–270 cycles at 3 GHz. Intel optimization manual; 7-cpu.com.

## 1.3 Speed, power and cost

**Source** Lesson 1, videos 9–10.

For a long time "better" simply meant "faster". Today the architect has to balance three quantities:

**Performance** how fast the machine runs its intended workload;

**Power** how much energy it consumes, which sets battery life for portable devices and cooling cost for servers;

**Cost** how much it costs to fabricate.

Which of these dominates depends on the product: weight and battery life for a phone, cost for an embedded controller, performance for a supercomputer.

Budgets: phone SoC 2–5 W, desktop CPU 65–125 W, server CPU up to 500 W (EPYC 9005, Xeon 6900P). Vendor data.

**Caution** "Faster" without a workload and a power/cost budget is not a meaningful design goal.

## 1.4 Power consumption

**Source** Lesson 1, videos 11–13; H&P §1.5.

The power dissipated by a processor has two components.

### 1.4.1 Dynamic (active) power

The power consumed by charging and discharging the capacitance of wires and gates whenever a signal switches is called *dynamic power*:

$$P_{\text{dyn}} = \tfrac{1}{2} C V^2 f \alpha, \tag{1.1}$$

where $C$ is the total capacitance being switched, $V$ the supply voltage, $f$ the clock frequency and $\alpha$ the *activity factor*, the fraction of the capacitance that actually switches each cycle.

$C$ grows with chip size and transistor count; $\alpha$ depends on the workload and on how much logic is left idle. H&P writes "frequency switched" for the product $f\alpha$.

Two facts follow directly from Equation (1.1).

- Power scales with the *square* of the voltage, so lowering $V$ is the most effective lever. However, the maximum frequency a circuit can sustain also drops as $V$ drops, so in practice $V$ and $f$ are lowered together and power falls roughly with $V^3$.
- Doubling the frequency at constant voltage doubles the power, and so does doubling the transistor count (more parallel hardware), through the doubled $C$. The difference is that a higher frequency usually also needs a higher voltage, which raises power quadratically, whereas parallel hardware can run at the same or a lower $V$ and $f$.[1]

[1] Until about 2005 each new process shrank $V$ together with the transistor, so a chip ran faster at constant power. Leakage stopped voltage scaling near 1 V: clock rates stalled at 3–4 GHz and the extra transistors went into more cores (Lesson 18).

**Example 1.2.** Reducing the supply voltage by 15 % while keeping $C$, $f$ and $\alpha$ fixed reduces dynamic power to $0.85^2 \approx 0.72$ of its original value, i.e. by about 28 %.

### 1.4.2 Static power

The power consumed even when no signal switches is called *static power* (leakage power); it arises because transistors are imperfect switches and leak current when off. Leakage grows as the supply voltage—and with it the transistor threshold voltage—is reduced. There is therefore a trade-off: lowering $V$ reduces dynamic power but increases static power, and the total $P = P_{\text{dyn}} + P_{\text{static}}$ has a minimum at some intermediate voltage (Figure 1.2).

> **Key point — Power**
> Dynamic power $\propto CV^2 f\alpha$ favors low voltage; static power penalizes it. Modern designs sit near the minimum of the sum and get further gains from reducing $C$, $\alpha$ and $f$ rather than $V$.

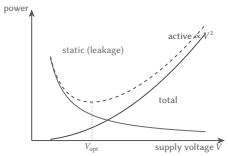



**Figure 1.2:** Dynamic and static power pull in opposite directions as the supply voltage changes.

## 1.5 Fabrication cost

Chips are manufactured on circular silicon *wafer*s, each of which is cut into many identical rectangular *die*s. The cost of processing a wafer is roughly fixed, so the cost of a single die is

$$\text{cost per die} \;=\; \frac{\text{cost per wafer}}{\text{dies per wafer} \times \text{yield}}. \tag{1.2}$$

**Source** Lesson 1, videos 15–17; H&P §1.6.

### 1.5.1 Yield

Not every die works: defects are scattered over the wafer, and a die that contains a defect is usually discarded. The *yield* is the fraction of dies that are functional. Because defects fall at random positions, a larger die is more likely to contain one, so yield *decreases* as die area increases.

H&P §1.6 gives an empirical model, yield $\approx 1/(1 + \text{defect density} \times \text{area})^N$; a mature process leaves about 0.1 defects per cm$^2$.

### 1.5.2 Consequences

Combining Equation (1.2) with the yield behaviour gives the central result of this section: halving the die area *more than* halves the cost per die, because it doubles the number of dies per wafer *and* raises the yield. Read together with Moore's law this offers the architect two options each technology generation:

- keep the die area constant and pack in twice the transistors—a more capable chip at roughly the same cost; or
- keep the design constant and let it shrink to half the area—the same chip at well under half the cost.

**Tip** Dies per wafer $\approx \pi(d/2)^2/A - \pi d/\sqrt{2A}$; the second term removes the partial dies at the wafer edge. H&P §1.6.

A 300 mm wafer offers 707 cm$^2$, and the largest die one lithography field can print is $26 \times 33$ mm $= 858$ mm$^2$; NVIDIA's H100 uses 814 mm$^2$ of it.

**Example 1.3.** Suppose a wafer costs \$5000 and holds 100 dies of a given design with a yield of 50 %: the cost per working die is $5000/(100 \times 0.5) = \$100$. A shrink that fits 200 dies per wafer and raises the yield to 70 % lowers the cost to $5000/(200 \times 0.7) \approx \$36$, less than half.

> **Key point — Lesson summary**
> Architecture is trade-off engineering under three constraints— performance, power and cost—driven by technology trends (Moore's law) and their exceptions (the memory wall). The next lesson makes "performance" precise.

Lesson 2

# Metrics and Evaluation

| | |
|---|---|
| **Lecture** | Lesson 2, videos 1–25 |
| **Textbook** | H&P [1] §1.8–1.9 |
| **Keywords** | latency, throughput, speedup, benchmark, geometric mean, iron law, CPI, Amdahl's law, diminishing returns |

Before we can improve performance we must be able to measure it, compare two machines, and predict the effect of a change. This lesson introduces the vocabulary (latency, throughput, speedup), the methodology (benchmarks and how to summarize them) and two analytical tools that recur throughout the course: the iron law of performance and Amdahl's law.

## 2.1 Performance

**Source** Lesson 2, videos 2–3; H&P §1.8.

"Performance" is ambiguous until we say what we are measuring.

**Definition 2.1** (Latency). The *latency* of a task is the time from its start to its completion.

**Definition 2.2** (Throughput). The *throughput* of a system is the number of tasks it completes per unit time.

Throughput is *not* simply the inverse of latency. When tasks overlap (pipelining, Lesson 3) or run in parallel (multiprocessors), a system can complete many tasks per second even though each individual task takes a long time.

**Caution** Throughput = 1/latency holds only when tasks are processed one at a time, strictly one after another.

DDR4-3200: first data 14 ns after a read (CL22), yet 25.6 GB/s sustained as accesses overlap (Lesson 15). JEDEC JESD79-4.

**Example 2.3.** A processor that takes 10 s for each of two programs run one after the other has latency 10 s per program and throughput 0.1 programs/s. A second processor that takes 15 s per program but runs two at once has a worse latency yet a throughput of $2/15 \approx 0.13$ programs/s.

## 2.2 Comparing performance

**Source** Lesson 2, videos 4–7; H&P §1.8.

To say that machine $X$ is "$N$ times faster" than machine $Y$ we use the *speedup*:

$$N = \frac{\text{speed}(X)}{\text{speed}(Y)} = \frac{\text{latency}(Y)}{\text{latency}(X)} = \frac{\text{throughput}(X)}{\text{throughput}(Y)}. \tag{2.1}$$

**Caution** A percentage is not a speedup: 30 % less time is $N \approx 1.43$, not 1.3.

**Tip** Always keep the ratio in the form old time/new time; most sign errors come from inverting it.

Note that latency appears *inverted*: a shorter time means a higher speed. A speedup greater than 1 means $X$ is better; a speedup below 1 means it is worse (a "speedup of 0.8" is a slowdown). Speedups compose multiplicatively: if $X$ is 2× faster than $Y$ and $Y$ is 3× faster than $Z$, then $X$ is 6× faster than $Z$.

## 2.3 Measuring performance

**Source** Lesson 2, videos 8–10; H&P §1.8.

The ideal measurement runs the user's actual workload on the actual machine. This is rarely possible—the machine may not exist yet, and the workload is usually unknown—so we resort to *benchmarks*: programs, together with their input data, that a community agrees to use as a proxy for real workloads. A collection of such programs is a *benchmark suite*.

### 2.3.1 Types of benchmarks

From most to least representative:

**Real applications** complete programs with realistic inputs. Most representative, but hardest to set up, and impossible on a simulator or an incomplete prototype.

**Kernels** the most time-consuming part of a real application (e.g. a matrix multiply), extracted so that it can be run on its own. Easier to run; still reflects the real work.

**Synthetic benchmarks** programs written to *behave like* real applications (similar mix of instructions and memory accesses) while being simpler to run. Useful early in a design.[1]

**Peak performance** the theoretical maximum rate the hardware can sustain (e.g. instructions per second if every unit were busy). A marketing number that no real program reaches.

A cycle-level simulator such as gem5 runs three to five orders of magnitude slower than the hardware it models.

[1] The classic examples are Whetstone (1972, floating point) and Dhrystone (Weicker, 1984, integer). Both lost their credibility once optimizing compilers began to delete the code whose results are never used, which is why a synthetic program is no longer accepted as a final measurement.

**Note 2.4.** Peak performance is not a benchmark result at all; it is an upper bound that says nothing about how close real programs get to it.

### 2.3.2 Benchmark standards

Because a vendor could pick benchmarks that flatter its own product, independent organizations define standard suites. SPEC (Standard Performance Evaluation Corporation) publishes CPU, graphics and server suites; TPC does the same for transaction-processing workloads; EEMBC for embedded systems. Members of these organizations agree on the programs, inputs and reporting rules.

**Source** Lesson 2, video 12; [4].

SPEC CPU2017 contains 43 programs in four suites: integer and floating point, each in a speed and a rate version. spec.org.

## 2.4 Summarizing performance

A suite yields one number per program; we usually want one number per machine.

- **Execution times** may be combined with the ordinary arithmetic mean: the average time is well defined, and the ratio of the average times of two machines equals the speedup of the total runtime.
- **Speedups** (ratios) must *not* be averaged arithmetically. The arithmetic mean of ratios depends on which machine is the reference, and it does not equal the ratio of the mean times. Use the *geometric mean*:

$$\overline{S} = \left(\prod_{i=1}^{n} S_i\right)^{1/n}. \tag{2.2}$$

**Source** Lesson 2, videos 13–14; H&P §1.8.

**Tip** Compute it as $\exp\left(\frac{1}{n}\sum_i \ln S_i\right)$: a product of many ratios overflows, a sum of logarithms does not.

**Example 2.5.** Program A takes 10 s on machine $X$ and 20 s on $Y$; program B takes 20 s on $X$ and 10 s on $Y$. The per-program speedups of $X$ over $Y$ are 2 and 0.5. The arithmetic mean, 1.25, suggests $X$ is faster; computed with $Y$ as the reference the arithmetic mean is also 1.25, suggesting $Y$ is faster—a contradiction. The geometric mean is $\sqrt{2 \times 0.5} = 1$ either way, correctly reporting that the machines are equivalent on this suite.

**Caution** The geometric mean is consistent, not predictive: it does not say how long the suite takes. For that, add the times.

> **Key point**
> Average *times* arithmetically; average *speedups* geometrically. The geometric mean of the speedups equals the ratio of the geometric means of the execution times, whichever machine is the reference.

## 2.5 The iron law of performance

The *iron law* decomposes CPU time into three factors that map onto the different parts of a computer system:

$$\text{CPU time} = \underbrace{\#\text{instructions}}_{\text{IC}} \times \underbrace{\frac{\text{cycles}}{\text{instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{seconds}}{\text{cycle}}}_{T_{\text{clk}}}. \tag{2.3}$$

**Source** Lesson 2, videos 15–18; H&P §1.9.

**Instruction count (IC)** determined by the algorithm, the compiler and the instruction set (ISA).

**Cycles per instruction (CPI)** determined by the ISA and by the processor's microarchitecture (pipelining, out-of-order execution, caches, …). Most of this book is about lowering CPI.

**Clock cycle time ($T_{\text{clk}}$)** determined by the microarchitecture (how much logic sits between two latches), the circuit design and the fabrication technology.

The value of the law is that it exposes trade-offs: a change that reduces one factor often increases another. A deeper pipeline lowers $T_{\text{clk}}$ but raises CPI; a richer ISA lowers IC but may raise CPI or $T_{\text{clk}}$.

Pentium 4 (Prescott): 31 stages, 3.8 GHz; Core 2 (Conroe, 2006): 14 stages, 2.93 GHz, yet faster. Intel data.

**Caution** Never evaluate a change by a single factor. "Higher clock frequency" is "faster" only if IC × CPI grows by a smaller factor than the frequency.

### 2.5.1 Unequal instruction times

When different instruction classes take different numbers of cycles, the CPI is a weighted average. With $n$ classes,

**Caution** Vendors quote IPC = 1/CPI. Averaging IPCs over a suite has the same defect as averaging speedups.

$$\text{CPU time} = \Big(\sum_{i=1}^{n} \text{IC}_i \times \text{CPI}_i\Big) \times T_{\text{clk}}, \tag{2.4}$$

where $\text{IC}_i$ and $\text{CPI}_i$ are the count and cycles per instruction of class $i$.

Realistic order of magnitude: SPEC integer programs execute about 25 % loads, 10 % stores and 15 % branches. H&P Appendix A.

**Example 2.6.** A program executes $10^9$ instructions on a 2 GHz processor: 60 % are ALU operations (1 cycle), 30 % loads/stores (2 cycles) and 10 % branches (3 cycles). The average CPI is $0.6 \cdot 1 + 0.3 \cdot 2 + 0.1 \cdot 3 = 1.5$, so the CPU time is $10^9 \times 1.5 \times 0.5\,\text{ns} = 0.75\,\text{s}$.

## 2.6 Amdahl's law

*Amdahl's law* predicts the overall speedup obtained by speeding up only *part* of the execution.

Source [5]; Lesson 2, videos 19–22; H&P §1.9.

**Definition 2.7** (Amdahl's law). If an enhancement speeds up a fraction $f_{\text{enh}}$ of the *original* execution time by a factor $s_{\text{enh}}$, the overall speedup is

$$S_{\text{overall}} = \frac{1}{(1 - f_{\text{enh}}) + \dfrac{f_{\text{enh}}}{s_{\text{enh}}}}. \tag{2.5}$$

Caution $f_{\text{enh}}$ is a fraction of the *original* time, not of the time after the change, and not a fraction of the instruction count.

**Example 2.8.** Floating-point instructions account for 40 % of the run time. A new FP unit makes them 4× faster. The overall speedup is $1/(0.6 + 0.4/4) = 1/0.7 \approx 1.43$. Making the FP unit infinitely fast would only reach $1/0.6 \approx 1.67$.[2]

[2] Amdahl's three-page paper (AFIPS 1967) argued in prose, without this formula, against large parallel machines. With 95 % of the time parallel, no core count exceeds 20×. Lesson 18 applies it to cores.

Source Lesson 2, videos 21–24.

### 2.6.1 Implications

Letting $s_{\text{enh}} \to \infty$ in Equation (2.5) gives the upper bound $S_{\text{overall}} \le 1/(1 - f_{\text{enh}})$. Two design principles follow.

> **Key point — Make the common case fast**
> A modest speedup of a large fraction of the time beats a huge speedup of a small fraction. Optimize what the program actually spends its time on.

Its converse is jokingly called *Lhadma's law*: *do not sacrifice the uncommon case too much* when speeding up the common one. If the 10 % that is left untouched by an optimization is made 10× slower as a side effect, it now takes as long as the whole original program, and the overall speedup becomes a slowdown.

Caution "Lhadma" is "Amdahl" spelled backwards. It is the lecture's own name for the principle, not standard terminology.

### 2.6.2 Diminishing returns

Repeatedly improving the same part of the execution yields *diminishing returns*: once that part has shrunk, it is no longer the common case, and the remaining time is dominated by everything else (Figure 2.1). After each improvement the architect should re-measure and ask what the common case is *now*.

**Example 2.9.** Starting from the previous example (FP = 40 % of time), the first 2× FP speedup gives $S = 1/(0.6 + 0.2) = 1.25$. A second 2× FP speedup on the new machine, where FP is now only $0.2/0.8 = 25$ % of the time, gives just $1/(0.75 + 0.125) \approx 1.14$.

Amdahl's law assumes a fixed problem; if it grows with the machine, the serial part shrinks (Gustafson 1988).

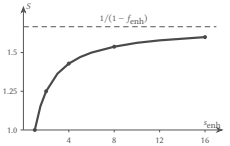



**Figure 2.1:** Overall speedup $S$ for $f_{\text{enh}} = 0.4$ as the enhanced part gets faster. Each further improvement of the same part buys less.

> **Key point — Lesson summary**
> Measure with latency or throughput as appropriate; compare with speedups and summarize them with the geometric mean. The iron law (Equation (2.3)) attributes time to IC, CPI and clock period; Amdahl's law (Equation (2.5)) bounds what any partial improvement can achieve.

# Pipelining

| | |
|---|---|
| **Lecture** | Lesson 3, videos 1–18 |
| **Textbook** | H&P [1] Appendix C.1–C.3 |
| **Keywords** | pipeline, stage, pipeline CPI, stall, flush, control dependence, data dependence, RAW, WAR, WAW, hazard, forwarding, pipeline depth |

The first and most fundamental technique for raising throughput is *pipelining*: instead of finishing one instruction before starting the next, the processor overlaps the execution of several instructions, each in a different stage of completion. This lesson reviews how a pipelined processor works, quantifies its ideal performance, and then studies the *dependences* between instructions that prevent the ideal from being reached.

## 3.1 The idea of pipelining

Executing an instruction involves a sequence of steps that use different pieces of hardware. If the steps are arranged as *stage*s separated by *latch*es, then as soon as instruction $I_1$ leaves stage 1 for stage 2, instruction $I_2$ can enter stage 1. With $k$ stages, up to $k$ instructions are in flight at once.

**Source** Lesson 3, videos 2–3; H&P §C.1.

The consequences for the two metrics of Lesson 2 are different:

- **Throughput** improves by up to a factor of $k$: in steady state one instruction completes every stage time instead of every $k$ stage times.
- **Latency** of an individual instruction does *not* improve. It is the same total work, and the latches between stages add a small overhead, so per-instruction latency slightly increases.

### 3.1.1 Pipelining in a processor

The classic *five-stage pipeline* (Figure 3.1) is used throughout the course as the reference design.

Current x86 cores have about 14–20 stages (Agner Fog, microarchitecture manual); the five-stage model captures every issue that matters here.

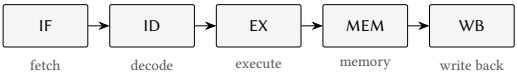



**Figure 3.1:** The five-stage instruction pipeline.

**IF** *instruction fetch*: read the instruction at the *program counter* (PC) from the instruction memory and increment the PC.
**ID** *instruction decode* and register read: determine the operation and read the source operands from the register file.
**EX** *execute*: perform the ALU operation, or compute the memory address for a load/store, or evaluate a branch condition.
**MEM** *memory*: access data memory for loads and stores.
**WB** *write back*: write the result into the register file.

Figure 3.2 shows five independent instructions flowing through the pipeline. After the first four cycles fill the pipeline, one instruction completes per cycle.

Each stage is assumed to take exactly one cycle, the memory access included—an assumption Lessons 12–15 undo.

**Caution** Separate instruction and data memory ports are assumed here; a single port would add *structural* hazards (H&P Appendix C).

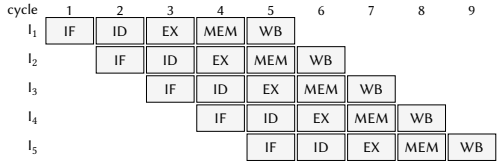



**Figure 3.2:** Instruction timing in a five-stage pipeline: from cycle 5 on, one instruction completes every cycle.

## 3.2 Pipeline CPI

**Source** Lesson 3, video 6; H&P §C.1.

In the ideal case of Figure 3.2 the pipeline completes one instruction per cycle, so the CPI of the iron law (Equation (2.3)) is 1. Two effects raise it above 1.

- *Fill and drain.* The first instruction takes $k$ cycles before anything completes. For a program of $n$ instructions the total is $k + (n-1)$ cycles, i.e. CPI $= (k+n-1)/n \to 1$ as $n$ grows. For real programs ($n$ in the billions) this is negligible.
- *Stalls.* Whenever the pipeline cannot advance an instruction to the next stage, a cycle passes without an instruction completing. These cycles are what actually determine the CPI of a pipelined processor:

$$\text{CPI} \;=\; 1 + \frac{\text{stall cycles}}{\text{instruction}}. \tag{3.1}$$

## 3.3 Stalls and flushes

**Source** Lesson 3, videos 7–8; H&P §C.2.

Two mechanisms lose cycles in a pipeline.

**Definition 3.1** (Stall). A *stall* holds an instruction (and everything behind it) in its current stage for one or more cycles while the instructions ahead of it continue. An empty slot, a *bubble*, travels down the pipeline in its place.

**Definition 3.2** (Flush). A *flush* discards instructions that have already been fetched into the pipeline because they should not have been executed at all, typically because they follow a branch whose outcome was guessed wrongly.

A stall of $c$ cycles costs $c$ cycles; a flush of $m$ instructions costs $m$ cycles, since each flushed instruction occupied a slot in which a useful instruction could have completed.

## 3.4 Control dependences

**Source** Lesson 3, videos 9–10; H&P §C.2.

A *control dependence* exists when whether an instruction executes at all depends on the outcome of an earlier branch. In the five-stage pipeline the branch condition and target are known only at the end of EX.[1] By then the two following instructions have already been fetched (Figure 3.3); if the branch is taken they must be flushed and the fetch restarted at the target.

[1] Early RISC pipelines exposed the wasted slots instead of flushing them: in MIPS and SPARC the instruction after a branch (the *delay slot*) is executed, and the compiler fills it. With many instructions in flight one slot buys little; MIPS Release 6 (2014) added compact branches without a delay slot, and RISC-V never had one.

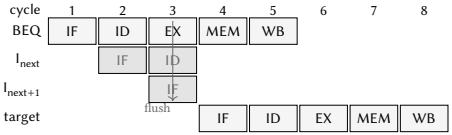



**Figure 3.3:** A taken branch resolved in EX wastes two fetch slots.

The 20 %/60 % below are round illustrative numbers; integer code really does branch about every fifth instruction (H&P ch. 3).

The cost of this on the CPI depends on how often it happens. Typical figures are that 20 % of instructions are branches and 60 % of branches are taken; with the simple policy of always fetching the fall-through instruction ("predict not taken"), only taken branches cause a flush:

$$\text{CPI} = 1 + 0.2 \times 0.6 \times 2 = 1.24. \tag{3.2}$$

**Example 3.3.** In a deeper pipeline where the branch is resolved in stage 10, each taken branch flushes nine instructions. With the same branch statistics the CPI becomes $1 + 0.2 \times 0.6 \times 9 = 2.08$—the processor completes less than one instruction every two cycles. This is why branch prediction (Lesson 4) matters more the deeper the pipeline.

**Tip** Cycles lost to control dependences = (fraction of instructions that are branches) × (fraction mispredicted) × (instructions flushed per misprediction). The last factor is (stage in which the branch resolves) − 1.

## 3.5 Data dependences

**Source** Lesson 3, videos 11–12; H&P §C.2.

A *data dependence* exists between two instructions when they access the same register or memory location and at least one of them writes it. There are three kinds, named by the order of the accesses in program order:

**RAW (read after write)** the later instruction *reads* a value that the earlier one *writes*. Also called a *true* or *flow* dependence: the data really has to flow from one to the other.

**WAR (write after read)** the later instruction *writes* a location that the earlier one *reads*. An *anti* dependence.

**WAW (write after write)** both instructions *write* the same location; the later write must be the one that survives. An *output* dependence.

WAR and WAW are called *false dependences* or *name* dependences: no data moves between the instructions; they merely reuse the same register name. They can be removed by renaming (Lesson 6). A RAW dependence cannot.

RAR—two reads of the same register—is not a dependence: order does not matter.

**Example 3.4.**

```
I1:  ADD R1, R2, R3      ; R1 <- R2 + R3
I2:  SUB R4, R1, R5      ; RAW on R1 with I1
I3:  MUL R1, R6, R7      ; WAW on R1 with I1, WAR with I2
```

## 3.6 Dependences and hazards

A dependence is a property of the *program*; whether it causes a problem depends on the *pipeline*.

**Source** Lesson 3, videos 13–14; H&P §C.2.

**Definition 3.5** (Hazard). A *hazard* is a situation in which a particular pipeline would violate a dependence if it continued unmodified—the dependent instruction would read a stale value or execute when it should not.

A hazard caused by a data dependence is a *data hazard*, one caused by a control dependence a *control hazard*.

In the five-stage pipeline:

- A **RAW** dependence is a hazard only if the instructions are close enough. Registers are written in WB and read in ID, so an instruction depends hazardously on the previous one and the one before it, but an instruction three or more slots behind reads the register after it has been written.
- **WAR** and **WAW** dependences never cause hazards here, because instructions read registers in ID and write in WB *in program order*; a later instruction cannot overtake an earlier one. They become hazards in out-of-order pipelines (Lesson 6).
- **Control** dependences are always hazards, because the pipeline fetches instructions after the branch before the branch has resolved.

This assumes the register file is written in the first half of a cycle and read in the second half (H&P §C.2); otherwise the instruction three slots behind is hazardous too.

> **Key point**
> Dependences are decided by the program; hazards by the pipeline. The same program has more hazards in a deeper or out-of-order pipeline.

## 3.7 Handling hazards

Once a hazard is detected, the pipeline has three options.

1. **Flush** the dependent instructions—for control hazards, where the wrong instructions were fetched.
2. **Stall** the dependent instruction until the value it needs has been written—always correct, but costs cycles.
3. **Fix** the value by *forwarding* (bypassing): route the result directly from the stage that produced it to the stage that needs it, without waiting for it to reach the register file (Figure 3.4).

**Source** Lesson 3, videos 15–16; H&P §C.2.

Lesson 9 adds store-to-load forwarding: a pending store hands its value to a later load inside the load-store queue.

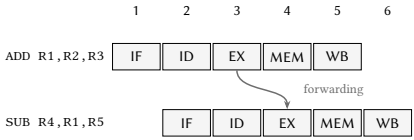



**Figure 3.4:** Forwarding: the ALU result of ADD is fed straight into the EX stage of SUB, avoiding a stall.

Forwarding removes most RAW stalls but not all of them: a value cannot be forwarded *backwards in time*. The result of a load is only available at the end of MEM, whereas the next

**Tip** Stall cycles = (first cycle in which the value can be used) − (cycle in which it is needed); $\leq 0$ means forwarding alone suffices.

instruction needs it at the start of its own EX, which is the same cycle. One stall cycle is therefore unavoidable for a *load-use* pair even with full forwarding.

**Example 3.6.**

```
LW  R1, 0(R2)   ; R1 available at end of MEM (cycle 4)
ADD R3, R1, R4  ; needs R1 in EX (cycle 4): 1 stall
```

With forwarding but no stall the ADD would use a stale R1; without forwarding it would stall two cycles.

On a Skylake core an L1 hit takes 4 cycles, so a load-use pair there costs about 3 stall cycles, not 1 (Intel optimization manual).

## 3.8 How many stages?

**Source** Lesson 3, video 17; H&P §C.1, §1.5.

If pipelining a $k$-stage design gives up to $k\times$ the throughput, why not use hundreds of stages? Three effects pull in different directions as the *pipeline depth* increases:

- **Clock period decreases** (less logic per stage), which lowers $T_{\text{clk}}$ in the iron law—the intended benefit. But the latch overhead per stage does not shrink, so the gain flattens out.
- **CPI increases**: more stages means more instructions in flight between a branch and its resolution, hence more instructions flushed per misprediction, and more distance over which RAW hazards need stalls.
- **Power increases**: more latches switch every cycle and the higher frequency raises dynamic power (Equation (1.1)).

**Caution** A deeper pipeline is not automatically faster: only the product CPI $\times T_{\text{clk}}$ decides (Lesson 2).

A mispredicted branch costs about 16–20 cycles on Skylake-class cores (Agner Fog, microarchitecture manual).

Considering performance alone (the product CPI $\times T_{\text{clk}}$), the optimum is deep—on the order of 30–40 stages. Once power is included, the optimum moves to roughly 10–15 stages, close to where current designs sit.[2]

[2] The deepest pipeline ever shipped in an x86 was the Pentium 4 "Prescott" of 2004: 31 stages, up to 3.8 GHz, with power stopping it there. Its successor, the Core microarchitecture of 2006, went back to 14 stages, and depths have stayed near 15–20 since.

> **Key point — Lesson summary**
> Pipelining improves throughput, not latency; ideal CPI is 1 and every stall or flush adds to it (Equation (3.1)). Control and RAW dependences turn into hazards that cost cycles; forwarding removes most RAW stalls, and branch prediction (next lesson) attacks the control hazards. Pipeline depth trades clock period against CPI and power.

# Branches

| | |
|---|---|
| **Lecture** | Lesson 4, videos 1–48 |
| **Textbook** | H&P [1] §3.3, §3.9, §C.2 |
| **Keywords** | branch prediction, misprediction penalty, predict not taken, branch target buffer, branch history table, two-bit predictor, history-based predictor, PShare, GShare, tournament predictor, hierarchical predictor, return address stack, predecoding |

Lesson 3 showed that control dependences are hazards in every pipeline: the processor has to fetch the instructions that follow a branch long before it knows where the branch goes, and every wrong choice flushes work. This lesson studies how the fetch stage makes that choice. Starting from the simplest policy—always continue with the next instruction—it develops the structures used in real processors: branch target buffers, saturating counters, predictors that learn patterns from branch history, combinations of several predictors, and a dedicated predictor for function returns. Along the way it shows why prediction accuracy matters more the deeper the pipeline.

## 4.1 Branches in a pipeline

**Source** Lesson 4, videos 1–3; H&P §C.2.

Consider a conditional branch `BEQ R1, R2, L` in the five-stage pipeline of Lesson 3. If R1 equals R2, execution continues at label L; the branch is then a *taken branch*, and the address of L is its *branch target address*.

Otherwise execution continues with the next instruction in memory, at PC+4, and the branch is a *not-taken branch*. The pipeline learns which case applies only when the branch has passed through three stages: IF fetches it, ID reads R1 and R2, and EX compares them and computes the target. The branch is *resolved* at the end of EX (Figure 4.1).

The fetch stage cannot wait for that. In cycle 2, while the branch is in ID, and again in cycle 3, it must load *some* instruction. When the branch resolves, two instructions are already in the pipeline behind it. If they are the instructions that really follow the branch, nothing is lost; if not, they are flushed and two cycles are wasted. Fetching nothing during these cycles would waste the same two cycles unconditionally, so it always pays to fetch the instruction that is most likely to be the right one.

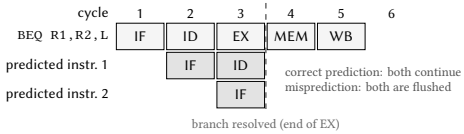



**Figure 4.1:** A branch in the five-stage pipeline. The two instructions fetched before the branch resolves are predictions; they are flushed if the prediction was wrong.

**Definition 4.1** (Branch prediction). The choice, made in the fetch stage, of the address from which the next instruction is fetched, before the current instruction has been decoded or executed, is called *branch prediction*.

**Definition 4.2** (Misprediction). A *branch misprediction* occurs when the branch, once resolved, turns out to go elsewhere than predicted. The instructions fetched from the wrong address are flushed and fetching restarts at the correct address.

### 4.1.1 What the predictor knows

At the time of the fetch the only available information is the PC of the instruction being fetched; the instruction itself has not even been decoded yet. From this address alone the predictor has to answer three questions:

1. Is the instruction a branch at all?
2. If it is, is it taken?
3. If it is taken, what is its target address?

Unconditional jumps, calls and returns are branches that are always taken; for them only questions 1 and 3 matter.

The answer is a single number, the next PC: PC+4 for a non-branch or a not-taken branch, the target address for a taken branch.

## 4.2 Prediction accuracy and performance

**Source** Lesson 4, videos 4–5; H&P §C.2.

With a predictor in place, the only cycles lost to control hazards are those spent on mispredicted paths. Specializing Equation (3.1) to branches gives

$$\text{CPI} = 1 + \frac{\text{mispredictions}}{\text{instruction}} \times \frac{\text{penalty}}{\text{misprediction}}. \tag{4.1}$$

**Definition 4.3** (Accuracy and penalty). The *branch prediction accuracy* is the fraction of branches whose outcome is predicted correctly, so that

$$\frac{\text{mispredictions}}{\text{instruction}} = \frac{\text{branches}}{\text{instruction}} \times (1 - \text{accuracy}),$$

assuming that non-branch instructions are always predicted correctly, with PC+4 as the next PC. The *misprediction penalty* is the number of cycles lost per misprediction; in a pipeline that holds one instruction per stage it equals the number of instructions flushed. A branch resolved in stage $r$ has a penalty of $r - 1$ cycles.

The two factors of Equation (4.1) have different origins. The first is set by the predictor (and by how many branches the program executes); the second is set by the pipeline, namely by how late branches are resolved. A better predictor shrinks the first factor, while a deeper pipeline enlarges the second, so the same predictor costs more in a deeper pipeline. A perfect predictor would reach the ideal CPI of 1 regardless of the depth.

**Example 4.4.** In a program 15 % of the instructions are branches and the predictor is 92 % accurate. Mispredictions per instruction are $0.15 \times 0.08 = 0.012$. If branches are resolved in EX (stage 3, penalty 2), $\text{CPI} = 1 + 0.012 \times 2 = 1.024$. In a pipeline that resolves branches in stage 11 (penalty 10), the same predictor gives $\text{CPI} = 1 + 0.012 \times 10 = 1.12$.

## 4.3 Predict not taken

**Source** Lesson 4, videos 6–8; H&P §C.2.

The alternative to predicting is to refuse to predict: the fetch stage simply waits until it knows the next PC. Since even whether an instruction is a branch is known only after ID, the wait affects every instruction. A non-branch is followed by a fetch two cycles after its own fetch, and a branch, taken or not, by a fetch three cycles later.

The simplest real predictor is to *predict not taken*: always fetch from PC+4. This needs no memory and no hardware beyond what the fetch stage already has, since it increments the PC anyway. It is right for every non-branch and every not-taken branch, and wrong only for taken branches, which cost three cycles as before. Table 4.1 compares the two policies; predict not taken is never worse.

Its accuracy, however, is poor. Most branches are taken—loop branches jump back most of the time, and unconditional jumps, calls and returns are always taken—so predict not taken mispredicts the majority of branches. With the typical statistics used in Lesson 3 it mispredicts on 12 % of all instructions, giving the CPI of Equation (3.2).

**Note 4.5** (Several predictions in flight). In a deep pipeline several branches can be between IF and the resolution stage at the same time, each carrying the prediction made when it was fetched. When a branch resolves as mispredicted, everything fetched after it is flushed, including later branches together with their predictions. A branch fetched on a wrong path therefore never causes a flush of its own: only mispredictions of branches on the correct path cost cycles.

**Table 4.1:** Cycles from the fetch of an instruction until the correct next instruction is fetched, in the five-stage pipeline.

| Instruction | Refuse to predict | Predict not taken |
|---|---|---|
| Taken branch | 3 | 3 |
| Not-taken branch | 3 | 1 |
| Not a branch | 2 | 1 |

## 4.4 The need for better prediction

How much a better predictor is worth depends on the pipeline. A processor that handles $w$ instructions per stage flushes $w$ times as many instructions per misprediction as a one-instruction-wide pipeline of the same depth, so the execution time relative to perfect prediction is

$$\frac{T}{T_{\text{perfect}}} = 1 + \frac{\text{mispredictions}}{\text{instruction}} \times (\text{instructions flushed per misprediction}), \tag{4.2}$$

which for $w = 1$ is exactly the CPI of Equation (4.1).

**Source** Lesson 4, videos 9–11; H&P §3.3.

**Example 4.6.** Let 15 % of the instructions be branches, 70 % of them taken, so that predict not taken is 30 % accurate. Table 4.2 evaluates Equation (4.2) for three predictors and three pipelines: a five-stage pipeline (2 instructions flushed), a pipeline resolving branches in stage 13 (12 flushed), and the same depth with 3 instructions per stage (36 flushed).

Real numbers: a Skylake-class core loses about 16–20 cycles per misprediction (Agner Fog, microarchitecture manual) and must refill a 224-entry reorder buffer (Intel optimization manual).

**Caution** Mispredictions are counted per *instruction*, not per branch: multiply 1 − accuracy by the fraction of instructions that are branches.

**Table 4.2:** Execution time relative to perfect prediction for the branch statistics of Example 4.6.

| Predictor | Mispred./instr. | Stage 3 | Stage 13 | Stage 13, 3 wide |
|---|---|---|---|---|
| Not taken (30 %) | 0.105 | 1.21 | 2.26 | 4.78 |
| 90 % accurate | 0.015 | 1.03 | 1.18 | 1.54 |
| 99 % accurate | 0.0015 | 1.003 | 1.018 | 1.054 |

Three observations follow. In the shallow pipeline even predict not taken costs only 21 %, and improving from 90 % to 99 % accuracy gains less than 3 %. In the deep pipeline predict not taken more than doubles the execution time, and the same improvement is worth a speedup of $1.18/1.018 \approx 1.16$. In the deep and wide pipeline it is worth $1.54/1.054 \approx 1.46$. Modern processors are both deep and wide, which is why they invest heavily in branch prediction.

**Tip** (mispredictions per instruction) × (instructions flushed) is the slowdown itself: $0.015 \times 12 = 0.18$, the 18 % of the middle row.

> **Key point**
> What matters is the *misprediction* rate, not the accuracy. Going from 90 % to 99 % accuracy looks like a small step but removes 90 % of the mispredictions, and the deeper and wider the pipeline, the more each remaining misprediction costs.

## 4.5 The branch target buffer

Predict not taken uses no information about the instruction being fetched. Better predictors use its *history*: what the instruction at this PC did the previous times it was executed. The next PC is predicted as a function of the PC and of this history. Because the instruction stored at an address does not change, whether it is a branch and where a direct branch jumps to are the same every time; and most branches go in the same direction most of the time.

**Source** Lesson 4, videos 12–15; H&P §3.9; [6].

**Definition 4.7** (Branch target buffer). A *branch target buffer* (BTB) is a table, indexed by the PC of the instruction being fetched, that stores the predicted next PC for that instruction.

In IF the BTB is read with the current PC, and its output is used as the next PC. When the branch resolves, the actual next PC is compared with the prediction. On a mismatch the wrongly fetched instructions are flushed and the actual target is written into the BTB entry, so that the next execution of the same instruction is predicted with the new target.

### 4.5.1 A realistic BTB

A BTB cannot have one entry for every possible instruction address. It must be read within the fetch cycle, in addition to the fetch itself, which limits it to a small table—hundreds to a few thousand entries. The PC is therefore mapped to an entry by a simple hash: a few of its low-order bits. Instructions executed close together in time, such as those of a loop, lie close together in memory; they share their high-order bits but differ in the low-order bits, which spreads them over different entries. Two instructions collide only if they are far apart in the code, and such instructions are less likely to be active at the same time.

Real capacities: about 4096 BTB entries on Intel Skylake (commonly reported); AMD's Zen 2 uses three levels of 16, 512 and 7168 entries (Agner Fog, microarchitecture manual).

With fixed-length 4-byte instructions aligned to word boundaries, the two lowest bits of every PC are zero and carry no information, so a BTB with $2^k$ entries is indexed by bits $k + 1$ to 2 (Figure 4.2). With variable-length instructions (x86) the index starts at bit 0.

**Tip** Entry number = (PC $\gg$ 2) & (entries − 1): drop the two always-zero bits, then keep the $k$ low bits.

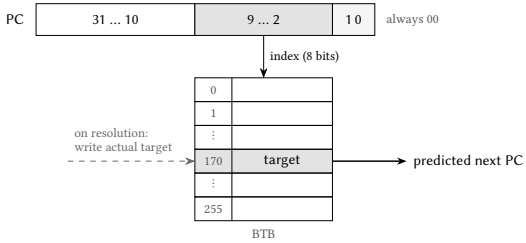



**Figure 4.2:** A 256-entry BTB indexed by PC bits 9–2. The entry is read in IF and written with the actual target when a branch resolves.

**Example 4.8.** In a 256-entry BTB, the instruction at `0x004012A8` uses bits 9–2 of its PC: $(\texttt{0x12A8} \gg 2)$ & `0xFF` = `0x4AA` & `0xFF` = `0xAA`, i.e. entry 170. The next instruction, at `0x004012AC`, uses entry 171; the instruction at `0x004016A8`, 256 instructions further on, shares entry 170.

## 4.6 Direction prediction

**Source** Lesson 4, videos 16–21; H&P §C.2.

A BTB alone answers the direction question implicitly: an entry that holds PC+4 means "not taken". This spends a full target address on every non-branch and every not-taken branch. It is cheaper to split the prediction: a *direction predictor* decides whether the instruction is a taken branch, and the BTB is consulted only when it says so.

**Definition 4.9** (Branch history table). A *branch history table* (BHT) is a direction predictor in the form of a table indexed by low-order PC bits. In its simplest form each entry is a single bit holding the outcome of the last execution of the branch: 0 selects PC+4 as the next PC, 1 selects the target from the BTB. This is the *one-bit predictor* (1BP).

When a branch resolves, its BHT entry is set to the actual outcome, and, if the branch was taken, the target is written into the BTB (Figure 4.3). This organization has three advantages. The BHT entry of a non-branch instruction is never written, stays 0, and correctly predicts PC+4. The BTB needs entries only for taken branches, which are far fewer than all instructions. And since a BHT entry is a single bit, the BHT can have many more entries than the BTB at the same cost, so that different branches rarely share an entry.

**Caution** Neither table is tagged: if a taken branch maps to the same entry, that non-branch is predicted taken too, and the error is caught only at decode.

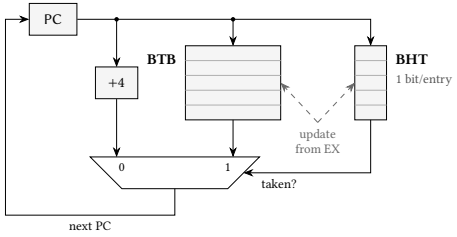



**Figure 4.3:** Next-PC prediction with a BHT and a BTB. The BHT bit selects between PC+4 and the BTB target; both tables are updated when the branch resolves.

**Example 4.10.** An inner loop processes three array elements each time it is entered:

```
0x0400  Inner: LW   R2, 0(R3)     ; load element
0x0404         ADD  R5, R5, R2    ; add to sum
0x0408         ADDI R3, R3, 4     ; next element
0x040C         ADDI R1, R1, -1    ; decrement count
0x0410         BNE  R1, R0, Inner ; T, T, N per run
```

With a 64-entry BHT (index bits 7–2) the five instructions use BHT entries 0 to 4; the BNE uses entry 4. With an 8-entry BTB (bits 4–2) the BNE uses BTB entry 4, which it shares with the instruction at `0x03F0`, eight instructions earlier. If that instruction is not a branch, its BHT

entry (entry 60) stays 0, so the BTB entry is used only by the branch. Table 4.3 follows the BNE over two runs of the loop, starting with all tables cleared.

**Table 4.3:** One-bit prediction of the BNE of Example 4.10 (BHT entry 4) over two runs of the loop.

| Execution | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| BHT[4] before | 0 | 1 | 1 | 0 | 1 | 1 |
| Predicted next PC | 0x414 | 0x400 | 0x400 | 0x414 | 0x400 | 0x400 |
| Outcome | T | T | N | T | T | N |
| Correct? | no | yes | no | no | yes | no |

The first execution also writes 0x400 into BTB entry 4; from then on the target is always right, but the direction is wrong in four of the six executions.

## 4.7 The two-bit predictor

The one-bit predictor works well for branches that always go the same way, but it has two weaknesses.

- A branch that almost always goes one way suffers *two* mispredictions per exception: one on the exception itself, and one on the next normal outcome, because the exception has flipped the bit.
- A branch whose direction changes often is predicted badly. An alternating branch (T, N, T, N, ...) is mispredicted every time, and the short loop of Table 4.3 two times out of three.

**Source** Lesson 4, videos 22–26; H&P §C.2; [7].

**Definition 4.11** (Two-bit predictor). A *two-bit predictor* (2BP, also called a two-bit counter, 2BC) stores in each BHT entry a two-bit counter that is incremented when the branch is taken and decremented when it is not taken.

The counter is a *saturating counter*: it stays at 11 when incremented and at 00 when decremented. Its more significant bit is the *prediction bit*, taken if 1. The less significant bit is the *hysteresis bit*. Together with the prediction bit it records how strongly the counter believes its prediction: the counter is in a strong state when the two bits are equal and in a weak state when they differ, so the four states are strong not taken (00), weak not taken (01), weak taken (10) and strong taken (11) (Figure 4.4). From a strong state, two consecutive outcomes against the prediction are needed to change it, so a single exception costs a single misprediction.

**Tip** Read the two bits as a number 0–3: T adds one, N subtracts one, both clamped, and the prediction is taken iff the value is at least 2.

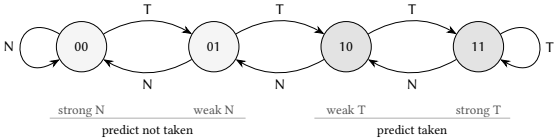



**Figure 4.4:** State diagram of the two-bit saturating counter. T (taken) moves right, N (not taken) moves left; the upper bit is the prediction.

**Table 4.4:** The loop branch of Example 4.10 over three runs of the loop with a two-bit counter starting at 00. Mispredictions are marked ×.

| Execution | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| Outcome | T | T | N | T | T | N | T | T | N |
| Counter before | 00 | 01 | 10 | 01 | 10 | 11 | 10 | 11 | 11 |
| Prediction | N | N | T | N | T | T | T | T | T |
| 2BP | × | × | × | × | | × | | | × |
| 1BP (for comparison) | × | | × | × | | × | × | | × |

Table 4.4 shows the effect on the short loop. Once the counter has reached strong taken (11, before execution 6), a single loop exit only moves it down to weak taken (10) and the next taken outcome restores it, so only the exit is mispredicted: one misprediction per run of the loop instead of two.

**Caution** From a *weak* state one contrary outcome is enough: a two-bit counter does not always need two mispredictions to change its prediction.

### 4.7.1 Initialization and more bits

A counter initialized to a strong state costs two mispredictions if the branch actually goes the other way; one initialized to a weak state costs only one. The weak states, however, are also where the counter is most vulnerable: an alternating branch T, N, T, N, … that starts in 01 moves between 01 and 10 and is mispredicted every time, whereas starting in 00 it is mispredicted only on every other execution. Neither choice is best for every branch: each initial state has outcome sequences for which it does worse. Counters are therefore simply reset to the simplest state, 00.

No predictor is immune: for any predictor there is an outcome sequence that always goes against its current prediction.

Counters with three or more bits add hysteresis: more consecutive exceptions are needed to change the prediction. This helps only for branches whose exceptions come in runs, it makes the counter slower to follow a branch that really changes its behaviour, and it costs more bits. The step from one to two bits is a large improvement; further bits gain little, which is why the two-bit counter is the standard building block.

## 4.8 History-based predictors

**Source** Lesson 4, videos 27–33; H&P §3.3; [8].

Some branches follow a regular pattern that counters cannot capture, because counters only track which direction is more frequent. A branch that tests whether a loop index is even alternates T, N, T, N; the loop branch of Example 4.10 repeats T, T, N. Both are perfectly predictable once the most recent outcomes are known.

**Definition 4.12** (History-based predictor). A *history-based predictor* records the most recent outcomes of a branch (its history) and uses them to select one of several counters, each of which learns what the branch does after one particular history.

### 4.8.1 One-bit history

The simplest version stores in each BHT entry one history bit, the last outcome of the branch, and two two-bit counters: one used when the last outcome was N, one when it was T. To predict, the history bit selects a counter and the counter's upper bit is the prediction. To update, the selected counter is incremented or decremented with the actual outcome, and the outcome becomes the new history bit. An entry needs $1 + 2 \times 2 = 5$ bits.

For the alternating branch, the counter selected after N sees only T and the counter selected after T sees only N. After each has been trained, the branch is predicted correctly every time. For the pattern T, T, N this is not enough: after a T the branch is sometimes taken and sometimes not, so the counter selected after T keeps mispredicting.

### 4.8.2 Two-bit and $N$-bit history

With two history bits and four counters (10 bits per entry, Figure 4.5), each of the four possible histories NN, NT, TN and TT has its own counter. For T, T, N the history TT is always followed by N, while TN and NT are always followed by T. Table 4.5 traces the loop branch: after the counters have been trained, every execution is predicted correctly. The counter for NN is never used again.

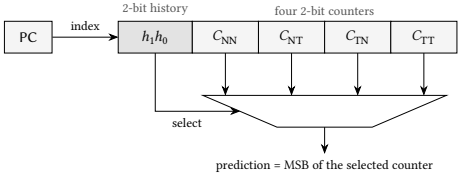



**Figure 4.5:** An entry of a two-bit history predictor. The history bits select one of four two-bit counters; after the branch resolves, the selected counter is updated and the outcome is shifted into the history.

**Definition 4.13** ($N$-bit history predictor). An *$N$-bit history predictor* keeps the last $N$ outcomes of each branch and $2^N$ two-bit counters per entry, one for each possible history. An entry costs

$$N + 2 \cdot 2^N \text{ bits.} \tag{4.3}$$

**Table 4.5:** Two-bit history predictor on the loop branch of Example 4.10, starting with history NN and all counters at 00. The history is written oldest outcome first; mispredictions are marked ×.

| Execution | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| History | NN | NT | TT | TN | NT | TT | TN | NT | TT | TN | NT | TT |
| Counter used | 00 | 00 | 00 | 00 | 01 | 00 | 01 | 10 | 00 | 10 | 11 | 00 |
| Prediction | N | N | N | N | N | N | N | T | N | T | T | N |
| Outcome | T | T | N | T | T | N | T | T | N | T | T | N |
| Mispredicted | × | × | | × | × | | × | | | | | |

After training, an $N$-bit history predictor predicts every repeating pattern whose period is at most $N+1$, because within such a pattern the last $N$ outcomes determine the next one. The price grows exponentially with $N$, and most of it is wasted: a branch with period $p$ uses at most $p$ of its $2^N$ counters. Longer histories also take longer to train, since every counter that is used has to be trained separately.

**Example 4.14.** A loop that always runs 10 iterations has a branch with the pattern T T T T T T T T T N, of period 10. Predicting it perfectly requires $N = 9$, i.e. $9 + 2 \cdot 512 = 1033$ bits per BHT entry, of which the branch uses 10 of the 512 counters. A BHT of 1024 such entries takes more than a megabit.

> **Key point**
> History lets a predictor learn patterns, not only biases. Giving every branch its own set of $2^N$ counters, however, makes long histories prohibitively expensive, and nearly all of those counters are never used.

## 4.9 Shared counters: PShare and GShare

**Source** Lesson 4, videos 34–39; H&P §3.3; [9].

The waste disappears if the counters are not private to a branch. The BHT keeps only an $N$-bit history per entry, and all branches share one large table of two-bit counters.

**Definition 4.15** (Pattern history table). A *pattern history table* (PHT) is a table of two-bit counters shared by all branches and indexed by a combination of the branch's history and its PC, typically their bitwise XOR.

A branch now occupies only as many counters as it has distinct histories. An always-taken branch has the history T T … T and uses a single counter, an alternating branch uses two, and the 10-iteration loop branch of Example 4.14 uses ten. Mixing in the PC bits sends different branches with the same history to different counters. Collisions remain possible, but with a large PHT they are rare.

**Example 4.16.** A BHT of 1024 entries holding 12-bit histories takes 12 288 bits, and a PHT of $2^{12} = 4096$ counters takes 8192 bits: 20 480 bits in total. A 1024-entry predictor with 12-bit history and private counters would need $1024 \times (12 + 2 \cdot 4096) = 8\,400\,896$ bits, about 410 times as much.

### 4.9.1 Private and global history

The history that indexes the PHT can be chosen in two ways (Figure 4.6).

**Definition 4.17** (PShare). The *private branch history* of a branch is the sequence of its own most recent outcomes, kept in its BHT entry. A *PShare predictor* indexes the shared PHT with the private history XORed with the PC.

**Definition 4.18** (GShare). The *global branch history* is the sequence of the most recent outcomes of all branches, kept in a single shift register into which every resolved branch shifts its outcome. A *GShare predictor* indexes the shared PHT with the global history XORed with the PC.

PShare is suited to branches with patterns of their own: alternating branches and short loops. GShare is suited to *correlated* branches, whose outcome depends on the outcomes of other branches executed just before.

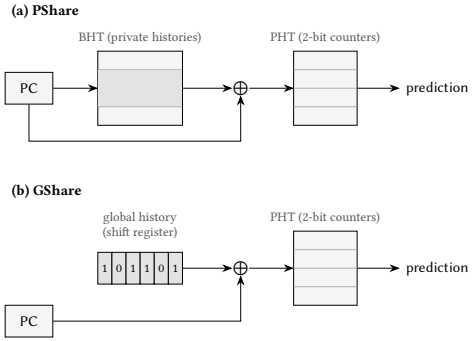



**Figure 4.6:** Predictors with shared counters. (a) PShare combines the branch's own history with the PC; (b) GShare combines the global history of all branches with the PC.

**Example 4.19.** In the fragment below the second branch is taken exactly when the first is not.

```
        BNE  R1, R0, Skip1   ; skip unless R1 == 0
        ADDI R2, R2, 1       ; count zeros
Skip1:  BEQ  R1, R0, Skip2   ; skip if R1 == 0
        ADD  R3, R3, R1      ; sum nonzero values
Skip2:  ...
```

If the value in R1 depends on the data, the private history of either branch looks random, and PShare predicts it poorly. The global history seen by the second branch contains the outcome of the first, so once the PHT counters for these histories have been trained, GShare predicts the second branch perfectly, barring aliasing in the PHT.

A branch whose outcome depends on data may be cheaper to remove than to predict: Lesson 5 replaces such branches with predication.

**Caution** Only GShare is a standard name (McFarling): what the lecture calls PShare is a local- or private-history predictor in the literature, as in Yeh and Patt's PAs schemes.

Neither is better for all branches. GShare can in principle learn a branch's own pattern as well, but the outcomes of the other branches executed in between are interleaved in the global history, so the same pattern needs a considerably longer history—and a larger PHT—than in PShare. Since programs contain both kinds of branches, the natural next step is to use both predictors.

## 4.10 Tournament and hierarchical predictors

**Source** Lesson 4, videos 40–43; H&P §3.3; [9].

Since different branches are best served by different predictors, a processor can combine several predictors and use, for each branch, the one that suits it. The combination can be symmetric or layered.

### 4.10.1 Tournament predictors

A *tournament predictor* runs two predictors, for example GShare and PShare, side by side and decides separately for each branch which of them to trust.

**Definition 4.20** (Meta-predictor). The *meta-predictor* of a tournament predictor is a table of two-bit counters, indexed by the PC, whose counter for a branch predicts which of the two component predictors will be right (Figure 4.7).

The Alpha 21264 combined 1024 local histories of 10 bits, 1024 three-bit local counters, a 4096-entry global table indexed by 12 history bits and a 4096-entry chooser: about 29 kilobits (H&P §3.3).

Both component predictors are trained on every branch as usual. The meta-predictor counter changes only when the components disagree in correctness: it moves towards the component that was right. When both are right or both are wrong, it stays unchanged.

### 4.10.2 Hierarchical predictors

A tournament predictor combines two good—and therefore expensive—predictors and updates both on every branch. Most branches, however, are easy to predict.

**Definition 4.21** (Hierarchical predictor). A *hierarchical predictor* combines a cheap predictor, which handles all branches, with one or more better and more expensive predictors that are used and trained only for the branches the cheap predictor handles poorly.

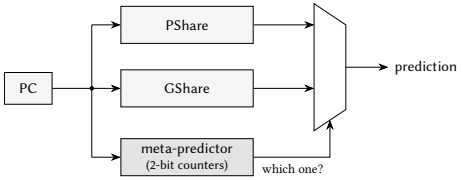



**Figure 4.7:** A tournament predictor. The meta-predictor selects, per branch, the component predictor whose prediction is used.

Because the expensive predictor has far fewer branches to learn, it can be much smaller than in a tournament predictor for the same accuracy, or more accurate for the same size.

A typical arrangement has three levels (Figure 4.8). A large table of two-bit counters indexed by the PC predicts every branch. Above it, a small local-history predictor and a small global-history predictor hold entries only for selected branches. Their entries are *tagged* with bits of the PC, so that a lookup can tell whether the branch has an entry. The prediction comes from the highest level with a matching entry. The counter table is always updated, and a local or global entry that matches the branch is updated as well; a branch that the counter table mispredicts is given an entry in the local predictor, and a branch that the local predictor mispredicts is given one in the global predictor. Branches that the counters predict well take entries in the expensive levels only rarely, so those small tables are left for the branches that need them.[1]

AMD's Zen 2 layers by speed instead: a fast perceptron predicts, a slower TAGE predictor overrides it—about 30 % fewer mispredictions than Zen 1 (AMD).

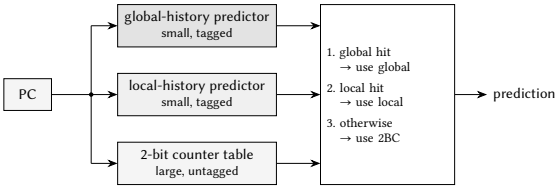



**Figure 4.8:** A hierarchical predictor. Tagged upper levels override the two-bit counter table for the branches they hold.

[1] Intel's Pentium M (2003) is the classic example: a table of two-bit counters covering all branches, backed by tagged local-history and global-history predictors, a loop detector that learns how many iterations a loop branch runs, and a separate predictor for indirect branches (Gochman et al., *Intel Technology Journal* 7(2), 2003).

Table 4.6 summarizes which kind of predictor handles which kind of branch.

**Table 4.6:** Branch behaviour and the simplest predictor that handles it well.

| Branch behaviour | Predictor |
|---|---|
| Almost always the same direction | Two-bit counter |
| Own repeating pattern (alternation, short loop) | Private history (PShare) |
| Correlated with preceding branches | Global history (GShare) |
| Mixture of the above in one program | Tournament or hierarchical |
| Function return | Return address stack (next section) |

## 4.11 The return address stack

A function return (RET) is an unconditional branch, so its direction is trivially predicted as taken. Its target, however, depends on where the function was called from. The BTB remembers only the most recent target, so when a function is called from different places in turn, every return is mispredicted.

**Source** Lesson 4, videos 44–47; H&P §3.9; [10].

**Example 4.22.** A function is called from two places:

```
0x1000          CALL Func     ; return address 0x1004
  ...
0x1040          CALL Func     ; return address 0x1044
  ...
0x2000  Func:   ...
0x2030          RET           ; to 0x1004, then to 0x1044
```

When the first return resolves, the BTB entry of the RET receives `0x1004`; that return is itself mispredicted, since the entry was still empty. The second return, which goes to `0x1044`, is mispredicted and overwrites the entry. If calls from the two places alternate, every return is mispredicted.

**Definition 4.23** (Return address stack). A *return address stack* (RAS) is a small hardware stack in the fetch stage. When a call is fetched, its return address (PC+4) is pushed; when a return is fetched, the top entry is popped and used as the predicted target.

Real return stacks hold 16 entries on Intel cores and 32 on AMD's Zen 2 (Agner Fog, microarchitecture manual).

Since calls and returns nest, the RAS predicts returns almost perfectly (Figure 4.9). Being small, it can fill up when calls nest deeply. When a call finds the RAS full, the push wraps around and overwrites the oldest entry. Returns occur in reverse order of the calls, so the entries needed soonest are the most recent ones, and most calls and returns happen deep in the call tree. Overwriting the oldest entry loses only the predictions for returns from the outermost functions, which are rare, whereas discarding the new return address would lose the predictions for the frequent calls at the deepest level.

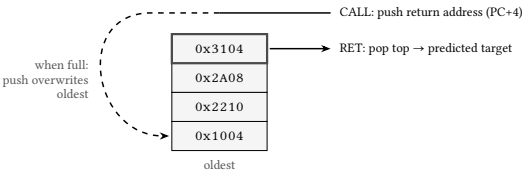



**Figure 4.9:** A four-entry return address stack. Calls push, returns pop; when the stack is full, a push wraps around and overwrites the oldest entry.

### 4.11.1 Recognizing a return

The RAS has to be used while the RET is being fetched, before it has been decoded. There are two ways to know that the instruction is a return.

- **Predict it.** The predictor tables store, per PC, whether the instruction is a return; the entry is set the first time the instruction is decoded. Since the instruction at a given address does not change, this prediction is almost always right.
- **Predecode it.** With *predecoding*, the processor partially decodes each instruction when it is brought from main memory into the processor's cache, and stores a few extra bits with the cached instruction, for example whether it is a branch or a return. The fetch stage reads these bits together with the instruction and knows its kind immediately.

> **Key point — Lesson summary**
> Every fetch is a prediction. A misprediction wastes the work of every instruction it flushes (Equation (4.2)), and the number of flushed instructions grows with pipeline depth and width. Predict not taken is free but inaccurate. A BTB predicts targets and a BHT predicts directions, best with two-bit saturating counters. History-based predictors learn patterns; sharing their counters in a PHT makes long histories affordable, with private history (PShare) for self-repeating branches and global history (GShare) for correlated ones. Tournament and hierarchical predictors combine several predictors, and a return address stack predicts function returns.

# Predication

| | |
|---|---|
| **Lecture** | Lesson 5, videos 1–12 |
| **Textbook** | H&P [1] §A.6, §C.2, §H.4, §H.6 |
| **Keywords** | predication, hard-to-predict branch, if-conversion, conditional move, MOVZ, MOVN, CMOVcc, condition code, break-even misprediction rate, full predication, Itanium, predicate register |

The predictors of Lesson 4 guess most branches correctly, but some branches remain hard to predict for any predictor, typically because their direction depends on the data being processed, and in a deep pipeline every misprediction flushes tens of instructions. This lesson studies the alternative for such branches: *predication*, which executes the work of both directions of a branch and keeps only the results of the correct one, so that nothing is left to mispredict. We examine which branches benefit, how a compiler removes a branch by if-conversion, the conditional move instructions this relies on, when the transformation pays off, and instruction sets in which every instruction can be made conditional.

## 5.1 Prediction and predication

A branch predictor guesses in the fetch stage where a branch goes, and the processor continues to fetch along the guessed path. A correct guess costs nothing. A wrong guess flushes every instruction fetched after the branch, which in a modern processor, several instructions wide and many stages deep, amounts to tens of instructions (Example 4.6). The cost of a branch is thus zero most of the time and large once in a while.

**Source** Lesson 5, videos 1–2; H&P §C.2.

**Definition 5.1** (Predication). Executing the work of *both* directions of a branch and throwing away the results of the direction that turns out to be wrong is called *predication*.

Predication has no misprediction penalty, since nothing is predicted, but it always wastes the work of the direction that is not needed: if both directions contain equally many instructions, half of the work is thrown away. Per execution of the branch, predication wastes the instructions of the unneeded direction, while prediction wastes the misprediction penalty multiplied by the probability of a misprediction. Which is smaller depends on the kind of branch (Table 5.1); predication can be the better choice only for a small if-then-else, whose directions contain far fewer instructions than a misprediction flushes.

**Caution** Nothing is guessed and nothing is undone: both directions run on purpose, and the code itself discards the unneeded results.

**Table 5.1:** Prediction and predication for different kinds of branches.

| Branch | Prediction | Predication | Better |
|---|---|---|---|
| Loop | accurate | also runs the code after the loop in every iteration | prediction |
| Call, return | always taken; BTB and RAS give the target | work on the other path is always wasted | prediction |
| Large if-then-else | occasionally flushes tens of instructions | wastes a large direction every time | prediction |
| Small if-then-else | flushes tens of instructions per misprediction | wastes a few instructions every time | predication, if hard to predict |

For a small if-then-else the choice depends on how well the branch is predicted. If a misprediction flushes 40 instructions and the unneeded direction contains 4, predication wastes less than prediction as soon as more than one execution in ten is mispredicted. Section 5.4 makes this comparison precise for the code that a compiler actually produces.

Real penalties are larger: about 17 cycles on a Skylake-class core (Lesson 4), some 70 instruction slots at four instructions per cycle.

> **Key point**
> Prediction costs nothing when it is right and tens of instructions when it is wrong; predication always wastes the unneeded direction. Branches are therefore predicted by default, and predication is worth considering only for small if-then-else branches that are hard to predict.

## 5.2 If-conversion

**Source** Lesson 5, video 3; H&P §H.4; [11].

Predication needs the work of both directions in the form of instructions that are executed one after the other, with no branch between them. Producing such code is the task of the compiler. Consider the following if-then-else, which sums the values below and above a threshold separately and counts the values below it. If the values of v fall on either side of t in no regular order, the branch is hard to predict.

```
if (v < t) {
  lo = lo + v;
  n  = n + 1;
} else {
  hi = hi + v;
}
```

**Definition 5.2** (If-conversion). The compiler transformation that removes such a branch by turning the control dependence (Lesson 3) of the statements of both directions into a data dependence on the branch condition is called *if-conversion*.

With ordinary instructions, the converted code (Figure 5.1) executes the statements of both directions unconditionally, writing their results into temporaries, and then uses the condition c as data to select the new value of every variable assigned in either direction. When both directions assign a variable x, the selection chooses between their two temporaries, `x = c ? x1 : x2`; when only one direction assigns it, as for each variable in this example, the other alternative is the old value. The converted code contains no branch.

**Caution** The unneeded direction also runs: it must not store to memory or fault, e.g. by loading via an invalid pointer.

Removing the branch also leaves one straight-line stretch of code, which the compiler can then schedule as a whole (Lesson 10).

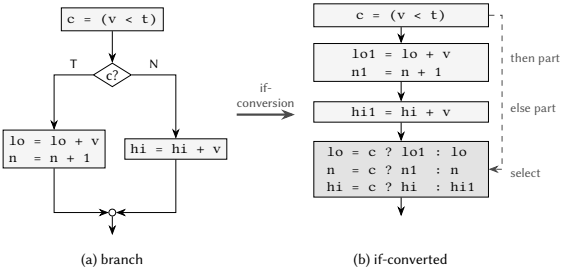



**Figure 5.1:** If-conversion of a small if-then-else. (a) The original code branches on c. (b) The converted code computes both directions into temporaries and uses c as data to select the results.

Plain arithmetic can also select without a branch, by masking both alternatives with AND and OR, at several instructions each.

The transformation gains something only if the selections can be carried out without a branch. Translated into ordinary branch instructions, each selection is itself an if-then-else on c, as hard to predict as the branch that was removed, so the converted code would still mispredict and in addition execute the work of both directions. If-conversion is cheap only if the instruction set can make the effect of an instruction conditional.

## 5.3 Conditional move instructions

**Source** Lesson 5, videos 4–5; H&P §H.4, §A.6.

The simplest such instruction makes only a copy conditional.

**Definition 5.3** (Conditional move). A *conditional move* is an instruction that copies a value into its destination if a condition holds and leaves the destination unchanged otherwise.

The instruction is executed in either case; only whether it writes depends on the condition. It does not change the flow of control, so there is nothing to predict.

The MIPS instruction set has two conditional moves whose condition is the contents of a register. The instruction *MOVZ* (move if zero) copies register Rs into register Rd if register Rt holds zero, and the instruction *MOVN* (move if not zero) copies it if Rt does not hold zero:

```
MOVZ Rd, Rs, Rt      ; if (Rt == 0) Rd <- Rs
MOVN Rd, Rs, Rt      ; if (Rt != 0) Rd <- Rs
```

A selection `x = c ? x1 : x2` becomes two moves into the register of x, of which exactly one writes. With c in R3, x1 and x2 in R1 and R2, and x in R4:

```
MOVN R4, R1, R3      ; if c != 0: x <- x1
MOVZ R4, R2, R3      ; if c == 0: x <- x2
```

The condition register is typically set by a comparison such as SLT, which writes 1 if the condition holds and 0 otherwise.

**Tip** After SLT the condition register is nonzero exactly when the condition holds: the MOVN carries the then part and the MOVZ the else part.

**Example 5.4.** With v in R1, t in R2, and lo, n, hi in R4, R5, R6, the code of Section 5.2 compiles with a branch into

```
      SLT  R3, R1, R2     ; R3 <- (v < t)
      BEQ  R3, R0, Skip   ; v >= t: go to else part
      ADD  R4, R4, R1     ; lo <- lo + v
      ADDI R5, R5, 1      ; n  <- n + 1
      J    Cont           ; skip else part
Skip: ADD  R6, R6, R1     ; hi <- hi + v
Cont: ...
```

and, if-converted, into

```
      SLT  R3, R1, R2     ; R3 <- (v < t)
      ADD  R7, R4, R1     ; then part into R7, R8
      ADDI R8, R5, 1
      ADD  R9, R6, R1     ; else part into R9
      MOVN R4, R7, R3     ; lo <- R7 if v < t
      MOVN R5, R8, R3     ; n  <- R8 if v < t
      MOVZ R6, R9, R3     ; hi <- R9 if v >= t
```

The temporaries R7–R9 are needed because R4–R6 must keep their old values until the moves decide which of them change. Each variable here is assigned in only one direction, so a single move per variable suffices.

Other instruction sets base conditional moves on flags rather than on the contents of a register.

**Definition 5.5** (Condition code). A *condition code*, or flag, is a one-bit status value, for example zero, negative, carry or overflow, that arithmetic and comparison instructions set as a side effect and that later instructions test.

The x86 instruction set has a family of conditional moves, *CMOVcc*, in which cc names a condition on the condition codes: CMOVZ moves if the zero flag is set, CMOVNZ if it is clear, CMOVG if the last comparison found "greater", CMOVL if it found "less", and so on. A common pattern assigns one alternative unconditionally and lets a single conditional move overwrite it with the other, as in this computation of $m = \max(a, b)$:

```
MOV   EAX, EBX      ; m <- a
CMP   EBX, ECX      ; set flags from a - b
CMOVL EAX, ECX      ; if a < b: m <- b
```

## 5.4 Performance of if-conversion

If-conversion is not free: the converted code executes the work of both directions and the conditional moves on every execution. To compare the two versions of a code fragment we count pipeline slots, as in Lesson 3: every executed instruction occupies one slot, and every misprediction wastes as many slots as it flushes instructions. Let the branch version execute $n_b$ instructions per execution on average and mispredict $m$ times per execution with a penalty of $P$ instructions, and let the converted version execute $n_c$ instructions. If-conversion pays off when

$$n_c < n_b + mP \qquad \Longleftrightarrow \qquad m > \frac{n_c - n_b}{P}. \tag{5.1}$$

**Source** Lesson 5, videos 6–7; H&P §C.2.

**Caution** $m$ is the rate of this one branch, per execution of the fragment, not the per-instruction rate of Lesson 4; $P$ counts instructions, not cycles.

The right-hand side is the *break-even misprediction rate*: the extra instructions of the converted code, measured in units of the misprediction penalty. Compared with the rough comparison of Section 5.1, the extra instructions include the conditional moves, less the branch and jump instructions that the conversion removes.

**Tip** Each extra instruction of the converted code raises the break-even rate by $1/P$: with $P = 20$, five percentage points of predictor accuracy apiece.

**Example 5.6.** The branch version of Example 5.4 executes five instructions when the then part applies (SLT, BEQ, ADD, ADDI, J) and three otherwise (SLT, BEQ, ADD). If both parts apply equally often, $n_b = 4$; the converted version always executes $n_c = 7$. Suppose the predictor

is 75 % accurate on this branch and $P = 20$. The branch version costs $4 + 0.25 \times 20 = 9$ slots per execution and the converted version 7, a speedup of $9/7 \approx 1.29$ for the fragment. By Equation (5.1) the break-even rate is $m = 3/20 = 0.15$: the branch version is faster only if the predictor is more than 85 % accurate. With $P = 40$ the break-even rate halves to 0.075, an accuracy of 92.5 %.

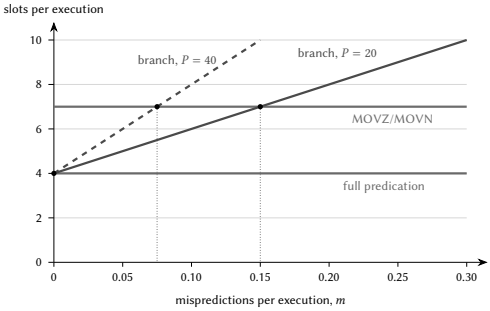



**Figure 5.2:** Slots per execution of the fragment of Example 5.6 as a function of the misprediction rate. Dots mark the break-even rates; the fully predicated version is discussed in Section 5.6.

Figure 5.2 shows the comparison as a function of $m$. The cost of the branch version grows linearly with the misprediction rate, with a slope equal to the penalty, while the converted version costs the same for every $m$. A deeper or wider pipeline raises the penalty and so favours if-conversion; larger then and else parts raise $n_c - n_b$ and favour the branch.

> **Key point**
> If-conversion replaces an occasional large penalty by a constant overhead. It pays off when the mispredictions per execution times the penalty exceed the extra instructions of the converted code (Equation (5.1)).

## 5.5 Benefits and costs of conditional moves

**Source** Lesson 5, video 8; H&P §H.4.

Removing branches with conditional moves has one benefit and several costs.

- **Benefit:** hard-to-predict branches, and with them their mispredictions, disappear.
- **Compiler support** is needed. The processor executes the code it is given; a branch is removed only if the compiler recognizes a suitable if-then-else and converts it.
- **More registers** are needed: every result is first computed into a temporary.
- **More instructions** are executed: the work of both directions, even on executions where the branch would have been predicted correctly, and the conditional moves that select the results.

Of these, the compiler support, the removal of the branch and the execution of both directions are inherent to predication; the extra registers and the moves are not. They arise because only the move is conditional: every result has to be computed into a temporary so that a conditional move can decide whether to keep it. If the instruction that computes a result could itself be made conditional, neither the temporaries nor the moves would be needed.

## 5.6 Full predication

**Source** Lesson 5, videos 9–12; H&P §H.4, §H.6; [12].

A conditional move is a separate opcode, and the only operation it makes conditional is a copy. The alternative is support in the instruction set for making every instruction conditional.

**Definition 5.7** (Full predication). An instruction set provides *full predication* if condition bits are added to every instruction: the instruction takes effect only if its condition holds, and has no effect otherwise.

The IA-64 instruction set of Intel's Itanium processors is an example (Figure 5.3). In most IA-64 instructions, the six least significant bits of the 41-bit instruction hold a *qualifying*

*predicate*: the number of one of 64 one-bit registers, each called a *predicate register*. The instruction is fetched and occupies its pipeline slot in either case, but it takes effect only if the selected predicate register holds 1; otherwise it is annulled: it writes no result and raises no exception. Predicate register 0 always holds 1, so an instruction whose qualifying predicate is 0 is unconditional. The predicate registers are set by compare instructions, which write a pair of predicates with complementary values, one for the condition and one for its negation.

A 128-bit bundle holds three 41-bit instructions and a 5-bit template; IA-64 also has 128 integer and 128 floating-point registers (Lesson 11; H&P §H.6).

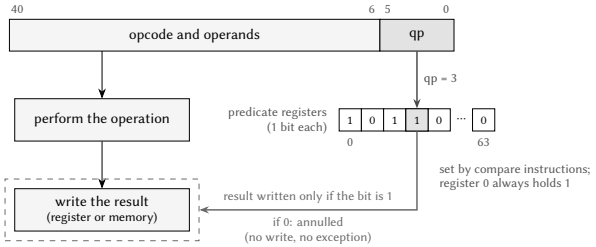



**Figure 5.3:** Full predication in IA-64. The qualifying predicate (qp) of an instruction selects one of 64 one-bit predicate registers. The instruction passes through the pipeline in either case, but its result is written only if that register holds 1; otherwise the instruction is annulled, with no write and no exception.

**Example 5.8.** With full predication, if-conversion guards each statement of both directions directly with the condition or its negation, and the code of Section 5.2 becomes

```
      CMPLT P1, P2, R1, R2 ; P1 <- (v < t), P2 <- !P1
(P1)  ADD   R4, R4, R1     ; lo <- lo + v  if v < t
(P1)  ADDI  R5, R5, 1      ; n  <- n + 1   if v < t
(P2)  ADD   R6, R6, R1     ; hi <- hi + v  if v >= t
```

where the qualifying predicate is written in parentheses before the instruction and CMPLT denotes a compare instruction that sets P1 to 1 and P2 to 0 if R1 is less than R2, and the reverse otherwise. The additions themselves are conditional, so they write directly into the variables, and the compare replaces both the SLT and the branch. Table 5.2 compares the three versions with the numbers of Example 5.6.

**Table 5.2:** Three versions of the example fragment with a 75 % accurate predictor and $P = 20$.

| Version | Instructions | Temporaries | Moves | Mispred. | Slots |
|---|---|---|---|---|---|
| Branch | 5 or 3 | 0 | 0 | 0.25 | 9 |
| MOVZ/MOVN | 7 | 3 | 3 | 0 | 7 |
| Full predication | 4 | 0 | 0 | 0 | 4 |

The fully predicated version executes the compare and the instructions of both directions, and nothing else.[1] Compared with the branch version it executes the unneeded direction but no branch and no jump; here the two balance, and its $n_c = 4$ equals the average $n_b$ of the branch version. Its break-even rate by Equation (5.1) is $m = (4 - 4)/20 = 0$: the branch version is not faster even with perfect prediction, and every misprediction makes it slower (Figure 5.2). In general the fully predicated version executes one instruction fewer than the MOVZ/MOVN version per conditional move, so with $M$ moves its break-even rate is lower by $M/P$; if the conditional moves test a register that already holds the condition, they need no compare while full predication still needs one, and the difference is $(M - 1)/P$. By lowering the break-even rate, full predication makes if-conversion pay off also for branches that are predicted too well to justify conditional moves.

[1] IA-64 was developed jointly by HP and Intel from 1994, and full predication was part of its plan to expose parallelism at compile time instead of in hardware. The first Itanium shipped in 2001; it never displaced x86-64, and Intel shipped the last one in July 2021 (Lesson 11; Intel press releases).

ARM's 32-bit A32 instructions carry a 4-bit condition field, so nearly all of them are predicated; AArch64 (2011) dropped it, keeping conditional select (CSEL) (Arm Architecture Reference Manual).

**Key point — Lesson summary**
Branch prediction costs nothing when right and tens of instructions when wrong; predication executes both directions of a branch and never mispredicts, at the price of always doing the unneeded work. Branches are therefore predicted by default, and compilers use predication only for small, hard-to-predict if-then-else branches, not for loops, calls, returns or large if-then-else branches. If-conversion removes such a branch by turning the control dependence into a data dependence on the condition. With ordinary instructions it computes both directions into temporaries and selects the results with conditional moves, such as MOVZ and MOVN in MIPS or CMOVcc on the condition codes in x86. If-conversion pays off when the mispredictions per execution times the penalty exceed its extra instructions (Equation (5.1)); deeper pipelines favour it. Full predication, with a condition on every instruction as in IA-64, avoids the temporaries and moves and so lowers the break-even misprediction rate.

# ILP

| | |
|---|---|
| **Lecture** | Lesson 6, videos 1–20 |
| **Textbook** | H&P [1] §3.1, §3.4; [13, 14] |
| **Keywords** | RAW dependence, WAW dependence, false dependence, architectural register, physical register, register renaming, register alias table, instruction-level parallelism, ideal processor, instructions per cycle, processor width, in-order execution, out-of-order execution |

Branch prediction (Lesson 4) and predication (Lesson 5) remove most of the cycles that control dependences cost a pipeline, which brings its CPI close to the ideal value of 1. Going below 1 requires executing several instructions in the same cycle, and what prevents this are the data dependences between instructions. This lesson shows that only read-after-write dependences are fundamental, that the others are removed by register renaming, and that the remaining dependences define the instruction-level parallelism of a program: the bound against which the instructions per cycle of real processors are measured.

## 6.1 All instructions in the same cycle

**Source** Lesson 6, videos 1–3; H&P §3.1.

A pipeline that never stalls or flushes completes one instruction per cycle, so its CPI is 1 (Equation (3.1)). A lower CPI means that more than one instruction completes per cycle. In the extreme, all $N$ instructions of a program would be fetched in the same cycle, decoded together in the next cycle, executed together in the cycle after that, and so on (Figure 6.1). A five-stage processor would complete the whole program in 5 cycles, a CPI of $5/N$, which tends to 0 for long programs.




**Figure 6.1:** The ideal: all $N$ instructions pass through the stages in the same five cycles.

A real program cannot be executed this way, because its instructions use the results of other instructions. Consider the following five instructions, the running example of the first half of this lesson:

```
I1:  ADD R3, R1, R2      ; R3 <- R1 + R2
I2:  SUB R5, R3, R4      ; R5 <- R3 - R4
I3:  AND R6, R1, R4      ; R6 <- R1 AND R4
I4:  OR  R4, R2, R7      ; R4 <- R2 OR R7
I5:  XOR R5, R1, R7      ; R5 <- R1 XOR R7
```

When all five instructions are in the execute stage in the same cycle, I2 computes R3 − R4 while I1 is still computing R3, so I2 uses the old value of R3. Forwarding (Figure 3.4) cannot correct this: it delivers a result computed in one cycle to an instruction that executes in a later cycle, whereas here the result and its use fall into the same cycle. How close a processor can come to the ideal is therefore determined by the data dependences of the program.

## 6.2 Dependences in same-cycle execution

**Source** Lesson 6, videos 4–6; H&P §3.1.

The read-after-write dependence of I2 on I1 must be respected: I2 can execute no earlier than one cycle after I1, when the result of I1 can be forwarded to it. I3, I4 and I5 read no register that an earlier instruction writes, so they can still execute in the same cycle as I1. Figure 6.2 shows the resulting timing: I2 waits one cycle, and I3 to I5 execute before I2 although they follow it in the program. RAW dependences therefore bound how far the CPI can fall even with unlimited width: if every instruction read the result of its predecessor, one instruction would complete per cycle and the CPI would remain 1.

Executing instructions ahead of an earlier one breaks the write-after-write dependence between I2 and I5. Both write R5, and in program order I5 writes last, so after these instructions R5 must hold the result of I5; every later instruction that reads R5 expects that value. In Figure 6.2, however, I5 writes R5 in cycle 5 and the delayed I2 overwrites it in cycle 6. To keep the correct value, I5 would also have to be delayed until after the write of I2, although its computation needs nothing from I2.

**Tip** The last word of the name is what the *earlier* instruction does: in write-after-read the earlier instruction reads.

A write-after-read dependence is at risk in the same way. I4 writes R4, which I2 reads, and program order requires I2 to use the old value of R4. Once I2 is delayed, the new value produced by I4 already exists when I2 executes, and I2 must be kept from using it. A single RAW delay thus spreads: every later instruction that writes a register the delayed instruction

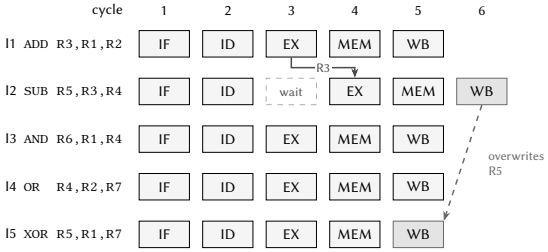



**Figure 6.2:** Same-cycle execution of the running example. The RAW dependence on R3 delays I2 by one cycle; as a consequence I2 writes R5 after I5, which violates their WAW dependence.

reads or writes must be delayed as well, so that every instruction still sees the register values of its position in the program.

## 6.3 Removing false dependences

**Source** Lesson 6, videos 7–8; H&P §3.1.

The three kinds of data dependence of Lesson 3 differ in nature. In a RAW dependence the later instruction uses the value computed by the earlier one: the data really flows from one instruction to the other, and no hardware can let the reader execute before the writer. WAW and WAR dependences are false dependences: no value passes between the two instructions. They exist only because the program stores different values in the same register, a consequence of the limited number of registers and of the fact that an instruction executed repeatedly, for example in every iteration of a loop, always writes the same register. In the example, the results of I2 and I5 are unrelated values that happen to be named R5.

**Caution** H&P calls these *name dependences*: WAR is an *antidependence*, WAW an *output dependence*.

False dependences therefore disappear if different values are kept in different places. One way to achieve this is to keep several versions of a register. When I2 and I5 both write R5, the processor stores both results instead of letting one overwrite the other, and each instruction that reads R5 uses the version selected by program order: an instruction located between I2 and I5 uses the result of I2, an instruction after I5 the result of I5. The order in which the two writes actually happen no longer matters, so I5 can execute as early as if I2 did not exist. Likewise, I2 reads the old version of R4 no matter when I4 produces the new one.

The difficulty is the bookkeeping. For every read, the processor must find the right version, the one produced by the closest preceding writer in program order, and it must recognize when a version is no longer needed. Register renaming performs this bookkeeping systematically.

## 6.4 Register renaming

**Source** Lesson 6, videos 9–11; H&P §3.1, §3.4; [13].

Renaming separates the registers that a program names from the storage in which the processor keeps their values.

**Definition 6.1** (Architectural and physical registers). A register defined by the instruction set, and therefore named in programs (for example R0 to R31), is an *architectural register*. A storage location in which the processor actually keeps a register value is a *physical register*. A processor that renames has many more physical registers than architectural ones.

**Definition 6.2** (Register renaming). Rewriting each instruction, as it is decoded, so that it reads and writes physical instead of architectural registers, with a newly allocated physical register for every result, is called *register renaming*.

**Definition 6.3** (Register alias table). The *register alias table* (RAT) has one entry per architectural register. The entry names the physical register that holds the most recent value of that architectural register, in program order, among the instructions renamed so far.

The lecture expands RAT as *register allocation table*; both names denote the same table.

Each instruction is renamed in three steps (Figure 6.3):

1. For each source register, read its RAT entry; the renamed instruction reads that physical register.
2. Allocate a free physical register, one that holds no value still needed, for the result; the renamed instruction writes it.

3. Overwrite the RAT entry of the destination register with the new physical register, so that later instructions reading this architectural register use the new result.

The sources must be looked up before the destination entry is overwritten; otherwise an instruction such as `ADD R1, R1, R2` would read its own result instead of the previous value of R1.

Lessons 7 and 8 show how renaming is combined with out-of-order execution in real designs.

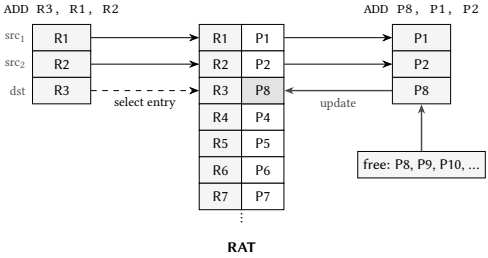



**Figure 6.3:** Renaming `ADD R3, R1, R2`. R1 and R2 are looked up in the RAT, the result is given the free physical register P8, and the RAT entry of R3, which held P3, is overwritten with P8.

The physical registers are exactly the versions of Section 6.3. Every write creates a new version, and the RAT records which version is current in program order, so a reader automatically uses the version produced by its closest preceding writer.[1]

[1] Renaming can even replace an instruction: Intel cores from Ivy Bridge on can execute many register-to-register moves in the RAT alone, pointing the destination entry at the source's physical register. The move then uses no execution unit and has no latency (Agner Fog, microarchitecture manual).

Real sizes: Skylake has 180 integer and 168 vector physical registers for 16 architectural ones of each kind (Intel; WikiChip).

**Example 6.4.** We rename the running example. Before I1, the RAT maps R1 to R7, the registers the example uses, to P1 to P7, and the other architectural registers to physical registers that the example does not use; P8, P9, ... are free and are allocated in increasing order. Table 6.1 lists each renamed instruction and the RAT entries after it. I2 reads P8, the result of I1, because I1 has already updated the entry of R3. I3 reads P4, the old R4, before I4 maps R4 to P11. The two writes to R5 go to P9 and P12, and at the end the RAT maps R5 to P12, so an instruction that follows reads the result of I5, as program order requires.

**Table 6.1:** Renaming the running example. Entries changed by an instruction are in bold; R1, R2 and R7 keep P1, P2 and P7 throughout.

| | Instruction | Renamed | R3 | R4 | R5 | R6 |
|---|---|---|---|---|---|---|
| | (initial) | | P3 | P4 | P5 | P6 |
| I1 | `ADD R3, R1, R2` | `ADD P8, P1, P2` | **P8** | P4 | P5 | P6 |
| I2 | `SUB R5, R3, R4` | `SUB P9, P8, P4` | P8 | P4 | **P9** | P6 |
| I3 | `AND R6, R1, R4` | `AND P10, P1, P4` | P8 | P4 | P9 | **P10** |
| I4 | `OR R4, R2, R7` | `OR P11, P2, P7` | P8 | **P11** | P9 | P10 |
| I5 | `XOR R5, R1, R7` | `XOR P12, P1, P7` | P8 | P11 | **P12** | P10 |

## 6.5 Dependences after renaming

**Source** Lesson 6, video 12; H&P §3.4.

In a renamed program every result is written into its own physical register. No two instructions write the same location, and no instruction writes a location that an earlier instruction reads, so WAW and WAR dependences between register accesses no longer exist. Every read, on the other hand, still refers to the result of the instruction that produced the value in the original program, so every RAW dependence is preserved. In Table 6.1 the only dependence left is that of I2 on I1 through P8.

Dependences through memory locations are not affected by register renaming; Lesson 9 treats them.

Same-cycle execution now only has to respect this one dependence. I1, I3, I4 and I5 execute in the first cycle and I2 in the second, with the result of I1 forwarded to it. I2 and I5 write P9 and P12, so the order of their writes is irrelevant, and I2 reads P4, which I4 does not touch. The five instructions execute in two cycles, and every later reader still finds the correct values through the RAT.

**Key point**
RAW dependences are the only true dependences between instructions. Register renaming removes all WAW and WAR dependences on registers, so the only constraint left on executing instructions in parallel is the flow of data itself.

## 6.6 Instruction-level parallelism

**Source** Lesson 6, videos 13–15; H&P ch. 3; [14].

Once false dependences are removed, the number of cycles needed to execute a program depends only on its RAW dependences. This number is a property of the program itself, not of a particular processor. It is measured on a hypothetical *ideal processor*, which

- renames registers, so that no false dependences exist;
- executes every instruction in the first cycle in which all the values it reads are available: the first cycle if it depends on no earlier instruction, otherwise the cycle after the latest of the instructions it depends on;
- has no limit on the number of instructions it executes in a cycle;
- has unlimited hardware, so that no instruction waits for a busy execution unit: there are no structural dependences;
- executes every instruction in exactly one cycle, whatever its operation.

**Definition 6.5** (Instruction-level parallelism). The *instruction-level parallelism* (ILP) of a program that executes $N$ instructions is

$$\mathrm{ILP} \;=\; \frac{N}{C}, \tag{6.1}$$

where $C$ is the number of cycles in which the ideal processor executes them.

The cycle $c(I)$ in which the ideal processor executes instruction $I$ follows directly from the RAW dependences:

$$c(I) \;=\; 1 + \max_{J \in D(I)} c(J), \qquad C = \max_{I} c(I), \tag{6.2}$$

where $D(I)$ is the set of instructions whose results $I$ reads, and the maximum over an empty set is 0. $C$ is thus the length of the longest chain of RAW dependences in the program. Since no processor can execute an instruction before the values it needs exist, the ILP is an upper bound on the rate, in instructions per cycle, at which any processor can execute this program (Section 6.8).

**Tip** Renaming need not be done on paper: for each register read, find the closest *preceding* writer; later writes do not matter.

**Example 6.6.** We compute the ILP of the following eight instructions:

```
I1:  ADD R1, R2, R3
I2:  MUL R4, R1, R5     ; R1 from I1; R5 written later
I3:  SUB R6, R2, R7     ; R2, R7 written only later
I4:  ADD R1, R6, R8     ; R6 from I3
I5:  AND R9, R1, R8     ; R1 from I4, not from I1
I6:  OR  R2, R8, R3
I7:  XOR R7, R4, R2     ; R4 from I2, R2 from I6
I8:  ADD R5, R3, R8
```

I1, I3, I6 and I8 read only registers not written before them and execute in cycle 1. I2 and I4 depend on instructions of cycle 1 and execute in cycle 2. I5 and I7 each depend on an instruction of cycle 2 and execute in cycle 3 (Figure 6.4). Hence $C = 3$ and $\mathrm{ILP} = 8/3 \approx 2.67$.[2] Two pitfalls are visible. R1 is written by both I1 and I4, and I5 reads the value of I4; taking I1 as its producer would wrongly place I5 in cycle 2. And I3 reads R2 and R7, which I6 and I7 write only later; these are WAR dependences, removed by renaming, and they do not delay I3.

[2] Real programs offer more, but not much more. Wall simulated benchmarks under models ranging from realistic to impossibly good, though not perfect, branch prediction, renaming and alias analysis, and found the average parallelism rarely above 7, with about 5 more common; only perfect techniques gave much more [14].

## 6.7 Structural and control dependences

**Source** Lesson 6, video 16; H&P §3.1.

**Caution** H&P calls this a structural *hazard*: it comes from the hardware, not from the program, so it is not really a dependence.

Structural dependences, in which an instruction waits for hardware that another instruction is using, do not arise on the ideal processor, which has enough hardware for every instruction that is ready. Branches, however, add control dependences: whether the instructions after a branch are executed depends on its outcome. The ideal processor that defines the ILP is also assumed to predict every branch perfectly. It therefore knows from the start which instructions are executed, and the instructions after a branch do not wait for the branch to execute. The

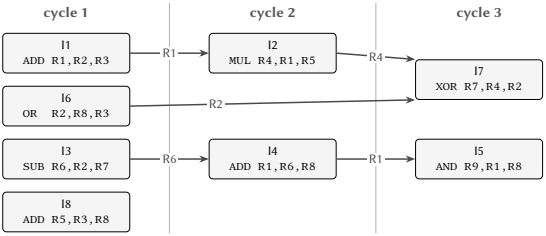



**Figure 6.4:** RAW dependences of Example 6.6, labelled with the register that carries the value. Each instruction is placed in the cycle in which the ideal processor executes it; the longest chain has length 3.

branch itself is an instruction like any other: it counts towards $N$ and executes one cycle after the instructions that produce its operands. Instructions skipped because of the branch outcome are not executed and do not count.

Predication (Lesson 5) removes the control dependence instead, at the price of executing both paths.

**Example 6.7.** In the following fragment the branch is taken, so I4 is not executed:

```
I1:         ADD R1, R2, R3     ; cycle 1
I2:         SUB R4, R1, R5     ; cycle 2: R1 from I1
I3:         BEQ R4, R6, Skip   ; cycle 3: R4 from I2
I4:         ADD R7, R8, R9     ; not executed
I5:  Skip:  XOR R8, R2, R6     ; cycle 1
I6:         AND R9, R8, R1     ; cycle 2: R8 from I5
```

With perfect branch prediction I5 executes in cycle 1 although the branch executes only in cycle 3. Five instructions execute in 3 cycles, so ILP $= 5/3 \approx 1.67$. If I5 and I6 had to wait for the branch, they would execute in cycles 4 and 5, and the ILP would drop to 1.

## 6.8 ILP and IPC

The ILP describes a program on the ideal processor; a real processor is described by the rate it actually achieves.

**Source** Lesson 6, videos 17–20; H&P ch. 3.

**Definition 6.8** (Instructions per cycle). The average number of instructions a processor completes per cycle is its *instructions per cycle* (IPC). It is the reciprocal of the CPI of the iron law (Equation (2.3)): IPC $= 1/\text{CPI}$.

**Caution** The IPC of a whole program is total instructions over total cycles, not the average of the IPCs of its phases.

For every processor and program, IPC $\leq$ ILP, because the ideal processor executes each instruction as early as the dependences allow. A real processor falls short of the ideal for several reasons.

**Width** It can execute only a limited number $w$ of instructions per cycle. A *narrow* processor executes one or two, a *wide* one four or more. Whatever the program, IPC $\leq w$.

Real widths: Skylake is designed for 4 instructions per clock, Ice Lake for 5 (Agner Fog, microarchitecture manual).

**Execution units** It has a limited number of adders, multipliers and other execution units, and operations such as multiplication may take several cycles. An instruction whose kind of unit is busy waits even when its inputs are ready: a structural dependence, which the ideal processor does not have.

**Execution order** A processor with *in-order execution* never executes an instruction before an instruction that precedes it in the program. Consecutive independent instructions may execute in the same cycle, but an instruction that has to wait for its inputs also holds back every instruction behind it. A processor with *out-of-order* execution may execute any instruction whose inputs are available (Lesson 7).

An out-of-order processor needs renaming to do so; without it, it would also have to respect the WAW and WAR dependences of Section 6.2. An in-order processor reads registers in program order and, if all its instructions take equally long, also writes them in program order, so it needs no renaming.

**Example 6.9.** We run the program of Example 6.6 on four processors, each executing every instruction in one cycle and having enough execution units of every kind; the out-of-order ones rename registers and, when more instructions are ready than the width allows, execute the oldest ones first. Table 6.2 lists the instructions executed in each cycle. The in-order

processors execute only I1 in cycle 1, because I2 must wait for R1 and blocks I3 to I8. The same blocking recurs at I4 (waiting for R6), at I5 (waiting for R1 from I4) and at I7 (waiting for R2), so these processors need 5 cycles whether two or four wide. The two-wide out-of-order processor executes two ready instructions in every cycle and needs 4 cycles; only the four-wide out-of-order processor reaches the ILP.

**Table 6.2:** Instructions of Example 6.6 executed in each cycle. The schedule of the ideal processor equals that of the four-wide out-of-order processor.

| Processor | Cycle 1 | Cycle 2 | Cycle 3 | Cycle 4 | Cycle 5 | IPC |
|---|---|---|---|---|---|---|
| In-order, 2 wide | 1 | 2, 3 | 4 | 5, 6 | 7, 8 | 1.60 |
| In-order, 4 wide | 1 | 2, 3 | 4 | 5, 6 | 7, 8 | 1.60 |
| Out-of-order, 2 wide | 1, 3 | 2, 4 | 5, 6 | 7, 8 | | 2.00 |
| Out-of-order, 4 wide | 1, 3, 6, 8 | 2, 4 | 5, 7 | | | 2.67 |

In the example both in-order processors are held back by the order: widening from two to four gains nothing. Out-of-order execution lifts the two-wide processor only to its width (from 1.60 to $w = 2$), but the four-wide one to the ILP (from 1.60 to 2.67). In general (Table 6.3), the width is the main limit of a narrow processor (with $w = 1$, IPC $\leq 1$ in any order), so out-of-order execution raises its IPC only moderately. In a wide in-order processor, waiting instructions block the ones behind them and most of the width stays unused. Wide processors therefore need out-of-order execution to use their width, and out-of-order execution pays off fully only in a wide processor.

**Table 6.3:** What limits the IPC of a processor relative to the ILP of the program.

| | In-order | Out-of-order |
|---|---|---|
| Narrow (1–2 wide) | mainly width | mainly width |
| Wide (4 or more) | mainly order; width unused | little: IPC approaches ILP |

**Key point — Lesson summary**

Executing many instructions in the same cycle would bring the CPI far below 1, but data dependences prevent it. Only RAW dependences are true dependences; WAW and WAR dependences come from reusing register names. Register renaming, driven by the register alias table, maps architectural registers to newly allocated physical registers and removes all false dependences between register accesses. The ILP (Equation (6.1)) of a program is the number of instructions per cycle of an ideal processor with renaming, unlimited width, no structural dependences and perfect branch prediction: the instruction count divided by the length of the longest RAW chain; it is a property of the program and bounds the IPC of every processor. Real processors fall below it, limited by their width, their execution units and in-order execution; approaching the ILP takes a wide out-of-order processor. The next lesson presents Tomasulo's algorithm, which executes instructions out of order with renaming.

Both kinds ship in the same phone: Arm's Cortex-A55 is 2 wide and in-order, the Cortex-A78 4 wide and out-of-order (Arm product pages).

Lesson 7

# Instruction Scheduling

| | |
|---|---|
| **Lecture** | Lesson 7, videos 1–30 |
| **Textbook** | H&P [1] §3.4–3.5; [15] |
| **Keywords** | out-of-order execution, Tomasulo's algorithm, instruction queue, reservation station, register alias table, tag, common data bus, issue, dispatch, write result, stale result, oldest first |

Lesson 6 showed that programs contain far more instruction-level parallelism than an in-order processor can use, and that approaching it requires a processor that renames registers and executes instructions out of order. This lesson describes how such a processor decides, cycle by cycle, which instructions to execute. The method is Tomasulo's algorithm, which combines register renaming with the detection of instructions whose operands are available and is still the basis of today's out-of-order processors. We follow an instruction through its three steps, examine the choices that arise when several instructions or results compete for the same hardware, trace a complete example, and derive a shortcut for computing when each instruction executes.

## 7.1 Improving IPC

**Source** Lesson 7, videos 1–2; H&P §3.1.

Since the ILP of real programs is well above 1, a processor whose IPC is to come close to it must deal with four kinds of dependences between instructions. Each has its own remedy:

**Control dependences** The instructions that follow a branch can enter the pipeline before the branch executes only if its outcome is guessed. Branch prediction (Lesson 4), and predication for hard-to-predict branches (Lesson 5), take care of them.

**False data dependences** WAW and WAR dependences exist only because instructions reuse register names. Register renaming (Lesson 6) removes them.

**True data dependences** RAW dependences cannot be removed: an instruction has to wait until the values it reads have been computed. What can be avoided is that instructions which do not depend on it wait as well.

**Structural dependences** Instructions also compete for hardware: reservation stations, execution units and the bus introduced below. The remedy is a wider processor (Lesson 6) with more of these resources, so that more instructions can be issued and executed in the same cycle.

**Caution** A dependence is a property of the program; a hazard is a property of the pipeline that has to resolve it. H&P §3.1.

**Definition 7.1** (Out-of-order execution). Executing each instruction as soon as the values it reads are available, even while instructions that precede it in program order are still waiting, is called *out-of-order execution*.

Out-of-order execution is combined with renaming: without it, an instruction could overtake only earlier instructions with which it has no WAW or WAR dependence, and much of the ILP would be lost. What remains is the scheduling problem: in every cycle the processor must determine which instructions have all their operands, and decide which of them to execute.

## 7.2 Tomasulo's algorithm

**Source** Lesson 7, video 3; H&P §3.4; [15].

The scheduling method used in out-of-order processors was introduced in the IBM System/360 Model 91.

**Definition 7.2** (Tomasulo's algorithm). The hardware algorithm published by Tomasulo in 1967, which renames registers and determines which instructions have their operands available so that each instruction executes as soon as its operands have been computed, is called *Tomasulo's algorithm*.

Scale today: Skylake holds 97 instructions in its scheduler and 224 in its reorder buffer (Lesson 8). Intel optimization manual.

Current processors schedule instructions in essentially the same way. The differences are listed in Table 7.1. The first two concern only the scope of the algorithm; the third changes its mechanism. Tomasulo's algorithm has no provision for exceptions: once instructions have executed out of order, the registers need not correspond to any point in the program at which an exception could be taken. Supporting exceptions requires an additional structure, the subject

of Lesson 8.[1]

[1] The Model 91 (1967) ran at a 60 ns clock and applied the algorithm to floating point only, where an add took 2 cycles and a multiply 3; its interrupts were imprecise for exactly this reason. The idea then saw little use until the mid-1990s, when the Pentium Pro and the PowerPC 604 brought it back.

**Table 7.1:** Tomasulo's algorithm and modern out-of-order processors.

| | Tomasulo's algorithm | Modern processors |
|---|---|---|
| Instructions scheduled | floating-point operations | all instructions |
| Instructions in flight at once | a few | hundreds |
| Exceptions | not supported | supported (Lesson 8) |

## 7.3 Organization

**Source** Lesson 7, video 4; H&P §3.4.

Figure 7.1 shows the hardware that carries out Tomasulo's algorithm. The fetch stage appends each fetched instruction to the *instruction queue*, a first-in first-out buffer that keeps the fetched instructions in program order until they can be moved on to the reservation stations.

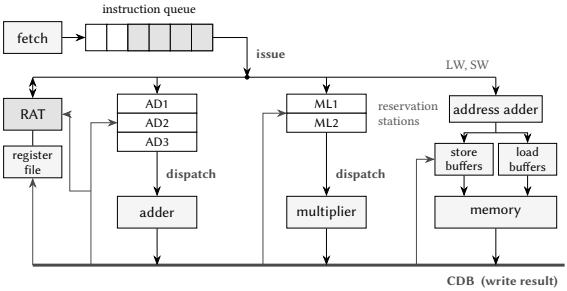



**Figure 7.1:** Organization of a processor with Tomasulo's algorithm. Issue moves the instruction at the head of the queue into a reservation station, using the RAT and the register file; dispatch sends a ready instruction to its execution unit; results return over the common data bus to the reservation stations, the RAT, the register file and the store buffers. Loads and stores take the path through the address adder and the load and store buffers.

**Definition 7.3** (Reservation station). A *reservation station* (RS) holds one instruction from the moment it leaves the instruction queue until its result has been broadcast. It records whether it is busy; the operation; for each operand either its value or, if the value has not been computed yet, the tag of the value; and whether the instruction has already been dispatched.

Reservation stations are grouped by the execution unit that serves them. The examples of this lesson use three stations AD1, AD2 and AD3 for additions and subtractions, served by one adder, and two stations ML1 and ML2 for multiplications and divisions, served by one multiplier. The name of a station is the *tag* under which the result of its instruction will be broadcast.

The 360/91 had exactly this arrangement: three reservation stations on its floating-point adder and two on its multiplier–divider [15].

The register alias table of Lesson 6 has one entry per register. An empty entry means that the register file holds the current value of the register; otherwise the entry holds the tag of the station whose instruction will produce that value. Tags thus play the role of the physical registers of Lesson 6: every result receives a new name, and an instruction that reads a register refers to the result of its closest preceding writer.

**Definition 7.4** (Common data bus). The *common data bus* (CDB) carries each result computed by an execution unit, together with its tag, to all reservation stations, to the RAT and to the register file.

Loads and stores use a separate path. An adder computes the memory address; a load then waits in a load buffer, which holds the address, and a store in a store buffer, which holds the address and the value to be stored, possibly received from the CDB. The value read by a load is placed on the CDB like any other result (Section 7.8).

Every instruction passes through three steps, marked in Figure 7.1:

**Issue** The instruction at the head of the instruction queue is placed in a free reservation station, with its operands and its destination resolved through the RAT. This step is

called *issue*.

**Dispatch** An instruction whose operands are all available is sent from its reservation station to an execution unit; this is its *dispatch*.

**Write result** The result is placed on the CDB, from which it reaches the waiting reservation stations, the RAT and the register file. This step is called *write result*, or broadcast.

## 7.4 Issue

Issue takes the instruction at the head of the instruction queue and performs the following steps. In the examples of this lesson one instruction is issued per cycle; a wider processor issues several per cycle, still in program order.

**Source** Lesson 7, videos 5–7; H&P §3.5.

1. Take the next instruction, in program order, from the head of the queue.
2. Determine where each operand comes from. If the RAT entry of the operand register is empty, the value is in the register file; otherwise the entry names the tag of the value.
3. Find a free reservation station for the operation. If every station of that kind is busy, the instruction is not issued in this cycle; it stays at the head of the queue, and the instructions behind it wait as well.
4. Place the instruction in the station: the operation, and for each operand the value from the register file or the tag from the RAT.
5. Set the RAT entry of the destination register to the name of the station.

Instructions are issued strictly in program order. The RAT then describes the register values seen at the position of the instruction being issued, so that each operand refers to its closest preceding writer. As in renaming, the operands are looked up before the destination entry is overwritten.

**Caution** Issue never waits for operands, only for a free reservation station.

**Example 7.5.** The following program is the running example of this lesson:

```
I1:  MUL R1, R2, R3     ; R1 <- R2 * R3
I2:  ADD R4, R1, R5     ; R4 <- R1 + R5
I3:  SUB R6, R2, R5     ; R6 <- R2 - R5
I4:  ADD R1, R5, R3     ; R1 <- R5 + R3
I5:  DIV R2, R6, R3     ; R2 <- R6 / R3
I6:  MUL R3, R4, R1     ; R3 <- R4 * R1
I7:  SUB R5, R3, R2     ; R5 <- R3 - R2
```

Initially the RAT is empty and the register file holds R1 = 1, R2 = 14, R3 = 4, R4 = 3, R5 = 6 and R6 = 7. In cycle 1, I1 is issued into ML1 with the operand values 14 and 4, and the RAT entry of R1 becomes ML1. In cycle 2, I2 is issued into AD1 (Figure 7.2). The RAT entry of R1 holds ML1, so the first operand is the tag ML1; the entry of R5 is empty, so the second operand is the value 6. The RAT entry of R4 becomes AD1. I3 and I4 follow in cycles 3 and 4. I4 writes R1 again and replaces ML1 with AD3 in the entry of R1: every instruction issued from now on that reads R1 uses the result of I4, not that of I1.

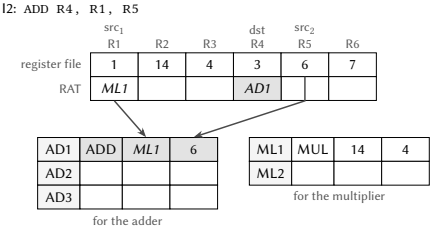



**Figure 7.2:** Issue of I2 in cycle 2 of Example 7.5. The RAT entry of R1 yields the tag ML1 (tags in italics), the empty entry of R5 lets the value 6 be taken from the register file, and the entry of R4 is set to AD1. Shaded cells are written in this step.

## 7.5 Dispatch

An instruction waits in its reservation station until all its operands are values. Whenever a result appears on the CDB, every reservation station compares the tags of its operands with

**Source** Lesson 7, video 8; H&P §3.5.

the tag on the bus and, where they match, replaces the tag with the value from the bus; this is called *capturing* the value. A station whose operands are all values holds a *ready* instruction. When the execution unit that serves the station is free, a ready instruction is dispatched: its operation and operand values are sent to the unit, which starts executing it. The station remains busy while the instruction executes, because its name is the tag under which the result will be broadcast.

**Caution** Other texts swap the names: *dispatch* for placing an instruction in a station and *issue* for sending it to a unit. H&P's steps are issue, execute and write result.

### 7.5.1 More than one ready instruction

Several instructions may be ready when an execution unit becomes free, and one of them must be chosen. Every choice is correct, since a ready instruction has all the values it needs; the choice affects only performance and hardware cost.[2]

**Source** Lesson 7, videos 9–10.

**Oldest first** Dispatch the instruction that was issued first. Other things being equal, more of the instructions in flight were issued after an older instruction and may be waiting for its result, so this approximates the next policy while requiring only that the order of issue be recorded.

**Most dependents first** Dispatch the instruction whose result is needed by the largest number of waiting instructions, so that as many instructions as possible become ready soon. This promises the best performance, but finding that instruction means comparing the tag of every candidate with the operands of all reservation stations, which is too slow and costly.

**Random** The simplest selection logic. It may keep an instruction on which many others depend waiting behind unimportant ones, so performance suffers.

[2] Wakeup, the comparison of the broadcast tag against every waiting operand, and select, the choice of one ready instruction per unit, must together fit in one cycle; otherwise dependent instructions cannot execute back to back. The cost of that loop grows with the number of stations, which is why schedulers hold about a hundred instructions where reorder buffers hold hundreds [16].

Oldest first is a good compromise between the two others and is the rule used in the examples of this lesson. In the running example, both I2 and I4 are ready when the adder becomes free in cycle 7 (Section 7.9); oldest first dispatches I2, although I4 has been ready two cycles longer.

## 7.6 Write result

**Source** Lesson 7, video 11; H&P §3.5.

When an execution unit has finished an instruction, the write-result step broadcasts its result.

1. The result and the tag of the instruction's reservation station are placed on the CDB.
2. Every operand that holds this tag captures the value (Section 7.5).
3. If the RAT entry of a register holds the tag, the value is written into that register in the register file and the RAT entry is cleared, so that instructions issued later read the value from the register file.
4. The reservation station is freed.

The station need not record the destination register, because the RAT entry that holds the tag identifies it. Figure 7.3 shows the broadcast of the result of I2 in the running example.

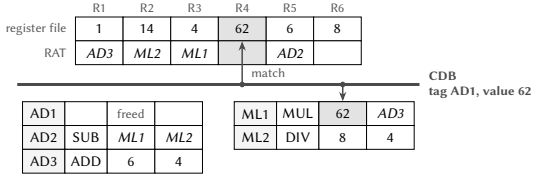



**Figure 7.3:** Write result of I2 in cycle 9 of the running example. ML1 (I6) captures 62 in place of the tag AD1; the RAT entry of R4 holds AD1, so R4 becomes 62 and the entry is cleared; AD1 is freed. No other operand or RAT entry holds AD1.

### 7.6.1 More than one result in a cycle

**Source** Lesson 7, video 12.

Two execution units may finish in the same cycle, but one bus carries one result at a time. One solution is a separate bus for each unit. Every operand of every reservation station must then compare its tag with each bus and select the value from the matching one, and the register file needs a write port per bus; the comparison logic, which is already large, grows in proportion to the number of buses. The alternative keeps a single bus and lets one result wait for the next cycle. The rule used here gives priority to the result of the *slower* unit: its instruction was dispatched longer ago, so it tends to be the older instruction, and more instructions are likely

to be waiting for its result. In the examples we assume that a unit whose result is waiting for the bus cannot start another instruction.

### 7.6.2 Stale results

By the time a result is broadcast, a later instruction that writes the same register may already have been issued. The RAT entry of the register then holds the tag of the later instruction, not the tag on the bus, and step 3 has no effect: neither the register file nor the RAT changes. The result is *stale* only for the register, and ignoring it there is correct. After both instructions the register must hold the later result, which arrives when the later instruction broadcasts. The earlier result is still needed, but only by the instructions issued between the two writers; they hold its tag and capture the value in step 2. Instructions issued after the later writer refer to the later tag.

**Source** Lesson 7, video 13; H&P §3.5.

**Example 7.6.** In the running example, I1 and I4 both write R1. When I1 broadcasts 56 under the tag ML1 in cycle 6, the RAT entry of R1 already holds AD3, because I4 was issued in cycle 4. R1 in the register file keeps its old value 1 and the entry keeps AD3, while AD1, which holds I2, captures 56. R1 becomes 10 only in cycle 11, when I4 broadcasts under AD3.

> **Key point**
> A broadcast delivers its value to every operand that waits for its tag, but it updates the register file and the RAT only if the RAT entry of the destination still holds that tag.

## 7.7 The steps in one cycle

For a single instruction, Tomasulo's algorithm is a sequence: issue, waiting and capturing in a reservation station, dispatch, execution, write result. For the processor as a whole, all steps take place in every cycle, each for different instructions: in the same cycle one instruction may be issued, several stations may capture a broadcast value, ready instructions may be dispatched to free units, and one result is broadcast.

**Source** Lesson 7, videos 14–19; H&P §3.5.

Whether two steps of the *same* instruction can take place in the same cycle is a matter of circuit design. In the simplest design every step examines the structures as they are at the start of the cycle, and all updates take effect at its end. Then:

- An instruction issued in cycle $C$ is in its reservation station only from cycle $C + 1$, so it can be dispatched in $C + 1$ at the earliest.
- An instruction that captures its last missing operand in cycle $C$ is ready from cycle $C + 1$ and can be dispatched in $C + 1$ at the earliest.
- A station freed by a broadcast in cycle $C$ can receive a new instruction in cycle $C + 1$.

The delay between capture and dispatch can be removed with additional logic, a dispatch that also considers operands arriving on the bus in the current cycle, and a station freed by a broadcast could similarly be reused in the same cycle. The additional logic lies in the same cycle as the step it extends and therefore lengthens the clock cycle. An instruction, however, is not dispatched in the cycle in which it is issued: while issue writes it into its station, dispatch does not yet see it there.

Two interactions between issue and write result in the same cycle need care.

- Both can change the RAT entry of the same register: the broadcast of an older writer clears it, and the issue of a newer writer sets it to a new tag. The issue must prevail, because it belongs to the later instruction in program order; equivalently, the write result is applied first and the issue second. Otherwise later instructions would read the register file and miss the result of the newly issued instruction.
- An instruction may be issued in the cycle in which one of its operands is broadcast. In the simplest design it finds the tag in the RAT, since the entry is cleared only at the end of the cycle, but enters its station after the value has passed on the bus; issue must then compare operand tags with the bus and take the value directly, or the instruction would wait for a broadcast that never comes. If the updates of the broadcast are visible to the issue in the same cycle, the issue simply reads the new value from the register file.

**Key point**
In the simplest design an instruction advances by at most one step per cycle: issued in cycle $C$, it is dispatched in $C + 1$ at the earliest, and a result broadcast in cycle $W$ lets the instructions waiting for it be dispatched in $W + 1$ at the earliest.

## 7.8 Load and store instructions

**Source** Lesson 7, video 20; H&P §3.5.

Loads and stores create dependences through memory that are analogous to the dependences through registers.

```
SW  R1, 0(R2)       ; MEM[R2] <- R1
LW  R3, 8(R4)       ; R3 <- MEM[R4 + 8]
```

If R4 + 8 equals R2, the load reads the value just stored, a RAW dependence through memory. Two stores to the same address have a WAW dependence, and a store to an address that an earlier load reads has a WAR dependence. The RAT cannot remove the false ones, because it renames registers, not memory locations, and whether two memory accesses conflict is known only once both addresses have been computed.

Skylake: 72 loads and 56 stores in flight. Intel optimization manual.

**Caution** "In program order" refers to the memory accesses; when each address is computed is a separate design choice (H&P §3.5 computes them in program order).

Tomasulo's algorithm therefore performs loads and stores in program order. They are issued like other instructions, with their operands resolved through the RAT, and the address adder computes the address. A load then waits in a load buffer and a store in a store buffer (Figure 7.1); a store whose value has not been computed yet captures it from the CDB. The memory accesses are made from these buffers in the order in which the loads and stores were issued. A load broadcasts the value it reads on the CDB under its own tag, exactly like an arithmetic result. Modern processors also execute loads and stores out of order, which requires the processor to detect and handle the memory dependences; Lesson 9 covers this.

Lesson 9 lets a load overtake a store whose address is not yet known, and checks afterwards whether the two addresses overlapped.

## 7.9 A complete example

**Source** Lesson 7, videos 21–26; H&P §3.5.

We now execute the program of Example 7.5 to the end. The assumptions are those of the simplest design of Section 7.7, together with the following:

- At most one instruction is issued per cycle, into the free station of the required kind with the lowest number.
- The adder executes ADD and SUB in 2 cycles; the multiplier executes MUL and DIV in 4 cycles. An instruction dispatched in cycle $D$ with an execution time of $k$ cycles executes in cycles $D$ to $D + k - 1$ and broadcasts in cycle $D + k$, in which its unit can already accept the next instruction.
- There is one CDB, and the multiplier has priority over the adder (Section 7.6.1); a unit whose result waits for the bus accepts no new instruction. Ready instructions are dispatched oldest first.

For scale: on Skylake a floating-point add or multiply takes 4 cycles, a double-precision divide 13–14 and an integer add 1. Agner Fog, *Instruction tables*.

The units here are not pipelined: each is busy in cycles $D$ to $D + k - 1$. Real adders and multipliers accept a new operation every cycle.

Table 7.2 lists what happens in each cycle, and Table 7.3 shows the reservation stations and the RAT after cycles 5, 6 and 7.

**Cycles 1–5.** One instruction is issued per cycle. I1 and I3 have all their operands at issue and are dispatched in the next cycle, I1 to the multiplier and I3 to the adder. I4 is ready from cycle 5, but the adder is busy with I3. I2 waits for the tag ML1 and I5 for AD2. After cycle 5 every station is busy.

**Cycle 6.** Both units finished in cycle 5. The bus takes the result of the slower multiplier, 56 under ML1, which is stale for R1 (Example 7.6); AD1 captures it and ML1 is freed. The result of I3 waits in the adder. I6 is not issued, because at the start of the cycle both multiplier stations are busy.

**Cycle 7.** I3 broadcasts 8 under AD2. The RAT entry of R6 holds AD2, so R6 becomes 8 and the entry is cleared; ML2 captures the value for I5, and AD2 is freed. The adder can take a new instruction, and both I2 and I4 are ready; oldest first dispatches I2. I6 is issued into ML1, freed in the previous cycle, and receives two tags, AD1 and AD3.

**Cycles 8–19.** I7 is issued into AD2 and waits for ML1 and ML2. I5 starts on the multiplier in cycle 8, and I4 on the adder in cycle 9, when I2 releases it. I6 needs the results of I2 (cycle 9)

**Table 7.2:** Events in each cycle of the complete example. Captures are listed with the broadcast that causes them.

| Cycle | Issue | Dispatch | Write result and remarks |
|---|---|---|---|
| 1 | I1 → ML1 | | |
| 2 | I2 → AD1 | I1 | |
| 3 | I3 → AD2 | | I2 waits for ML1 |
| 4 | I4 → AD3 | I3 | RAT entry of R1 becomes AD3 |
| 5 | I5 → ML2 | | all stations busy; I4 ready, adder busy |
| 6 | | | I1: ML1 = 56, stale for R1; AD1 captures; I3 finished, waits for the bus; no free station for I6 |
| 7 | I6 → ML1 | I2 | I3: AD2 = 8, R6 ← 8; ML2 captures; I2 and I4 ready, I2 older |
| 8 | I7 → AD2 | I5 | |
| 9 | | I4 | I2: AD1 = 62, R4 ← 62; ML1 captures |
| 10 | | | |
| 11 | | | I4: AD3 = 10, R1 ← 10; ML1 captures |
| 12 | | I6 | I5: ML2 = 2, R2 ← 2; AD2 captures |
| 13–15 | | | I6 executes |
| 16 | | | I6: ML1 = 620, R3 ← 620; AD2 captures |
| 17 | | I7 | |
| 18 | | | |
| 19 | | | I7: AD2 = 618, R5 ← 618 |

**Table 7.3:** Reservation stations and RAT at the end of cycles 5, 6 and 7. Tags are in italics; "exec" marks an instruction that has been dispatched. An empty RAT entry is shown with the register-file value.

| | After cycle 5 | After cycle 6 | After cycle 7 |
|---|---|---|---|
| AD1 | I2: *ML1*, 6 | I2: 56, 6 | I2: 56, 6 (exec) |
| AD2 | I3: 14, 6 (exec) | I3: 14, 6 (exec) | free |
| AD3 | I4: 6, 4 | I4: 6, 4 | I4: 6, 4 |
| ML1 | I1: 14, 4 (exec) | free | I6: *AD1*, *AD3* |
| ML2 | I5: *AD2*, 4 | I5: *AD2*, 4 | I5: 8, 4 |
| R1 | *AD3* | *AD3* | *AD3* |
| R2 | *ML2* | *ML2* | *ML2* |
| R3 | 4 | 4 | *ML1* |
| R4 | *AD1* | *AD1* | *AD1* |
| R5 | 6 | 6 | 6 |
| R6 | *AD2* | *AD2* | 8 |

and I4 (cycle 11) and the multiplier, which I5 releases in cycle 12; I7 needs the result of I6, broadcast in cycle 16. After the last broadcast, in cycle 19, the register file holds R1 = 10, R2 = 2, R3 = 620, R4 = 62, R5 = 618 and R6 = 8, exactly the values produced by executing the program in order.

IPC here is $7/19 \approx 0.37$: even with unlimited stations and units the chain I1→I2→I6→I7 would need 17 cycles.

> **Key point**
> The instructions of the example broadcast in the order I1, I3, I2, I4, I5, I6, I7, and each executes as soon as its operands, a free station and a free unit allow. Because every operand is resolved through a tag at issue and stale results never reach the register file, the final register values are those of sequential execution.

## 7.10 Timing without tracing the structures

**Source** Lesson 7, videos 27–29; H&P §3.5.

Tracing every structure cycle by cycle is laborious. When only the timing matters, it suffices to determine the cycles of issue, dispatch and write result of each instruction. Let $I_n$, $D_n$ and $W_n$ denote these cycles for instruction $n$, and $k_n$ its execution time. Under the assumptions of Section 7.9:

**Issue** $I_n$ is the first cycle after $I_{n-1}$ in which a station of the required kind is free. A station freed by the write result of instruction $m$ is free from cycle $W_m + 1$.

**Dispatch** $D_n$ is the first cycle that satisfies

$$D_n \;\geq\; 1 + \max\Big(I_n,\; \max_{p \in P(n)} W_p\Big), \tag{7.1}$$

where $P(n)$ is the set of instructions that produce the operands of $n$ (its closest preceding writers), and in which the execution unit is free: not held by another instruction $m$, which holds it in cycles $D_m$ to $W_m - 1$, and not taken by an older instruction that is also ready.

Tip $P(n)$: for each register $n$ reads, its closest preceding writer (Lesson 6).

**Write result** $W_n = D_n + k_n$, or later if the bus is taken in that cycle by a result with priority. A unit or the bus may be taken by a *later* instruction that became ready earlier, so the rows cannot simply be completed in program order. Units and the bus are assigned in cycle order across all instructions: in each cycle a free unit takes the oldest of the instructions ready in that cycle, and the bus the result with priority. A unit never stays idle for an older instruction that is not yet ready.

Tip Each step waits for one resource: issue for a free station, dispatch for the operands and a free unit, write result for the bus.

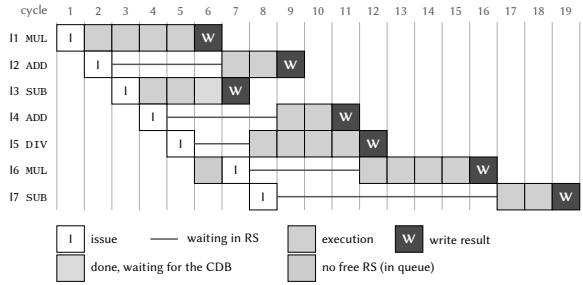



**Figure 7.4:** Timing of the running example. Out-of-order execution lets I3 run while I2 waits and I5 start while I4 waits; the delays of I3 (bus), I4 (adder) and I6 (reservation station) are those derived in Example 7.7.

**Example 7.7.** Table 7.4 applies these rules to the running example, and Figure 7.4 draws the result. I3, I4 and I6 show every rule. I3 is dispatched in cycle 4, one cycle after its issue, and finishes executing in cycle 5; its write result would be cycle 6, but the bus then carries the result of I1, so $W_3 = 7$. Whether the adder is free for I2 in cycle 7 thus depends on the row of the later I3. I4 is ready from cycle 5, but the adder is occupied by I3, whose result waits for the bus, until its write result in cycle 7, and then by the older I2 until cycle 9, so $D_4 = 9$ and $W_4 = 11$. I6 finds both multiplier stations busy until ML1 is freed by $W_1 = 6$, so $I_6 = 7$; its operands come from I2 and I4, and Equation (7.1) gives $D_6 \geq 1 + \max(7, 9, 11) = 12$, which is also the cycle in which the multiplier is released by I5, so $D_6 = 12$ and $W_6 = 16$. The table agrees with the full trace of Table 7.2.

Tip With a single bus no two instructions broadcast in the same cycle: two equal entries in the $W_n$ column mean that a bus conflict was missed.

**Table 7.4:** Issue, dispatch and write-result cycles of the running example.

| | Instruction | $I_n$ | $D_n$ | $W_n$ | What determines the timing |
|---|---|---|---|---|---|
| I1 | `MUL R1, R2, R3` | 1 | 2 | 6 | ready at issue |
| I2 | `ADD R4, R1, R5` | 2 | 7 | 9 | $W_1 = 6$; adder busy with I3 until $W_3 = 7$ |
| I3 | `SUB R6, R2, R5` | 3 | 4 | 7 | ready at issue; bus taken in 6 |
| I4 | `ADD R1, R5, R3` | 4 | 9 | 11 | adder: I3, then the older I2 |
| I5 | `DIV R2, R6, R3` | 5 | 8 | 12 | $W_3 = 7$ |
| I6 | `MUL R3, R4, R1` | 7 | 12 | 16 | no station until $W_1 = 6$; $W_4 = 11$; multiplier until $W_5 = 12$ |
| I7 | `SUB R5, R3, R2` | 8 | 17 | 19 | $W_6 = 16$ |

**Key point — Lesson summary**

Tomasulo's algorithm combines register renaming with out-of-order execution, in which each instruction executes as soon as its operands are available. Issue moves instructions in program order from the instruction queue into reservation stations, taking each operand either as a value from the register file or as a tag from the RAT, and points the RAT entry of the destination to the station. A station captures values from the common data bus until its instruction is ready; dispatch then sends it to a free execution unit, choosing the oldest when several are ready. Write result broadcasts the value and tag, frees the station, and updates the register file and the RAT only if the result is not stale. One bus serves one result per cycle, the slower unit having priority. Loads and stores are performed in program order. With one step per cycle, the issue, dispatch and write-result cycles of every instruction follow from its operands, the available stations and units, and the bus. Tomasulo's algorithm does not support exceptions; the reorder buffer of the next lesson adds this.

Issue, dispatch and write result stay exactly as described; Lesson 8 adds a fourth step, commit, which writes the register file in program order.

# ReOrder Buffer

| | |
|---|---|
| **Lecture** | Lesson 8, videos 1–40 |
| **Textbook** | H&P [1] §3.6, §3.8, §C.4; [17] |
| **Keywords** | exception, precise exception, phantom exception, reorder buffer, commit pointer, issue pointer, commit, branch misprediction recovery, register alias table, unified reservation station, superscalar, out-of-order execution |

Tomasulo's algorithm (Lesson 7) executes each instruction as soon as its operands are available and writes its result into the register file when the result is broadcast. Renaming keeps the computed values correct, but two events that every real processor must handle break the scheme: exceptions and branch mispredictions. In both cases instructions that follow the offending instruction in program order may already have overwritten registers, and these writes cannot be undone. This lesson introduces the reorder buffer, which lets instructions execute out of order but updates the registers strictly in program order. It traces the resulting processor in detail and ends with unified reservation stations, superscalar execution, and a precise account of which steps of an out-of-order processor actually work out of order.

## 8.1 Exceptions in out-of-order execution

**Source** Lesson 8, videos 1–2; H&P §C.4, §3.6.

**Caution** Some texts, following the MIPS convention, reserve *interrupt* for an event that no instruction causes, such as a timer tick or an I/O completion; H&P §C.4 calls both *exceptions*. The mechanism of this chapter serves both.

An exception is an event, such as a division by zero or an access to an invalid memory address, that an instruction raises when it executes. The processor saves the address of the excepting instruction and transfers control to an exception handler, usually part of the operating system. When the handler has dealt with the cause, it may return to the saved address, so that the excepting instruction is executed again and the program continues after it. For the handler to work, and for the program to continue correctly after the return, the registers and memory must look as they would in a processor that executes one instruction at a time.

**Definition 8.1** (Precise exception). An exception is a *precise exception* if, when the handler starts, every instruction that precedes the excepting instruction in program order has completed and updated the registers and memory, and neither the excepting instruction nor any instruction that follows it has changed them.

Tomasulo's algorithm does not provide precise exceptions. Consider:

```
I1:  DIV R1, R2, R3     ; R3 = 0: division by zero
I2:  ADD R2, R4, R5     ; overwrites R2, read by I1
I3:  ADD R6, R6, R7     ; R6 <- R6 + R7
```

I2 and I3 do not depend on I1, so they execute and write R2 and R6 within a few cycles of being issued. The division takes many cycles, and in the model used here the divider reports the division by zero when the operation completes (Figure 8.1). By then both later results are in the register file, and three things go wrong.

- The handler sees values of R2 and R6 that no sequential execution stopped at I1 could produce.
- After the return, I1 is executed again and now reads the value of R2 written by I2, which in program order it must never see.
- I2 and I3 are also executed a second time. I2 merely recomputes the same value, but I3 adds R7 to R6 twice.

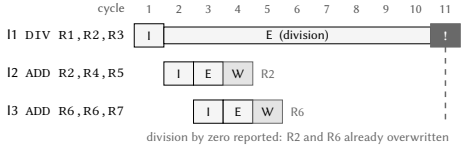



**Figure 8.1:** An exception under Tomasulo's algorithm (I: issue, E: execute, W: write result). I2 and I3 write R2 and R6 long before the division I1 reports the division by zero.

## 8.2 Branch mispredictions

Branch mispredictions cause the same problem. In the following program the branch predictor lets the processor fetch and issue I3 to I6 while I2 waits for the result of the division:

**Source** Lesson 8, video 3; H&P §3.6.

```
I1:        DIV R1, R2, R3     ; takes many cycles
I2:        BEQ R1, R4, Skip   ; predicted not taken
I3:        ADD R5, R6, R7     ; I3-I6: executed only
I4:        LW  R8, 0(R5)      ;   if not taken
I5:        MUL R9, R5, R5
I6:        SUB R6, R9, R7
           ...
     Skip: ...
```

I2 needs R1 and cannot execute before the division completes, whereas I3 has its operands at once and I4 a cycle after I3. Under Tomasulo's algorithm R5 and R8 are written long before the branch is resolved. If the branch turns out to be taken, the instructions of the wrong path can be removed from the reservation stations and fetching can restart at `Skip`, but R5 and R8 have been overwritten with values that the program never computes, and their correct values are lost.

A second problem arises when an instruction on the wrong path raises an exception. The program uses R5 as an address only when the branch is not taken; when it is taken, the value R6 + R7 need not be a valid address, and the load I4 may raise an exception long before I2 executes.

**Definition 8.2** (Phantom exception). An exception raised by an instruction that lies on a mispredicted path, and therefore is never executed in a sequential execution of the program, is called a *phantom exception*.

A phantom exception must not reach the handler: in program order, the instruction that raised it does not exist.

**Caution** *Phantom exception* is this course's term; H&P speaks of an exception raised by a speculated instruction, buffered and discarded with it.

## 8.3 Correct out-of-order execution

Both problems have the same cause. An instruction changes the registers as soon as it has computed its result, before it is known that every earlier instruction has completed without an exception or a misprediction. The remedy is to separate computing a result from making it visible:

- instructions still *execute* out of order, as soon as their operands are ready;
- results are still *broadcast* out of order, so that waiting instructions receive them without delay;
- but results are *written into the registers* in program order, and only once it is certain that the instruction belongs to the program's execution.

Holding the results until they may be written requires a structure that remembers the program order of the instructions after they have been issued. The reservation stations cannot do this: an instruction leaves its reservation station as soon as its result is broadcast, and the stations are allocated in no particular order.

**Source** Lesson 8, video 4; H&P §3.6; [17].

> **Key point**
> Execute out of order, broadcast out of order, but write the registers in program order.

Smith and Pleszkun [17] give three more ways to the same end: a result shift register, a history buffer and a future file.

## 8.4 The reorder buffer

The structure that remembers the program order is a queue of the issued instructions.

**Source** Lesson 8, videos 5–10; H&P §3.6.

**Definition 8.3** (Reorder buffer). The *reorder buffer* (ROB) holds, in program order, every instruction that has been issued but has not yet passed the commit step of Definition 8.4, in which its result is written into the register file. Each entry has three fields:
**Reg** the destination register of the instruction;
**Value** its result, once computed;
**Done** a bit that is set when Value holds the result.

These three fields suffice for the examples of this chapter. An entry must also record whether the instruction raised an exception or is a mispredicted branch, and the address of

the instruction, which commit needs in order to take the exception or to restart fetching (Sections 8.5 and 8.6).

The ROB is a circular buffer managed by two pointers (Figure 8.2). The *issue pointer* names the entry that the next issued instruction receives, and it advances by one entry for every issued instruction. The oldest instruction that has not yet committed is found at the *commit pointer*. The entries from the commit pointer up to, but not including, the issue pointer are in use and hold instructions in program order; all others are free. Since equal pointers would describe both an empty and a full buffer, an occupancy count or a full bit distinguishes the two. In normal operation both pointers only advance, wrapping around from the last entry to the first; recovery moves the issue pointer back (Section 8.5). When every entry is in use, no instruction can be issued until the oldest one leaves the ROB. Every issued instruction receives an entry, including a branch, which writes no register: its entry keeps its place in the program order and, as shown below, records whether it was mispredicted.

ROBs have grown from 40 entries on the Pentium Pro to 224 on Skylake (Lesson 7), 256 on AMD Zen 3 and 512 on Intel Golden Cove. Intel optimization manual; AMD Family 19h software optimization guide.

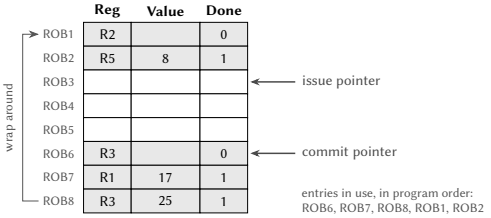



**Figure 8.2:** A reorder buffer with eight entries. The shaded entries, from the commit pointer up to the issue pointer, are in use; here this range wraps around from ROB8 to ROB1.

### 8.4.1 Issue, dispatch and write result

With a ROB, the three steps of Tomasulo's algorithm change as follows.

**Issue** The instruction needs a free reservation station (RS) and the free ROB entry at the issue pointer; if either is missing, issue waits. The ROB entry receives the destination register and a cleared Done bit, and the issue pointer advances. Each operand is then looked up in the RAT, whose entries now name ROB entries instead of RSs:

- if the RAT entry points to the register file, the value is copied from the register;
- if it names a ROB entry whose Done bit is set, the value is copied from that entry;
- otherwise the RS records the ROB entry as the tag of the operand and waits for a broadcast with that tag.

Finally, the RAT entry of the destination register is set to the new ROB entry, and the RS records that entry as the tag of its result.

**Dispatch** As before, an instruction whose operands are all available starts executing. Its RS is freed at this point.

**Write result** The result is broadcast on the CDB together with its tag, the ROB entry. Waiting RSs capture it as before, and the ROB entry stores it and sets its Done bit. Neither the register file nor the RAT is changed.

The second lookup case is new. A result stays in the ROB from write result until commit, and during that time the RAT keeps naming the ROB entry; an instruction issued in that interval must find the value there, because the broadcast it would otherwise wait for has already happened.

The earlier release of RSs is also new. In Tomasulo's algorithm an RS must stay allocated until write result, because the station itself is the tag under which the result is broadcast. With a ROB the tag is the ROB entry, so the RS is no longer needed once the operation has started, and the same number of RSs can hold more waiting instructions.

### 8.4.2 Commit

A fourth step completes every instruction.

**Definition 8.4** (Commit). In the *commit* step the processor examines the entry at the commit pointer. If its Done bit is set, the value is written into the destination register, and if the RAT entry of that register still names this ROB entry, the RAT entry is reset to point to the register file. The commit pointer then advances, which frees the entry.

If the instruction at the commit pointer is not done, nothing commits, even if later entries are: results enter the register file only in program order. The RAT entry is reset only when it still names the committing entry; otherwise a later instruction writes the same register, and the RAT must keep naming that instruction's entry (Section 8.7). Commit is also called *retirement*.

*Retire* is the word of the vendor manuals: Intel's counter `INST_RETIRED.ANY` counts commits, and measured IPC (Lesson 2) is derived from it.

### 8.4.3 Hardware organization

Figure 8.3 shows the organization of Lesson 7 with a ROB added. The CDB now delivers results to the RSs and to the ROB, the RAT names ROB entries, and the register file is written only by commit.[1] An instruction passes through six steps: fetch, decode, issue, execute (from dispatch until the operation completes), write result and commit.

[1] Keeping results in the ROB, as here, is the design of Intel's P6 cores (Pentium Pro, 1995). Since Sandy Bridge (2011) every result is written straight into the physical register file of Lesson 6, and the ROB entry records only which physical register the instruction was given; commit then changes a mapping instead of copying a value, and recovery frees the registers allocated after the branch.

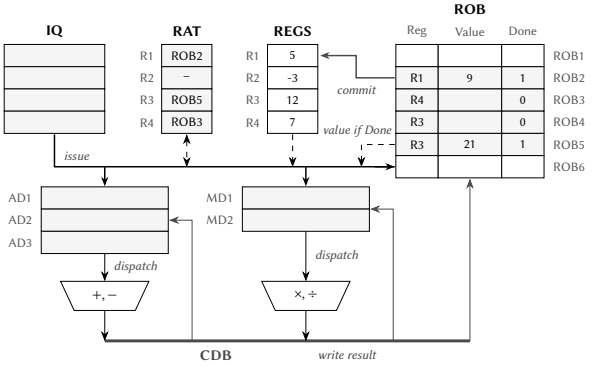



**Figure 8.3:** Tomasulo's organization with a reorder buffer. Issue allocates an RS and a ROB entry and renames the operands through the RAT; operand values come from the register file or from a Done ROB entry. Results go from the CDB to the RSs and the ROB; commit copies the value at the commit pointer into the register file.

**Note 8.5.** At any moment the most recent value of an architectural register, in the program order of the instructions issued so far, is in one of three places: in the register file, when the RAT points there; in a ROB entry whose Done bit is set; or not yet computed, in a ROB entry whose Done bit is clear. The register file itself always holds the values as of the last committed instruction.

## 8.5 Branch misprediction recovery

With a ROB, instructions on a mispredicted path cannot change the register file as long as they are removed before they commit. In the program of Section 8.2, I3 to I6 are issued while I2 waits for R1, and all of them except I6, which is still executing, complete: I3 and I5 write their results into their ROB entries, and the load I4 raises a phantom exception, which is recorded in its entry (Section 8.6). When I2 finally executes and the processor finds that the branch was mispredicted, the misprediction is recorded in the ROB entry of I2, and nothing else happens yet. Instructions before the branch are unaffected and continue to execute and commit.

**Source** Lesson 8, videos 11–12; H&P §3.6.

When the branch reaches the commit pointer, every instruction before it has committed, so the register file holds exactly the values that a sequential execution has after the branch. Every instruction still in the processor follows the branch and is on the wrong path. The processor now performs *branch misprediction recovery*, which consists of four actions (Figure 8.4):

1. discard all ROB entries by setting the issue pointer equal to the commit pointer, which leaves the ROB empty;
2. reset every RAT entry to point to the register file;
3. free all RSs and abandon the operations in the execution units, whose results could only go to discarded ROB entries;
4. restart fetching at the correct successor of the branch.

A correctly predicted branch simply commits: the commit pointer advances and nothing else happens.[2]

The branch can be held up for a long time: a 32-bit integer division alone takes 26 cycles on Skylake (Agner Fog, *Instruction tables*).

[2] Recovering at commit is the simplest scheme and the one used in this chapter. Real processors start recovery as soon as the mispredicted branch executes, discarding only the ROB entries after the branch and restarting fetching at once, so that no cycles are lost while the branch waits to commit (H&P §3.6). The RAT must then be brought back to its state at the branch, for example from a copy saved when the branch was issued.

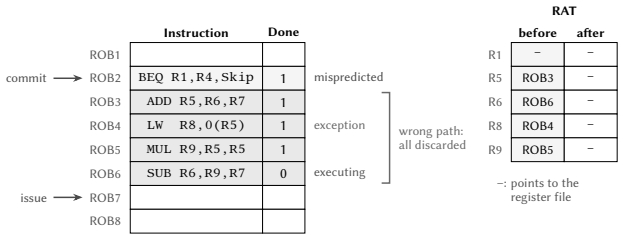



**Figure 8.4:** Recovery when the mispredicted branch I2 reaches the commit pointer. I1 has already committed; the ROB entries of I3 to I6 are discarded, including the exception recorded for I4, and all RAT entries return to the register file.

## 8.6 Exceptions with a reorder buffer

**Source** Lesson 8, videos 13–15; H&P §3.6.

Exceptions are handled by treating an exception as one more result of the instruction that raises it. When an instruction raises an exception during execution, the processor records the exception in the instruction's ROB entry instead of invoking the handler. When that entry reaches the commit pointer, the exception is taken: the processor discards the ROB entries, resets the RAT, and frees the RSs and execution units exactly as in misprediction recovery, saves the address of the excepting instruction, and starts fetching the exception handler.

This removes both problems of Sections 8.1 and 8.2.

**Precise exceptions** When the excepting instruction reaches the commit pointer, all earlier instructions have committed and no later instruction has written the register file. Stores likewise write memory only when they commit (Lesson 9), so memory is also unchanged by later instructions. The handler sees exactly the state of Definition 8.1; after the return, the later instructions are executed from that state.

Demand paging rests on this: the handler fetches the missing page, and the faulting load is then executed again (Lesson 13).

**No phantom exceptions** An exception on a mispredicted path is recorded in an entry that follows the mispredicted branch. The branch commits first, and its recovery discards the entry together with the exception, which is never taken. In Figure 8.4 the exception of I4 disappears when I2 commits.

### 8.6.1 The outside view of execution

The ROB separates two notions of when an instruction is executed. Inside the processor, an instruction executes when an execution unit performs its operation, in any order. The programmer, the operating system and the exception handler, however, see only the register file, memory and the program counter, and for them an instruction is executed when it commits. Commits happen one instruction after another in program order, exactly as in a processor that completes each instruction before starting the next. An instruction whose result is in the ROB but has not committed has, seen from outside, not been executed yet; if its result is discarded, it has never been executed at all. The example of Section 8.8 ends with an instance.

## 8.7 RAT updates on commit

**Source** Lesson 8, video 16; H&P §3.6.

The conditional RAT update of Definition 8.4 is needed because a register may have several pending writers. Suppose the instructions in ROB2 and ROB5 both write R4, and ROB2 commits while ROB5 is still in the ROB. The RAT entry of R4 names ROB5, because that instruction was issued later.

- Commit writes the value of ROB2 into R4. This is the correct architectural value after ROB2, and it is the value an exception handler must see if an instruction between ROB2 and ROB5 raises an exception.
- The RAT entry must nevertheless keep naming ROB5. An instruction issued now follows ROB5 in program order and must use the value of ROB5, not the older value that has just entered the register file.

**Tip** Only the youngest writer of a register ever resets its RAT entry; an older one writes the register file and leaves the RAT alone.

When ROB5 commits, the RAT entry of R4 still names ROB5 unless an even later writer has been issued in the meantime, and only then is it reset. The opposite mistake, leaving a RAT entry that names a committed entry, is equally wrong: the entry is freed at commit and may soon be given to an unrelated instruction.

## 8.8 A reorder buffer example

**Source** Lesson 8, videos 17–30; H&P §3.6.

We now trace a complete program through a processor with a ROB. The processor has three RSs for addition and subtraction (AD1 to AD3), two for multiplication and division (MD1, MD2) and a ROB of eight entries. ADD and SUB take 1 cycle, MUL 4 cycles and DIV 6 cycles, and there are enough execution units that no ready instruction waits for one. The steps follow these conventions:

- one instruction is issued per cycle, into the free RS of the required kind with the lowest number;
- an RS freed by a dispatch in cycle $C$, and a ROB entry freed by a commit in cycle $C$, can be given to a new instruction from cycle $C + 1$;
- an instruction is dispatched no earlier than the cycle after its issue and the cycle after the broadcast of each operand it waits for;
- an operation dispatched in cycle $D$ with an execution time of $k$ cycles executes in cycles $D$ to $D + k - 1$ and writes its result in cycle $D + k$; there is one CDB, and in this example no two results are written in the same cycle;
- an instruction commits no earlier than the cycle after its write result, and at most one instruction commits per cycle.

One commit per cycle is an assumption of this example: Skylake retires up to four micro-operations per cycle and AMD Zen 3 up to eight. Intel optimization manual; AMD Family 19h software optimization guide.

**Example 8.6.** The program is

```
I1:  DIV R1, R2, R3      ; R1 <- R2 / R3
I2:  ADD R4, R5, R6      ; R4 <- R5 + R6
I3:  MUL R2, R4, R4      ; R2 <- R4 * R4
I4:  SUB R5, R1, R6      ; R5 <- R1 - R6
I5:  ADD R4, R4, R3      ; R4 <- R4 + R3
I6:  ADD R6, R2, R4      ; R6 <- R2 + R4
```

with initial values R1 = 5, R2 = 48, R3 = 4, R4 = 2, R5 = 7 and R6 = 3. All RAT entries point to the register file, and both ROB pointers are at ROB1.

**Cycles 1–6: issue.** One instruction is issued per cycle, I$k$ into ROB$k$. I1 copies 48 and 4 from the register file into MD1 and sets the RAT entry of R1 to ROB1. In cycle 2 I1 is dispatched and frees MD1, and I2 is issued into AD1 with the values 7 and 3. I3, issued in cycle 3, finds ROB2 in the RAT entry of R4; ROB2 is not done, so both operands of I3 wait for ROB2. I3 can use MD1 again, which under Tomasulo's algorithm alone would stay occupied by I1 until cycle 8. I4 is issued into AD1, freed by I2 in cycle 3; its first operand waits for ROB1, its second is 3. In cycle 4 I2 writes its result: ROB2 receives 10, and I3 captures it. In cycle 5 I5 is issued into AD2 and finds the RAT entry of R4 still naming ROB2, which is done, so it copies 10 from the ROB, not from the register file, which still holds the old value 2; I5 then renames R4 to ROB5. In cycle 6 I6 is issued into AD3, because AD2, freed by the dispatch of I5 in the same cycle, can be used only from cycle 7; I6 waits for ROB3 and ROB5. Tables 8.1 and 8.2 show the state at the end of cycle 6: AD1 holds I4 and AD3 holds I6, all other RSs are free, and the register file is unchanged.

**Tip** Follow one register at a time: its RAT entry changes when a writer issues, and a reader takes whatever the RAT names in its own issue cycle.

**Tip** Entries in use = (issue − commit) mod 8 (0 means empty or full): at the end of cycle 6 that is $(7 - 1) \bmod 8 = 6$, namely ROB1 to ROB6.

**Table 8.1:** ROB of Example 8.6 at the end of cycles 6 and 10. At the end of cycle 6 the commit pointer is at ROB1, at the end of cycle 10 at ROB3; the issue pointer is at ROB7 in both.

| | | | Cycle 6 | | Cycle 10 | |
|---|---|---|---|---|---|---|
| Entry | Instruction | Reg | Value | Done | Value | Done |
| ROB1 | `DIV R1, R2, R3` | R1 | | 0 | committed | |
| ROB2 | `ADD R4, R5, R6` | R4 | 10 | 1 | committed | |
| ROB3 | `MUL R2, R4, R4` | R2 | | 0 | 100 | 1 |
| ROB4 | `SUB R5, R1, R6` | R5 | | 0 | 9 | 1 |
| ROB5 | `ADD R4, R4, R3` | R4 | | 0 | 14 | 1 |
| ROB6 | `ADD R6, R2, R4` | R6 | | 0 | | 0 |

**Cycles 7–8: results, but no commit.** I5 writes 14 into ROB5 in cycle 7 and I1 writes 12 into ROB1 in cycle 8; I6 and I4 capture these values. Although ROB2 has been done since cycle 4 and ROB5 since cycle 7, nothing commits, because ROB1, at the commit pointer, is not done until the end of cycle 8.

**Table 8.2:** RAT and register file of Example 8.6. A dash means that the RAT entry points to the register file.

| | Cycle 6 | | Cycle 10 | | End |
|---|---|---|---|---|---|
| Register | RAT | Value | RAT | Value | Value |
| R1 | ROB1 | 5 | – | 12 | 12 |
| R2 | ROB3 | 48 | ROB3 | 48 | 100 |
| R3 | – | 4 | – | 4 | 4 |
| R4 | ROB5 | 2 | ROB5 | 10 | 14 |
| R5 | ROB4 | 7 | ROB4 | 7 | 9 |
| R6 | ROB6 | 3 | ROB6 | 3 | 114 |

**Cycles 9–10: the first commits.** In cycle 9 I1 commits: R1 becomes 12 and, since the RAT entry of R1 still names ROB1, it is reset to the register file. In the same cycle I3 writes 100 into ROB3 and I4 is dispatched. In cycle 10 I2 commits and R4 becomes 10, but the RAT entry of R4 names ROB5 and is left unchanged, as Section 8.7 requires. I4 writes 9 and I6 is dispatched.

**Cycles 11–14: the remaining commits.** I6 writes 114 in cycle 11. I3, I4, I5 and I6 commit in cycles 11 to 14, one per cycle. Each is the last writer of its register, so each resets its RAT entry. At the end the ROB is empty, all RAT entries point to the register file, and the registers hold R1 = 12, R2 = 100, R3 = 4, R4 = 14, R5 = 9 and R6 = 114, the values of a sequential execution.

At the end of cycle 10 the register file holds 10 in R4 while the RAT sends readers of R4 to ROB5, which holds 14. Both are right: 10 is the value of R4 after I2, the last committed instruction, and 14 is the value after I5, the last issued writer.

The example also illustrates the outside view of Section 8.6.1. Suppose that I4 raises an exception when it executes in cycle 9. The exception is recorded in ROB4, I2 and I3 still commit in cycles 10 and 11, and in cycle 12 I4 reaches the commit pointer and the exception is taken. The handler finds R1 = 12, R2 = 100 and R4 = 10, but not R4 = 14, although I5 wrote 14 into ROB5 in cycle 7, before I1 had even written its result: from outside, execution stopped exactly before I4.

## 8.9 Timing with a reorder buffer

**Source** Lesson 8, videos 31–34; H&P §3.6.

The cycle-by-cycle state of a ROB processor can be summarized by the cycle in which each instruction passes each step, as for Tomasulo's algorithm in Lesson 7, with one additional column for commit. We keep the notation of Lesson 7: $I_n$, $D_n$ and $W_n$ are the cycles of issue, dispatch and write result of instruction $n$, $k_n$ is its execution time, and $P(n)$ is the set of instructions that produce its operands; $C_n$ denotes its commit cycle. Under the conventions of Section 8.8, $I_n$ is the first cycle after $I_{n-1}$ in which an RS of the required kind and a ROB entry are free. An RS released by the dispatch of instruction $m$ is free from cycle $D_m + 1$, and a ROB entry released by the commit of $m$ from cycle $C_m + 1$. With enough execution units and no competition for the CDB, the other cycles follow from

$$D_n = 1 + \max\Big(I_n,\ \max_{p \in P(n)} W_p\Big), \qquad W_n = D_n + k_n, \qquad C_n = \max\big(W_n + 1,\ C_{n-1} + 1\big), \tag{8.1}$$

**Tip** Fill in the table row by row in program order, each row from left to right, with Commit last.

where the first equation is the bound of Equation (7.1) with a free execution unit. Unlike execution, commit depends on the preceding instruction even when the two share no data. The point at which RSs are released and the number of commits per cycle are design assumptions that change the table. If an RS is released only at write result, as in Lesson 7, it is free from cycle $W_m + 1$; if two instructions may commit in the same cycle, $C_n = \max(W_n + 1,\ C_{n-1},\ C_{n-2} + 1)$.

Table 8.3 lists the result for the example. The Execute and Write result columns are out of order: I5 writes its result before I1, I3 and I4, which precede it. The Commit column is increasing. I2 and I5 commit six cycles after writing their results, held up by the slow division at the head of the ROB and, after it commits in cycle 9, by the limit of one commit per cycle: every later instruction commits exactly one cycle after its predecessor. This waiting delays no computation: I3 and I5 obtained the result of I2, and I6 those of I3 and I5, without waiting for any commit. The ROB delays only the moment at which results become part of the architectural state.

A ROB is sized to cover the longest stall it should hide: at four instructions per cycle, 224 entries hold some 56 cycles—an L3 hit on Skylake takes 42 cycles, a DRAM access about 200 (7-cpu.com).

**Table 8.3:** Timing of Example 8.6. The Execute column gives the cycles $D_n$ to $D_n + k_n - 1$ in which the operation is performed.

| | Instruction | Issue | Execute | Write result | Commit |
|---|---|---|---|---|---|
| I1 | `DIV R1, R2, R3` | 1 | 2–7 | 8 | 9 |
| I2 | `ADD R4, R5, R6` | 2 | 3 | 4 | 10 |
| I3 | `MUL R2, R4, R4` | 3 | 5–8 | 9 | 11 |
| I4 | `SUB R5, R1, R6` | 4 | 9 | 10 | 12 |
| I5 | `ADD R4, R4, R3` | 5 | 6 | 7 | 13 |
| I6 | `ADD R6, R2, R4` | 6 | 10 | 11 | 14 |

The ROB can nevertheless limit execution when it is too small. With only four entries, I5 could not be issued before cycle 10, the cycle after I1 commits, and it would execute several cycles later than in Table 8.3 although its operands are available from cycle 5. A ROB must therefore be large enough to hold all instructions issued while a slow instruction waits to commit.

## 8.10 Unified reservation stations

In Figure 8.3 each kind of execution unit has its own group of RSs, and the size of each group is fixed at design time. Programs differ in their mix of operations, so a program with many additions can find all adder RSs occupied, and issue must wait, while the multiplier RSs are empty. Because instructions are issued in program order, even a multiplication that follows the waiting addition cannot use the free multiplier RSs.

**Source** Lesson 8, video 35; H&P ch. 3.

**Definition 8.7** (Unified reservation stations). A *unified reservation station* is a single pool of RSs shared by all operations: an instruction can be issued into any free RS of the pool.

Issue then waits only when all RSs are occupied, so the same number of RSs is used more effectively. The cost lies in dispatch: instead of choosing among the RSs of one group for one execution unit, the dispatch logic must find, for each free execution unit, the ready instructions in the whole pool whose operation that unit performs, and select among them.

Intel's schedulers are unified (97 entries on Skylake, Lesson 7); AMD's Zen cores use several distributed integer schedulers and a separate floating-point scheduler. Vendor optimization manuals.

## 8.11 Superscalar processors

Lesson 6 showed that the IPC of a processor is limited by the number of instructions it can handle per cycle (Section 6.8).

**Source** Lesson 8, video 36; H&P §3.8.

**Definition 8.8** (Superscalar processor). A *superscalar* processor performs every step on more than one instruction per cycle: it fetches, decodes and issues several instructions, dispatches several to execution units, writes several results and commits several instructions in each cycle.

Every step needs its own hardware for this. Fetch and decode must handle several instructions at once; issue must rename several instructions in the same cycle, each seeing the RAT updates of the ones before it; dispatch needs several execution units; writing several results needs several CDBs, and every RS operand must compare its tag against all of them; commit must examine several entries at the commit pointer and write several registers.

The steps form a chain through which every instruction passes, so the processor sustains no more instructions per cycle than its most restrictive step handles. A processor that fetches, decodes and dispatches four instructions per cycle but issues three and commits two completes, in the long run, at most two instructions per cycle, however much ILP the program has. Widening one step pays off only if that step is the bottleneck.

## 8.12 Terminology

The names used here for the steps of an out-of-order processor are not universal. Papers, textbooks and processor documentation use several alternatives (Table 8.4). Some of them call commit *complete*; in this chapter, by contrast, an operation completes when its execution ends.

**Source** Lesson 8, video 37; H&P ch. 3.

**Caution** The same word can name different steps: in some texts *dispatch* is what this course calls issue, and *issue* what it calls dispatch.

**Table 8.4:** Names of the steps in this course and in other texts.

| This course | Names in other texts |
|---|---|
| Issue | issue, allocate, dispatch |
| Dispatch | dispatch, execute, issue |
| Commit | commit, complete, retire, graduate |

## 8.13 What is out of order

**Source** Lesson 8, videos 38–40; H&P §3.6.

A processor with out-of-order execution does not perform all of its steps out of order (Figure 8.5).

**Fetch and decode** take instructions in program order, along the predicted path.

**Issue** is in program order. Renaming requires it, because each instruction must see the RAT as left by all instructions before it, and ROB entries are allocated in program order.

**Execute and write result** are out of order: an instruction starts as soon as its operands are ready and finishes when its operation completes. These are the only steps in which a later instruction overtakes an earlier one.

**Caution** The ROB reorders nothing: entries enter and leave it in program order. The name refers to its effect on execution, not to the queue itself.

**Commit** is in program order again; the ROB restores the order that execution relaxed.

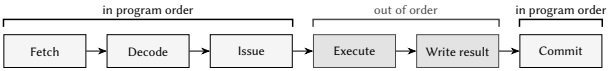



**Figure 8.5:** The steps of an out-of-order processor with a ROB. Only execution and write result take place out of program order.

**Key point — Lesson summary**

Tomasulo's algorithm writes results into the registers as soon as they are broadcast, so exceptions are not precise, mispredicted instructions corrupt registers, and wrong-path instructions can raise phantom exceptions. The reorder buffer keeps issued instructions in program order between a commit pointer and an issue pointer. The RAT names ROB entries, results are broadcast to the RSs and the ROB, RSs are freed at dispatch, and only commit, one ROB entry after another in program order, writes the register file and resets a RAT entry that still names the committing entry. A mispredicted branch or an excepting instruction is acted on when it commits: the ROB is emptied, the RAT points back to the register file, the RSs and execution units are cleared, and fetching restarts at the correct address. Timing tables gain a commit column that follows Equation (8.1). Unified reservation stations share one RS pool among all operations; a superscalar processor handles several instructions per cycle in every step and is limited by its narrowest step. Only execution and write result are out of order.

# Memory Ordering

| | |
|---|---|
| **Lecture** | Lesson 9, videos 1–14 |
| **Textbook** | H&P [1] §3.5–3.6 |
| **Keywords** | memory dependence, load, store, commit, load-store queue, LSQ, store-to-load forwarding, out-of-order load-store execution, in-order load-store execution, re-execution, reorder buffer, reservation station |

With the reorder buffer of Lesson 8, an out-of-order processor handles every dependence that passes through registers and keeps exceptions precise. Loads and stores, however, also depend on one another through memory, and whether a load reads the location that an earlier store writes is known only once both addresses have been computed, which is why Lesson 7 performed loads and stores in program order. This lesson shows how they can execute out of order: stores write memory only at commit, a load-store queue keeps the uncommitted loads and stores in program order, and each load obtains its value either from memory or directly from the store that wrote it.

## 9.1 Dependences through memory

**Source** Lesson 9, videos 1–2; H&P §3.5–3.6.

The processor of Lesson 8 deals with control dependences and with every dependence that passes through a register. Branch prediction (Lesson 4) removes most of the cost of control dependences, and the ROB cancels the instructions of a mispredicted path; renaming (Lesson 6) removes the false (WAR and WAW) dependences that arise from reusing a register name; the scheduling of Tomasulo's algorithm (Lesson 7) makes each instruction wait until the register values it reads have been computed; and the ROB makes exceptions precise.

Loads and stores remain. They depend on one another through memory locations in the same way as instructions depend on one another through registers (Section 7.8):

About a third of the instructions in typical integer programs are loads or stores, roughly two loads per store (H&P Appendix A, SPEC instruction mix).

```
SW  R1, 0(R2)      ; MEM[R2] <- R1
ADD R4, R5, R6     ; R4 <- R5 + R6
LW  R3, 0(R4)      ; R3 <- MEM[R4]
```

The load has a RAW dependence on the store exactly when R4 equals R2. Two properties distinguish such a dependence from a dependence through a register.

- It is not visible in the instructions themselves. The register names say nothing about the memory location, which is known only after the address has been computed, often several cycles after the instruction was issued.
- Nothing renames memory. The RAT maps register names only, so the processor must find, for every load, the store whose value the load has to read, or determine that there is none.

**Caution** Memory has WAR and WAW dependences too, but stores write memory in program order (Section 9.2), so only RAW needs machinery.

The rest of the lesson answers two questions: when a store actually writes memory, and where a load finds its value.

## 9.2 When memory is written

**Source** Lesson 9, video 3; H&P §3.6.

A store executes like any other instruction once its operands are available, but it does not write memory at that time. Executing the store computes the address and determines the value; the write itself takes place when the store commits. A write to memory can no more be undone than a write to a register. If a store wrote memory when it executed, an exception raised afterwards by an earlier instruction would let the handler see a location that, in program order, has not yet been written, and a store on a mispredicted path would change memory that the program never changes. At commit, in contrast, the store is known to belong to the execution of the program, and stores commit in program order.[1]

Deferring the write creates a problem for loads. A load could read memory as soon as it has computed its address, but a store that precedes it in program order may have executed and still be waiting to commit, or may not even have computed its address yet. If that store writes the location the load reads, memory still holds the old value, and the load must instead obtain the value of the store. The processor must therefore keep, in program order, the address and value of every store that has not yet committed, and every load must consult them.

[1] Commit is when the write becomes irrevocable, not when it reaches the cache: real cores move the value into a store buffer that drains into the L1 cache afterwards. Other cores may see it late, which is what the consistency models of Lesson 21 constrain.
In the pipeline of Lesson 3, the MEM stage performs loads and stores in program order.

**Key point**
Stores execute out of order but write memory at commit, in program order. Until then their values exist only inside the processor, and loads must look for them there.

## 9.3 The load-store queue

**Source** Lesson 9, videos 4–5; H&P §3.6.

The uncommitted loads and stores are kept in a queue that works like the ROB.

**Definition 9.1** (Load-store queue). The *load-store queue* (LSQ) holds, in program order, every load and store that has been issued but has not yet committed. Each entry has four fields:
**L/S** whether the instruction is a load or a store;
**Addr** the memory address, once it has been computed;
**Value** for a store, the value to be written; for a load, the value it has obtained;
**C** set when the entry is complete: a load has obtained its value, or a store has recorded both its address and its value.

Like the ROB, the LSQ is a circular buffer with an issue pointer and a commit pointer (Figure 9.1). A load or store receives the entry at the issue pointer when it is issued, in addition to its ROB entry, and its LSQ entry is freed when it commits. Other instructions do not use the LSQ. A load records its address and then its value. A store records its address as soon as its base register is available and its value as soon as the value register is, in either order. Because a store keeps its entry until it commits and writes memory only then, the LSQ contains exactly the stores whose values are not yet in memory.

The ROB also holds the loads and stores, but a separate queue keeps the address searches short.

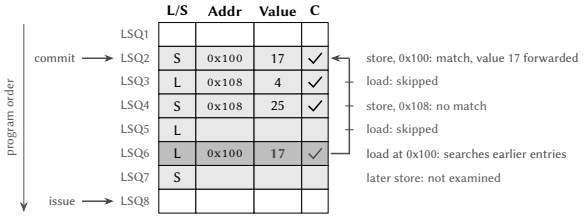



**Figure 9.1:** A load-store queue with eight entries; LSQ2 to LSQ7 are in use. The load in LSQ6 has computed the address `0x100`. Its search skips loads and the store to `0x108` and takes the value 17 from the nearest earlier store to `0x100`.

When a load has computed its address, it records the address in its LSQ entry and searches the earlier entries, from its own entry back towards the commit pointer. Loads are skipped, because reading a location does not change it; only stores matter. The search ends at the first store with the same address or at the commit pointer.

1. If it finds a store with the same address, this nearest matching store is the last write to the location before the load in program order, so its value is the one the load must read. The load copies the value from the store's LSQ entry, waiting for it if the store has not recorded its value yet, and does not access memory at all. Supplying a load with the value of an uncommitted store in this way is called *store-to-load forwarding*.
2. If it reaches the commit pointer without finding a matching store or a store with an unknown address, no uncommitted store writes the location before the load, so memory already holds the correct value, and the load reads memory.

If, before it ends, the search passes a store whose address has not been computed yet, that store may or may not write the location the load reads, and the processor cannot yet tell which. This case is the difficult one, and the next section presents the ways of dealing with it.

**Caution** Not the forwarding of Lesson 3: there a result is bypassed between pipeline stages, here a value is copied from one instruction's LSQ entry into another's.

## 9.4 Out-of-order and in-order load-store execution

**Source** Lesson 9, videos 5–8; H&P §3.6.

For a load whose search passes a store with an unknown address, there are three options.

1. Loads and stores execute in program order among themselves, as in Lesson 7.
2. The load waits until every earlier store has computed its address, so that its search never meets an unknown address.
3. The load executes anyway, and a wrong value is detected and corrected later.

The pattern of branch prediction (Lesson 4): proceed on a guess, check it later, squash what followed if the guess was wrong.

The first two options make loads wait; the third does not, but needs a check. A store's address is unknown until its base register is available, which can take long when that register is computed by a slow instruction. In the following program the address of the store depends on a multiplication, whereas the address of the load is available almost at once:

```
I1:  MUL R1, R2, R3      ; R1 <- R2 * R3
I2:  ADD R1, R1, R4      ; R1 <- R1 + R4 (an address)
I3:  SW  R5, 0(R1)       ; MEM[R1] <- R5
I4:  ADD R6, R6, R7      ; R6 <- R6 + R7 (an address)
I5:  LW  R8, 0(R6)       ; R8 <- MEM[R6]
I6:  SUB R9, R8, R5      ; R9 <- R8 - R5
```

We use the timing conventions of Section 8.8. MUL takes 4 cycles; ADD, SUB, loads and stores take 1 cycle, in which a load computes its address and obtains its value and a store computes its address; the value R5 is available from the start. I1 writes its result in cycle 6 and I2 in cycle 8, so the store I3 executes, and its address becomes known, only in cycle 9. I4 writes its result in cycle 6, so the load I5 could execute in cycle 7 (Figure 9.2).




**Figure 9.2:** The program of Section 9.4 under two policies. (a) I5 executes before the store I3 knows its address; I3 checks I5 in cycle 9. (b) I5 waits until I3 has computed its address. I1, I2 and I4 behave as in (a).

### 9.4.1 Out-of-order load-store execution

**Definition 9.2** (Out-of-order load-store execution). Under *out-of-order load-store execution* (option 3), a load executes as soon as its address can be computed, whether or not the earlier stores have computed theirs. Its LSQ search passes over stores with unknown addresses as if their addresses differed.

In the example I5 executes in cycle 7 and I6, which uses its value, in cycle 9. In return, every store must check its own address against the loads that have overtaken it. When I3 computes its address in cycle 9, it searches the later LSQ entries for loads with the same address whose C bit is set, that is, loads that have already obtained a value.

- If the address of I5 differs, nothing happens: I5 has obtained the correct value, and I6 writes its result in cycle 10.
- If the addresses are equal, I5 has obtained a value that I3 overwrites in program order, so every instruction that used this value, directly or indirectly, has computed a wrong result. The processor cancels I5 and all instructions after it and executes them again, starting by fetching I5. None of them has committed, since I3 precedes them, so the ROB discards them like the instructions of a mispredicted path (Section 8.5). When I5 executes again, its search finds I3, and it receives the value by forwarding.

**Tip** A load looks back for stores, a store looks forward for loads: each search skips its own kind.

Cancelling costs a pipeline restart, comparable to a branch misprediction, which Agner Fog's microarchitecture manual puts at 15–20 cycles on Skylake-class cores.

### 9.4.2 In-order load-store execution

**Definition 9.3** (In-order load-store execution). Under *in-order load-store execution* (option 1), loads and stores execute in program order among themselves: a load or store waits until every

earlier load and store has executed, where a store counts as executed once its address is known. Other instructions still execute out of order.

In the example I5 waits for I3 and executes in cycle 10, and I6 executes in cycle 12 and writes its result in cycle 13. No load ever obtains a wrong value, and nothing is re-executed. But I5 waits three cycles also when its address differs from that of I3, and then the wait is pure loss. Option 2 gives the same timing here, because no load precedes I5; in general it waits less, since a load does not depend on an earlier load.

### 9.4.3 Comparison

Out-of-order load-store execution gains cycles whenever a load does not read a location written by an earlier store whose address is still unknown, and loses cycles when it does, because cancelling and re-executing the load and the instructions after it costs more than waiting would have. Options 1 and 2 delay every load that follows a store with an unknown address, whether the two conflict or not. In real programs few loads actually read what such a store writes, so out-of-order load-store execution performs better, and it is the policy that high-performance processors use.[2]

[2] Cores decide per load rather than once for all: a predictor says whether this load is likely to conflict. The Alpha 21264 makes only those loads wait whose PC appears in its table of past squashes, cleared periodically so that a load can become speculative again; store sets [18] pair each load with the stores it has conflicted with. Intel calls its version memory disambiguation and has shipped it since 2006.

> **Key point**
> In-order load-store execution never re-executes a load but makes loads wait for stores they usually do not depend on. Out-of-order load-store execution lets loads run at once; correctness then relies on each store checking the completed later loads when it computes its address.

## 9.5 Store-to-load forwarding

Store-to-load forwarding relates each load to the store whose value it must read, and a processor with out-of-order load-store execution uses this relation from both ends.

**Source** Lesson 9, video 9; H&P ch. 3.

**For a load** the value comes from the nearest earlier store with the same address, the last store to write the location before the load in program order. An older store to the same address does not qualify: its value is overwritten by the nearer one. If the LSQ holds no earlier store with this address, the value comes from memory.

**For a store** the value goes to every later load with the same address, up to the next store with that address. The loads after that store receive the value of that store instead.

**Caution** Search from the load backwards: a search forward from the commit pointer finds the oldest matching store.

Hardware forwards only a load that lies entirely inside one store; on a partial overlap the load waits for the store to reach L1 instead, a dozen or so cycles on Skylake-class cores (Agner Fog, microarchitecture manual).

A load applies the first rule in its search (Section 9.3), and a store applies the second in its check of the completed later loads (Section 9.4). Figure 9.3 shows the relation for seven loads and stores to two addresses.

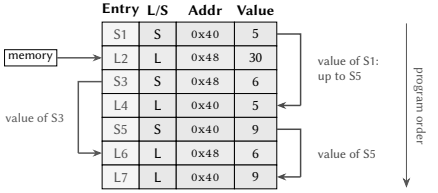



**Figure 9.3:** Store-to-load forwarding for seven loads and stores to `0x40` and `0x48`, with the values of a sequential execution. L4 receives the value of S1, not of S5, which follows it; L7 receives the value of S5, which overwrites that of S1. L2 precedes S3 and reads memory.

## 9.6 A load-store queue example

We now trace the loads and stores of Figure 9.3 (Example 9.4) under out-of-order load-store execution.

**Source** Lesson 9, video 10; H&P §3.6.

**Example 9.4.** The loads and stores of a program are, in program order,

```
S1:  SW  R1, 0(R2)      ; MEM[0x40] <- R1
L2:  LW  R3, 8(R2)      ; R3 <- MEM[0x48]
```

```
S3:  SW  R4, 8(R10)     ; MEM[R10+8] <- R4
L4:  LW  R5, 0(R2)      ; R5 <- MEM[0x40]
S5:  SW  R6, 0(R7)      ; MEM[R7] <- R6
L6:  LW  R8, 8(R2)      ; R8 <- MEM[0x48]
L7:  LW  R9, 0(R2)      ; R9 <- MEM[0x40]
```

with R2 = 0x40, R1 = 5, R4 = 6 and R6 = 9 available from the start. R10 = 0x40 and R7 = 0x40 are computed by slow instructions that precede S3 and S5, so these two stores learn their addresses late, in the order of Table 9.1. Memory initially holds 20 at 0x40 and 30 at 0x48.

**Tip** To check a trace, write down the values of a sequential execution first: a load that ends with any other value marks a missing re-execution.

**Table 9.1:** Execution of Example 9.4. In each step one entry computes its address: a load searches the earlier stores; a store checks the later loads whose C bit is set.

| Step | Entry | Addr | Search or check | Outcome |
|---|---|---|---|---|
| 1 | S1 | 0x40 | no later load completed | value 5 recorded |
| 2 | L2 | 0x48 | S1: other address | memory: 30 |
| 3 | L4 | 0x40 | S3: unknown; S1: match | from S1: 5 |
| 4 | S3 | 0x48 | L4: other address | no conflict |
| 5 | L6 | 0x48 | S5: unknown; S3: match | from S3: 6 |
| 6 | L7 | 0x40 | S5: unknown; S3: other; S1: match | from S1: 5 |
| 7 | S5 | 0x40 | L6: other address; L7: match | L7 re-executed |
| 8 | L7 | 0x40 | S5: match | from S5: 9 |

L4 and L6 pass over stores whose addresses are still unknown, and the later checks of S3 and S5 find no conflict with them. L7 also passes over S5, but in step 7 S5 computes 0x40 and finds L7 completed with this address, with no store to 0x40 between them. The value 5 that L7 obtained from S1 is overwritten by S5 in program order, so L7 and every instruction after it are cancelled and executed again, and in step 8 L7 receives 9 from S5.

After the entries commit in program order, memory holds 9 at 0x40 and 6 at 0x48, and the loads have delivered R3 = 30, R5 = 5, R8 = 6 and R9 = 9, the results of a sequential execution. With in-order load-store execution L4 would have waited for S3, and L6 and L7 for S5, and nothing would have been executed twice.

## 9.7 The LSQ, the ROB and the reservation stations

Loads and stores pass through the same steps as other instructions, and the LSQ adds work to each step (Table 9.2). A load or store is issued only when a ROB entry and an LSQ entry are both available; the LSQ entry takes the place of the reservation station that other instructions need with their ROB entry. A load broadcasts its value on the CDB under the name of its ROB entry, exactly like an arithmetic result. A store produces no register value, so nothing is broadcast, and its ROB entry is done as soon as the store has recorded both its address and its value.

**Source** Lesson 9, videos 11–14; H&P §3.6.

Real sizes: Skylake keeps 72 loads and 56 stores in flight against a 224-entry ROB, Ice Lake 128 and 72 against 352 (Intel optimization manual).

Both queues are in program order, so the load or store at the commit pointer of the ROB is also the oldest entry of the LSQ, and the two entries are freed together. A store leaves the LSQ exactly when its value is written to memory, so a later load always finds that value in one of the two places it looks: in the store's LSQ entry before the commit, in memory after it.

A multicore adds a rule: a load that has already read must also be squashed if another core writes its address before it commits (Lesson 21).

**Key point — Lesson summary**
Loads and stores depend on one another through memory addresses that are known only after execution, and memory is not renamed. Stores therefore write memory at commit, in program order. The load-store queue holds the uncommitted loads and stores in program order, with their addresses, values and completion bits. A load takes its value from the nearest earlier store with the same address by store-to-load forwarding, or from memory if there is none. Earlier stores with unknown addresses are handled either by in-order load-store execution, where the load waits for them, or by out-of-order load-store execution, where the load proceeds and a store that later computes a matching address has the load and all instructions after it re-executed; the latter is faster because such conflicts are rare. A load or store needs a ROB entry and an LSQ entry at issue, where other instructions need a ROB entry and an RS.

**Table 9.2:** The steps of a load and a store with an LSQ.

| Step | Load | Store |
|---|---|---|
| Issue | needs a free ROB entry and LSQ entry (no RS); the base register is looked up in the RAT, and the RAT entry of the destination register is set to the load's ROB entry | needs a free ROB entry and LSQ entry (no RS); the base and value registers are looked up in the RAT; no RAT entry changes |
| Execute | computes the address and records it in the LSQ; searches the earlier stores; copies a forwarded value or reads memory; sets C | records the address when the base register is available and checks the later loads whose C is set; records the value when it is available; sets C once both are recorded |
| Write result | broadcasts the value on the CDB; the ROB entry receives it | nothing to broadcast |
| Commit | the value is written to the register file, and the RAT entry is reset if it still names this ROB entry; the ROB and LSQ entries are freed | the value is written to memory; the ROB and LSQ entries are freed |

# Compiler ILP

| | |
|---|---|
| **Lecture** | Lesson 10, videos 1–20 |
| **Textbook** | H&P [1] §3.2, Appendix H |
| **Keywords** | instruction-level parallelism, instructions per cycle, dependence chain, tree height reduction, instruction scheduling, in-order execution, if-conversion, loop unrolling, function call inlining, code bloat, instruction count, software pipelining, trace scheduling |

Lessons 6–9 raised the instructions per cycle (IPC) of a processor with hardware: register renaming, out-of-order execution with a reorder buffer, and out-of-order load-store execution let the IPC approach the instruction-level parallelism (ILP) of the program. The hardware, however, can exploit only the parallelism that the program contains and that lies within its reach. This lesson shows how the compiler improves both: tree height reduction shortens the dependence chains that bound the ILP, instruction scheduling places independent instructions next to each other, and if-conversion, loop unrolling and function call inlining remove the branches, calls and returns that keep them apart.

**Caution** "Compiler ILP" is not a second kind of ILP: the ILP is a property of the program (Lesson 6), and the compiler changes the program.

## 10.1 How the compiler can raise IPC

**Source** Lesson 10, videos 1–2; H&P §3.2.

The IPC of every processor is bounded by the ILP of the program it executes (Section 6.8). Two limits on the IPC are not determined by the processor alone, and the compiler influences both.

**The ILP of the program** The ILP is the number of instructions divided by the length of the longest chain of RAW dependences (Equations (6.1) and (6.2)). How long the chains are depends on how the compiler has translated the source program: the same computation can often be expressed by instructions that form shorter chains.

**The parallelism a processor can find** A processor looks for instructions to execute only among a limited number of instructions that follow the oldest unexecuted one. An out-of-order processor considers the instructions that fit into its reorder buffer (Lesson 8); an in-order processor considers only the next few and stops at the first one that must wait (Lesson 6). Independent instructions that lie farther apart in program order than this window cannot execute in the same cycle, however high the ILP. The compiler can place such instructions close together, and it can remove the branches, calls and returns that stand between them.

The compiler sees far more of the program than a hardware window, and it runs before the program executes, so it can afford analyses that a processor could never carry out within a clock cycle. Table 10.1 lists the techniques of this lesson.

The compiler's window is a whole function, and after inlining the whole call tree; the processor's is its reorder buffer (Lesson 8).

**Table 10.1:** Limits on the IPC and the compiler techniques that address them.

| Limit | Compiler technique | Section |
|---|---|---|
| long dependence chains lower the ILP | tree height reduction | 10.2 |
| independent instructions lie far apart | instruction scheduling | 10.3 |
| branches separate instructions | if-conversion, trace scheduling | 10.4, 10.7 |
| loop branches separate iterations | loop unrolling, software pipelining | 10.5, 10.7 |
| calls and returns separate instructions | function call inlining | 10.6 |

## 10.2 Tree height reduction

**Source** Lesson 10, videos 3–4; H&P Appendix H.

A compiler translates an arithmetic expression by way of its *expression tree*, in which every operation is a node whose children are its operands. Each operation becomes one instruction, and the instruction of a node reads the results of the instructions of its children. The longest chain of RAW dependences among these instructions therefore equals the *height* of the tree, the number of operations on the longest path from the root to a leaf, and on the ideal processor of Lesson 6 each level of the tree takes one cycle.

One cycle per level assumes one-cycle operations. With a multi-cycle operation each level costs its latency, and the height matters even more.

A straightforward translation follows the order in which the source program writes the operands. For a sum of six variables held in R1 to R6, with the result in R10, it produces

```
ADD R10, R1, R2      ; cycle 1
ADD R10, R10, R3     ; cycle 2
ADD R10, R10, R4     ; cycle 3
ADD R10, R10, R5     ; cycle 4
ADD R10, R10, R6     ; cycle 5
```

Every addition uses the result of the previous one. The tree degenerates into a chain of height 5 (Figure 10.1a): the five instructions take five cycles on any processor, and their ILP is 1.

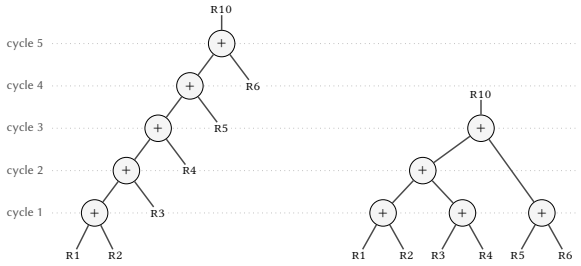
**Definition 10.1** (Tree height reduction). Rewriting an expression into an equivalent one whose expression tree has fewer levels, by using the associativity and commutativity of its operations, is called *tree height reduction*. The number of operations, and hence of instructions, does not change.

**Example 10.2.** Grouped as ((R1 + R2) + (R3 + R4)) + (R5 + R6), the same sum becomes

```
ADD R10, R1, R2      ; cycle 1
ADD R11, R3, R4      ; cycle 1
ADD R12, R5, R6      ; cycle 1
ADD R10, R10, R11    ; cycle 2
ADD R10, R10, R12    ; cycle 3
```

The three additions of pairs depend on nothing and execute in cycle 1, the sum of the first two pairs follows in cycle 2 and the final addition in cycle 3 (Figure 10.1b). There are still five instructions, now in three cycles: the ILP rises from 1 to $5/3 \approx 1.67$. The price is two further registers, R11 and R12, for intermediate results.

Registers are the budget for regrouping: x86-64 has 16 architectural integer registers, A64 and RISC-V 31; beyond them the compiler spills to the stack.




(a) left to right: height 5 (b) balanced: height 3

**Figure 10.1:** Expression trees of the sum of R1 to R6. (a) The left-to-right translation is a chain of five additions. (b) The balanced tree of Example 10.2 has height 3. Each level of additions executes in one cycle of the ideal processor.

For $n$ operands combined by one associative operation, the left-to-right translation has height $n-1$, whereas a balanced tree has height $\lceil \log_2 n \rceil$. Addition and multiplication are associative, and so are AND, OR and XOR. Subtraction is not, since $(a-b)-c \neq a-(b-c)$, but an expression with subtractions can still be regrouped when the signs are adjusted. Because $a-b = a+(-b)$, the terms can be grouped freely as long as each keeps its sign, and the signs inside a group that follows a minus sign are inverted: for example $a-b-c-d = (a-b)-(c+d)$, which reduces the height from 3 to 2.

**Caution** Floating-point addition is not associative (rounding); compilers regroup it only when permitted, as by `-ffast-math`.

> **Key point**
> Tree height reduction changes the dependences between instructions, not their number. It shortens the chain that bounds the ILP, which pays off on processors that can execute several instructions per cycle.

## 10.3 Instruction scheduling

**Source** Lesson 10, videos 5–7; H&P §3.2.

A processor executes independent instructions in the same cycle only if it finds them. A compiler usually translates a program statement by statement, so the instructions of one statement are followed by those of the next. When each statement forms a chain, the independent instructions of different statements are separated by the rest of the chain. An out-of-order

processor whose window covers both chains finds them anyway, but an out-of-order processor with a shorter window does not, and neither does an in-order processor, which executes the next instructions only up to the first one that must wait. Three compiler techniques make independent instructions easier to find: instruction scheduling, loop unrolling (Section 10.5) and trace scheduling (Section 10.7).

**Definition 10.3** (Instruction scheduling). Reordering the instructions of a program, without changing its results, so that instructions that can execute in the same cycle are adjacent in program order is called *instruction scheduling*.

For an in-order processor, scheduling fills the cycles in which the processor would otherwise wait with instructions that do not depend on the waiting one. To schedule for a processor that executes up to $w$ instructions per cycle, the compiler can determine for every instruction the cycle in which the ideal processor would execute it (Equation (6.2)) and emit the instructions in the order of these cycles, in groups of at most $w$. Every instruction then still follows the instructions whose results it reads, but the reordered program must also compute the same results as the original. A move that breaks a WAW or WAR dependence changes a value that some instruction uses; the compiler then either keeps the original order or compensates:

**Destination register** When a moved instruction would overwrite a register whose old value a later instruction still reads, it writes its result to another, otherwise unused register instead, and the instructions that read the result are changed accordingly.

**Address offset** When an instruction that adds a constant to a register, such as an index increment, moves ahead of an instruction that uses this register as the base of an address, the displacement of the address is reduced by the constant: `SW R3, 0(R1)` placed after `ADDI R1, R1, 4` becomes `SW R3, -4(R1)`.

**Caution** Memory holds the compiler back as well: a load may be moved ahead of a store only if the two addresses can be proved different. Two pointers of the same type may point to the same location, so unless the compiler can prove otherwise, the order stands.

**Example 10.4.** With $a$ to $h$ in R1 to R8, the statements $x = (a+b)\times c - d$ and $y = (e-f)\times g + h$ compile into

```
I1:  ADD R9, R1, R2      ; a + b
I2:  MUL R9, R9, R3      ; (a + b) * c
I3:  SUB R9, R9, R4      ; x <- (a + b) * c - d
I4:  SUB R10, R5, R6     ; e - f
I5:  MUL R10, R10, R7    ; (e - f) * g
I6:  ADD R10, R10, R8    ; y <- (e - f) * g + h
```

**Tip** Moving an instruction past an `ADDI` of $c$ subtracts $c$ from every displacement on that register; moving it the other way adds $c$.

The instructions form two chains, I1–I3 and I4–I6, so the ideal processor executes I1 and I4 in cycle 1, I2 and I5 in cycle 2, and I3 and I6 in cycle 3: the ILP is 2. A two-wide in-order processor, with every instruction taking one cycle, executes only I1 in cycle 1, because I2 must wait for it, and only I2 in cycle 2. In cycle 3 it executes I3 together with I4, but then I5 and I6 again need a cycle each. It takes five cycles, an IPC of $6/5 = 1.2$ (Figure 10.2a). Scheduled in the order I1, I4, I2, I5, I3, I6, every pair of adjacent instructions is independent, and the same processor needs three cycles, an IPC of 2, equal to the ILP (Figure 10.2b). The two statements use different registers, so the reordering breaks no WAW or WAR dependence.

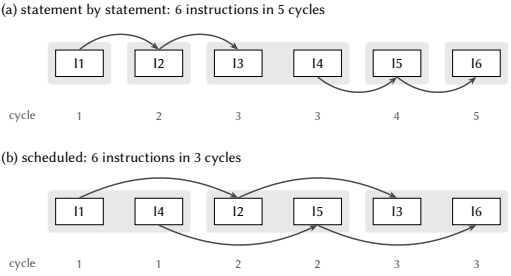



**Figure 10.2:** The program of Example 10.4 on a two-wide in-order processor. Arcs are RAW dependences; shaded groups execute in the same cycle. (a) In the order of the statements, the chain I1–I3 holds back I4. (b) After scheduling, both chains advance in every cycle.

**Example 10.5.** The loop

```
for (i = 0; i < n; i++)
    a[i] = a[i] + x;
```

adds $x$ to each of the $n$ elements of an integer array. With R1 pointing to the current element, R2 holding $x$, and R5 holding the address just past the last element, it compiles into

The compiled loop has no *i*: the compiler keeps the address as the only induction variable and compares it against the end in R5 (*strength reduction*).

```
Loop: LW   R3, 0(R1)      ; R3 <- a[i]
      ADD  R3, R3, R2     ; R3 <- a[i] + x
      SW   R3, 0(R1)      ; a[i] <- R3
      ADDI R1, R1, 4      ; next element
      BNE  R1, R5, Loop   ; repeat until the end
```

A four-wide in-order processor with perfect branch prediction executes the LW alone, because the ADD waits for it, and then the ADD alone, because the SW waits for it. The SW and the ADDI execute together, and the BNE, which waits for the ADDI, executes together with the LW of the next iteration. Every iteration takes three cycles, a CPI of $3/5 = 0.6$. Moved up behind the LW, the ADDI no longer waits behind the chain, and the SW, which now follows the increment, needs the displacement $-4$:

```
Loop: LW   R3, 0(R1)
      ADDI R1, R1, 4
      ADD  R3, R3, R2
      SW   R3, -4(R1)     ; R1 already advanced
      BNE  R1, R5, Loop
```

**Caution** The $-4$ is in bytes, not elements: a displacement is corrected by exactly the constant the base register gained, whatever the element size.

This is the order of the ideal cycles, with the branch kept at the end of the iteration. The LW and the ADDI now execute together, the ADD follows, and the SW, the BNE and the LW and ADDI of the next iteration fill all four places of the next cycle. Every iteration takes two cycles, a CPI of $2/5 = 0.4$.

A static schedule fixes an assumed latency: the order is right while the loads hit in the cache. On a miss an in-order processor stalls, and only the hardware of Lessons 7–9 reacts.

Scheduling thus matters for in-order processors, for out-of-order processors whose window is short compared with the chains in the program, and for out-of-order processors whenever more instructions are ready than their width, since the order decides which of them execute first (Lesson 7).

## 10.4 Scheduling and if-conversion

**Source** Lesson 10, videos 8–9; H&P §3.2.

Instruction scheduling can move instructions only within a stretch of code that is executed as a whole, and a conditional branch ends such a stretch. An instruction that follows the branch is executed on only one of its paths. Moving it ahead of the branch would execute it on the other path as well, and instructions of the two paths cannot be placed in the same group, because only one of the paths is executed. The compiler therefore cannot group instructions across a branch, even when the instructions before and after it are independent.

Such a stretch is a *basic block*. With a branch every five instructions or so in integer code (Lesson 3), one block alone offers little to schedule.

If-conversion (Lesson 5) removes this boundary for a small if-then-else. The converted code computes the condition and both parts unconditionally and selects the results with conditional moves, all in one branch-free sequence, which the compiler can schedule like any other: the instructions that compute the condition and the two parts do not depend on the selections, so they are scheduled into the first groups according to their own dependences, and the selections follow. The if-converted code of Example 5.4 already has this shape. Its first four instructions are independent of each other and the three conditional moves depend only on them, so a four-wide in-order processor executes all seven instructions in two cycles.

### 10.4.1 Loops

A loop ends every iteration with a branch back to its beginning, and this branch cannot usefully be if-converted. Removing it would mean executing the body as many times as the loop could ever iterate, and each copy would need its own predicate telling whether its iteration should have been executed; the instructions that compute and apply these predicates would cost more than scheduling could gain. Loop branches are, moreover, predicted accurately, and predicating one would execute the code that follows the loop in every iteration (Table 5.1). The loop branch therefore stays, and it confines instruction scheduling to a single iteration: the scheduled body of Example 10.5, with two cycles per iteration on the four-wide in-order processor, is the best order for that processor. When the body is short, most of the independent instructions of the loop lie in different iterations, where the compiler cannot bring them together. Loop unrolling addresses this limitation.

## 10.5 Loop unrolling

In a loop with a short body, a large fraction of the instructions executed are loop overhead, and the work is spread over many iterations separated by the loop branch. In the loop of Example 10.5, three of the five instructions do the work; the ADDI and the BNE are overhead.

**Source** Lesson 10, videos 10–15; H&P §3.2.

**Definition 10.6** (Loop unrolling). Rewriting a loop so that each iteration of the new loop performs the work of $k$ iterations of the original loop, by repeating the body $k$ times with adjusted indices and advancing the index by $k$, is called *loop unrolling*. The number $k$ is the *unrolling factor*: unrolling a loop once gives $k = 2$, unrolling it twice $k = 3$.

Unrolled with $k = 2$, for an even $n$, the loop becomes

```
for (i = 0; i < n; i += 2) {
    a[i]   = a[i]   + x;
    a[i+1] = a[i+1] + x;
}
```

and compiles into

```
Loop: LW   R3, 0(R1)       ; a[i] <- a[i] + x
      ADD  R3, R3, R2
      SW   R3, 0(R1)
      LW   R3, 4(R1)       ; a[i+1] <- a[i+1] + x
      ADD  R3, R3, R2
      SW   R3, 4(R1)
      ADDI R1, R1, 8       ; two elements further
      BNE  R1, R5, Loop
```

Each iteration now processes two elements with eight instructions, of which the index increment and the branch are executed once. The two copies of the body are adjacent and separated by no branch, so the compiler can schedule them together once the second copy keeps its intermediate result in a register of its own (Section 10.5.2).

If $n$ is not a multiple of $k$, the compiler adds code, typically a copy of the original loop, for the remaining elements.

### 10.5.1 Instruction count and ILP

Unrolling does not affect the clock period, but it changes the other two factors of the iron law (Equation (2.3)). The original loop executes 5 instructions per element, the unrolled loop 8 per two elements, or 4 per element: the overhead instructions are executed half as often, and the instruction count falls by 20 %. With a factor $k$ the loop executes $(3k + 2)/k$ instructions per element, which approaches 3, the work alone.

**Tip** Read it as $3 + 2/k$: the overhead per element halves with every doubling of $k$, so most of the gain lies in the first few steps.

The ILP rises as well. On the ideal processor (Equation (6.2)) the ADDIs of successive iterations form a chain of one cycle per iteration. Every LW of an iteration reads R1 from the ADDI of the previous iteration and executes in the same cycle as the ADDI of its own iteration, and its ADD and SW follow in the next two cycles; the reuse of R3 by the second copy creates WAW and WAR dependences that renaming removes. The ideal processor thus executes one iteration, or $k$ elements, per cycle, and the ILP of a long loop approaches the number of instructions per iteration: 5 for the original loop, 8 for the unrolled one. This relies on iterations that depend on each other only through the index: in a loop that accumulates, such as `s = s + a[i]`, the additions to `s` form a chain of one cycle per element in either version.

**Tip** Such a reduction is unrolled with $k$ separate accumulators, added together after the loop; for floating point this needs the reassociation permission of Section 10.2.

### 10.5.2 Cycles per instruction

Whether the CPI falls as well depends on the processor. Table 10.2 compares four versions of the loop on the four-wide in-order processor of Example 10.5 and on the ideal processor, both with perfect branch prediction. The first two versions are those of Example 10.5. Unrolled but not scheduled, the loop still executes one LW–ADD–SW chain after the other: the second LW joins the first SW, the ADDI joins the second SW, and the BNE joins the LW of the next iteration, so every iteration of eight instructions takes five cycles. The CPI rises from 0.6 to $5/8 = 0.625$, although the time per element falls from 3 to 2.5 cycles with the instruction count. Scheduled in the order of the ideal cycles, the unrolled body becomes

A loop branch is mispredicted about once per run, at the exit; a remainder loop adds at most one more.

```
Loop: LW   R3, 0(R1)
      LW   R4, 4(R1)       ; second copy uses R4
      ADDI R1, R1, 8
      ADD  R3, R3, R2
```

```
ADD  R4, R4, R2
SW   R3, -8(R1)     ; R1 already advanced
SW   R4, -4(R1)
BNE  R1, R5, Loop
```

Both compensations of Section 10.3 are needed. The second LW now precedes the ADD and the SW of the first copy, which still need R3, so the second copy computes its value in R4; and both stores follow the increment, so their displacements are reduced by 8. The processor executes both LWs and the ADDI in the first cycle and both ADDs in the second, and the two SWs, the BNE and the first LW of the next iteration fill the third. From then on the groups no longer coincide with the iterations, and in the steady state 16 instructions, two iterations, take 5 cycles: 1.25 cycles per element, a CPI of $5/16 = 0.3125$. Compared with the scheduled original loop, the time per element falls from 2 to 1.25 cycles, a speedup of 1.6, of which $5/4 = 1.25$ comes from the instruction count and $0.4/0.3125 = 1.28$ from the CPI.

**Table 10.2:** The loop of Example 10.5 before and after unrolling with $k = 2$, each as compiled and scheduled: instructions per element, and cycles per element and CPI on two processors with perfect branch prediction.

| | | Four-wide in-order | | Ideal | |
|---|---|---|---|---|---|
| Version | Instructions | Cycles | CPI | Cycles | CPI |
| Original | 5 | 3.00 | 0.600 | 1.00 | 0.200 |
| Scheduled | 5 | 2.00 | 0.400 | 1.00 | 0.200 |
| Unrolled | 4 | 2.50 | 0.625 | 0.50 | 0.125 |
| Unrolled and scheduled | 4 | 1.25 | 0.3125 | 0.50 | 0.125 |

On the in-order processor, unrolling thus lowers the CPI through the scheduling it makes possible: the loop branch no longer keeps the instructions of two iterations apart, and the compiler interleaves them. On the ideal processor the order of the instructions does not matter. There the CPI falls because the ILP rises from 5 to 8, and together with the lower instruction count the time per element halves; an out-of-order processor with a large window profits in the same way, up to its width. A processor that completes one instruction per cycle has a CPI of 1 in every version and gains only the factor 1.25 from the instruction count. Loop unrolling therefore lowers the instruction count on every processor, and the CPI on processors that execute several instructions per cycle.

### 10.5.3 Downsides of unrolling

The body grows linearly with $k$, from 5 instructions to $3k + 2$, whereas the gain levels off: the instructions per element approach 3, and a real processor cannot raise its IPC beyond its width, however high the ILP. Going from $k = 4$ (14 instructions, 3.5 per element) to $k = 8$ (26 instructions, 3.25 per element) nearly doubles the body to execute only 7 % fewer instructions, an instance of the diminishing returns of Lesson 2.

**Definition 10.7** (Code bloat). The growth of the program size caused by a transformation such as loop unrolling is called *code bloat*.

The front end is the first limit: Skylake caches 1536 decoded micro-operations and Zen 2 4096, and a loop unrolled past that falls back to the decoders. Intel optimization manual; AMD Family 17h software optimization guide.

Larger code occupies more memory, and, as the lessons on caches will show, it causes more cache misses when its instructions are fetched, which can cancel the gain. Two further problems concern the number of iterations. When $n$ is not a multiple of $k$, the remaining iterations need additional code. And unrolling presumes that the number of remaining iterations is known when an iteration of the unrolled loop starts. A loop whose exit is decided inside the body, such as a search that stops at the first zero element, cannot simply execute $k$ copies of the body per iteration, because the later copies could perform iterations that the original loop would not; it needs additional exit checks. Compilers therefore unroll with small factors, and only loops whose bodies are short.

**Key point**
Loop unrolling executes the index increment and the loop branch once per $k$ iterations of the original loop, which lowers the instruction count on every processor. It also lowers the CPI on processors that execute several instructions per cycle: on in-order processors because the compiler can schedule the instructions of several iterations together, on processors that find the parallelism themselves because the ILP of loops with independent iterations rises. Its costs are code that grows with $k$ and extra code when the number of iterations is not a multiple of $k$ or not known in advance.

## 10.6 Function call inlining

A call separates the instructions of the caller from those of the called function, much as a branch separates the two paths of an if-then-else. In addition, every call executes instructions that perform none of the function's work: the call and the return themselves, the copying of the arguments into the registers prescribed by the calling convention and of the result out of them, and often the saving and restoring of registers on the stack.

**Source** Lesson 10, videos 16–18; H&P Appendix A.

**Definition 10.8** (Function call inlining). Replacing a call by a copy of the body of the called function, adapted to the operands of the caller, is called *function call inlining*.

Real conventions pass more in registers: the System V AMD64 ABI the first six integer arguments, AArch64's the first eight. The rest go on the stack.

**Caution** In C and C++ `inline` is a linkage keyword, not an order: GCC inlines functions not declared `inline` from `-O2` on, and only `always_inline` forces it.

**Example 10.9.** The function

```
int sqdiff(int x, int y) { int d = x - y; return d * d; }
```

is called as `c = sqdiff(a, b)`, with $a$, $b$ and $c$ in R8, R9 and R10. The calling convention passes the arguments in R4 and R5 and returns the result in R2:

```
        ADD  R4, R8, R0     ; x <- a
        ADD  R5, R9, R0     ; y <- b
        CALL SqDiff
        ADD  R10, R2, R0    ; c <- result
        ...
SqDiff: SUB  R2, R4, R5     ; d <- x - y
        MUL  R2, R2, R2     ; result <- d * d
        RET
```

Inlined, the call becomes

```
        SUB  R10, R8, R9    ; c <- a - b
        MUL  R10, R10, R10  ; c <- c * c
```

Each call executes seven instructions, the inlined code two. The argument and result copies also lengthen the dependence chain from $a$ and $b$ to $c$: on the ideal processor the call takes four cycles, the inlined code two (Figure 10.3).

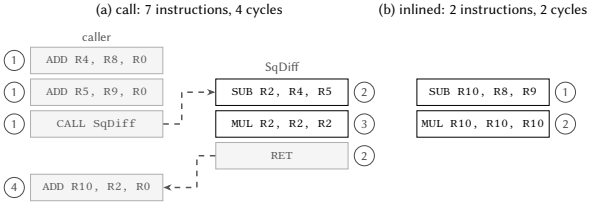



**Figure 10.3:** Example 10.9 before and after inlining. Shaded instructions only implement the call; dashed arrows are the transfers of control; circled numbers give the cycle in which the ideal processor executes each instruction.

Inlining thus lowers the instruction count, since the call, the return and the copying of arguments and results disappear, and it shortens dependence chains, since the body reads the caller's operands directly and delivers its result where the caller needs it. Moreover, the body becomes part of the caller's code: the compiler can schedule its instructions together with those of the caller and apply tree height reduction across the former call, and the processor no longer has to predict the call and the return (Lesson 4).[1]

[1] Inlining is above all an *enabling* transformation: with the body in the caller, the arguments are often known constants there, so constant propagation, dead code elimination and the techniques of this lesson apply across the former boundary. Compilers therefore inline early and optimize afterwards.

### 10.6.1 Code bloat from inlining

Inlining places a separate copy of the body at every call site, and for large functions called from many sites the code grows considerably.

**Example 10.10.** A function with a body of 20 instructions and a return is called from 10 sites, each of which spends 4 instructions on the call (two argument copies, the call and a result copy). Without inlining the code holds $10 \times 4 + 21 = 61$ instructions, and each call executes $4 + 21 = 25$. Inlined at all sites, the function and the call sequences disappear, but every site receives the 20 instructions of the body: 200 instructions, more than three times the code, to save 5 of the 25 instructions (20 %) of each call. For the function of Example 10.9, whose body has two instructions, the same 10 sites need $10 \times 4 + 3 = 43$ instructions without inlining and only $10 \times 2 = 20$ with it: the code shrinks, and each call executes 2 instructions instead of 7.

First-level instruction caches have stayed small—32 KB on both Skylake and Zen 3—and every inlined copy competes for them (Intel and AMD optimization manuals).

As with loop unrolling, the larger code causes more cache misses and can cancel the gain. Compilers therefore inline small functions, whose call overhead is large compared with their body, and functions called from a single site, whose one copy replaces the original; large functions called from many sites remain calls.

> **Key point**
> Function call inlining removes the call, the return and the passing of arguments and results, lowering the instruction count and shortening dependence chains, and it lets the compiler optimize the body together with the caller. It pays off for small functions; for large functions called from many sites, code bloat outweighs the gain.

## 10.7 Other compiler techniques

**Source** Lesson 10, videos 19–20; [19]; [20].

The techniques of this lesson are examples of a larger body of compiler transformations that raise the IPC. Two further techniques carry instruction scheduling across the boundaries that confine it; they are only outlined here. In *software pipelining* the compiler treats a loop like a pipeline: it schedules the instructions of successive iterations so that the iterations overlap, and every cycle combines independent instructions of several iterations while the results stay those of the original loop [19]. The second, *trace scheduling*, schedules across branches without removing them: the compiler uses a profile to select the most frequently executed path through several branches, schedules this path as one block of code, and adds checks that branch to compensation code, which corrects the results, when execution leaves the path [20]. Carried to its end, scheduling by the compiler leads to processors that leave the grouping of operations into parallel steps entirely to the compiler: the VLIW processors of Lesson 11.[2]

[2] IA-64, the best-known VLIW-style instruction set (Lesson 11), supports software pipelining in hardware: its *rotating* registers rename a register by iteration, rotating predicates switch the stages of the pipeline on and off, and a dedicated loop branch fills and drains it, so one copy of the body does what unrolling needs several copies for.

> **Key point — Lesson summary**
> The IPC is limited by the ILP of the program and by the independent instructions a processor can find within its window; the compiler influences both. Tree height reduction regroups expressions so that their instructions form shorter chains, raising the ILP at the same instruction count. Instruction scheduling makes independent instructions adjacent, changing destination registers and address offsets where instructions move past each other; it helps in-order and small-window processors. Branches confine it, and if-conversion removes them from small if-then-else constructs. Loop unrolling executes the index increment and the loop branch once per $k$ iterations, lowering the instruction count; it also lowers the CPI, because the compiler can schedule the instructions of several iterations together and because the ILP of loops with independent iterations rises. Function call inlining removes calls, returns and argument copying. Both cause code bloat, so compilers reserve them for short loops and small functions. Software pipelining and trace scheduling extend scheduling across loop iterations and across branches.

# VLIW

| | |
|---|---|
| **Lecture** | Lesson 11, videos 1–9 |
| **Textbook** | H&P [1] §3.7, Appendix H |
| **Keywords** | very long instruction word, VLIW, out-of-order superscalar, in-order superscalar, NOP, backward compatibility, irregular code, code bloat, full predication, branch hint, stop bit, Itanium, EPIC, digital signal processor |

Lesson 10 showed that the compiler can raise the number of instructions a processor completes per cycle by rewriting the program: it shortens chains of dependences, places independent instructions next to each other, and removes branches and calls. In a very long instruction word (VLIW) processor this idea is taken to its end point: the compiler alone decides which operations execute in parallel and encodes the decision in the instructions, so that the hardware needs none of the dependence-tracking machinery of the out-of-order processors of Lessons 7–9. This lesson compares VLIW with superscalar designs, weighs its advantages against its disadvantages, and shows where the approach has failed and where it has succeeded.

## 11.1 Superscalar and VLIW processors

A superscalar processor (Lesson 8) executes up to $N$ instructions per cycle. Designs differ in how many instructions they examine to find $N$ that can execute together, and in how much of that search they leave to the compiler (Figure 11.1 and Table 11.1). An out-of-order design finds the parallelism of any program by itself, at the price of complex hardware for renaming, window scheduling and the reorder buffer (Lessons 6–8). An in-order design stops at the first instruction that must wait for another, so it needs the compiler to place independent instructions next to each other (Lesson 10). A VLIW processor executes, without any checking, one instruction of up to $N$ operations that the compiler has made independent.

**Source** Lesson 11, videos 1–3; H&P §3.7; [21].

**Caution** A wrong grouping is not slow but wrong: nothing checks, so two dependent operations in one instruction are executed anyway. Correctness rests on the compiler.

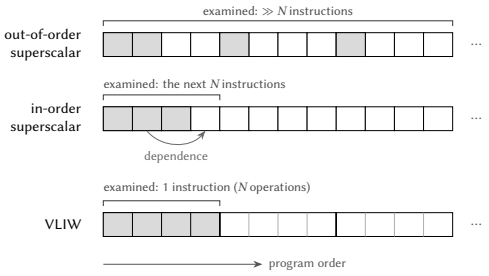



**Figure 11.1:** What each design examines to fill one cycle with up to $N = 4$ operations. Shaded boxes are executed in this cycle; the in-order design stops at the first instruction that depends on an earlier one in its group.

**Definition 11.1** (Very long instruction word). A *very long instruction word* (VLIW) architecture is an instruction set architecture in which each instruction consists of up to $N$ operations, each equivalent to one instruction of an ordinary ISA, that are executed in parallel in the same cycle. Choosing the operations that form an instruction, and ensuring that they are independent, is the task of the compiler; the processor performs no dependence checking.

Real width: an IA-64 *bundle* is 128 bits—three 41-bit operations and a 5-bit template that gives their execution-unit types and the stop positions (Intel Itanium architecture manual).

An instruction that carries $N$ operations is about $N$ times as long as an ordinary one, which gives the architecture its name. Since it does the work of up to $N$ ordinary instructions, a VLIW processor that completes one instruction per cycle is comparable to a superscalar processor with an IPC of up to $N$ (Lesson 6).

**Caution** Instruction counts and CPI of a VLIW program are not comparable with those of an ordinary ISA; compare operations per cycle.

**Example 11.2** (Packing operations into instructions). The fragment below adds four array elements and stores the sum. The comments list its RAW dependences (Lesson 3); the four loads only read R8 and are independent of each other.

**Table 11.1:** Superscalar and VLIW processors that execute up to $N$ operations per cycle.

| | Out-of-order | In-order | VLIW |
|---|---|---|---|
| Executes per cycle | $\leq N$ instructions | $\leq N$ instructions | 1 instr. ($\leq N$ ops) |
| Examines | $\gg N$ instructions | next $N$ instructions | 1 instruction |
| Hardware | complex | simpler | simplest |
| Compiler | can help | must help | does all the work |

```
I1: LW  R1, 0(R8)
I2: LW  R2, 4(R8)
I3: LW  R3, 8(R8)
I4: LW  R4, 12(R8)
I5: ADD R5, R1, R2    ; RAW with I1, I2
I6: ADD R6, R3, R4    ; RAW with I3, I4
I7: ADD R7, R5, R6    ; RAW with I5, I6
I8: SW  R7, 16(R8)    ; RAW with I7
```

A VLIW compiler takes the operations in program order and places each one in the earliest instruction that follows all the operations it depends on and still has a free slot; slots left over are filled with NOPs (Table 11.2). With four slots the eight operations occupy four instructions, hence four cycles, where an ordinary ISA needs eight instructions; with two slots they occupy five.

**Tip** Lower bound: $\lceil n/N \rceil$ instructions, and never fewer than the longest dependence chain has operations—$\max(2, 4) = 4$ here. The bound need not be reachable: with two slots it is 4, yet no schedule does better than five, because I5 and I6 together need all four loads, which fill the first two instructions.

**Table 11.2:** The fragment of Example 11.2 compiled for VLIW ISAs with four and with two operation slots. Each row is one instruction; sizes assume 32-bit slots.

| Instruction | 4 slots | 2 slots |
|---|---|---|
| 1 | I1, I2, I3, I4 | I1, I2 |
| 2 | I5, I6, NOP, NOP | I3, I4 |
| 3 | I7, NOP, NOP, NOP | I5, I6 |
| 4 | I8, NOP, NOP, NOP | I7, NOP |
| 5 | — | I8, NOP |
| NOP slots | 8 of 16 | 2 of 10 |
| Code size | 512 bits | 320 bits |

## 11.2 Advantages and disadvantages

**Source** Lesson 11, videos 4–5; H&P §3.7, Appendix H.

Leaving the search for parallelism to the compiler has three advantages.

- **The compiler has plenty of time.** An out-of-order processor must find independent instructions within a fraction of every clock cycle and only in its window; the compiler runs once and can analyze the whole program.
- **The hardware is simpler.** Without renaming, window scheduling and a reorder buffer, a VLIW processor needs less area and power for the same operations per cycle, so it can be more energy efficient.
- **Loops and regular code work well.** When arrays are processed in loops, the dependences are few and visible at compile time, and loop unrolling (Lesson 10) removes the branches between iterations, so that their independent operations fill the slots.

It also has four disadvantages.

- **Latencies are not known at compile time.** The compiler groups operations as if each completes within its cycle. If one takes longer, e.g. a load with a cache miss (Lesson 12), all later operations wait for it, even independent ones, whereas an out-of-order processor keeps executing instructions that do not depend on it.

The number the compiler must assume: on Skylake a load takes 4 cycles from L1, 12 from L2 and about 40 from L3 (Intel optimization manual), and about 200–300 cycles (≈ 60–80 ns) from DRAM (7-cpu.com).

**Tip** The first two are what the compiler cannot know: the actual latencies, the actual addresses and the actual branch directions.

- **Irregular code is hard.** The compiler sees the program, not its execution. When it cannot prove two operations independent—a store and a load through pointers that may address the same location, or operations on either side of a data-dependent branch—it must place them in different instructions, whereas an out-of-order processor knows the actual addresses and predicts the branches at run time (Lessons 4 and 9).
- **Code bloat.** Whenever fewer than $N$ independent operations are available, NOPs fill the slots and the program grows: the four-slot version in Table 11.2 is half NOPs and occupies 512 bits, against 256 bits for eight ordinary 32-bit instructions. Compaction (Section 11.3) reduces this overhead.

- **A new processor needs recompiled programs.** The grouping of operations and the number of slots are fixed at compile time, which undermines backward compatibility (below).

**Definition 11.3** (Backward compatibility). A processor offers *backward compatibility* when it runs, unmodified, the programs compiled for earlier processors of its family.

For superscalar processors backward compatibility comes with a performance gain: a wider out-of-order processor finds more parallelism in an *old* binary without recompilation. A new VLIW processor with more slots either cannot run an old VLIW binary at all, because the instruction format has changed, or runs it with no more operations per cycle than the processor it was compiled for.[1]

**Example 11.4** (An old binary on a wider VLIW processor). A four-slot processor that still accepts the two-slot format executes the two-slot version of Example 11.2 in five cycles, as the old processor does; only the recompiled four-slot version takes four. The eight ordinary instructions, counted the same way, take five cycles on a two-wide and four on a four-wide out-of-order processor, without recompilation.

> **Key point**
> VLIW trades hardware complexity for dependence on the compiler. It pays off when the code is regular and compiled for the exact processor; it loses when parallelism or operation latencies become known only at run time, or when old binaries must run faster on new processors.

[1] Software can supply the missing compatibility. Transmeta's Crusoe (2000) executed no x86 instruction directly: a translator, the code morphing software, compiled blocks of x86 code into 128-bit VLIW instructions of four operations and cached them [22]; the later Efficeon used 256-bit instructions. Nvidia's Denver (2014) decodes Arm instructions in hardware and translates only frequently executed Arm code into its native VLIW instructions.

## 11.3 VLIW instructions

**Source** Lesson 11, video 6; H&P Appendix H.

A VLIW instruction is a sequence of $N$ operation slots (Figure 11.2). Each slot is encoded like an instruction of an ordinary ISA, with its own opcode from the full set of ordinary opcodes and its own operands.

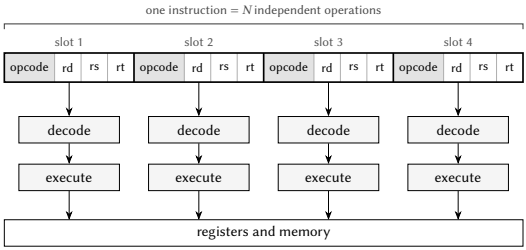



**Figure 11.2:** A VLIW instruction with four operation slots. Each slot is decoded and executed independently of the others; no hardware checks for dependences between slots.

The operations of one instruction must be independent: none may use the result of another, and no two may write the same location. The hardware does not check this, so correct grouping is the responsibility of the compiler. Operations in *consecutive* instructions may depend on each other, as I5 depends on I1 in Table 11.2.

**Caution** Independence is required only within an instruction, not between instructions.

The compiler exposes the parallelism with instruction scheduling, as in Example 11.2, helped by the other techniques of Lesson 10: tree height reduction shortens dependence chains so that more operations can share an instruction, and loop unrolling and function call inlining remove the branches, calls and returns that would separate operations. A VLIW processor has no renaming hardware, so the compiler itself removes false dependences (Lesson 3) by writing results into otherwise unused registers, of which the instruction set provides many.

IA-64 gives the compiler 128 integer and 128 floating-point registers to rename into, against 32 in MIPS and 16 in x86-64 (Lesson 5).

Branches limit the grouping, since the operations after a branch execute only on one of its paths. A VLIW instruction set therefore provides good support for predication (Lesson 5), ideally full predication, so that the compiler can if-convert branches and group the operations of both paths. A branch that remains can carry a *branch hint*, with which the compiler tells the processor which way it expects the branch to go.

Fixed slots waste space whenever fewer than $N$ independent operations are available. Compacted instructions store the operations one after another without NOPs, and a *stop bit* marks

the last operation of each group that executes together. In the four-slot program of Table 11.2, I4, I6, I7 and I8 carry a stop bit; at 33 bits per operation the code occupies 264 bits instead of 512.

## 11.4 VLIW processors in practice

**Source** Lesson 11, videos 7–9; H&P §3.7, Appendix H.

Two kinds of processors show where the VLIW approach fails and where it succeeds.

### 11.4.1 Itanium

IA-64 uses stops in the same way to end groups of independent instructions (Section 11.3).

The most prominent attempt to bring VLIW-style instructions to general-purpose computing was *Itanium*, the processor family Intel built for the IA-64 instruction set; its designers called the approach EPIC, explicitly parallel instruction computing [23]. The instruction set, however, accumulated many additional features whose hardware became very complex, removing the main advantage of VLIW. The compiler still found little parallelism in irregular code, and Itanium did not displace out-of-order processors.

### 11.4.2 Digital signal processors

H&P lists the TI C6x DSPs as an example of the VLIW approach.

**Caution** The C6x marks the complement of a stop bit: the lowest bit of an operation set to 1 means *in parallel with the next one*, so a group ends where that bit is 0; eight 32-bit operations make a 256-bit packet (TI C6000 reference guide).

A *digital signal processor* (DSP) is a processor specialized for digital signals such as audio, video and radio. This market plays to the strengths of VLIW (Section 11.2).

- Loops that apply the same computation to many samples dominate the code: regular code in which the compiler finds plenty of parallelism.
- Programs are usually compiled for the particular DSP of the device they run in, so backward compatibility of binaries matters little.
- DSPs sit in power-limited devices, where energy-efficient hardware pays off.

> **Key point — Lesson summary**
> An out-of-order superscalar processor examines many more than $N$ instructions and needs no help from the compiler; an in-order one examines $N$ and needs that help; a VLIW processor examines one instruction of up to $N$ independent operations and relies on the compiler entirely. VLIW hardware is simple and energy efficient and performs well on regular loop code, but binaries must be recompiled for each new processor, irregular code yields little parallelism, a cache miss on one load delays all later operations, and NOPs bloat the code unless it is compacted with stop bits. Itanium failed in general-purpose computing; DSPs are where VLIW fits. Lesson 12 turns from instructions per cycle to memory.

# Cache Review

| | |
|---|---|
| **Lecture** | Lesson 12, videos 1–45 |
| **Textbook** | H&P [1] §2.1, §B.1–B.2 |
| **Keywords** | locality principle, temporal locality, spatial locality, cache hit, cache miss, AMAT, miss penalty, block offset, tag, valid bit, direct-mapped cache, set-associative cache, LRU, dirty bit |

Lessons 3–11 made processors complete ever more instructions per second, yet every instruction has to be fetched from memory, many also load or store data, and Lesson 1 showed that memory speed falls ever further behind processor speed (the memory wall). This lesson reviews the mechanism that bridges the gap: the cache, a small and fast memory that serves most accesses without waiting for the large and slow main memory, because programs access memory with locality. It covers how cache performance is measured, how a cache is organized and searched, which block is evicted on a miss, and how writes are handled; virtual memory (Lesson 13) and the advanced cache techniques of Lesson 14 build directly on this material.

## 12.1 The locality principle

Caches work because the memory accesses of a program are far from random.

**Source** Lesson 12, videos 1–6; H&P §2.1; [24].

**Definition 12.1** (Locality principle). The observation that what a program does in the near future is likely to be close to what it has done in the recent past is called the *locality principle*.

For memory references, "close" has two meanings. Suppose a program has just accessed address $X$.

**Definition 12.2** (Temporal and spatial locality). A program exhibits *temporal locality* when it is likely to access the same address $X$ again soon. It exhibits *spatial locality* when it is likely to access addresses near $X$ soon.

Both kinds follow from the way programs are structured. A loop executes the same instructions over and over (temporal locality), and those instructions are fetched from consecutive addresses (spatial locality). A variable updated in every iteration is accessed at the same address again and again; the elements of an array processed in order lie at consecutive addresses.

**Example 12.3.** The loop below copies R6 bytes from the buffer at address R4 to the buffer at address R5.

```
Loop:  LB   R3, 0(R4)      ; R3 <- source byte
       SB   R3, 0(R5)      ; destination byte <- R3
       ADDI R4, R4, 1      ; next source byte
       ADDI R5, R5, 1      ; next destination byte
       ADDI R6, R6, -1     ; one byte less to copy
       BNE  R6, R0, Loop   ; repeat for all bytes
```

The six instruction fetches have both kinds of locality: they go to consecutive addresses, and the same six addresses are fetched in every iteration. The data accesses form two streams, one through each buffer. Every byte is read or written exactly once, so the data has no temporal locality. It has spatial locality nevertheless: the accesses alternate between two buffers that may be far apart, but each of them is adjacent to the previous access to the same buffer. Spatial locality only requires accesses near *recently* accessed addresses, not near the one accessed last.

**Caution** Spatial locality is about distance, not regularity: a stream of accesses with a large constant stride is perfectly regular and has none.

## 12.2 Caches

Locality suggests how to give the processor fast access to a large memory. Going to main memory for every fetch, load and store is slow. Making all of main memory as fast as the processor is not possible either: fast memory is expensive per bit, and a larger memory is inherently slower to access. The remaining option is to keep a small amount of fast memory inside the processor itself and to place in it the data that was accessed recently, together with the data around it. By temporal and spatial locality, most accesses then find their data there.

**Source** Lesson 12, videos 7–9; H&P §B.1; [25].

**Definition 12.4** (Cache). A *cache* is a small, fast memory that holds copies of the recently accessed parts of a larger and slower memory, so that most accesses can be served from the copies.

Every memory access of the processor first looks for its data in the cache (Figure 12.1). If the data is there, the access is a *cache hit*, and the data is read or written in the cache.

Otherwise the access is a *cache miss*: the data is obtained from main memory and copied into the cache, so that further accesses to it, and by spatial locality to its neighbours, will hit. Because the cache is consulted on every access, the lookup itself must be very fast, which is a second reason, besides cost, why the cache has to be small. A small cache cannot hold everything, and misses are therefore unavoidable.[1]

[1] The idea is Wilkes's "slave memory" of 1965 [25]. The first commercial machine to have one was the IBM System/360 Model 85 of 1968, whose 16–32 KB buffer was named the *cache* in the paper describing it [26].

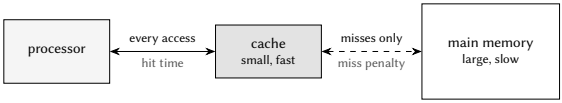



**Figure 12.1:** Every memory access goes to the cache; only misses go on to main memory.

## 12.3 Cache performance

**Source** Lesson 12, videos 10–12; H&P §B.2.

The benefit of a cache is measured by the average time a memory access takes. The following quantities enter it.

**Definition 12.5** (Hit rate, miss rate). The *hit rate* is the fraction of memory accesses that are cache hits; the *miss rate* is the fraction that are misses. Hit rate + miss rate = 1.

**Definition 12.6** (Hit time, miss penalty). The *hit time* is the time needed to look up the cache and access the data in it. The *miss penalty* is the *additional* time a miss takes: fetching the data from main memory and placing it in the cache.

A miss also pays the hit time, since the cache has to be searched before the miss is detected; a miss therefore takes the hit time plus the miss penalty. Weighting both cases by their frequency gives the *average memory access time* (AMAT):

$$\begin{aligned}\text{AMAT} &= (1 - \text{miss rate}) \times \text{hit time} + \text{miss rate} \times (\text{hit time} + \text{miss penalty}) \\ &= \text{hit time} + \text{miss rate} \times \text{miss penalty}.\end{aligned} \tag{12.1}$$

**Caution** The miss penalty is the *extra* time of a miss: subtract the hit time from the total time of a miss.

A good AMAT requires three things.

- A low hit time: the cache must be small and simple.
- A low miss rate: the cache must be large enough, and organized cleverly enough, to hold the data the program is about to use.
- A low miss penalty. The miss penalty is dominated by the access time of main memory and is large; Lesson 14 reduces it with further levels of cache.

The first two goals conflict: a larger cache misses less often but takes longer to search. The best cache size balances them, and AMAT is the measure by which they are balanced.

At 3 GHz this penalty is 180 cycles; a 4-wide processor (Lesson 6) loses some 700 instruction slots while it waits.

**Example 12.7.** Main memory adds a miss penalty of 60 ns. Three candidate caches are compared:

| Cache | Hit time | Miss rate | AMAT |
|---|---|---|---|
| Small | 1 ns | 6 % | $1 + 0.06 \times 60 = 4.6$ ns |
| Medium | 1.5 ns | 3 % | $1.5 + 0.03 \times 60 = 3.3$ ns |
| Large | 3 ns | 2 % | $3 + 0.02 \times 60 = 4.2$ ns |

The medium cache is best. The large cache misses less often, but its longer hit time costs more than the misses it saves. All three are an order of magnitude better than accessing main memory on every access.

**Note 12.8.** A cache pays off only if a hit is much faster than an access to main memory and misses are rare. With a hit time close to the access time of main memory, even a hit is about as slow as an access without a cache; with frequent misses, the miss penalty dominates. In both cases AMAT approaches or even exceeds the access time of main memory alone.

### 12.3.1 Caches in real processors

Real processors have more than one level of cache. The cache that is consulted first, and that directly serves the reads and writes of the processor, is the smallest and fastest; only its misses go on to further, larger cache levels (L2, L3; Lesson 14) and finally to main memory. This first level is called the *L1 cache*. L1 caches are typically 16–64 KB (1 KB = $2^{10}$ bytes): large enough that about 90 % of accesses hit, and small enough that a hit takes only one to three clock cycles.

**Source** Lesson 12, video 13; H&P §2.1.

Sizes in recent processors: Skylake has a 32 KB L1 per core, a 256 KB L2 and an 8 MB shared L3; Zen 3 has 32 KB, 512 KB and 32 MB (7-cpu.com).

## 12.4 Blocks and lines

To determine quickly whether data is in the cache, the cache is organized as a table of fixed-size entries. Data is moved between main memory and the cache not byte by byte but in units called blocks.

**Source** Lesson 12, videos 14–18; H&P §B.1.

**Definition 12.9** (Cache block). A *cache block* is a run of consecutive bytes of memory that is copied into the cache, and evicted from it, as a unit. The number of bytes in a block is the *block size*.

The block size trades spatial locality against cache space.

- If blocks are very small (in the extreme, one byte), an access to the next byte, which spatial locality makes likely, misses again, and each byte needs its own bookkeeping.
- If blocks are very large, most of a block is data that will not be used soon. It occupies space in which other recently used data could have been kept, and fewer blocks fit into the cache.

Typical block sizes are 32–128 bytes. The number of blocks a cache can hold is its size divided by the block size: a 16 KB cache with 64-byte blocks holds $2^{14}/2^6 = 256$ blocks.

Real block sizes: 64 bytes on current x86-64 processors and most Arm cores, 128 bytes on Apple's M1 and M2 (Intel and Arm optimization manuals; Apple `hw.cachelinesize`).

### 12.4.1 Where blocks start

Blocks do not start at arbitrary addresses. If they did, a block covering addresses 100–227 and another covering 200–327 could both be in the cache, an address could lie in several cached blocks, and finding out whether an address is cached would require comparing address ranges. Instead, memory is divided into fixed, non-overlapping blocks that start at multiples of the block size: with 128-byte blocks, at addresses 0, 128, 256, … (Figure 12.2). Every address then belongs to exactly one block.

**Definition 12.10** (Cache line). A *cache line* is a slot in the cache that can hold one block. Memory is divided into blocks and the cache into lines; a line has the size of a block, and holds different blocks at different times.

Block sizes, and therefore line sizes, are always powers of two. Only then can the block of an address and the position within the block be found by selecting bits of the address, as the next section shows, rather than by a division.

**Caution** This chapter keeps *block* (in memory) and *line* (its slot in the cache) apart; H&P uses block and line as synonyms, and vendor manuals say cache line for both.

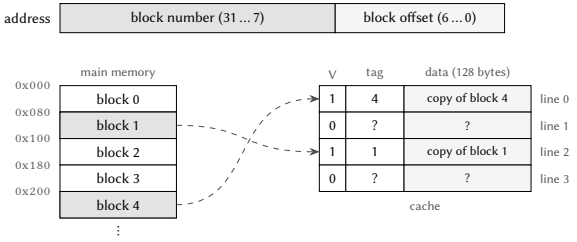



**Figure 12.2:** Main memory divided into 128-byte blocks and a cache of four lines. Lines 0 and 2 hold copies of blocks 4 and 1; lines 1 and 3 hold nothing valid.

## 12.5 Block number, tag and valid bit

With a block size of $2^k$ bytes, an address $A$ splits into two fields.

**Source** Lesson 12, videos 19–23; H&P §B.1.

**Definition 12.11** (Block offset and block number). The low-order $k$ bits of the address are its *block offset*, the position of the byte within its block. The remaining high-order bits are its *block number*, which identifies the block:

$$\text{block number} = \lfloor A/2^k \rfloor, \qquad \text{block offset} = A \bmod 2^k. \tag{12.2}$$

**Example 12.12.** With 128-byte blocks ($k = 7$), the address `0x0040A7C6` has block offset `0xC6 & 0x7F = 0x46 = 70` and block number `0x0040A7C6 >> 7 = 0x814F`. The byte is at position 70 of block `0x814F`, which covers the addresses `0x0040A780` to `0x0040A7FF`.

### 12.5.1 Tags

Since a line holds different blocks at different times, each line must record which block it currently holds. This record is the *tag* of the line: the block number of the block in the line, or, as the following sections show, the part of the block number that is not already implied by the position of the line. A lookup computes the block number of the address and compares it with the tags of the lines in which the block could be. A match is a hit, and the block offset then selects the byte within the line.

### 12.5.2 The valid bit

When the processor is turned on, the lines of the cache contain arbitrary bits, including arbitrary tags. A tag that happens to match a block number would produce a hit on data that was never loaded.

**Definition 12.13** (Valid bit). The *valid bit* of a line indicates whether the line holds a block. All valid bits are cleared at start-up, and a line's valid bit is set when a block is loaded into it.

Emptying a line, or the whole cache, thus only requires clearing valid bits; the tags and data can be left as they are. The one exception is a block that has been written in the cache but not yet in main memory (Section 12.11): it must be written back first.

x86 has both operations: `WBINVD` writes dirty blocks back before clearing the valid bits, `INVD` clears them and discards the data.

A lookup is a hit only if the line is valid and its stored tag matches the tag field of the address:

$$\text{hit} \;=\; V \wedge (\text{stored tag} = \text{tag of the address}). \tag{12.3}$$

As long as no part of the block number selects the line, as in this section, the tag field is the whole block number; the organizations of the following sections use some of its bits to select the line and keep only the remaining bits in the tag.

> **Key point**
> An address is a block number and an offset. The tag identifies which block a line holds, the valid bit whether it holds one at all, and the offset picks the byte. Where a block may be placed, the subject of the next section, decides *which* lines have to be searched.

## 12.6 Where a block can be placed

**Source** Lesson 12, video 24; H&P §B.1.

Caches differ in how many of their lines a given block may be placed in. This number is the *associativity* of the cache. It determines how many tags a lookup must compare, and how often different blocks compete for the same lines. Three organizations are distinguished (Figure 12.3):
**Direct-mapped** each block can be placed in exactly one line (Section 12.7).
**Set-associative** each block can be placed in any of $N$ lines that form a set (Section 12.8).
**Fully associative** each block can be placed in any line of the cache (Section 12.9).

Typical associativities: Skylake L1 8-way, L2 4-way, L3 16-way; Zen 3 L1 8-way, L2 8-way, L3 16-way (7-cpu.com).

## 12.7 Direct-mapped caches

**Source** Lesson 12, videos 25–28; H&P §B.1.

The simplest of the three organizations gives every block a single line of the cache. In a *direct-mapped cache* every block has exactly one line in which it can be placed, and that line is selected by the low-order bits of the block number.

**Definition 12.14** (Index). The *index* is the group of address bits that selects where in the cache a block may be placed. In a direct-mapped cache with $2^n$ lines it consists of the $n$ low-order bits of the block number.

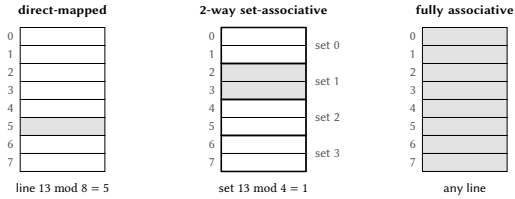



**Figure 12.3:** The lines in which block number 13 may be placed in an 8-line cache of each organization.

The address is thus split into three fields (Figure 12.4): the offset in the $k$ lowest bits, above it the index in the next $n$ bits, and the tag in all remaining bits. The line stores only these tag bits, not the whole block number: all blocks that can be placed in line $i$ have index $i$, so the index part of their block numbers is known from the line itself.

A lookup reads the one line selected by the index, compares its tag with the tag of the address, and checks the valid bit (Equation (12.3)).

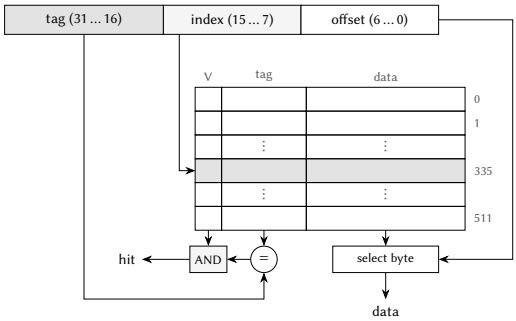



**Figure 12.4:** Lookup in a direct-mapped 64 KB cache with 128-byte blocks. The index selects one of 512 lines; a single comparator checks its tag.

**Example 12.15.** For a direct-mapped 64 KB cache with 128-byte blocks and 32-bit addresses there are $512 = 2^9$ lines: offset bits 6–0, index bits 15–7, tag bits 31–16. The address `0x0040A7C6` of Example 12.12, block number `0x814F`, has index $\texttt{0x814F} \,\&\, \texttt{0x1FF} = \texttt{0x14F} = 335$ and tag $\texttt{0x814F} \gg 9 = \texttt{0x0040}$. It hits if line 335 is valid and holds tag `0x0040`.

### 12.7.1 Advantages and disadvantages

The strength of a direct-mapped cache is that only one line has to be examined. A single comparator suffices, so the lookup is fast, cheap and energy-efficient: the hit time is as low as it can be for a cache of a given size.

Its weakness is that blocks with the same index compete for a single line. If a program alternates between two such blocks, each access evicts the block the other access needs, and every access misses, even when the rest of the cache is empty. These conflicts raise the miss rate.

Misses caused by this competition are *conflict misses*, one of the three causes of misses classified in Lesson 14.

**Example 12.16.** An 8-line direct-mapped cache with 16-byte blocks uses offset bits 3–0, index bits 6–4 and tag bits 31–7. A program alternately accesses addresses `0x1000` (block `0x100`) and `0x1084` (block `0x108`). Both blocks have index 0, with tags `0x20` and `0x21`. Each access replaces the block loaded by the previous one, so every access is a miss, however long the program continues.

## 12.8 Set-associative caches

A set-associative organization relaxes the one-line restriction.

**Source** Lesson 12, videos 29–31; H&P §B.1.

**Definition 12.17** (Set-associative cache). A *set-associative cache* divides its lines into *sets* of $N$ lines each; a block can be placed in any line of one particular set. Such a cache is called $N$-way set-associative.

The index now selects a set rather than a line. A cache with $2^s$ sets has an $s$-bit index, and the tag again consists of all bits above the index. A lookup compares the tag of the address with the tags of all $N$ lines of the selected set, in parallel, and hits if any valid line matches (Figure 12.5); the matching line supplies the data.

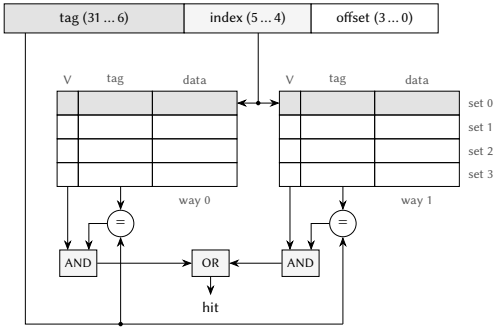



**Figure 12.5:** Lookup in a 2-way set-associative cache with four sets, the organization of Example 12.18. The index (here 0, as in that example) selects a set; each of its two lines hits if it is valid and its tag matches.

**Example 12.18.** Organize the 8 lines of Example 12.16 as a 2-way set-associative cache. It has 4 sets, so the index is bits 5–4 and the tag bits 31–6. Block `0x100` goes to set 0 with tag `0x40`, and block `0x108` to set 0 with tag `0x42`. The two blocks still share a set, but the set has two lines: after the first access to each block has missed and loaded it, every further access hits.

Why 32 KB 8-way L1 caches are common: 64 sets of 64-byte blocks span 4 KB, so index and offset fit in the page offset and the lookup can start before translation (Lesson 14).

Compared with a direct-mapped cache of the same size, an $N$-way set-associative cache needs $N$ comparators and a selection of the matching line's data, which makes the hit time longer, but it suffers fewer conflicts and so has a lower miss rate.

## 12.9 Fully associative caches

**Source** Lesson 12, videos 32–33; H&P §B.1.

At the other extreme, a block may be placed anywhere.

**Definition 12.19** (Fully associative cache). In a *fully associative cache* any block can be placed in any line.

There is nothing to select, so there is no index: the tag is the entire block number, and a lookup compares it with the tag of every line in the cache. Two blocks never have to compete for a line while another line is free, but a comparator is needed for every line, which makes a large fully associative cache slow and expensive.

Fully associative structures stay small: many L1 TLBs of 32–64 entries (Lesson 13) are fully associative; caches are not.

### 12.9.1 One scheme for all three

Direct-mapped and fully associative caches are the two extremes of set-associative caches. A direct-mapped cache is 1-way set-associative, with as many sets as lines; a fully associative cache has a single set containing all lines. For every organization, with addresses of $a$ bits,

$$\begin{aligned} \text{offset bits} &= \log_2(\text{block size}),\\ \text{index bits} &= \log_2(\text{number of sets}) = \log_2\big(\text{lines}/N\big),\\ \text{tag bits} &= a - \text{index bits} - \text{offset bits}, \end{aligned} \tag{12.4}$$

where $N = 1$ for a direct-mapped cache (index bits $= \log_2(\text{lines})$) and $N =$ lines for a fully associative cache (0 index bits). For the 64 KB cache with 128-byte blocks of Example 12.15, the tag grows from 16 bits when direct-mapped to 25 bits when fully associative; Table 12.4 in Section 12.12 lists the intermediate cases.

**Tip** A cache of $C$ bytes has $a - \log_2(C/N)$ tag bits. Each doubling of $N$ moves one bit from the index to the tag.

**Key point**
Associativity trades hit time against miss rate. More ways per set means more tag comparisons on every access, but fewer conflicts, because a block is evicted only when every line of its set is occupied by another block.

## 12.10 Cache replacement

**Source** Lesson 12, videos 34–36; H&P §B.1.

On a miss, the new block must be placed in one of the lines allowed for it. If one of those lines is not valid, that line is used. If all of them hold valid blocks, one block has to be evicted. A direct-mapped cache has no choice; in set-associative and fully associative caches the victim is chosen by the *replacement policy*.

**Random** evict a randomly chosen block of the set. Simple, but it ignores how the blocks are being used.

**FIFO** (first in, first out) evict the block that has been in the cache longest, regardless of whether it is still in use.

**LRU** evict the block whose most recent access lies furthest in the past.

**Definition 12.20** (Least recently used). The *least recently used* (LRU) policy evicts, among the blocks that may be replaced, the one that has not been accessed for the longest time.

LRU exploits temporal locality: a block accessed recently is likely to be accessed again soon, so the least recently used block is the one least likely to be needed.

### 12.10.1 Implementing LRU with counters

LRU can be implemented with a counter per line. In an $N$-way set-associative cache each line has a $\log_2 N$-bit counter. At start-up the counters of each set are initialized to the values $0, 1, \ldots, N-1$, in any order, and the update rule below keeps it that way: every value occurs exactly once in each set. The line with counter 0 is the least recently used and the one with counter $N-1$ the most recently used. On every access to a line whose counter is $c$:

1. every counter in the set that is greater than $c$ is decremented;
2. the counter of the accessed line is set to $N-1$.

On a miss, the victim is the line with counter 0; once the new block is in it, the access is treated like any other and the counters are updated as above. Table 12.1 traces one 4-way set.

**Tip** The counters of a set stay a permutation of $0, \ldots, N-1$: a value that appears twice in a row of a trace means an update was missed.

**Caution** Update the counters on *every* access, hits included; updating them only when a block is brought into the cache gives FIFO.

**Table 12.1:** LRU counters of one 4-way set holding blocks A–D, shown as block:counter for each of its four lines.

| Access | Result | Line 0 | Line 1 | Line 2 | Line 3 |
|---|---|---|---|---|---|
| (initial) | | A:0 | B:1 | C:2 | D:3 |
| C | hit | A:0 | B:1 | C:3 | D:2 |
| A | hit | A:3 | B:0 | C:2 | D:1 |
| E | miss, B evicted | A:2 | E:3 | C:1 | D:0 |
| D | hit | A:1 | E:2 | C:0 | D:3 |
| B | miss, C evicted | A:0 | E:1 | B:3 | D:2 |

The cost of this implementation is $\log_2 N$ bits per line and, more importantly, activity on every access: even a hit may change all $N$ counters of the set. For highly associative caches this costs considerable hardware and energy, which motivates the cheaper approximations of LRU in Lesson 14.

## 12.11 Write policy

**Source** Lesson 12, videos 37–40; H&P §B.1.

Writes raise two questions that reads do not.

1. *Is a block brought into the cache on a write miss?* A cache that brings it in follows the *write-allocate policy*; a no-write-allocate cache writes the value to main memory only. Most caches are write-allocate: by locality, a block that has just been written is likely to be read or written again soon.
2. *When does main memory receive the written value?* Here two policies exist.

**Definition 12.21** (Write-through and write-back). Under the *write-through policy* a write updates the block in the cache and immediately also the block in main memory.

Under the *write-back policy* a write updates only the block in the cache; main memory is updated when the block is evicted from the cache.

**Caution** The two choices are independent: write-through or write-back says *when* memory is updated, write-allocate or not *what a write miss does.*

Write-through keeps main memory always up to date, but every write then pays the access time of main memory, which is what the cache was meant to avoid. Caches therefore use write-back: many writes to a block while it is in the cache cost at most a single write to main memory, at eviction. A write-back cache is normally also write-allocate; otherwise every write to a block that is not in the cache would go to main memory, however often the block is written.

Writing every evicted block back would, however, also write blocks that were only read and are identical to their copies in memory.

**Definition 12.22** (Dirty bit). The *dirty bit* of a line indicates whether its block has been written since it was loaded. It is cleared when a block is loaded and set when the block is written. On eviction, a dirty block is written to main memory; a clean block is simply discarded.

**Example 12.23.** Blocks X and Y map to the same line of a direct-mapped cache, which is initially invalid. Table 12.2 follows a sequence of accesses under write-back with write-allocate. The two consecutive writes to Y cost a single memory write, at step 6, and evicting Y at step 8 costs none, because Y was not written after it was loaded at step 7.[2] Write-back needs 3 memory writes in total; write-through would need one per write, i.e. 4.

[2] Write-through is not extinct: the Pentium 4 and AMD's Bulldozer wrote their L1 data caches through to the next level and absorbed the writes in a buffer. Such an L1 needs no dirty bits and never writes back; both families later returned to write-back L1 caches (Intel and AMD optimization manuals).

**Table 12.2:** A single line under the write-back policy (Example 12.23). D is the dirty bit after the access.

| Step | Access | Result | Memory operations | Line after |
|---|---|---|---|---|
| 1 | read X | miss | read X | X, D=0 |
| 2 | write X | hit | — | X, D=1 |
| 3 | read Y | miss | write X back, read Y | Y, D=0 |
| 4 | write Y | hit | — | Y, D=1 |
| 5 | write Y | hit | — | Y, D=1 |
| 6 | write X | miss | write Y back, read X | X, D=1 |
| 7 | read Y | miss | write X back, read Y | Y, D=0 |
| 8 | read X | miss | read X | X, D=0 |

## 12.12 Putting it together

**Source** Lesson 12, videos 41–45; H&P §B.1.

A cache is fully described by a few design choices, from which its structure follows.
**Cache size and block size** give the number of lines and the offset bits.
**Associativity** gives the number of sets, hence the index and tag bits (Equation (12.4)).
**Replacement policy** LRU adds a $\log_2 N$-bit counter to every line (none for direct-mapped caches).
**Write policy** write-back adds a dirty bit to every line.
Every line thus stores a valid bit, a dirty bit, an LRU counter and a tag in addition to its data. Figure 12.6 shows how these are used on one access. A hit reads or writes the data and updates the LRU counters. A miss first chooses a victim, writes it back if it is valid and dirty, and loads the new block; the access then proceeds as a hit.

**Example 12.24.** Consider the 2-way set-associative cache of Example 12.18 (16-byte blocks, index bits 5–4, tag bits 31–6), now with LRU replacement and the write-back policy. Table 12.3 follows its set 1 through five accesses; both lines of the set are valid throughout. The first access hits and only updates the counters. The second misses, and its victim is way 1, whose counter is 0; that block is dirty and is written back before the new block is loaded. The fourth access is a write miss whose victim is clean, so nothing is written back, and the loaded block becomes dirty at once. The last access evicts the block with tag `0x40`, which the third access has written, and writes it back.

**Tip** Memory traffic under write-back: one read per miss plus one write per eviction of a dirty block; hits cost nothing.

**Example 12.25.** Table 12.4 works out the organization of a 64 KB cache with 128-byte blocks, 32-bit addresses, LRU replacement and write-back, for several associativities. For the 8-way cache, the address `0x0040A7C6` (block number `0x814F`) has index $\texttt{0x814F} \,\&\, \texttt{0x3F} = 15$ and tag $\texttt{0x814F} \gg 6 = \texttt{0x205}$. Its 512 lines need $512 \times 24 = 12\,288$ bits of metadata, i.e. 1536 bytes, about 2.3 % of the 64 KB of data.

Metadata is a reason not to shrink blocks: with 16-byte blocks the same 8-way cache would have 4096 lines of 24 bits, 12 KB or 19 % of its data.

Lesson 13 applies the same ideas—blocks, tags and replacement—to the translation of virtual

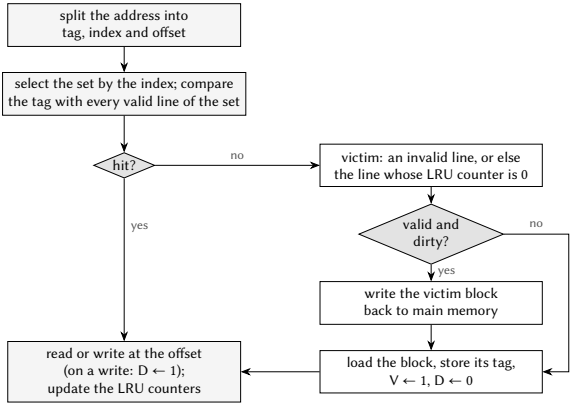



**Figure 12.6:** One memory access in a set-associative cache with LRU replacement and the write-back policy.

**Table 12.3:** Set 1 of a 2-way set-associative cache with LRU replacement and write-back (Example 12.24): tag, dirty bit and LRU counter of each way after each access. All tags, including those of the memory operations, are hexadecimal.

| | | Way 0 | | | Way 1 | | | |
|---|---|---|---|---|---|---|---|---|
| Access | Result | Tag | D | LRU | Tag | D | LRU | Memory operations |
| (initial) | | 40 | 0 | 0 | 43 | 1 | 1 | |
| read 0x1018 | hit | 40 | 0 | 1 | 43 | 1 | 0 | — |
| read 0x1054 | miss | 40 | 0 | 0 | 41 | 0 | 1 | write back 43, load 41 |
| write 0x101C | hit | 40 | 1 | 1 | 41 | 0 | 0 | — |
| write 0x1098 | miss | 40 | 1 | 0 | 42 | 1 | 1 | load 42 |
| read 0x1058 | miss | 41 | 0 | 1 | 42 | 1 | 0 | write back 40, load 41 |

addresses, and Lesson 14 turns to techniques that lower the hit time, the miss rate and the miss penalty of the basic cache described here.

**Table 12.4:** Address fields and metadata bits per line (valid + dirty + LRU counter + tag) of a 64 KB cache with 128-byte blocks and 32-bit addresses.

| Organization | Sets | Offset | Index | Tag | Metadata per line |
|---|---|---|---|---|---|
| Direct-mapped | 512 | 7 | 9 | 16 | $1 + 1 + 0 + 16 = 18$ |
| 2-way | 256 | 7 | 8 | 17 | $1 + 1 + 1 + 17 = 20$ |
| 8-way | 64 | 7 | 6 | 19 | $1 + 1 + 3 + 19 = 24$ |
| Fully associative | 1 | 7 | 0 | 25 | $1 + 1 + 9 + 25 = 36$ |

**Key point — Lesson summary**

Programs access memory with temporal and spatial locality, so a small, fast cache of recently used blocks serves most accesses. Its benefit is measured by AMAT = hit time + miss rate × miss penalty (Equation (12.1)). Memory is divided into power-of-two blocks placed into cache lines; an address splits into tag, index and offset, and a line hits when it is valid and its tag matches. Direct-mapped, set-associative and fully associative caches differ in how many lines a block may occupy, trading hit time against miss rate. On a miss LRU evicts the least recently used block, and the write-back policy with a dirty bit writes a block to memory only when a modified block is evicted.

# Virtual Memory

| | |
|---|---|
| **Lecture** | Lesson 13, videos 1–27 |
| **Textbook** | H&P [1] §B.4–B.5 |
| **Keywords** | virtual memory, physical memory, page, page frame, page table, page fault, virtual page number, page offset, multi-level page table, page size, internal fragmentation, TLB, TLB miss, page table walk |

Lesson 12 treated every address produced by the processor as a location in main memory. In a real system the addresses that programs use are *virtual:* every program sees a large address space of its own, and the operating system and the processor together map the parts of it that the program actually uses onto the memory that is installed. This lesson explains why virtual memory is needed, how page tables translate virtual addresses into physical ones, how large these tables are and how multi-level page tables keep them small, and why translation would slow down every memory access if the processor did not cache recent translations in a translation lookaside buffer, which Lesson 14 combines with the caches of Lesson 12.

## 13.1 Why virtual memory

**Source** Lesson 13, videos 1–3; H&P §B.4; [27].

Hardware and software see memory differently. To the hardware, memory is what is installed in the machine: an array of as many bytes as the memory modules provide. To a program, memory is an address space of $2^{32}$ bytes on a 32-bit processor or $2^{64}$ bytes on a 64-bit processor, and all of it belongs to that program: code, data and stack are placed at whatever addresses the program and its compiler choose, regardless of how much memory is installed and of which other programs are running at the same time.

The two views disagree in two respects.

- The address space of a single program is much larger than the installed memory. A 32-bit address space is 4 GB, which a machine may or may not have; a 64-bit address space is 16 EB (1 EB = $2^{60}$ bytes), many orders of magnitude more than any machine has.
- Many programs run at the same time, and each of them uses the same addresses, as if it owned the entire address space.

**Caution** No processor translates all 64 bits. x86-64 requires *canonical* addresses, in which bits 63–48 repeat bit 47, so the usable space is 256 TB rather than 16 EB.

**Definition 13.1** (Virtual memory). The mechanism that gives every program the view of a large memory of its own, and maps the parts of that memory the program actually uses onto the memory installed in the machine, is called *virtual memory*.

The operating system decides where each part of each program is kept, and the processor applies these decisions to every address on every memory access. Programs can thus be written without knowing the size of memory or the other programs they will run with, a program cannot access the memory of another unless the operating system arranges it, and together the programs can use more memory than is installed, the excess being kept on disk (Section 13.4).[1]

[1] The idea is that old: the Manchester Atlas of 1962 presented its 16K words of core store and 96K words of magnetic drum as one address space of 512-word pages, and translated addresses through 32 associative *page address registers* — the ancestor of the TLB of Section 13.10.
**Source** Lesson 13, videos 4–5; H&P §B.4.

## 13.2 Physical memory and virtual address spaces

The two views correspond to two kinds of addresses.

**Definition 13.2** (Physical memory). The memory actually installed in the machine is its *physical memory*. Its bytes are numbered consecutively, from 0 up to the size of physical memory minus one, and the number that identifies a byte is its *physical address*.

The size of physical memory is determined by the installed modules, not by the width of the processor's addresses. A 32-bit machine can address 4 GB but may have only 2 GB installed; a 64-bit machine has perhaps tens of gigabytes, a tiny fraction of the $2^{64}$ bytes it could address. In the examples of this lesson a physical address has just as many bits as the installed memory requires: 31 bits for 2 GB.

**Definition 13.3** (Virtual address space). The *virtual address space* of a program is the full range of addresses the program can use, from 0 to $2^n - 1$ on an $n$-bit processor, independently of physical memory. An address in it is a *virtual address*.

All addresses a program works with are virtual: the program counter, the pointers stored in its data, and the addresses the processor computes for loads and stores. A program uses only a few regions of its virtual address space (Figure 13.1, left and right):

- its *code* and its global *data* at low addresses;
- above them the *heap*, which grows toward higher addresses as the program allocates memory dynamically;
- the *stack* at the top of the address space, which grows toward lower addresses.

Between the heap and the stack lies a huge range of addresses that the program never uses. Every running program has its own virtual address space laid out in this way, so different programs use the same virtual addresses for entirely different contents.

## 13.3 Pages and page tables

A mapping recorded separately for every virtual address would need as many table entries as there are addresses. Virtual memory therefore maps fixed-size ranges of addresses, just as a cache moves blocks rather than individual bytes.

**Source** Lesson 13, videos 6–7; H&P §B.4.

**Definition 13.4** (Page, page frame). The virtual address space is divided into consecutive ranges of equal size, each called a *page*. Physical memory is divided into ranges of the same size, each called a *page frame*, or frame for short. The size is a power of two, typically 4 KB, and any page fits exactly into any frame.

**Definition 13.5** (Page table). The *page table* of a program records, for every page of its virtual address space, whether the page is in physical memory and, if it is, which frame holds it.

The operating system maintains one page table for every running program and updates it whenever it places one of the program's pages into a frame or removes one. The page table of the running program is found through a register set by the operating system and is read to translate the program's addresses, usually by the processor itself and in some systems by the operating system (Section 13.10). Figure 13.1 illustrates the resulting mapping for two programs.

That register is CR3 on x86-64 and TTBR0/TTBR1 on AArch64; a context switch reloads it, and the whole mapping changes at once.

- Any page may be placed in any frame. Pages that are adjacent in a virtual address space need not be adjacent in physical memory, and the frames of different programs are interleaved.
- Pages that a program never uses are not mapped and occupy no frame; only the used regions of the address space consume physical memory.
- The page tables of two programs may map pages to the same frame. This lets programs share memory, for example a single copy of the code of a program that is running twice.
- Not every mapped page has to be in physical memory; the others are on disk (Section 13.4).

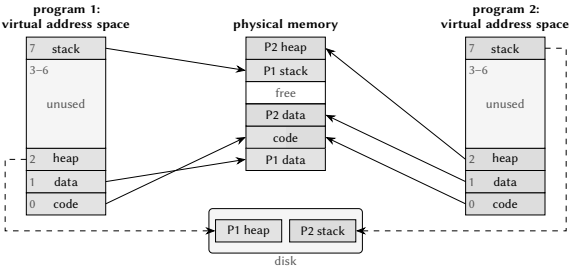



**Figure 13.1:** The virtual address spaces of two programs (pages 0–7, low addresses at the bottom) mapped onto six frames of physical memory. Page 0 of both programs is mapped to the same frame; each program has one page on disk (dashed); the unused pages 3–6 are not mapped.

A page table describes *every* page of the virtual address space, used or not; in its simplest form (Section 13.6) it has an entry for each. The number of pages it describes is therefore set by the size of the virtual address space and the size of a page; the size of physical memory only determines how many frames an entry can refer to.

**Caution** A page table is indexed by virtual page, not by frame. With less physical memory there are fewer frames, but the page table must still describe just as many pages.

## 13.4 Page faults

**Source** Lesson 13, video 8; H&P §B.4.

Together, the running programs usually map more pages than physical memory has frames. The pages that do not fit are kept on disk, and physical memory holds the pages that have been used recently.

**Definition 13.6** (Page fault). An access to a page that the page table does not map to a frame causes a *page fault*: an exception that transfers control to the operating system.

If the page is on disk, the operating system handles the page fault in three steps.

1. It chooses a frame for the page. If no frame is free, it evicts a page that has not been used recently; if that page has been written since it was loaded, it is first written back to disk. The page table of the evicted page is updated to show that the page is no longer in memory.
2. It reads the faulting page from disk into the frame and updates the page table to map the page to this frame.
3. It restarts the instruction that caused the fault, which now finds its page in memory; this requires precise exceptions (Lesson 8).

At 3 GHz a 10 ms disk seek is 30 million cycles; even an NVMe SSD read of 100 µs is 300 000.

A disk access takes milliseconds, i.e. millions of clock cycles, so a page fault is enormously more expensive than a cache miss. Page faults must be rare, and it pays to let the operating system choose the page to evict with a careful algorithm in software.

A third case touches no disk: Linux counts a fault that only has to allocate a zeroed frame or copy a shared page as a *minor* fault, against *major* faults that read from storage.

Not every page fault involves the disk: if the address lies outside every region that the program has been allocated, there is nothing to load, and the operating system treats the access as an error, usually by terminating the program.

**Note 13.7.** Physical memory acts as a cache of the pages on disk. Pages correspond to the blocks of Lesson 12, frames to lines, page faults to cache misses, and writing only modified pages back to disk to a write-back cache with a dirty bit. Placement is fully associative, since any page may be placed in any frame, yet no tags have to be searched, because the page table is indexed directly by the page.

## 13.5 Address translation

**Source** Lesson 13, videos 9–10; H&P §B.4.

Because pages start at multiples of the page size, a virtual address splits into two fields, just like the block number and block offset of a cache address.

**Definition 13.8** (Virtual page number, page offset). With pages of $2^k$ bytes, the low-order $k$ bits of a virtual address form its *page offset*, the position of the byte within its page. The remaining high-order bits form its *virtual page number* (VPN), which identifies the page.

Frames are numbered in the same way: the frame that starts at physical address $j \times 2^k$ has the *physical frame number* (PFN) $j$. The page table maps the VPN to the PFN, and translation replaces the one by the other while keeping the offset (Figure 13.2):

$$\begin{aligned} \text{VPN} &= \left\lfloor VA/2^k \right\rfloor, & \text{offset} &= VA \bmod 2^k, \\ \text{PFN} &= \text{page table[VPN]}, & PA &= \text{PFN} \times 2^k + \text{offset}, \end{aligned} \tag{13.1}$$

where $VA$ and $PA$ are the virtual and physical addresses. The offset passes through unchanged because a page and its frame have the same size and the byte keeps its position within them; in binary, the physical address is the PFN bits followed by the offset bits. The VPN has $k$ bits fewer than a virtual address and the PFN $k$ bits fewer than a physical address, so the two numbers differ in width whenever virtual and physical addresses do. If the page table holds no PFN for the page, the access causes a page fault instead.

**Caution** Translation never moves a byte out of its page, but an unaligned access can straddle a page boundary: its two halves lie in different pages and need two translations, into frames that may be far apart.

**Example 13.9.** A 32-bit processor with 4 KB pages ($k = 12$) has 1 GB of physical memory, hence 30-bit physical addresses. The page offset is bits 11–0, the VPN bits 31–12 (20 bits), and a PFN has 18 bits. The virtual address `0x0040A7C6` has VPN `0x0040A` and offset `0x7C6`. The page table of Figure 13.2 maps page `0x0040A` to frame `0x1F3C5`, so the physical address is $\texttt{0x1F3C5} \times 2^{12} + \texttt{0x7C6} = \texttt{0x1F3C57C6}$. The virtual address `0x000017FF` lies in page `0x00001`, which the same page table marks as not in memory: an access to it causes a page fault.

**Tip** With 4 KB pages and addresses written in hexadecimal, the last three digits are the offset; translation replaces the digits in front of them.

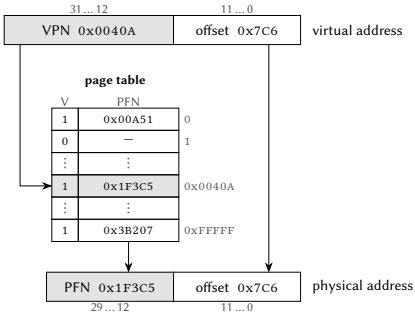



**Figure 13.2:** Translation of the virtual address 0x0040A7C6 with 4 KB pages and 30-bit physical addresses (Example 13.9). The VPN selects a page table entry; its PFN replaces the VPN, and the offset is copied. An entry with V = 0 would cause a page fault.

> **Key point**
> Translation replaces the virtual page number by a physical frame number and keeps the page offset. The page table, maintained by the operating system, supplies the frame number; if it holds none for the page, the access causes a page fault.

## 13.6 The size of a flat page table

The simplest page table is a single array.

**Source** Lesson 13, videos 11–12; H&P §B.4.

**Definition 13.10** (Flat page table). A *flat page table* is an array with one entry for every virtual page, indexed directly by the VPN; its element for a page is the *page table entry* (PTE) of that page.

A PTE holds the PFN of its page and a few status bits: a valid bit that is set only if the page is in physical memory, and bits that record how the page may be accessed (read, write, execute). A PTE is typically about as large as an address, 4 bytes on a 32-bit processor and 8 bytes on a 64-bit processor, which leaves room for the PFN and the status bits. Since every virtual page has an entry,

An x86-64 PTE is 8 bytes: the PFN plus a present bit, read/write, user/supervisor, accessed and dirty bits, and a no-execute bit at the top.

$$\text{size of a flat page table} \;=\; \frac{\text{size of the virtual address space}}{\text{page size}} \times \text{PTE size}. \qquad (13.2)$$

Two quantities are absent from Equation (13.2). The size of physical memory only affects how many bits of an entry the PFN occupies. The amount of memory the program actually uses does not matter at all, because unused pages have entries too. Every running program needs a page table of this size.

**Tip** Work in exponents: with an $n$-bit address space, $2^k$-byte pages and $2^s$-byte entries the table is $2^{n-k+s}$ bytes — subtract the page bits, add the entry bits.

**Example 13.11.** With 32-bit virtual addresses, 4 KB pages and 4-byte entries, a flat page table has $2^{32}/2^{12} = 2^{20}$ entries and occupies $2^{20} \times 4$ bytes $= 4$ MB for each running program, even for a program that uses only a few kilobytes. With 64-bit virtual addresses, 4 KB pages and 8-byte entries it would have $2^{52}$ entries and occupy $2^{55}$ bytes (32 PB), more than any memory or disk holds. Even if only 48 of the 64 address bits are used, the flat page table occupies $2^{36} \times 8 = 2^{39}$ bytes, i.e. 512 GB per program.

## 13.7 Multi-level page tables

A flat page table is large because it has entries for the whole virtual address space, although programs use only the bottom and the top of it. A multi-level page table avoids storing entries for the unused range in between.

**Source** Lesson 13, videos 13–17; H&P §B.4–B.5.

**Definition 13.12** (Multi-level page table). A *multi-level page table* is a tree of small tables. The VPN is split into one field per level. The first field indexes the outer page table, whose entries point to tables of the next level; the entries of the tables of the last level, the inner page

tables, hold PFNs. An entry whose part of the address space is not used at all is marked invalid, and no tables exist below it.

With two levels, a virtual address is split into an outer index $p_1$, an inner index $p_2$ and the page offset (Figure 13.3), and the translation takes two steps:

1. entry $p_1$ of the outer page table gives the location of an inner page table;
2. entry $p_2$ of that inner page table gives the PFN, which is combined with the offset as in Equation (13.1).

If either entry is invalid, no PFN is found and the access causes a page fault. The tables are usually sized so that each fits exactly into one page.

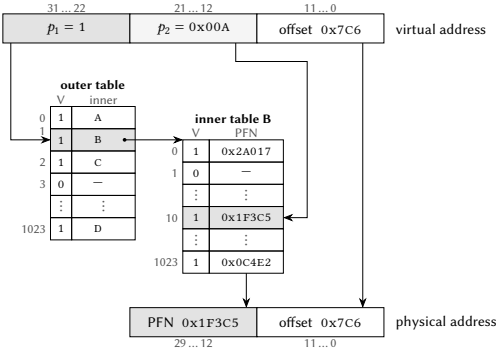



**Figure 13.3:** Two-level translation of the virtual address `0x0040A7C6` (Example 13.13). Outer entry 1 points to inner table B. Entries A, C and D point to inner tables that are not drawn; invalid outer entries, such as entry 3, have no inner table at all.

**Example 13.13.** The 20-bit VPN of Example 13.9 is split into a 10-bit outer index (bits 31–22) and a 10-bit inner index (bits 21–12). With 4-byte entries every table has $2^{10}$ entries and occupies 4 KB, exactly one page. The virtual address `0x0040A7C6` has VPN `0x0040A` = 0000000001 $0000001010_2$, hence $p_1 = 1$ and $p_2 = \texttt{0x00A} = 10$. Outer entry 1 points to inner table B, and entry 10 of B holds PFN `0x1F3C5`. The physical address is `0x1F3C57C6`, as with the flat page table: a multi-level page table stores the same mapping in a different form.

### 13.7.1 Size of a multi-level page table

The outer page table always exists, but an inner page table exists only for a part of the address space in which at least one page is used. With the organization of Example 13.13, each inner table covers $2^{10}$ pages, i.e. 4 MB of the virtual address space.

**Example 13.14.** A program uses 10 MB at the bottom of its 32-bit address space for code, data and heap (addresses `0x00000000`–`0x009FFFFF`) and 1 MB at the top for its stack (`0xFFF00000`–`0xFFFFFFFF`). With the two-level page table of Example 13.13, the bottom 10 MB lie in the ranges of outer entries 0, 1 and 2, of 4 MB each, and the stack in the range of outer entry 1023. The page table consists of the outer table and four inner tables, $5 \times 4$ KB = 20 KB, instead of the 4 MB of a flat page table.

A two-level page table is larger than a flat one only if every outer entry needs an inner table, i.e. if at least one page is used in each of the 1024 ranges of 4 MB. It then occupies 4 KB + $1024 \times 4$ KB $\approx$ 4.004 MB, more than the flat table by the size of the outer table. Real programs use a few contiguous regions of their address space and need only a few inner tables.[2]

[2] A different answer to the same problem is the *inverted* page table: one entry per frame rather than one per page, located by hashing the VPN. Its size follows physical memory and does not grow with the address space, but collisions cost a chain walk and shared pages need extra machinery. PowerPC and PA-RISC used it; H&P §B.4 describes the idea.

### 13.7.2 More levels for 64-bit addresses

Two levels do not suffice for 64-bit address spaces. A 4 KB table of 8-byte entries has $2^9$ entries, so each level of tables that fit into one page can consume only 9 bits of the VPN. With 48-bit virtual addresses the VPN has 36 bits. A two-level page table with 9-bit inner tables would need a 27-bit outer index and an outer table of $2^{27} \times 8$ bytes = 1 GB for every program. Splitting the 36 bits into four 9-bit fields instead yields a four-level page table in which every table fits

into one page; x86-64 processors use this organization. Each further level makes room for 9 more bits of virtual address.

x86-64 calls the four levels PML4, PDPT, PD and PT. Processors from Ice Lake on offer a fifth (CR4 . LA57) for 57-bit addresses, i.e. 128 PB.

**Example 13.15.** In such a four-level page table, a table of the last level covers $2^9$ pages, i.e. 2 MB of the address space; a table of the third level covers $2^9$ times as much, 1 GB; a table of the second level 512 GB; and the single table of the first level all $2^{48}$ bytes (256 TB). A program that uses 8 MB at the bottom and 8 MB at the top of its address space needs, for each end, four tables of the last level and one of each of levels 3 and 2, and all of them hang below the single first-level table: $1 + 2 \times (4 + 1 + 1) = 13$ tables, or $13 \times 4$ KB $= 52$ KB, whereas its flat page table would occupy 512 GB (Example 13.11).

> **Key point**
> A page table must describe the entire virtual address space, but a multi-level page table needs storage only for the parts that are used. The price is one table read per level on every translation.

## 13.8 Choosing the page size

**Source** Lesson 13, video 18; H&P §B.4.

How large pages should be is a compromise between two effects.

- **Larger pages** mean fewer pages: fewer PTEs and a smaller page table, and, as Section 13.10 shows, more memory covered by each translation that the processor caches.
- **Smaller pages** waste less memory in partly used pages.

The *page size*, the number of bytes in a page and in a frame, must balance the two.

**Definition 13.16** (Internal fragmentation). Memory is mapped a whole page at a time. A region whose size is not a multiple of the page size therefore leaves part of its last page unused, and no other program can use that part. The memory lost in this way is *internal fragmentation*.

A region of arbitrary size leaves on average half a page unused, so internal fragmentation grows in proportion to the page size.

**Caution** Paging removes *external* fragmentation, the waste left between allocations of different sizes, because any page fits any frame. What it leaves is the internal kind, inside the last page.

**Example 13.17.** A program on a 32-bit processor with 4-byte PTEs has three regions (code and data, heap, stack), each ending in a partly used page. Table 13.1 compares three page sizes. Going from 4 KB to 4 MB pages shrinks the flat page table by a factor of $2^{10}$, from 4 MB to 4 KB, but raises the expected internal fragmentation by the same factor, from 6 KB to 6 MB.

**Table 13.1:** Page table size and expected internal fragmentation for the program of Example 13.17 (half a page per region).

| Page size | Pages | Flat page table | Internal fragmentation |
|---|---|---|---|
| 4 KB | $2^{20}$ | 4 MB | $3 \times 2$ KB $= 6$ KB |
| 64 KB | $2^{16}$ | 256 KB | $3 \times 32$ KB $= 96$ KB |
| 4 MB | $2^{10}$ | 4 KB | $3 \times 2$ MB $= 6$ MB |

Pages are much larger than cache blocks (Lesson 12), because every page needs its own PTE and every page brought in from disk costs a disk access. The usual page size is 4 KB. Processors also support much larger pages, which the operating system can use for large regions that a program fills completely, so that internal fragmentation hardly matters.

4 KB has been the x86 page size since the 386 of 1985. AArch64 instead lets the operating system choose a granule of 4, 16 or 64 KB. x86-64 also offers 2 MB and 1 GB pages.

## 13.9 Memory access time with translation

**Source** Lesson 13, videos 19–21; H&P §B.4.

Every memory access a program makes, whether it fetches an instruction, loads or stores, uses a virtual address, which must be translated before physical memory can be accessed. The page table is itself kept in memory, so translation consists of memory reads. With a two-level page table, the processor performs a load such as `LW R1, 4(R2)` in four steps.

1. It adds 4 to R2 to obtain the virtual address and takes its VPN.
2. It computes the physical address of the outer PTE (the outer table address plus $p_1$ times the PTE size) and reads the PTE.
3. It computes the address of the inner PTE in the same way from the table address in the outer PTE and $p_2$, and reads the PTE.
4. It appends the offset to the PFN and accesses the data at this physical address through the cache.

Each PTE read is an ordinary memory access that goes through the caches and may hit or miss; if PTEs are not kept in the cache, each costs a full main-memory access. With a page table of $L$ levels, every access of the program turns into $L + 1$ memory accesses. If PTE reads have the same average memory access time (AMAT) as data accesses,

$$\text{access time with translation} \approx (L + 1) \times \text{AMAT}. \tag{13.3}$$

Besides the ordinary caches, x86 processors hold upper-level entries in dedicated *paging-structure caches*, so a walk usually needs only the last-level read. Intel SDM vol. 3.

**Example 13.18.** A cache has a hit time of 2 cycles, a miss rate of 5 % and a miss penalty of 40 cycles, so AMAT $= 2 + 0.05 \times 40 = 4$ cycles (Equation (12.1)). Without translation a memory access takes 4 cycles on average; with a flat page table it takes $2 \times 4 = 8$ cycles, and with a four-level page table $5 \times 4 = 20$ cycles. Individual accesses vary widely: a load whose three upper-level PTEs hit ($3 \times 2$ cycles), whose last-level PTE misses ($2 + 40$ cycles) and whose data hits (2 cycles) takes 50 cycles, 25 times as long as the cache hit it would be without translation.

The few upper-level entries are read by every translation and are usually cached; the numerous last-level entries miss more often. Either way, without a faster way to translate, virtual memory would slow down every instruction fetch, load and store by a large factor.

## 13.10 The translation lookaside buffer

**Source** Lesson 13, videos 22–24; H&P §B.4.

A program that accesses a page is likely to access the same page again soon, so the translations it needs at any time are few and are used over and over. The processor therefore caches them.

**Definition 13.19** (Translation lookaside buffer). A *translation lookaside buffer* (TLB) is a small, fast cache of recent translations. Each entry holds a valid bit, a tag taken from the VPN (the whole VPN in a fully associative TLB), the PFN to which that page is mapped, and the permission bits of its PTE. If the VPN of an address matches a valid entry, the PFN is taken from the entry and no page table is read.

A TLB is a cache in the sense of Lesson 12, but it can be much smaller than the data cache, for three reasons.

- It stores only translations, not data; an entry takes a few bytes.
- One entry serves a whole page rather than a single cache block, so a few entries cover the memory a program is actively using.
- It stores the final result of the translation, not the entries of the individual levels. A TLB hit takes a single lookup, however many levels the page table has.

**Tip** The memory a TLB maps at one time is its *reach*: entries × page size. 64 entries of 4 KB reach 256 KB; the same 64 entries of 2 MB pages reach 128 MB.

**Example 13.20.** A 128 KB data cache with 64-byte blocks has 2048 lines. With 4 KB pages, $128/4 = 32$ TLB entries cover as much memory as the cache holds, and 2048 are needed if every cached block lies in a different page; spatial locality keeps the actual need far below this upper bound.

### 13.10.1 TLB misses

A *TLB miss* occurs when no valid TLB entry matches the VPN of an address. The translation must then be obtained from the page table.

**Definition 13.21** (Page table walk). A *page table walk* reads the page table entries of an address level by level, from the outer page table down to the entry that holds the PFN, as described in Section 13.7.

After a TLB miss, the walk finds the PFN, the translation is written into the TLB, replacing an existing entry if the TLB is full, and the access proceeds (Figure 13.4).

**Caution** A TLB miss is not a page fault: the walk usually finds a valid PTE.

The TLB must agree with the page tables: when the operating system changes a mapping, e.g. by evicting a page, it invalidates the TLB entry of the page, and when the processor switches to another program, the entries of the previous program are invalidated or tagged with a program identifier.[3]

[3] That tag is a PCID on x86 and an ASID on Arm; with it a context switch need not flush the TLB. Invalidating a single mapping is the harder case: on a multicore processor every core that may hold the entry must drop it, so the operating system sends an interprocessor interrupt and waits for the acknowledgements. This *TLB shootdown* makes unmapping memory far more expensive than mapping it.

The walk can be performed in two ways.

**In software** the TLB miss raises an exception, and a handler in the operating system walks the page table and writes the translation into the TLB. The hardware is simple and the page tables may be organized in any way, but every TLB miss pays for an exception. MIPS processors handle TLB misses this way.

**In hardware** the processor contains a page table walker, logic that reads the PTEs and fills the TLB itself without involving the operating system. TLB misses are handled much

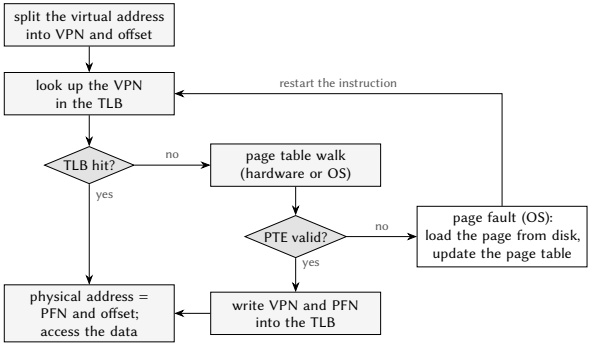



**Figure 13.4:** Translation on a memory access. A TLB hit supplies the PFN at once; a TLB miss requires a page table walk; an invalid PTE causes a page fault; once the page is in memory, the instruction is restarted.

faster, but the page table must have exactly the format that the hardware expects. High-performance processors, such as those of the x86 family, walk the page table in hardware.

> **Key point**
> Every memory access needs a translation. The TLB makes the common case, a TLB hit, a single fast lookup; a page table walk, in hardware or by the operating system, is needed only on a TLB miss, and the page-fault handler only when the walk finds no PFN.

## 13.11 TLB organization and performance

**Source** Lesson 13, videos 25–26; H&P §B.4.

The organization of a TLB follows from its role.

**Associativity** Every TLB miss costs a walk of several memory reads, so TLBs are fully associative or highly set-associative; for so few entries the comparators are affordable, and translations rarely compete for entries.

**Size** The TLB should hit about as often as the cache, so it must cover the pages of the data the cache holds (Example 13.20); this takes about 64 to 512 entries. Consulted on every access, it must also be about as fast as an L1 cache hit, which keeps it from growing larger.

Skylake, per Intel's optimization manual: 64 first-level data TLB entries for 4 KB pages, 128 for instructions, and a 1536-entry 12-way second-level TLB behind both.

**Levels** If that does not cover enough memory, a larger and slower second-level TLB with hundreds to thousands of entries is placed behind the first, as caches are organized in levels (Lesson 14). It is looked up only on a miss in the first, and a walk is needed only if both TLBs miss.

**Example 13.22.** The processor of Example 13.18 (AMAT of 4 cycles for every memory read, four-level page table) is given a TLB that is looked up in 1 cycle and misses on 1 % of the accesses. Every access pays for the lookup and for the data access, and one in a hundred also for a walk of $4 \times 4 = 16$ cycles, so an access takes on average $1 + 0.01 \times 16 + 4 = 5.16$ cycles instead of 20. Translation now costs 1.16 cycles instead of 16, mostly for the lookup, which Lesson 14 overlaps with the cache lookup.

This has the form of Equation (12.1): lookup time + miss rate × penalty, with the walk as the penalty. A TLB is a cache and is evaluated as one.

> **Key point — Lesson summary**
> Virtual memory maps the pages of each program's virtual address space onto frames of physical memory and keeps the rest on disk; an access to a page without a frame is a page fault. Translation replaces the virtual page number by a frame number from the page table and keeps the offset (Equation (13.1)). Multi-level page tables avoid the entries a flat table spends on unused address space (Equation (13.2)); the page size trades table size against internal fragmentation. Every level adds a memory read to each access, so a small, highly associative TLB caches translations, and the page table is walked only on a TLB miss.

# Advanced Caches

| | |
|---|---|
| **Lecture** | Lesson 14, videos 1–40 |
| **Textbook** | H&P [1] §2.3, §B.1, §B.3 |
| **Keywords** | pipelined cache, VIPT cache, aliasing, way prediction, NMRU, pseudo-LRU, conflict miss, prefetching, loop interchange, miss-under-miss support, MSHR, cache hierarchy, global miss rate, inclusion property |

Lesson 12 described the basic cache and measured it by the average memory access time, and Lesson 13 added the address translation that every memory access needs in a system with virtual memory. This lesson collects the techniques that make real caches fast. They are organized by the term of the average memory access time that they reduce: the hit time (pipelined caches, the placement of the cache relative to the TLB, way prediction and cheap replacement policies), the miss rate (larger blocks, prefetching and loop interchange) and the miss penalty (servicing overlapping misses and, above all, multilevel cache hierarchies).

## 14.1 Improving cache performance

**Source** Lesson 14, videos 1–2; H&P §2.3.

The quantity to be improved is the average memory access time (AMAT) of Lesson 12:

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}. \tag{14.1}$$

Each of the three terms offers its own line of attack, and Table 14.1 lists the techniques of this lesson by the term they target. Few of them are free. A technique that lowers one term often raises another—a smaller cache has a shorter hit time but a higher miss rate—and it is worthwhile only if AMAT as a whole decreases.

**Table 14.1:** Techniques of this lesson, grouped by the term of Equation (14.1) they reduce.

| Term | Techniques | Sections |
|---|---|---|
| Hit time | pipelined caches, VIPT caches, way prediction, NMRU and PLRU replacement | 14.2–14.6 |
| Miss rate | larger blocks, prefetching, loop interchange | 14.7–14.10 |
| Miss penalty | miss-under-miss support, cache hierarchies | 14.11–14.14 |

## 14.2 Pipelined caches

**Source** Lesson 14, videos 3–4; H&P §2.3.

The most direct way to lower the hit time is to keep the cache small and simple, for example direct-mapped: fewer lines have to be decoded and fewer tags compared. A small cache, however, misses more often, and a direct-mapped cache suffers more conflicts, so the time saved on hits is easily lost again on misses. The techniques of this and the next four sections reduce the hit time, or its cost, without giving up the miss rate.

**Caution** The L1 latencies vendors publish are load-to-use, not the array access alone: Skylake quotes 4 cycles, and 5 with an indexed address. Intel optimization manual.

Lesson 12 noted that an L1 hit takes one to three clock cycles. If a lookup that needs several cycles occupied the cache until it completed, each memory access would have to wait for the previous one: the hit time seen by an access would be its own lookup time plus this wait, and consecutive loads and stores would stall the processor. A cache whose lookup fits into one cycle has no such problem. An L1 cache that takes two or three cycles is usually pipelined.

**Definition 14.1** (Pipelined cache). A *pipelined cache* divides a lookup into stages separated by latches, so that a lookup takes several clock cycles but a new lookup can start in every cycle.

For a set-associative cache the lookup divides naturally into three steps, each of which can become a stage (Figure 14.1):

1. the index selects the set, and the valid bits and tags of its lines are read;
2. the tags are compared with the tag of the address, which decides between hit and miss and identifies the matching way, and the read of the data begins;
3. the data read completes, the data of the matching way is selected, and the block offset picks the requested bytes out of the block, which are delivered.

A miss is known at the end of the second stage. Pipelining does not reduce the number of cycles one lookup takes. It prevents that number from limiting the rate of accesses: while one access compares its tags, the next one already reads its set, exactly as instructions overlap in the pipeline of Lesson 3, and the clock period needs to accommodate only the slowest stage instead of the whole lookup.

Reading the data of every way in parallel with the tag check is what keeps the lookup short. Large L2 and L3 caches read the tags first and then only the matching way: slower, much less energy.

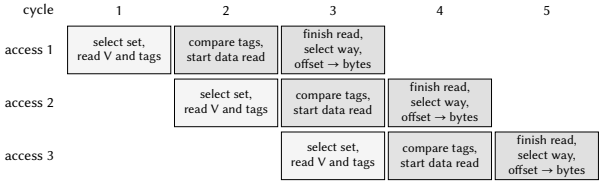



**Figure 14.1:** Three consecutive accesses to a cache whose lookup is split into three stages. Each access takes three cycles, but a new one starts in every cycle.

**Example 14.2.** The lookup logic of an L1 cache has a delay of 0.75 ns, and the other stages of the processor allow a clock period of 0.3 ns. Without pipelining, either the clock period grows to 0.75 ns or more for the whole processor, or each access occupies the cache for three cycles and memory accesses can start only in every third cycle. Split into three stages of about 0.25 ns each, which leaves room for the latch overhead, the lookup fits the 0.3 ns clock: the data arrives three cycles after the access starts, and an access can start in every cycle.

Pipelining gives one access per cycle; ports and banks give more. Skylake's L1 data cache serves two loads and one store per cycle. Intel optimization manual.

## 14.3 Address translation and the hit time

With virtual memory (Lesson 13) the processor issues virtual addresses, whereas data in main memory is located by physical addresses. The cache can be looked up after the translation, before it, or partly in parallel with it (Figure 14.2), and the choice affects the hit time.

**Source** Lesson 14, videos 5–7; H&P §B.3.

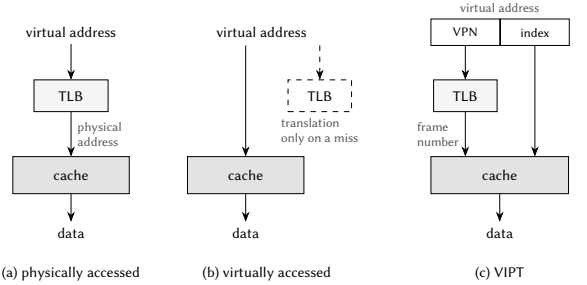



**Figure 14.2:** Three ways to combine the TLB with the cache lookup. In (b) translation is needed only on a miss, so the TLB is not in the path of a hit (it is still consulted for the permissions); in (c) it translates the virtual page number while the index already selects the set.

### 14.3.1 Physically accessed caches

A cache that is looked up with the physical address is a *physically accessed cache*. Every access first translates its virtual address in the TLB, and only the resulting physical address is used for the lookup (Figure 14.2a). The hit time therefore includes the translation:

$$\text{hit time} = \text{TLB lookup time} + \text{cache lookup time}. \tag{14.2}$$

The TLB has to be tiny to keep up: Skylake has 64 first-level data TLB entries for 4 KB pages, and 1536 second-level entries behind them. Intel optimization manual.

The TLB is small and fast, but its delay is paid on every access, hits included, and it adds to exactly the quantity we are trying to reduce.

### 14.3.2 Virtually accessed caches

**Definition 14.3** (Virtually accessed cache). A *virtually accessed cache* takes both the index and the tag from the virtual address. The address is translated only on a miss, when the block has to be fetched from main memory.

On a hit the TLB is not in the path, and the hit time is just the cache lookup time (Figure 14.2b). This advantage comes with problems.

- **Permissions.** The TLB entry of a page also records whether the page may be read, written or executed, and the permission has to be checked on every access. In practice the TLB is therefore consulted on hits as well.
- **Context switches.** Each process has its own virtual address space, so the same virtual address denotes different data in different processes. When the operating system switches the processor from one process to another (a context switch), the blocks in the cache are identified by virtual addresses of the old process and must not be found by the new one. Either the cache is flushed by clearing all valid bits, or every tag is extended with an identifier of the address space (H&P §B.3). Context switches occur many times per second, and with flushing the process that runs next starts each time with an empty cache and a burst of misses.
- **Aliasing**, the subject of Section 14.4.

### 14.3.3 Virtually indexed, physically tagged caches

A combination of the two approaches keeps the short hit time and avoids the flushes.

**Definition 14.4** (VIPT cache). A *virtually indexed physically tagged cache* (VIPT cache) takes the index from the virtual address and the tag from the physical address.

The index bits of the virtual address select the set at once (Figure 14.3). While the valid bits and tags of the set are being read, the TLB translates the virtual page number into the physical frame number, and the tags of the set, which are physical, are compared with the frame number from the TLB.

- The hit time is essentially the cache lookup time. The TLB lookup runs in parallel with the selection of the set and only has to deliver the frame number by the time the tags are compared, which a small TLB usually manages.
- The cache need not be flushed on a context switch. A tag names the physical memory its block came from, and that remains correct whichever process is running.
- The TLB is consulted on every access anyway, so the permissions are checked at no extra cost.

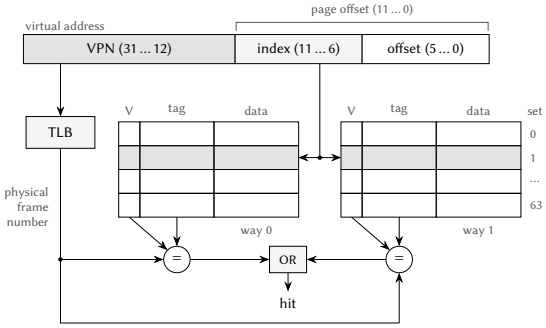



**Figure 14.3:** Lookup in a 2-way VIPT cache with 4 KB pages, 64-byte blocks and 64 sets (8 KB). The index comes from the page offset of the virtual address; the tags are physical frame numbers and are compared with the output of the TLB. A line hits if its valid bit is set and its tag matches.

## 14.4 Aliasing

**Source** Lesson 14, videos 8–11; H&P §B.3.

Virtual memory allows the same physical memory to appear at more than one virtual address.

**Definition 14.5** (Aliasing). The situation in which two or more virtual addresses, in the same or in different address spaces, are mapped to the same physical address is called *aliasing*. The virtual addresses are then aliases of each other.

Aliasing is common: processes that share memory see the same frames, usually at different virtual addresses, and a single process can have one frame mapped at two addresses. In a virtually accessed cache two aliases are simply two different addresses. They generally have different indexes and tags, so the cache can hold two copies of the same physical block, one per alias, and nothing connects the copies.

**Example 14.6.** The virtual addresses V1 and V2 are aliases of the physical address P, and in a virtually accessed cache they have different indexes. The memory at P contains the value 5. Table 14.2 follows four accesses. The last one reads 5 through V2, although 7 has been written to the same memory through V1. With the write-through policy main memory would hold 7, but the stale copy for V2 would still be returned.

**Table 14.2:** Two aliases in a virtually accessed cache with the write-back policy (Example 14.6): contents of the two cached copies and of main memory after each access.

| Step | Access | Result | Copy for V1 | Copy for V2 | Memory at P |
|---|---|---|---|---|---|
| 1 | read V1 | miss | 5 | — | 5 |
| 2 | read V2 | miss | 5 | 5 | 5 |
| 3 | write 7 to V1 | hit | 7 | 5 | 5 |
| 4 | read V2 | hit, returns 5 | 7 | 5 | 5 |

To be correct, every write would have to find all other cached copies of the same physical block and update or invalidate them, and the cache has no efficient way of locating them. Aliasing is thus a further problem of virtually accessed caches.[1]

[1] The operating system can prevent the second copy instead: page coloring maps aliases only at virtual addresses that agree in the index bits, so they select the same set. The price is less freedom in choosing which frame backs a page (H&P §B.3).

### 14.4.1 Aliasing in VIPT caches

In a VIPT cache the tag is physical, so two copies of a block would carry the same tag. They can nevertheless end up in different sets, because the index is taken from the virtual address. Whether that is possible depends on where the index bits lie. Translation replaces the virtual page number by the frame number and leaves the page offset unchanged; aliases therefore always have the same page offset, although their virtual page numbers differ.

- If the index includes bits of the virtual page number, two aliases can select different sets, the cache can hold two copies, and the problem of Example 14.6 remains.
- If the index consists of page-offset bits only, aliases always select the same set. There they find a single line with the physical tag of their block, so at most one copy exists and aliasing is harmless. The index bits of the virtual address are then also those of the physical address, and the cache behaves exactly like a physically accessed cache, only faster.

### 14.4.2 Avoiding aliasing

A VIPT cache is thus free of aliasing when its block offset and index fit into the page offset:

$$\text{offset bits} + \text{index bits} \;\le\; \text{page offset bits}. \tag{14.3}$$

The left side is $\log_2(\text{sets} \times \text{block size})$ and the right side $\log_2(\text{page size})$, so the condition reads sets × block size ≤ page size. Multiplying both sides by the associativity $N$ gives a bound on the cache size:

$$\text{cache size} \;\le\; N \times \text{page size}. \tag{14.4}$$

The block size has cancelled out: only the associativity and the page size limit a VIPT cache.

**Tip** Count bits, not bytes: with 4 KB pages only bits 0–11 survive translation, so offset bits + index bits ≤ 12.

**Example 14.7.** With 4 KB pages the page offset has 12 bits. A VIPT cache with 64-byte blocks (6 offset bits) can use at most 6 index bits, i.e. 64 sets. With 2 ways this is the 8 KB cache of Figure 14.3; with 4 ways it is 16 KB. A 2-way cache of 16 KB would need 128 sets, and its seventh index bit would be bit 12 of the virtual address, the lowest bit of the virtual page number, so aliases could be placed in different sets.

### 14.4.3 Real VIPT caches

The page size is fixed by the architecture and the operating system, and is typically 4 KB. By Equation (14.4), an L1 cache that is to be both VIPT and free of aliasing can therefore grow only by becoming more associative: a direct-mapped cache is limited to 4 KB, a 32 KB cache needs 8 ways. The Intel Core 2, Nehalem, Sandy Bridge and Haswell processors all have 8-way L1 data caches of 32 KB, exactly at this bound. Real L1 caches are consequently more associative than their hit time alone would justify, which makes the techniques of the next two sections important.

A larger page raises the bound: Apple's M-series uses 16 KB pages, and the M1 performance core has a 128 KB L1 data cache. Apple silicon CPU optimization guide.

> **Key point**
> A VIPT cache indexes with the virtual address while the TLB translates in parallel, and compares physical tags. It has nearly the hit time of a virtually accessed cache, needs no flush on a context switch, and is immune to aliasing as long as cache size $\leq N \times$ page size.

## 14.5 Way prediction

**Source** Lesson 14, videos 12–15; H&P §2.3.

Associativity moves the hit time and the miss rate in opposite directions. More ways per set mean fewer conflicts and a lower miss rate—and for a VIPT cache a larger permissible size (Equation (14.4))—but a lookup has to compare more tags and select among more lines, so the hit time grows. A direct-mapped cache has the shortest hit time and the highest miss rate. The goal is the miss rate of a set-associative cache with the hit time of a direct-mapped one.

**Definition 14.8** (Way prediction). A set-associative cache with *way prediction* guesses which line of the selected set holds the requested block and first checks only that line. If its tag matches, the access completes in the hit time of a direct-mapped cache; otherwise the other lines of the set are checked as in an ordinary set-associative cache.

The first check of an $N$-way cache of size $C$ examines one line per set, so it behaves like a direct-mapped cache of size $C/N$ with the same sets, and the hit rate of such a cache estimates how often the guess is right. If the guess is the line accessed most recently in the set, the estimate is exact: that line holds the block accessed most recently in the set, which is precisely the block a direct-mapped cache of size $C/N$ would hold. A wrong guess costs the quick check plus the normal lookup, and a miss is detected only after the normal lookup as well. If $p$ is the fraction of accesses in which the predicted line hits, $t_1$ the time to check one line and $t_N$ the lookup time of the whole set, the average lookup time is

$$t = p\,t_1 + (1-p)(t_1 + t_N) = t_1 + (1-p)\,t_N. \tag{14.5}$$

**Example 14.9.** An L1 cache is followed by an L2 cache that gives it a miss penalty of 20 ns. Three L1 designs are compared. The 4 KB direct-mapped cache is the subset of the 8-way cache that way prediction checks first, so the predicted line hits in $p = 80\,\%$ of all accesses, and the remaining 20 % (hits in other lines and all misses) take $0.8 + 1.5 = 2.3$ ns.

| L1 cache | Lookup time | Miss rate | AMAT |
|---|---|---|---|
| 4 KB direct-mapped | 0.8 ns | 20 % | $0.8 + 0.20 \times 20 = 4.8$ ns |
| 32 KB 8-way | 1.5 ns | 5 % | $1.5 + 0.05 \times 20 = 2.5$ ns |
| 32 KB 8-way, predicted | 0.8 or 2.3 ns | 5 % | $1.1 + 0.05 \times 20 = 2.1$ ns |

By Equation (14.5) the average lookup time with prediction is $0.8 + 0.2 \times 1.5 = 1.1$ ns. Way prediction keeps the miss rate of the 8-way cache and brings its average lookup time close to that of the direct-mapped cache.

## 14.6 Replacement policy and hit time

**Source** Lesson 14, videos 16–19; H&P §B.1.

The replacement policy affects the hit time as well as the miss rate. Two of the policies of Lesson 12 mark the extremes.

**Random** replacement does nothing on a hit and picks a victim at random on a miss. It adds nothing to the hit time, but it sometimes evicts a block that is about to be used, so its miss rate is higher.

**LRU** has a lower miss rate, but it keeps a $\log_2 N$-bit counter in every line, and an access, hit or miss, may change the counters of all $N$ lines of the set. This activity on every access costs logic, energy and hit time, the more so the higher the associativity.

The policies below approach the miss rate of LRU with far less bookkeeping.

### 14.6.1 Not most recently used

**Definition 14.10** (NMRU). The *not most recently used* (NMRU) policy records for each set only which line was accessed most recently, and on a miss evicts one of the other lines, chosen at random.

By temporal locality, the most recently used block is the one most likely to be accessed next. Evicting it is the most harmful choice random replacement can make, and NMRU excludes exactly that choice. The cost is one $\log_2 N$-bit pointer per set, and an access changes at most this pointer: an 8-way cache needs 3 bits per set instead of the $8 \times 3 = 24$ bits of LRU. NMRU misses somewhat more often than LRU, because its victim may be a block that was used only a little less recently than the most recent one.

### 14.6.2 Pseudo-LRU

**Definition 14.11** (PLRU). The *pseudo-LRU* (PLRU) policy keeps one bit per line, initially 0. An access sets the bit of its line; if this sets the last bit of the set that was still 0, the bits of all other lines are cleared. A miss evicts a line whose bit is 0, chosen at random.

**Caution** Elsewhere "pseudo-LRU" usually names the tree variant: $N-1$ bits per set steer a binary tree to the victim. The bit per line here is a different approximation.

The bit of a line tells whether the line has been accessed since the last reset. The bit of the most recently used line is always set, so the candidates of PLRU are always among those of NMRU, and PLRU is at least as good as NMRU. How closely it approximates LRU depends on how many bits are set. Right after a reset only the most recently used line is excluded, as in NMRU. With $N-1$ bits set, the only candidate is the line whose last access lies furthest in the past, which is exactly the choice of LRU. The cost is $N$ bits per set, and an access sets a single bit, except at a reset.

**Example 14.12.** A 4-way set holds the blocks A, B, C and D in lines 0–3; D was used most recently, and the PLRU bits were last reset by the access to D. Table 14.3 follows the set under the three policies. At the miss on E, LRU must evict A, NMRU may evict A, C or D, and PLRU may evict A or C; here all three evict A. If the next access, to F, also misses, LRU evicts C, NMRU may evict B, C or D, and PLRU, whose only line with bit 0 is line 2, also evicts C.

**Table 14.3:** State kept by LRU, NMRU and PLRU for one 4-way set (Example 14.12): the LRU order from least to most recently used block, the most recently used line recorded by NMRU, and the PLRU bits of lines 0–3.

| Access | Result | Lines 0–3 | LRU order | NMRU | PLRU bits |
|---|---|---|---|---|---|
| (initial) | | A B C D | A B C D | 3 | 0 0 0 1 |
| B | hit | A B C D | A C D B | 1 | 0 1 0 1 |
| D | hit | A B C D | A C B D | 3 | 0 1 0 1 |
| B | hit | A B C D | A C D B | 1 | 0 1 0 1 |
| E | miss | E B C D | C D B E | 0 | 1 1 0 1 |

**Key point**
Work done on every access lengthens every hit. Way prediction checks one line before the others, and NMRU and PLRU replace the counters of LRU by one pointer per set (NMRU) or one bit per line (PLRU); each keeps most of the miss-rate benefit of associativity or LRU at a fraction of the hit-time cost.

## 14.7 Causes of misses

Reducing the miss rate starts with knowing why misses occur. Misses are classified into three kinds, often called the three Cs.

**Source** Lesson 14, video 20; H&P §B.3; [28].

**Definition 14.13** (Compulsory, capacity and conflict misses). A *compulsory miss* is a miss on the first access to a block; it occurs in any cache, even an infinitely large one.

A *capacity miss* is a miss that would occur even in a fully associative cache of the same size: the cache cannot hold all the blocks the program is using, and the block has been evicted for lack of space.

A *conflict miss* is a miss that occurs only because of limited associativity: a fully associative cache of the same size would still hold the block, but in the actual cache it was evicted by another block competing for the same set.

A fourth C is added in multiprocessors: coherence misses, caused by another core's write invalidating the block (Lesson 19).

The three counts can be measured by simulating the same sequence of accesses on three caches. The misses of an infinite cache are the compulsory misses; a fully associative cache of the actual size adds the capacity misses; the actual cache adds the conflict misses.

**Example 14.14.** For one program, an infinite cache misses 1500 times, a fully associative 32 KB cache 6000 times, and the actual 4-way 32 KB cache 7800 times. Of the 7800 misses, 1500 are compulsory, $6000 - 1500 = 4500$ are capacity misses and $7800 - 6000 = 1800$ are conflict misses.

The classification points to the remedies. A larger cache reduces capacity misses, and a higher associativity or a better replacement policy reduces conflict misses, but all of these lengthen the hit time. The following sections present techniques that reduce the miss rate without lengthening hits.

## 14.8 Larger cache blocks

**Source** Lesson 14, videos 21–22; H&P §B.3.

For a cache of a given size, a larger block size affects the miss rate in two opposite ways.

- Where spatial locality is good, the data around an accessed address is used soon. A larger block brings more of it into the cache with a single miss, and later accesses to that data, which would otherwise have missed, now hit.
- Where spatial locality is poor, most of a larger block is never used. The cache holds fewer blocks, hence fewer of the blocks that are actually needed, and capacity and conflict misses increase.

As the block size grows, the miss rate therefore first falls and then rises again (Figure 14.4). The minimum lies at a larger block size for a larger cache, for example at about 64 bytes for a 4 KB cache and at about 256 bytes for a 256 KB cache: a large cache has room for many blocks even when they are large, so the space wasted on unused data hurts it less.

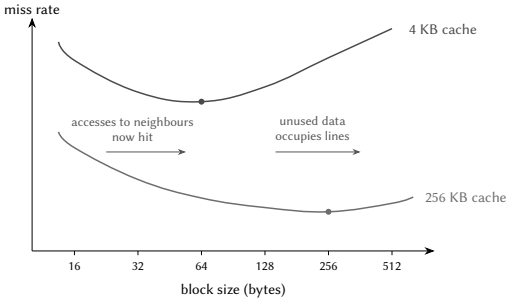



**Figure 14.4:** Miss rate as a function of the block size for a 4 KB and a 256 KB cache (shape only). The dots mark the best block sizes.

## 14.9 Prefetching

**Source** Lesson 14, videos 23–26; H&P §2.3.

A miss can be avoided altogether if the block is brought into the cache before the program asks for it.

**Definition 14.15** (Prefetching). Loading a block into the cache before it is accessed, because it is expected to be accessed soon, is called *prefetching*.

Prefetching is a guess, and its effect depends on whether the guess is right.

- A correct guess turns the first access to the block into a hit. The fetch from memory still takes place, but earlier, while the processor is doing other work, so the miss penalty is not paid by the access.

- A wrong guess wastes the fetch. Worse, the prefetched block has taken the place of another block, which may be needed later and then misses: the wrong guess *pollutes* the cache.

Prefetching pays off only if most guesses are right.

### 14.9.1 Prefetch instructions

The program often knows which data it will access next. A *prefetch instruction* is an instruction of the ISA that asks the processor to bring the block containing a given address into the cache; it changes no data. The compiler or the programmer places it in front of accesses that are known in advance, typically in loops over arrays [29]:

```
for (i = 0; i < n; i++) {
  prefetch(&a[i + D]);    /* D iterations ahead */
  sum = sum + a[i];
}
```

The difficulty is the prefetch distance $D$.

- If $D$ is too small, the prefetch has not completed when the loop reaches element $i + D$; the access still waits for the block, and little is gained.
- If $D$ is too large, the block arrives long before it is used. It occupies a line in the meantime and may be evicted again before it is needed.

The right distance is about the time a fetch takes divided by the time of one iteration. Both depend on the machine, so a program compiled with a distance that suits one processor prefetches too late or too early on another.

**Caution** A prefetch distance is tuned for one machine. A faster processor with the same memory needs a larger distance.

**Example 14.16.** A prefetched block is in the cache 50 ns after the prefetch instruction, and one iteration of the loop above takes 4 ns. The distance must be at least $50/4 = 12.5$, so a distance of 16 leaves a small margin. On a processor that executes each iteration in 2 ns with the same memory, the loop reaches element $i + 16$ only 32 ns after its prefetch, 18 ns before the block arrives. The loop then has to wait for memory: roughly every 16th access waits 18 ns, so 16 iterations take about 50 ns instead of 32 ns, and the faster processor runs the loop at only about 3.1 ns per iteration. It needs a distance of at least 25.

### 14.9.2 Hardware prefetching

In *hardware prefetching* the processor itself observes the sequence of memory accesses and guesses which blocks will be needed. The program need not be changed, and the guesses adapt to the machine the program runs on. Three designs exploit different access patterns (Figure 14.5).

**Stream buffer** A *stream buffer* [30] is a sequential prefetcher. When blocks are accessed in order, it fetches the next few blocks into a small buffer beside the cache, from where an access takes them without a memory access, and it fetches further blocks as the buffer is used up.

**Stride prefetcher** A *stride prefetcher* detects accesses that advance by a fixed distance, the *stride*, as when a loop visits every $k$-th element of an array, and prefetches the address one or more strides ahead.

**Correlating prefetcher** A *correlating prefetcher* records, for a block, which block was accessed after it, and when the block is accessed again prefetches the recorded successor. It handles patterns without a regular spacing, such as a linked list whose nodes are scattered in memory and that is traversed repeatedly.

## 14.10 Loop interchange

The compiler can also lower the miss rate by changing the order in which a program accesses its data. C stores a two-dimensional array row by row: `m[j][i]` and `m[j][i+1]` are adjacent, whereas `m[j+1][i]` lies a whole row further on. The following loop nest walks the array column by column:

**Source** Lesson 14, video 27; H&P §2.3.

```
double m[2000][1000];
for (i = 0; i < 1000; i++)
  for (j = 0; j < 2000; j++)
    m[j][i] = 2 * m[j][i];
```

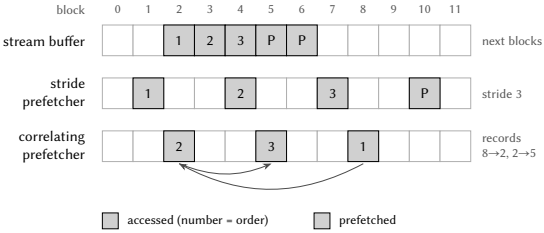



**Figure 14.5:** Access patterns exploited by the three hardware prefetchers. After the three accesses shown, the stream buffer fetches the following blocks, the stride prefetcher the block one stride ahead, and the correlating prefetcher block 2 the next time block 8 is accessed.

Each access of the inner loop jumps 1000 elements, 8000 bytes, beyond the previous one. There is no spatial locality: every access of the inner loop touches a different block, and the 2000 blocks visited by one pass of the inner loop are far more than an L1 cache can hold, so nearly every access misses.

**Definition 14.17** (Loop interchange). The compiler transformation that exchanges the order of nested loops, so that the innermost loop runs over the last index of the array and the data is accessed sequentially, is called *loop interchange*.

After interchange the outer loop runs over `j` and the inner loop over `i`. The inner loop now visits adjacent elements, one block serves several consecutive accesses, and a sequential or stride prefetcher can anticipate the next block. The compiler may interchange the loops only if it can prove that the result is unchanged. Here each iteration reads and writes only its own element, so the iterations can be executed in any order.

> **Key point**
> Prefetching can avoid even the miss on the first access to a block, but it helps only if its guesses are both correct and timely. Loop interchange attacks misses from the program side, by making the order of accesses match the layout of the data in memory.

## 14.11 Overlapping misses

**Source** Lesson 14, videos 28–30; H&P §2.3.

The miss penalty can be larger than the time needed to fetch a block. An out-of-order processor (Lesson 7) has many instructions in progress, and several of its loads and stores may access memory within a few cycles. Fetching a missing block takes many cycles, and in the meantime another access may miss as well.

**Definition 14.18** (Overlap miss). An *overlap miss* is a cache miss that occurs while an earlier miss is still being serviced.

After a load misses, an out-of-order processor continues with other instructions. The load cannot commit, however, so the processor eventually runs out of ROB entries or reservation stations and stops; a second miss issued before that point overlaps the first. How much it costs depends on the cache.

- A *blocking cache* does nothing else while a miss is being serviced. A later access is not even looked up until the earlier miss completes, so an overlapping miss pays the remaining time of the earlier miss in addition to its own penalty (Figure 14.6a).
- A *non-blocking cache* serves hits while a miss is in progress (*hit-under-miss*) and can also start further misses. Several requests are then in progress in memory at the same time, which exploits *memory-level parallelism*, provided that memory can service requests in parallel.

**Definition 14.19** (Miss-under-miss support). A cache with *miss-under-miss support* can service several misses at the same time: it keeps track of each miss in progress and sends their requests to the next level of the memory hierarchy concurrently.

Each miss in progress is recorded in a *miss status handling register* (MSHR). An access that misses in the cache first compares its block address with the MSHRs.

**No match** The access is a new miss. It allocates a free MSHR, which records the block and the instruction to be woken up, and the request is sent to memory.

**Match** The block is already on its way; this is a *half-miss*. No second request is sent, and the instruction is only added to the matching MSHR.

When the data arrives, it is placed in the cache, all instructions recorded in the MSHR are woken up, and the MSHR is released. Already two MSHRs give a large improvement over a blocking cache, four are better still, and because memory accesses take so long, even 16 to 32 MSHRs continue to help.

Real machines track fewer: Intel cores from Nehalem to Haswell have ten line-fill buffers, so at most ten L1 data misses are outstanding per core. Intel optimization manual.

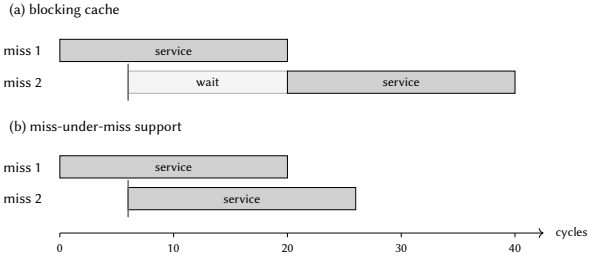



**Figure 14.6:** Two misses with a miss penalty of 20 cycles; the second occurs 6 cycles after the first (vertical line). (a) In a blocking cache the second miss takes 34 cycles instead of 20. (b) With miss-under-miss support each miss takes 20 cycles.

Miss-under-miss support costs additional hardware. It pays off in processors that issue many memory accesses concurrently, above all out-of-order processors.

## 14.12 Cache hierarchies

The miss penalty of an L1 cache is dominated by the access time of main memory. It can be reduced with the same idea that led to the L1 cache in the first place: place another cache between L1 and main memory. A miss in L1 then looks up the *L2 cache*, which is larger and slower than L1 but still much faster than main memory, and only misses in L2 go further.

**Source** Lesson 14, videos 31–34; H&P §B.3.

**Definition 14.20** (Cache hierarchy). A *cache hierarchy* consists of several levels of cache, L1, L2, L3, ..., each looked up only when the previous level misses (Figure 14.7). The level immediately before main memory is the *last-level cache* (LLC).

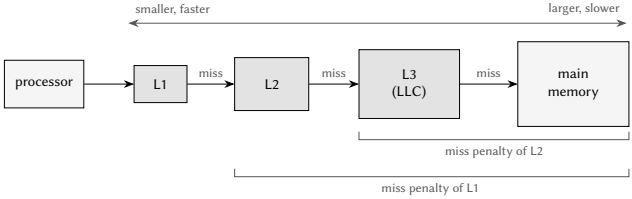



**Figure 14.7:** A three-level cache hierarchy. The miss penalty of each level is the average access time of the levels behind it.

### 14.12.1 AMAT of a cache hierarchy

The miss penalty of a level is the average time it takes to obtain the block from the levels behind it, which is the AMAT of the rest of the hierarchy. Applying Equation (14.1) level by level gives

$$\begin{aligned} \text{AMAT} &= h_1 + m_1 \times P_1, \\ P_1 &= h_2 + m_2 \times P_2, \\ &\vdots \\ P_{\text{LLC}} &= \text{access time of main memory}, \end{aligned} \tag{14.6}$$

where $h_i$ is the hit time, $m_i$ the miss rate and $P_i$ the miss penalty of level $i$. The miss rate $m_i$ is counted over the accesses that reach level $i$ (Section 14.13).

### 14.12.2 L1 versus L2

In Equation (14.6) the hit time $h_1$ of L1 is paid by every access, whereas the hit time $h_2$ of L2 is paid only by the fraction $m_1$ of accesses that miss in L1. L1 is therefore designed for a short hit time, with the techniques of the previous sections, while L2 can trade a longer hit time for a lower miss rate.

Skylake, per Intel's optimization manual: L1 data 32 KB, 4 cycles; L2 256 KB, 12 cycles; L3 about 2 MB per core, 44 cycles; main memory 60–100 ns, i.e. hundreds of cycles.

**Example 14.21.** Main memory takes 50 ns per access. A 32 KB cache has a hit time of 1 ns and a miss rate of 10 %; a 256 KB cache has a hit time of 5 ns and a miss rate of 3 %. Assume that in a hierarchy the 256 KB cache misses the same accesses as it would alone, i.e. $0.03/0.10 = 30\,\%$ of the accesses that reach it.

| Configuration | AMAT |
|---|---|
| No cache | 50 ns |
| 32 KB cache only | $1 + 0.10 \times 50 = 6$ ns |
| 256 KB cache only | $5 + 0.03 \times 50 = 6.5$ ns |
| L1 32 KB, L2 256 KB | $1 + 0.10 \times (5 + 0.30 \times 50) = 3$ ns |

The hierarchy is twice as fast as either cache alone. It combines the short hit time of the small cache, paid by every access, with the low miss rate of the large one, whose longer hit time is paid by only 10 % of the accesses.

## 14.13 Local and global miss rates

**Source** Lesson 14, videos 35–37; H&P §B.3.

In Example 14.21 the L2 cache misses 30 % of the accesses that reach it, three times the rate of L1. This does not make L2 a poor cache. L2 sees only the accesses that L1 has missed, which are the ones with the weakest locality; the accesses that are easy to serve never get past L1. The miss rate of a lower level must therefore be stated with its reference.

**Definition 14.22** (Local and global rates). The *local hit rate* of a cache is the number of its hits divided by the number of accesses that reach it, and its *local miss rate* is 1 − local hit rate.

The *global miss rate* of a cache is the number of its misses divided by the number of all memory accesses of the processor, and its *global hit rate* is 1 − global miss rate.

For L1 the local and global rates coincide. For a lower level the accesses that reach it are the misses of the level above, so the global miss rate of level $n$ is the product of the local miss rates:

$$\text{global miss rate}_n = \prod_{i=1}^{n} \text{local miss rate}_i. \tag{14.7}$$

The miss rates in Equation (14.6) are local. The global miss rate of a level says what fraction of all accesses has to go beyond it, which makes it the right measure for comparing levels and hierarchies.

**Definition 14.23** (Misses per 1000 instructions). The metric *misses per 1000 instructions* (MPKI) counts the misses of a cache per 1000 executed instructions.

Like the global miss rate, MPKI counts all misses of a level, but relative to the instructions instead of the memory accesses:

$$\text{MPKI} = 1000 \times \text{global miss rate} \times \frac{\text{memory accesses}}{\text{instruction}}. \tag{14.8}$$

**Tip** For a data cache: about a third of instructions load or store in typical code, so MPKI ≈ 330× global miss rate (H&P instruction mixes).

It thus states directly how often per instruction a program pays the miss penalty of that level.

**Example 14.24.** A program executes 1 000 000 instructions with 1 500 000 memory accesses. L1 misses 90 000 times and L2 27 000 times.

- L1: local and global miss rate $90\,000/1\,500\,000 = 6\,\%$; MPKI $90\,000/1000 = 90$.
- L2: local miss rate $27\,000/90\,000 = 30\,\%$ (local hit rate 70 %); global miss rate $27\,000/1\,500\,000 = 1.8\,\% = 0.06 \times 0.30$ (global hit rate 98.2 %); MPKI 27.

**Key point**
AMAT is computed with local miss rates. Levels and hierarchies are compared with global miss rates or MPKI, because a low local hit rate is normal for a level that sees only the misses of the level above.

## 14.14 The inclusion property

**Source** Lesson 14, videos 38–40; H&P §B.3; [31].

A block in L1 may or may not also be present in L2. Three relationships between the levels are possible:
**Inclusion** every block in L1 is also in L2.
**Exclusion** no block in L1 is also in L2.
**Neither** some blocks in L1 are also in L2 and others are not.

**Definition 14.25** (Inclusion property). A cache hierarchy has the *inclusion property* if every block in a level is also present in the next, larger level.

It might seem that a larger L2 with LRU replacement keeps every block of L1 automatically. It does not. Hits in L1 never reach L2, so a block that is used often, and therefore stays in L1, looks unused to the LRU counters of L2; eventually it becomes the least recently used block of L2 and is evicted there, while it remains in L1.

**Example 14.26.** L1 holds 2 blocks and L2 holds 3 blocks; both are fully associative with LRU replacement and initially empty, and a block that misses in both levels is loaded into both. Table 14.4 follows the accesses P, Q, P, R, P, S. The later accesses to P hit in L1, so L2 last sees P at the first access. At the miss on S, P is the least recently used block of L2 and is evicted, although it is still in L1.

**Table 14.4:** A two-level hierarchy with LRU replacement in both levels (Example 14.26). Contents are listed from least to most recently used; the last access breaks inclusion.

| Access | L1 | L2 | L1 after | L2 after |
|---|---|---|---|---|
| P | miss | miss | P | P |
| Q | miss | miss | P Q | P Q |
| P | hit | — | Q P | P Q |
| R | miss | miss | P R | P Q R |
| P | hit | — | R P | P Q R |
| S | miss | miss | P S | Q R S |

Inclusion therefore has to be enforced if it is wanted. One way is an *inclusion bit* in every line of L2, set while the block is also in L1; L2 never chooses a line with this bit set as its victim.[2]

This lesson completes the discussion of caches in a single processor (Lesson 19 returns to caches in multiprocessors). Lesson 15 turns to the memory technology from which caches and main memory are built.

**Key point — Lesson summary**
AMAT = hit time + miss rate × miss penalty, and each term has its own techniques. The hit time is reduced by pipelining the lookup, by VIPT caches, which index with the virtual address while the TLB translates and avoid aliasing if cache size $\leq N \times$ page size, by way prediction, and by the cheap replacement policies NMRU and PLRU. Misses are compulsory, capacity or conflict misses; the miss rate is reduced by a suitable block size, by prefetching (prefetch instructions, stream buffers, stride and correlating prefetchers) and by loop interchange. The miss penalty is reduced by miss-under-miss support, which tracks the misses in progress in MSHRs, and by cache hierarchies, whose AMAT is computed level by level with local miss rates, while global miss rates and MPKI describe how many accesses go past a level. With LRU in each level, inclusion does not hold automatically.

[2] Both choices are in use. Intel's client L3 up to Skylake is inclusive and doubles as a filter: a block that is not in L3 is in no core's cache, so the other cores need not be asked (Lesson 19). The L3 of Skylake-SP and of AMD's Zen is non-inclusive and holds mainly the blocks evicted from the private caches, so its capacity is not spent on copies.

# Memory

| | |
|---|---|
| **Lecture** | Lesson 15, videos 1–14 |
| **Textbook** | H&P [1] §2.2 |
| **Keywords** | SRAM, DRAM, memory cell, word line, bit line, sense amplifier, destructive read, trench cell, refresh, row decoder, RAS and CAS, fast page mode, front-side bus, northbridge |

Lessons 12–14 treated main memory as a large, slow device hidden behind the caches, and Lesson 1 showed that the gap between processor and memory speed, the memory wall, keeps widening. This lesson looks inside the memory itself. It describes the two technologies from which memories are built, static RAM for caches and dynamic RAM for main memory, down to the circuit that stores a single bit; it then shows how DRAM cells are organized into a chip and accessed, and how the chip is connected to the processor. Together these explain why main memory is slow and why it is not simply built with the faster technology of the caches.

## 15.1 Memory technologies

**Source** Lesson 15, videos 1–3; H&P §2.2.

Caches and main memory are both random-access memories.

**Definition 15.1** (Random access). A memory provides *random access* if any location can be read or written directly through its address, in about the same time for every location.

The opposite is sequential access (Lesson 16), in which a medium such as a magnetic tape has to be moved past all the data that lies between the current position and the requested one. Random-access memory (RAM) is built in two technologies, named after what happens to a stored bit while the memory is powered.

**Definition 15.2** (SRAM and DRAM). In *static random-access memory* (SRAM) a stored bit keeps its value for as long as the memory is powered. In *dynamic random-access memory* (DRAM) a stored bit loses its value after a short time, even with power on, unless it is periodically rewritten.

The names say nothing about what happens without power: both technologies are volatile and lose their entire contents when power is removed.

The two technologies differ in the circuit that stores a bit, and this difference determines their speed and cost. SRAM uses several transistors per bit, six in the usual design (Section 15.2); DRAM uses a single transistor (Section 15.3). SRAM therefore takes more chip area per bit and is more expensive per bit, but it is faster, because its cell actively drives the bit lines, whereas a DRAM cell only holds a small charge that has to be sensed, written back and refreshed. DRAM packs more bits into the same area and is cheaper per bit, but slower. Caches, which must be fast and can be small, are built from SRAM; main memory, which must be large and cheap, is built from DRAM.

The division is not absolute: IBM's POWER7 (2010) built its 32 MB L3 cache from DRAM cells—trench eDRAM—on the processor die (IBM POWER7 documentation).

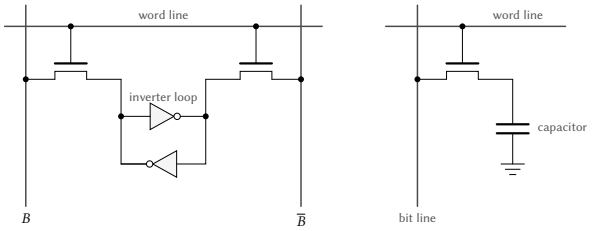
## 15.2 The SRAM cell

**Source** Lesson 15, video 4; H&P §2.2.

A memory stores its bits in a two-dimensional array of identical circuits. The circuit that stores one bit is called a *memory cell*. The cells are arranged in rows and columns and are reached through two sets of wires that cross the array.

**Definition 15.3** (Bit line and word line). A *bit line* runs along a column of cells and carries the value that is read from or written to a cell of the column. A *word line* runs along a row of cells; when it is activated, it connects every cell of the row to the bit line of its column.

In an SRAM cell (Figure 15.1a) the bit is held by two inverters connected in a loop, the output of each driving the input of the other. If the first inverter outputs 1, the second outputs 0, which in turn keeps the first at 1. The loop is stable in either of its two states and holds it for as long as the inverters are powered, which is what makes SRAM static. An access transistor, whose gate is connected to the word line, connects one node of the loop to the bit line.




(a) SRAM cell (6 transistors) (b) DRAM cell (1 transistor)

**Figure 15.1:** One memory bit. (a) An SRAM cell holds the bit in a loop of two inverters, whose nodes are connected to the complementary bit lines $B$ and $\overline{B}$ by two access transistors. (b) A DRAM cell holds the bit as charge on a capacitor behind a single access transistor.

**Write** The bit line is driven strongly to the new value and the word line is activated. The bit line overpowers the small inverter that drives the node, the loop settles in the state that matches the bit line, and it keeps this state after the word line has been deactivated.

**Read** The word line is activated and the loop drives the bit line. The transistors of the inverters are small, because the cell has to be small, whereas the bit line is a long wire attached to every cell of the column. A single cell can change the voltage of the bit line only slowly.

To speed up the read, the other node of the loop, which holds the complement of the bit, is connected through a second access transistor to a second bit line, $\overline{B}$. When the word line is activated, the cell pulls the two bit lines in opposite directions, and the value is found by comparing them.

**Definition 15.4** (Sense amplifier). A *sense amplifier* detects a small voltage change on a bit line, or a small difference between a pair of bit lines, and amplifies it to a full logic 0 or 1.

With a complementary pair, the sense amplifier only compares the two bit lines. The cell moves them in opposite directions, so the sign of their difference is clear after a very small swing and no reference voltage is needed; a single bit line would have to move much further before its value could be decided reliably. The pair helps writes as well, since the value and its complement act on both sides of the loop at once. The resulting cell has six transistors, two in each inverter and two access transistors, and it needs a word line, two bit lines and the supply connections of its inverters.

Every cache array of Lessons 12–14 is built from these cells: the data of a 32 KB L1 cache alone is 262 144 bits, some 1.6 million transistors.

## 15.3 The DRAM cell

A DRAM cell stores its bit as electric charge on a capacitor (Figure 15.1b). A single access transistor, controlled by the word line, connects the capacitor to the bit line; a charged capacitor represents 1 and a discharged one 0. To write, the bit line is set to the value and the word line is activated, so that the capacitor charges or discharges through the transistor. When the word line is deactivated, the transistor turns off and isolates the charge.

**Source** Lesson 15, videos 5–6; H&P §2.2; [32].

A transistor that is off is not a perfect switch, however. A small current still flows through it—the same imperfection that causes static power (Lesson 1)—and the charge of a stored 1 gradually leaks away until the cell can no longer be distinguished from a 0. This is what makes DRAM dynamic: a bit survives only if it is rewritten before too much of its charge has been lost.

**Caution** The "static" of SRAM is about retention, not power: SRAM cells leak as well, and the caches are among the largest contributors to a processor's static power.

**Definition 15.5** (Refresh). Reading the bits of DRAM cells and writing them back with full charge before the charge has leaked away is called *refresh*. Every cell must be refreshed periodically, whether or not its data is being used.

Reading a DRAM cell is delicate as well. The bit line is first set to a voltage halfway between 0 and 1, and then the word line is activated. The capacitor shares its charge with the bit line, whose voltage rises slightly if the cell held 1 and falls slightly if it held 0. The capacitor of a cell is tiny compared with the capacitance of the long bit line, so the change is very small, and a sense amplifier is needed to detect it. Once the charge has been shared, the capacitor is left near the intermediate voltage: the stored value is gone.

**Definition 15.6** (Destructive read). A *destructive read* is a read that destroys the stored value.

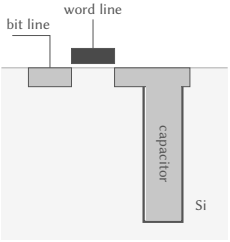



**Figure 15.2:** Cross-section of a trench cell (schematic).

The other common design, used in most current DRAMs, stacks the capacitor above the access transistor.

Every read of a DRAM cell is therefore followed by writing the value that was read back into the cell.

An SRAM read is not destructive, because the inverter loop keeps driving its nodes and restores their values by itself.

The capacitor also raises a problem of area. Built flat on the surface of the chip, a capacitor large enough to be sensed reliably would occupy far more area than the transistor, and much of the density advantage of a one-transistor cell would be lost.

**Definition 15.7** (Trench cell). A *trench cell* is a DRAM cell whose capacitor is formed in a deep, narrow trench etched vertically into the silicon next to the access transistor (Figure 15.2).

The trench gives the capacitor a large plate area while using little of the chip surface, so the cell occupies little more area than the transistor. It also needs fewer wires than an SRAM cell: one bit line instead of two, and no supply connections for inverters. Table 15.1 summarizes the comparison of the two cells.

**Table 15.1:** SRAM and DRAM compared.

| | SRAM | DRAM |
|---|---|---|
| Storage element | inverter loop | capacitor |
| Transistors per bit | 6 | 1 |
| Bit lines per cell | 2 | 1 |
| Keeps its value when powered | yes | only if refreshed |
| Read | non-destructive | destructive, then write-back |
| Speed | fast | slow |
| Area and cost per bit | high | low |
| Used for | caches | main memory |

> **Key point**
> DRAM stores a bit as a tiny charge behind one transistor, which makes it dense and cheap but slow: every read destroys the charge and must be followed by a write-back, and leakage forces a periodic refresh. SRAM avoids both at the price of six transistors per bit.

## 15.4 Memory chip organization

**Source** Lesson 15, video 7; H&P §2.2.

The cells of a memory chip form an array of $2^r$ rows and $2^c$ columns (Figure 15.3), and an address of $r+c$ bits is split into a row address of $r$ bits and a column address of $c$ bits, for instance the upper and the lower bits of the address. A read of one bit of a DRAM chip proceeds in four steps.

1. The *row decoder* turns the row address into the activation of exactly one of the $2^r$ word lines.
2. Every cell of the selected row is connected to its bit line. One sense amplifier per bit line detects the values, and the whole row is stored in a row latch.
3. The *column multiplexer* uses the column address to select one bit of the latch and delivers it to the data output.
4. The read was destructive for every cell of the row, so the contents of the latch are written back into the row over the bit lines.

A write uses the same path. The row is read into the latch as above, the bit selected by the column address is replaced in the latch by the value to be written, and the write-back then stores the modified row. In a DRAM chip a write is thus a read whose latch is changed before it is written back.

Real devices are 4, 8 or 16 bits wide rather than one: an 8-bit-wide DDR4 chip holds 1 KB in a row, and the eight chips of a rank open their rows together (JEDEC DDR4).

**Note 15.8.** A row latch that is written back when the row is closed is the simplified model used throughout this lesson. In real DRAM chips the sense amplifiers themselves latch the row: while the word line is still active they drive the bit lines to full levels, which restores the cells of the row, and a bit written into the latch reaches its cell at once. Closing the row turns the word line off and prepares the bit lines for the next read.

The two parts of the address take very different amounts of time. Selecting a row involves decoding, driving a long word line and waiting for the sense amplifiers to detect tiny charges

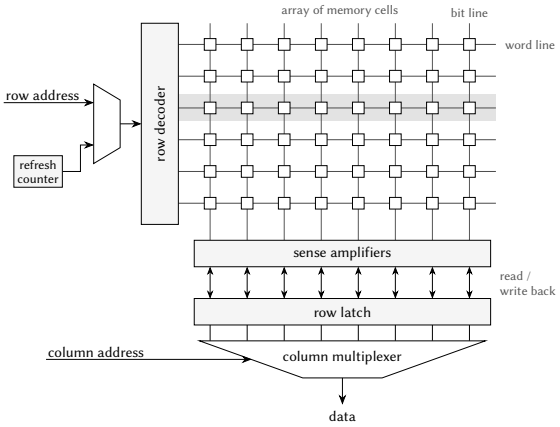



**Figure 15.3:** Organization of a DRAM chip. The row decoder activates the word line selected by the row address or by the refresh counter (Section 15.5); the sense amplifiers read the whole row into the row latch, the column multiplexer selects one bit, and the latch is written back into the row.

on long bit lines. Selecting a column only switches a multiplexer between bits that are already in the latch. Sections 15.6 and 15.7 exploit this difference.

Datasheets call the latched row the row buffer, and an access that finds its row already open a "page hit" (Section 15.7).

**Note 15.9.** An SRAM chip is organized in the same way, with a row decoder, sense amplifiers and a column multiplexer. Its reads are not destructive, however, so no write-back follows a read, and its cells need no refresh.

## 15.5 Refresh

Every row of a DRAM chip has to be refreshed at least once in every interval $T$, typically a few tens of milliseconds. A refresh uses the normal read path: it is a read (Section 15.4) whose latch is written back without using the output.

**Source** Lesson 15, videos 8–9; H&P §2.2.

A read by the program refreshes the row it touches, but such reads cannot be relied upon, because some rows are not read for long periods or not at all. All rows are therefore refreshed systematically. A refresh counter holds a row number. Periodically the row decoder is given the value of the counter instead of the row address (Figure 15.3); that row is read and written back, and the counter advances to the next row, wrapping around after the last one. With $R$ rows, each of which must be refreshed once per interval $T$, the chip must refresh one row, and advance the counter, at least once in every interval of length

JEDEC DDR4 fits 8192 refresh commands into a 64 ms window, one every 7.8 µs, and halves the interval above 85 °C.

$$t_{\text{row}} = \frac{T}{R}. \tag{15.1}$$

While a row is being refreshed, the chip cannot serve a read or a write, and an access that arrives in the meantime has to wait. Refresh is thus one more reason why DRAM is slow, and its cost grows with the number of rows.

**Example 15.10.** A DRAM chip has 8192 rows whose cells must be refreshed every 64 ms. By Equation (15.1) the chip must refresh one row in every 64 ms/8192 ≈ 7.8 µs, i.e. 128 000 rows per second. If refreshing one row takes 50 ns, the chip spends 128 000 × 50 ns = 6.4 ms of every second, or 0.64 % of the time, on refresh. A chip with four times as many rows and the same $T$ spends 2.56 %.

**Tip** The refresh overhead is $t/(T/R) = Rt/T$, with $t$ the time one row refresh occupies the chip; how the refreshes are spread over $T$ does not matter.

## 15.6 Address multiplexing

A chip with $2^{r+c}$ cells needs $r + c$ address bits, and every bit that is sent in parallel costs a pin on the package and a wire to the chip. The row address and the column address are not needed at the same time, however. The row address is needed first, to start the slow selection of the

**Source** Lesson 15, video 10; H&P §2.2.

row; the column address is needed only once the row is in the latch. DRAM chips therefore receive both parts over the same pins, one after the other, and two control signals tell the chip which part is present.

**Definition 15.11** (RAS and CAS). When the *row address strobe* (RAS) is asserted, the chip takes the value on the address pins as the row address, activates the word line and reads the row into the latch. When the *column address strobe* (CAS) is asserted, the chip takes the value on the address pins as the column address and selects that bit of the latch.

On a read the selected bit appears at the data output. If a write-enable signal is asserted together with CAS, the bit in the latch is instead replaced by the value at the data input. Deasserting RAS ends the access: the latch is written back and the row is closed (in the model of Note 15.8). Figure 15.4a shows two accesses of this kind.

In real chips RAS and CAS are active low; Figure 15.4 draws asserted signals high.

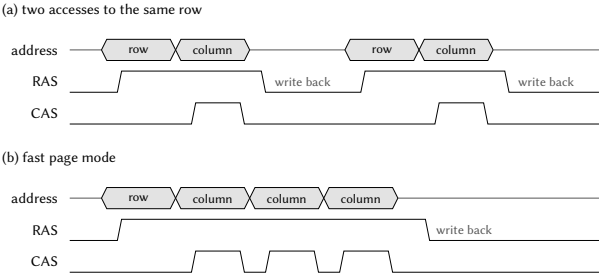



**Figure 15.4:** Signals of a DRAM chip with multiplexed address pins. (a) Each access opens the row with RAS, selects a column with CAS and closes the row. (b) In fast page mode (Section 15.7) the row is opened once, three columns are accessed, and the row is written back when RAS is deasserted.

Multiplexing reduces the address pins from $r + c$ to $\max(r, c)$, half as many when rows and columns are equally many, which makes the package smaller and cheaper and reduces the wiring between the chip and the memory controller that drives it (Section 15.8). A chip with $2^{28}$ cells in 16 384 rows and 16 384 columns, for example, needs 14 address pins instead of 28.

## 15.7 Fast page mode

**Source** Lesson 15, videos 11–12.

In the sequence of Figure 15.4a every access opens a row, uses one column and closes the row again, even when the next access is to the same row. Once a row has been opened, however, all its bits are in the latch, and reaching another bit of the row requires only a new column address.

**Definition 15.12** (Fast page mode). In *fast page mode* a row, once opened with RAS, stays open while several column addresses are applied with successive CAS pulses, each reading or writing one bit of the row. The row is written back only once, when RAS is finally deasserted (Figure 15.4b).

**Caution** This "page" is a row of the chip, not a virtual-memory page (Lesson 13).

Fast page mode pays for the expensive steps—selecting and sensing the row, and writing it back—once for a whole group of accesses, and for each access only the quick selection of a column.[1] Accesses to nearby addresses, which spatial locality (Lesson 12) makes common, usually differ only in the lower bits of the address and therefore share a row. The controller that drives RAS and CAS decides how long a row stays open. It can keep the row open as long as the following accesses use the same row, and it must close the row before it accesses another row or refreshes one. The savings therefore depend on how many consecutive accesses share a row.

[1] Fast page mode was the first of a line of improvements: extended data out (EDO) kept the data valid while the next column address was applied, synchronous DRAM (SDRAM, first shipped in 1992) replaced the strobes with a clock and a command protocol, and DDR added a transfer on both clock edges. Inside the chip the row-and-column structure is unchanged.

**Example 15.13.** In a DRAM chip, opening a row (RAS, decoding and sensing) takes 30 ns, selecting a column with CAS 5 ns, and closing the row (write-back) 10 ns. Five accesses arrive in the order of Table 15.2, where the pairs give (row, column). Without fast page mode every access opens and closes its row, which takes $5 \times 45\text{ ns} = 225\text{ ns}$. With fast page mode, rows are opened and closed three times, and the table adds up to 145 ns. Had the access to row 22 come last, the four accesses to row 7 would have shared a single opening, for a total of 105 ns.

**Caution** Module timings such as 22-22-22 are bus clocks, not nanoseconds: on DDR4-3200 opening a row, reading a column and closing a row each take 13.75 ns. JEDEC speed bins.

**Table 15.2:** The accesses of Example 15.13 in fast page mode.

| Access | Steps | Time (ns) |
|---|---|---|
| read (7, 12) | open row 7, CAS column 12 | 35 |
| read (7, 13) | CAS column 13 | 5 |
| write (7, 14) | CAS column 14 with write enable | 5 |
| read (22, 3) | close row 7, open row 22, CAS column 3 | 45 |
| read (7, 15) | close row 22, open row 7, CAS column 15 | 45 |
| — | close row 7 | 10 |
| Total | | 145 |

## 15.8 Connecting DRAM to the processor

**Source** Lesson 15, videos 13–14.

The time the processor waits for main memory is not just the access time of the DRAM chips. Between the two lies a path through other chips and buses, shown in its traditional form in Figure 15.5a.

**Definition 15.14** (Front-side bus and northbridge). The *front-side bus* (FSB) connects the processor to the *northbridge*, a chip that links the processor with main memory, with fast devices such as the graphics adapter, and with the southbridge, to which the slower I/O devices are attached (Lesson 16).

The northbridge contains the *memory controller*. It translates each memory access that arrives over the FSB into the signals of the DRAM chips: it splits the address into the row and the column address and drives RAS, CAS and write enable over the memory bus. A memory access on a cache miss therefore travels from the processor over the FSB to the northbridge and from there over the memory bus to the DRAM chips, and the data returns the same way. Each of these steps adds latency, and the FSB carries all other traffic between the processor and the devices behind the northbridge as well.

For scale: a 1333 MT/s front-side bus 64 bits wide carries about 10.7 GB/s for all traffic together, while one DDR4-3200 channel delivers 25.6 GB/s.

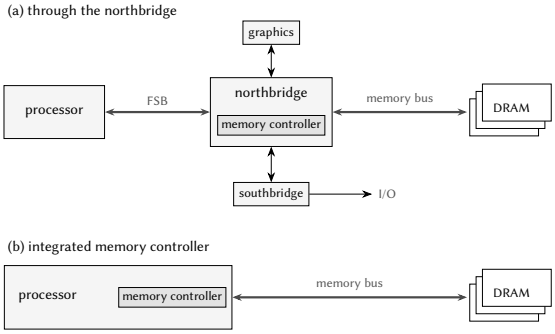



**Figure 15.5:** The path between processor and DRAM. (a) The memory controller in the northbridge is reached over the front-side bus. (b) With the memory controller integrated into the processor, the memory bus is attached to the processor directly.

Newer processors integrate the memory controller on the processor chip and attach the memory bus to the processor directly (Figure 15.5b).[2] The FSB and the northbridge then disappear from the path to memory, which lowers the latency of every access to main memory. Even so, a DRAM access remains far slower than the processor, for the reasons collected in this lesson, and the caches of Lessons 12–14 remain indispensable. Lesson 16 turns to storage devices, which are slower still but keep their data without power.

[2] AMD's Athlon 64 (2003) was the first mainstream x86 processor with the memory controller on the processor die, and Intel followed with Nehalem in 2008. The rest of the northbridge—the graphics and PCI Express links—moved onto the processor as well, leaving a single chipset for the slow devices. Each processor chip now has its own memory, and an access to the memory of another chip crosses the interconnect: the NUMA machines of Lesson 18.

**Key point — Lesson summary**
RAM is built as SRAM, whose six-transistor cell holds a bit in an inverter loop, or as DRAM, whose one-transistor cell (for example a trench cell) holds it as charge on a capacitor. SRAM is fast and expensive and is used for caches. DRAM is dense and cheap and is used for main memory, but its reads are destructive and are followed by a write-back, and leakage forces every row to be refreshed, so that one row must be refreshed in every interval $T/R$. A DRAM chip selects a row with the row decoder, reads it with the sense amplifiers into a latch and selects a bit with the column multiplexer. The row and column addresses share the address pins and are marked by RAS and CAS, and fast page mode serves several accesses to one row with a single opening. A memory access traditionally travels over the front-side bus to the memory controller in the northbridge and on over the memory bus; newer processors integrate the memory controller and drive the memory bus directly.

# Storage

| | |
|---|---|
| **Lecture** | Lesson 16, videos 1–14 |
| **Textbook** | H&P [1] §2.2, §D.1–D.2 |
| **Keywords** | magnetic disk, platter, cylinder, sector, seek time, rotational latency, optical disk, magnetic tape, sequential access, flash memory, solid-state disk, hybrid disk, I/O bus, mezzanine bus |

Lesson 15 showed that main memory is built from DRAM, which loses its contents as soon as power is removed. Programs and data that must survive have to be kept on storage devices, which also hold the pages that virtual memory (Lesson 13) cannot fit into physical memory. This lesson describes the main storage technologies—magnetic disks, optical disks, magnetic tape and semiconductor storage based on DRAM or flash memory—with emphasis on what determines their access time and how it evolves, and it shows how storage and other I/O devices are connected to the processor.

## 16.1 Storage

Storage keeps data *persistently*: unlike main memory, a storage device retains its contents when the computer is switched off. It serves two purposes.

**Source** Lesson 16, videos 1–2; H&P §D.1.

**Files** Programs, the data they work on and the settings of the system and its applications are kept in files, which must survive from one session to the next.

**Virtual memory** The pages that do not fit into physical memory are kept on storage, and a page fault reads such a page back into memory (Lesson 13).

Two properties of a storage device matter to the architect. Its *performance* is described by the two metrics of Lesson 2: throughput, the amount of data or the number of requests handled per second, and latency, the time a single access takes. Storage throughput improves over time, but more slowly than processor speed; storage latency improves even more slowly, more slowly than the latency of DRAM, so the gap between the processor and storage is wider still than the memory wall of Lesson 1. Its *reliability* matters at least as much: storage is where data is expected to be safe, and a lost file is often worse than a crashed program. Reliability is the subject of Lesson 17.

Read latency: DRAM 60–90 ns, a flash SSD about 100 µs, a disk about 10 ms (7-cpu.com; vendor data sheets).

## 16.2 Magnetic disks

A *magnetic disk* (hard disk) records bits as the direction of magnetization of tiny regions of a magnetic coating. It is built from rotating disks covered with this coating and from heads that move over them (Figure 16.1).

**Source** Lesson 16, video 3; H&P §D.1.

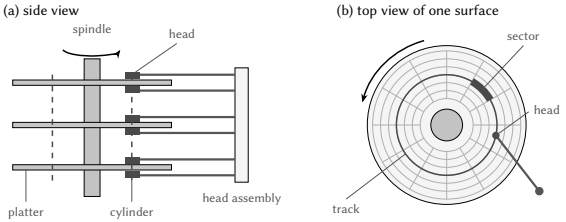



**Figure 16.1:** Structure of a magnetic disk. (a) Platters on a spindle, with one head per surface; the arms of all heads form the head assembly, and the dashed lines mark one cylinder. (b) One surface, divided into concentric tracks and each track into sectors; one track and one of its sectors are highlighted.

The coating covers both surfaces of each *platter*, a rigid circular disk. Several platters are stacked on a common *spindle*, which a motor turns at a constant speed, so that all platters rotate together (Figure 16.1a).

Each surface is read and written by its own *disk head*, which is mounted at the end of an arm and floats a tiny distance above the surface without touching it; a head that touches the rotating surface damages it. All arms are attached to one actuator and move together as the *head assembly*, which can place the heads anywhere between the outer edge of the platters and the spindle.

With the head assembly held in one position, each head passes over a circle of its surface as the platter rotates. This circle is a *track*, and a surface holds many concentric tracks (Figure 16.1b). Because the heads move together, they are always positioned over tracks at the same distance from the spindle, one on each surface. These tracks together form a *cylinder*: moving the head assembly selects a cylinder, and selecting one of the heads selects a surface, and therefore a single track, within it.

Along the track the data is divided into blocks of fixed size. Such a block, a *sector*, is the smallest unit that the disk reads or writes. A sector begins with a short preamble, which lets the disk recognize the start of the sector and synchronize with its bits; the data follows, and the sector ends with an error-correcting code with which the disk checks, and where possible corrects, the data it reads (Lesson 17).

The capacity of a disk follows directly from this organization:

$$\text{capacity} = \#\text{platters} \times 2 \times \frac{\text{tracks}}{\text{surface}} \times \frac{\text{sectors}}{\text{track}} \times \frac{\text{bytes}}{\text{sector}}, \tag{16.1}$$

where the factor 2 counts the two surfaces of a platter and the number of tracks per surface equals the number of cylinders.

Real disks place more sectors on the longer outer tracks; sectors per track is then an average.

**Example 16.1.** A disk has two platters with 100 000 tracks per surface, on average 2000 sectors per track, and 4096 bytes per sector. By Equation (16.1) it holds $2 \times 2 \times 100\,000 \times 2000 = 8 \times 10^8$ sectors, or $8 \times 10^8 \times 4096 \approx 3.3 \times 10^{12}$ bytes, i.e. about 3.3 TB.

**Caution** Disk capacities are decimal: 1 TB is $10^{12}$ bytes, not $2^{40}$, so a 4 TB drive holds only 3.64 TiB.

## 16.3 Disk access time

**Source** Lesson 16, videos 4–5; H&P §D.1.

To read or write a sector, the disk has to bring the right head over the right place of the rotating surface, and then the data has to travel to main memory. For a disk whose platters are already spinning, the time of one access consists of five parts.

**Seek time** The *seek time* is the time the head assembly needs to move the heads to the cylinder that contains the sector. It grows with the distance the heads have to travel.

**Rotational latency** The *rotational latency* is the time from the arrival of the head at the track until the start of the sector rotates under the head.

**Data read time** The sector passes under the head at the speed of rotation, and it is read (or written) during this time; an access to several consecutive sectors takes correspondingly longer.

**Controller time** The disk controller processes the request, for example checking the error-correcting code of each sector read.

**I/O bus time** The data is transferred between the disk and main memory over the I/O bus (Section 16.9).

$$t_{\text{access}} = t_{\text{seek}} + t_{\text{rot}} + t_{\text{read}} + t_{\text{controller}} + t_{\text{bus}}. \tag{16.2}$$

If the disk is still busy with earlier requests, a new request also waits in a queue before its own access begins. This queueing delay, the time until the earlier accesses have finished and the heads can move again, is often significant.

**Tip** In milliseconds a revolution takes 60 000/RPM: at 7200 RPM it is 8.33 ms, so the average rotational latency is about 4.2 ms.

The two rotation-dependent terms follow from the rotation speed, usually given in revolutions per minute (RPM). One revolution takes 60/RPM seconds. The start of the sector may be anywhere on the track when the head arrives, so the rotational latency lies between zero and one revolution and is half a revolution on average. Reading $n$ consecutive sectors takes the fraction $n/(\text{sectors per track})$ of a revolution:

$$\overline{t_{\text{rot}}} = \frac{1}{2} \cdot \frac{60}{\text{RPM}}, \qquad t_{\text{read}} = \frac{n}{\text{sectors per track}} \cdot \frac{60}{\text{RPM}}. \tag{16.3}$$

**Example 16.2.** A disk rotates at 10 000 RPM and has an average seek time of 4 ms. A request reads 100 consecutive sectors of a track that holds 500 sectors; the controller needs 0.3 ms and the transfer over the I/O bus 0.5 ms. One revolution takes $60\,000/10\,000 = 6$ ms, so by Equation (16.3) the average rotational latency is 3 ms and the data read time is $100/500 \times 6$ ms $= 1.2$ ms. By Equation (16.2) the access takes on average $4 + 3 + 1.2 + 0.3 + 0.5 = 9$ ms (Figure 16.2). Had the 100 sectors been scattered over the disk, each would have needed its own seek and rotational latency, and the request would have taken more than 700 ms.

A drive serves one access at a time: 9 ms is about 110 requests per second. Vendors quote this as IOPS; a flash SSD reaches 100 000 and more.

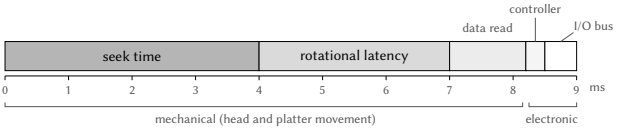



**Figure 16.2:** The access of Example 16.2, drawn to scale. Mechanical movement accounts for 8.2 ms of the 9 ms.

**Tip** Read 1 ms as one million cycles per gigahertz: the 9 ms above is 27 million cycles on a 3 GHz processor.

**Key point**
A disk access is dominated by mechanical movement: moving the head assembly and waiting for the platter to turn. It takes milliseconds, i.e. millions of clock cycles of a processor, which is why a page fault (Lesson 13) is so expensive.

## 16.4 Trends for magnetic disks

**Source** Lesson 16, video 6; H&P §D.2.

The components of a magnetic disk improve at very different rates.

**Capacity** depends on how densely bits are recorded on the surface. For many years it doubled roughly every one to two years.

**Seek time** is typically 5 ms to 10 ms and improves only very slowly. What improvement there is comes mainly from smaller platter diameters, which shorten the distance the heads travel.

This growth has slowed considerably since about 2010.

**Rotation speed** rose slowly, from about 5000 RPM to 10 000–15 000 RPM for the fastest drives; most drives spin at 5400–7200 RPM. A faster rotation requires stronger materials and produces more noise, heat and power consumption.

**Caution** A quoted seek time is an average over random seeks; a track-to-track seek takes under 1 ms, a full-stroke seek about twice the average.

**Controller and bus time** are electronic and improve at a good rate, but they are a small part of the access time (Figure 16.2).

The latency of a disk access is therefore set by mechanics and materials rather than by the progress of transistor technology that Moore's law describes (Lesson 1).[1] It improves far more slowly than processor speed, even though capacity grew quickly for many years. Higher recording density does shorten the data read time of a given amount of data, since more bytes pass under the head in each revolution, but it does not affect the seek time or the rotational latency.

[1] The magnetic disk is the oldest computer component still in use. IBM's 350 (1956) held five million characters (about 3.75 MB) on fifty 24-inch platters turning at 1200 RPM; its single access arm needed some 0.6 s to reach a track. Capacity has since grown by six orders of magnitude, the access time by less than two.

## 16.5 Optical disks

**Source** Lesson 16, video 7.

An *optical disk* is a single rotating disk like one platter of a magnetic disk, with data along a track, but it records bits as marks that change how the surface reflects light, and it reads them with a laser instead of a magnetic head. The laser works at a distance from the disk and focuses through a transparent protective layer, so dust and fingerprints on the surface are far less harmful than on a magnetic platter, which must be sealed inside its drive. An optical disk can therefore be removed from its drive and carried to another one: it is *portable*.

On CDs, DVDs and Blu-ray discs the data lies along a single spiral track.

Portability has a price. A disk written in one drive must be readable in every other drive, so the recording format has to be fixed by an industry standard: CD, then DVD, then Blu-ray. A denser recording technology becomes useful only after a new standard has been agreed on and drives, media and content for it have become widespread, which takes years. Optical disks therefore improve in large, infrequent steps. The platters of a magnetic disk never leave their drive; a manufacturer can adopt a better recording technology in its next product as long as the drive still connects through the same standard interface.

CD (1982) 700 MB, DVD (1996) 4.7 GB, Blu-ray (2006) 25 GB per layer: each came 10–14 years after the last with five to seven times the capacity.

## 16.6 Magnetic tape

**Source** Lesson 16, videos 8–9.

A *magnetic tape* records data on a long, thin strip coated with magnetic material, which is kept wound up in a cartridge and pulled past the read/write head of the drive. A tape has a large capacity, and its cartridges are replaceable: one drive can read and write any number of cheap cartridges. Its drawback is the way data is reached.

**Definition 16.3** (Sequential access). A medium provides *sequential access* if data can be reached only in order: to access a location, the medium has to be moved past all the data between the current position and the requested one, so the access time grows with that distance.

A disk, by contrast, reaches any sector directly, without passing over the data in between: one seek and at most one revolution take at most some tens of milliseconds, wherever the sector lies (Example 16.4). Its access time still depends on the seek distance, so a disk provides random access (Lesson 15) only approximately, but far more closely than a tape. Sequential access suits one use particularly well: *backup*, where large amounts of data are written in one pass and read back rarely, and then usually in one pass as well. It is unsuitable for virtual memory, whose page faults read pages from arbitrary locations. Tape is therefore used as backup storage rather than as the storage from which programs and files are used.

An LTO-9 cartridge (2021) holds 18 TB and streams at 400 MB/s, so reading it end to end takes about 12 hours (LTO consortium).

**Example 16.4.** A tape of 1000 m is wound at 5 m/s. With the tape at its beginning, data in the middle is reached after 100 s and data at the end after 200 s. A disk whose longest seek takes 15 ms and whose revolution takes 6 ms reaches any sector within about 21 ms, about 9500 times faster than the tape reaches its end. For reading a whole backup from start to finish, the winding time does not matter.

Tape is gradually being displaced. Its production volume is small, so its cost per gigabyte falls more slowly than that of magnetic disks, which are made in huge numbers; disks, for example external disks attached over USB, have become cheap enough to take over much of the backup role.

## 16.7 Solid-state disks

**Source** Lesson 16, video 10; H&P §2.2, §D.1.

Semiconductor memory has no moving parts, and an access to it takes nanoseconds to a fraction of a millisecond instead of the milliseconds of a disk. Its cost per gigabyte, however, is much higher. DRAM costs on the order of a hundred times more per gigabyte than a magnetic disk, while its latency is about 100 000 times lower (tens of nanoseconds against milliseconds). Storage built from semiconductor memory is used where speed justifies this cost.

**Definition 16.5** (Solid-state disk). A *solid-state disk* (SSD) is a storage device built from semiconductor memory, without moving parts, that is used like a disk: it stores blocks of data and connects through a disk interface such as SATA or directly over PCI Express.

An SSD can be built from either of two kinds of memory.

**DRAM with a battery** DRAM is volatile, so a battery keeps it powered when the computer is off. Such an SSD is extremely fast and extremely expensive per gigabyte, and its data is safe only as long as the battery lasts, which makes it unsuitable for keeping data over long periods.

**Flash memory** Most SSDs use *flash memory*, a semiconductor memory that keeps its contents without power: its bits are stored as charge trapped in transistors, where it remains for years.

**Caution** Flash memory ("flash") has nothing to do with a pipeline flush (Lesson 3).

Flash cannot be overwritten in place: a block of several MB must be erased first, and survives only some thousand erases.

A flash SSD is slower than DRAM but still much faster than a magnetic disk, needs little power, and retains its data without a battery. Its cost per gigabyte lies between those of DRAM and disk, so for the same price it offers much less capacity than a disk: for a long time, typical flash SSDs held gigabytes where disks held terabytes.

## 16.8 Hybrid magnetic–flash disks

**Source** Lesson 16, videos 11–12.

Magnetic disks and flash memory have complementary strengths. A magnetic disk offers a huge capacity at a low cost per gigabyte, but it is slow, consumes considerable power while its platters spin, and is vulnerable to shocks during operation. Flash memory is fast, consumes little power and has no moving parts, but it costs more per gigabyte.

**Definition 16.6** (Hybrid disk). A *hybrid disk* combines a magnetic disk with a smaller amount of flash memory in one device. The magnetic disk holds all data, and the flash memory serves as a cache for the data that is used most frequently (Figure 16.3).

Temporal and spatial locality (Lesson 12) apply to files as they do to memory accesses: the operating system, the applications in daily use and the files being worked on account for most accesses. If they are kept in flash, most requests are served at the speed of flash and without mechanical movement, while the capacity and the cost per gigabyte remain close to those of

the disk. When requests are served from flash, the disk can even stop spinning, which saves power and makes the device less sensitive to shocks; a request that then does need the disk waits, often for seconds, until the platters spin again.

The average access time behaves like the AMAT of a cache (Equation (12.1)). A request served from flash takes the hit time, and a request served by the disk takes the total time of a miss. The controller knows where each block lies, so a miss does not read the flash first, and the miss penalty is the difference between the two times.

**Example 16.7.** A hybrid disk combines flash with an access time of 0.1 ms and the disk of Example 16.2, which is kept spinning. For the request of that example, a hit also pays the controller and I/O bus times, $0.1 + 0.3 + 0.5 = 0.9$ ms, and a miss takes 9 ms, so the miss penalty is 8.1 ms. If 95 % of the requests find their data in flash, the average access time is $0.9 + 0.05 \times 8.1 \approx 1.3$ ms, about 7 times shorter than with the disk alone, although the flash holds only a small part of the data. The controller and the I/O bus now account for most of the time of a hit.

Table 16.1 summarizes the technologies of this and the previous section. Which one is preferable depends on the requirement that dominates: the lowest cost for a large amount of data favors a magnetic disk, speed, low power and robustness favor flash, and a hybrid disk approaches both for workloads whose accesses concentrate on a small part of the data.

**Table 16.1:** Magnetic disks and semiconductor storage compared.

| | Magnetic disk | DRAM SSD | Flash SSD | Hybrid disk |
|---|---|---|---|---|
| Access | slow (ms) | fastest | fast | fast for most |
| Cost per GB | low | very high | medium | low |
| Power | high | high | low | lower |
| Moving parts | yes | no | no | yes |
| Keeps data unpowered | yes | while battery lasts | yes | yes |

**Key point**
A hybrid disk applies the principle of caching to storage: flash in front of a magnetic disk gives most requests the speed and low power of flash at nearly the cost per gigabyte of the disk.

## 16.9 Connecting I/O devices

Storage devices, like other input/output (I/O) devices, are not attached directly to the processor. They are connected through buses defined by industry standards.

**Definition 16.8** (I/O bus). An *I/O bus* is a standardized bus through which I/O devices are connected to a computer. The standard fixes the connectors, the electrical signals and the protocol; PCI Express, SATA, SCSI and USB are examples.

Standardization lets any device built to the standard work with any computer that provides the bus, whoever manufactured either of them, and a computer maker can offer one kind of interface for a wide range of devices. The price is a limited rate of improvement. A new version of a standard has to be agreed on by many companies, and devices built for the older version should keep working, so a bus cannot adopt a faster technology as soon as it becomes available.[2] The more devices depend on a standard, the more slowly it changes.

I/O devices also differ enormously in the throughput they need: a graphics adapter moves huge amounts of data, a disk much less, a keyboard almost none. A single fast bus would force every device to have an expensive fast interface, and a single slow bus would throttle the fast devices. The buses are therefore arranged in a hierarchy (Figure 16.4).

**Definition 16.9** (Mezzanine bus). A *mezzanine bus* is a fast, short I/O bus between the processor and the slower I/O buses; today it is PCI Express. Fast devices such as the graphics adapter are connected to it directly, and the controllers of the slower buses are attached to it.

A SATA or SCSI controller on the mezzanine bus provides a SATA (or SCSI) bus for disks and optical drives, and a USB host controller provides a USB bus, extended by hubs, for external devices such as keyboards and flash drives. These buses serve naturally slower devices, for which a stable standard matters more than speed; they are slower than the mezzanine bus and

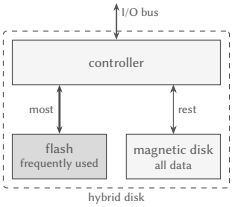



**Figure 16.3:** A hybrid disk. Most requests are served by the flash cache.

**Tip** The hit time is a floor: however high the hit rate, this disk cannot beat $9/0.9 = 10$ times the disk alone (Amdahl's law, Lesson 2).

**Caution** The flash is a cache, not extra capacity: a 2 TB hybrid disk with 8 GB of flash is sold and behaves as a 2 TB drive.

**Source** Lesson 16, videos 13–14.

[2] PCI Express doubles its per-lane bandwidth each generation: 2.5 GT/s in 2003, then 5, 8 (doubling through a leaner 128b/130b encoding), 16 (a seven-year wait), 32 and 64 GT/s in 2022. Compatibility runs both ways: an old card works in a new slot and a new card in an old slot, at the slower rate.

SATA 3 carries 6 Gb/s (600 MB/s), USB 3.2 Gen 2 10 Gb/s and a 16-lane PCIe 5.0 slot some 63 GB/s; a keyboard needs a few hundred bytes per second (SATA-IO, USB-IF, PCI-SIG specifications).

change less often. In PC chipsets the SATA and USB controllers are integrated into one chip, the *southbridge*, which is connected to the northbridge (Lesson 15).

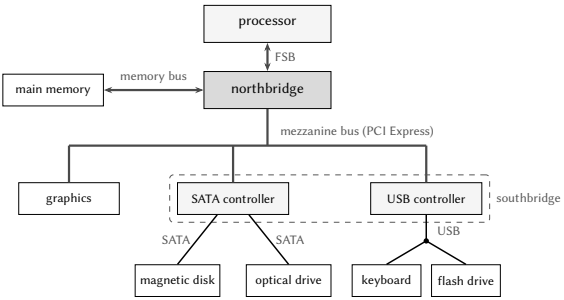



**Figure 16.4:** The hierarchy of buses. The graphics adapter is connected directly to the mezzanine bus; disks and other slower devices are reached through the SATA and USB controllers on it, which provide slower standard I/O buses. In PC chipsets these controllers form the southbridge (dashed).

Each controller translates between the bus it provides and the mezzanine bus. The data of a disk read travels from the disk over SATA to the SATA controller, over the mezzanine bus and through the northbridge into main memory; this path is the I/O bus time of Equation (16.2). Fast devices sit close to the processor, and the many slow devices are confined to their own buses behind the controllers, where they do not slow down the fast ones.

Newer processors integrate the northbridge and connect to the southbridge directly.

Whatever the technology and however it is connected, storage is where data is expected to survive, yet every storage device eventually fails. Lesson 17 shows how a system can keep its data and its service despite such failures.

**Key point — Lesson summary**

Storage holds files and the pages of virtual memory persistently, and its latency improves even more slowly than that of DRAM. A magnetic disk stores data in sectors on the tracks of rotating platters (Equation (16.1)); an access (Equation (16.2)) is dominated by the seek time and the rotational latency, half a revolution on average, and this mechanical latency improves only slowly, while capacity grew quickly for many years. Optical disks are portable but improve slowly because of standardization; magnetic tape offers cheap capacity with sequential access and serves for backup. Solid-state disks built from DRAM or flash memory avoid mechanical movement at a higher cost per gigabyte, and a hybrid disk uses flash as a cache for a magnetic disk. I/O devices are connected through standardized I/O buses arranged in a hierarchy: fast devices sit directly on a fast, short mezzanine bus such as PCI Express, and slower devices on slower, more stable buses such as SATA and USB, whose controllers are attached to the mezzanine bus.

# Fault Tolerance

| | |
|---|---|
| **Lecture** | Lesson 17, videos 1–32 |
| **Textbook** | H&P [1] §1.7, §D.2–D.3 |
| **Keywords** | dependability, fault, error, failure, reliability, availability, MTTF, MTTR, fault tolerance, checkpointing, N-module redundancy, parity, ECC, RAID |

Lesson 16 described the devices that store data persistently. Every component of a computer, storage devices included, eventually deviates from its intended behavior, and a system that returns wrong results or loses data whenever one component does so cannot be relied on. This lesson introduces the vocabulary for reasoning about such problems—faults, errors and failures; reliability and availability—classifies the faults that occur in practice, and presents the general techniques for tolerating them. The second half applies these ideas to disks: redundant arrays of independent disks (RAID) combine several disks into one, and their redundant organizations keep data intact and accessible even when an entire disk fails.

## 17.1 Dependability

**Source** Lesson 17, videos 1–2; H&P §1.7; [33].

A computer is useful only if its users can trust the service it provides. To make this precise, two descriptions of the system's behavior are distinguished. The *specified service* is the behavior the system should have according to its specification; the *delivered service* is the behavior it actually exhibits.

**Definition 17.1** (Dependability). The *dependability* of a system is the quality of its delivered service that justifies relying on the system to provide that service.

A system is built from components, or *modules*: processors, memory modules, disks and, at a finer level, the circuits and program functions inside them. Each module has a specified behavior of its own. Dependability problems begin when some module deviates from its specification; the next section describes how such a deviation can, but need not, turn into a deviation of the whole system.

## 17.2 Faults, errors and failures

**Source** Lesson 17, videos 3–5; H&P §1.7, §D.3.

A problem that starts inside one module passes through up to three stages before the users of the system can observe it. The stages are named separately because a dependable system tries to stop the problem at one of the transitions between them.

**Definition 17.2** (Fault, error, failure). A *fault* is a deviation of a module from its specified behavior, such as a defective transistor or a mistake in a program.

An *error* exists when the actual behavior inside the system—its internal state—differs from the specified behavior, such as a wrong value stored in a variable.

A *failure* occurs when the delivered service of the system deviates from the specified service.

A fault can exist for a long time without any effect. Until the faulty module is used in a way that exposes the deviation, the fault is a *latent* error. When the module is used in that way, the fault is *activated* and becomes an *effective* error: the internal state is now wrong. An effective error causes a failure only if it propagates to the service the system delivers. Not every fault is activated, and not every error becomes a failure (Figure 17.1).

**Example 17.3.** A function that multiplies two integers contains a programming mistake: it returns the correct product for all arguments except 6 and 9, for which it returns 45 instead of 54. The mistake is a fault from the moment the function is written, and it stays latent as long as no caller multiplies 6 by 9. When a program calls the function with these arguments and stores the result, the fault has been activated and the program state contains an error. If the result is the number of seats a booking system reserves, 45 seats are reserved instead of 54: the error has reached the delivered service, and a failure has occurred. If instead the program only tests whether the product is positive, the wrong value leads to the same decision as the correct one, and no failure occurs.

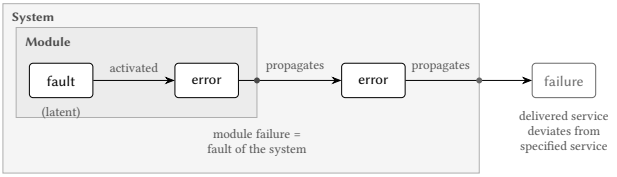



**Figure 17.1:** A fault becomes an error when activated. An error that leaves a module is a failure of the module and, at the same time, a fault of the system that contains it; an error that reaches the delivered service is a failure of the system.

The three terms are relative to the boundary under consideration. A failure of a module is, seen from the system that contains the module, a fault of that system. A disk whose head has crashed has failed as a disk; for an array of disks that stores redundant copies of the data (Section 17.8), the same event is a fault that need not lead to a failure of the array.

## 17.3 Reliability and availability

**Source** Lesson 17, videos 6–7; H&P §1.7.

With respect to its specified service, a system is at any time in one of two states. In *service accomplishment* the delivered service matches the specified service; in *service interruption* it does not. A failure moves the system from the first state to the second, and a restoration (repair) moves it back (Figure 17.2). The time spent in each state gives two different measures of dependability.

The model admits no third state: a service that is slow but still correct, or interrupted for some users only, has to be counted as one or the other.

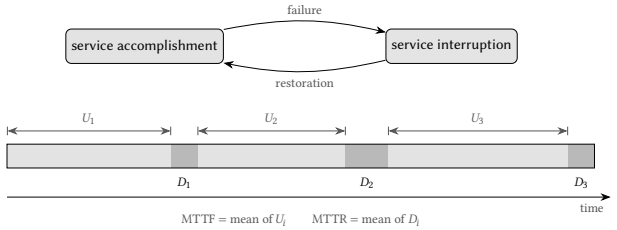



**Figure 17.2:** The two service states and a history that alternates between them. $U_i$ are the periods of service accomplishment, $D_i$ the periods of service interruption.

**Definition 17.4** (Reliability). The *reliability* of a system is a measure of *continuous* service accomplishment. It is usually quantified by the *mean time to failure* (MTTF): the average length of a period of service accomplishment, i.e. the average time from the start of service until the next failure.

**Definition 17.5** (Availability). The *availability* of a system is the fraction of time it spends in service accomplishment. With the *mean time to repair* (MTTR), the average length of a period of service interruption,

$$\text{availability} \;=\; \frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}}. \tag{17.1}$$

The sum of the two means is the average length of one complete failure–repair cycle, the *mean time between failures*:

$$\text{MTBF} \;=\; \text{MTTF} + \text{MTTR}. \tag{17.2}$$

**Caution** Disk and server datasheets print as "MTBF" the quantity defined here as MTTF: no repair time is included in it.

**Tip** Availability is quoted in "nines". Of the 8766 h in a year, 99.9 % leaves 8.8 h of interruption, 99.99 % 53 min and 99.999 % 5 min.

When repairs are short compared with the periods of service, the MTBF and the MTTF are nearly equal.

**Example 17.6.** A server runs for 1200 h, is down for 6 h, runs for 900 h, is down for 10 h, runs for 1500 h and is down for 8 h. Its MTTF is $(1200 + 900 + 1500)/3 = 1200$ h, its MTTR is $(6 + 10 + 8)/3 = 8$ h, its MTBF is 1208 h, and its availability is $1200/1208 \approx 99.34\,\%$. A second server that fails ten times as often (MTTF = 120 h) but is restored within 0.8 h on average has the same availability but only one tenth of the MTTF, so it is far less reliable.

## 17.4 Kinds of faults

Faults are classified in two ways: by their cause and by how long they last.

**Source** Lesson 17, videos 8–9; H&P §D.3.

### 17.4.1 By cause

**Hardware** A *hardware fault* occurs when hardware fails to perform as designed, e.g. a transistor that no longer switches or a disk head that touches the platter.

**Design** A *design fault* is a mistake in the design itself: a software bug, or a hardware design error such as the division bug of the early Intel Pentium, which was present in every chip built from the flawed design.

**Operation** An *operation fault* is a mistake made by the operators or users of the system, e.g. deleting the wrong file or disconnecting the wrong cable.

**Environment** An *environmental fault* originates outside the system: fire, flooding, a power outage, extreme temperature, or deliberate sabotage.

### 17.4.2 By duration

**Permanent** A *permanent fault* persists once it has occurred until the faulty part is repaired or replaced, e.g. a chip damaged by overheating.

**Intermittent** An *intermittent fault* lasts for a while, disappears, and recurs, e.g. a loose connector, or a processor running above its rated clock frequency that miscalculates only when it becomes hot.

**Transient** A *transient fault* occurs once and goes away by itself, e.g. a bit of memory flipped by a high-energy particle; rewriting the bit restores correct operation.

The two classifications describe different properties of a fault. A hardware fault, for instance, can be permanent (a burnt-out chip), intermittent (a cracked solder joint that loses contact only when warm) or transient (a flipped memory bit). A design fault, in contrast, is present in every copy of the design until the design is corrected, so it is permanent, even if it is activated only occasionally.

**Caution** Most memory errors are not of this kind: a field study of Google's servers found over 8 % of DIMMs hit by a correctable error per year, dominated by hard faults rather than particle strikes [34].

Five entries of the 2048-entry lookup table in the Pentium divider held a wrong value; replacing the chips cost Intel $475 million (Edelman, SIAM Review 39(1), 1997).

## 17.5 Improving reliability and availability

There are three ways to make a system more dependable.

**Avoid faults** Preventing faults from occurring in the first place is called *fault avoidance*: for example, using components with lower failure rates, keeping them cool, or keeping liquids away from the machines.

**Tolerate faults** Preventing faults from becoming failures is called *fault tolerance*: the system keeps delivering its specified service although some of its modules are faulty. Fault tolerance is built on redundancy, for instance error-correcting codes in memory (Section 17.7) or redundant disks (Section 17.8).

**Repair faster** Shortening the time from a failure to the restoration of service, e.g. by keeping spare parts on site.

> **Key point**
> Fault avoidance and fault tolerance reduce the number of failures, so they raise the MTTF and thereby both reliability and availability. Faster repair lowers only the MTTR; for a system that stops whenever a module fails it improves availability but not reliability.

**Source** Lesson 17, video 10; H&P §1.7.

## 17.6 Fault-tolerance techniques

General-purpose fault tolerance combines two ingredients: a way to *detect* errors and a way to *recover* from them.

**Source** Lesson 17, videos 11–13.

A processor does the same on a small scale: branch misprediction recovery (Lesson 8) discards the work done after the branch and resumes from the committed register state. Only the granularity differs.

### 17.6.1 Checkpointing

Saving the complete state of a system at regular intervals, so that execution can later resume from a saved state, is called *checkpointing*. When an error is detected, the system restores the most recent checkpoint and executes again from there. The re-execution is likely to succeed if the fault was transient, and often if it was intermittent; a permanent fault is activated again and cannot be tolerated this way. Recovery also takes time. If restoring the checkpoint and

**Caution** "Transient" describes the fault, not the error it leaves behind: the wrong value persists until something overwrites it. Re-execution helps only because the checkpoint predates the error.

repeating the lost work is quick, users perceive only a delay; if it takes too long, the recovery itself counts as a service interruption.

### 17.6.2 Redundancy

**Definition 17.7** (*N*-module redundancy). In *N-module redundancy* *N* identical modules perform the same computation on the same inputs, and their outputs are compared or put to a majority vote (Figure 17.3).

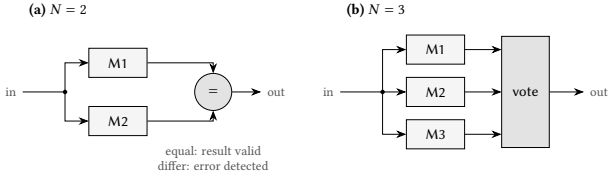



**Figure 17.3:** *N*-module redundancy. (a) Two modules and a comparator detect that one module is faulty, but not which. (b) Three modules and a majority voter mask the wrong output of any one module.

With two modules (*dual-module redundancy*), a mismatch reveals that one of them is faulty, but not which one, so the error is detected but cannot be corrected by the comparison alone. Combined with checkpointing, the system rolls back and repeats the computation. With three modules (*triple-module redundancy*), the correct result is the one produced by at least two, and a single faulty module is outvoted without any rollback—even if its fault is permanent. In general a majority vote among $N$ modules masks up to $\lfloor (N-1)/2 \rfloor$ faulty modules (Table 17.1). The price is $N$ times the hardware and energy.

**Caution** All *N* modules share one design, so a design fault is not tolerated.

**Table 17.1:** Faulty modules handled by *N*-module redundancy.

| *N* | Effect of faulty modules |
|---|---|
| 2 | one faulty module is detected (the outputs differ) but not corrected |
| 3 | one faulty module is outvoted 2:1 |
| 5 | one faulty module is outvoted 4:1, two are outvoted 3:2 |

With one faulty module, a five-module system still masks one more fault. With two faulty modules, the three correct ones still outvote them 3:2, but no margin is left: a third faulty module would outvote the two correct ones, and the vote would produce a wrong result. Operation is therefore stopped before that can happen. The Space Shuttle, with its five flight computers, followed this policy: after one computer failed the mission continued normally, and after a second one failed it was cut short.[1]

[1] Only four of the five identical IBM AP-101 machines voted on the same program; the fifth ran software written independently, against a shared design fault.

## 17.7 Codes for memory and storage

**Source** Lesson 17, video 14; H&P ch. 2.

For memory and storage, replicating whole modules is overkill. A fault typically corrupts a few of the stored bits, and those can be protected with far fewer extra bits by an error detection or correction code.

**Definition 17.8** (Parity). The *parity* of a group of data bits $d_1, \ldots, d_n$ is their exclusive OR,

$$p = d_1 \oplus d_2 \oplus \cdots \oplus d_n, \tag{17.3}$$

and a parity bit is one extra bit that stores $p$ together with the data.

When the data is read, the parity is computed again and compared with the stored bit. Flipping any single bit, including the parity bit itself, changes the result, so every single-bit error is detected. The parity does not reveal which bit flipped, so the error cannot be corrected, and two flipped bits cancel each other and go unnoticed. For example, the byte `10110010` has four ones and parity 0; if it is read back as `10100010`, the recomputed parity is 1 and the error is detected.

**Definition 17.9** (Error-correcting code). An *error-correcting code* (ECC) adds enough check bits to a group of data bits that a limited number of flipped bits can be both located and corrected.

Memory modules with ECC typically use a SECDED code (single error correction, double error detection), for example 8 check bits for every 64 data bits: any single flipped bit in a word is corrected, and any two flipped bits are detected. Disks protect each sector with considerably stronger codes, such as Reed–Solomon codes, that correct many flipped bits, in particular bursts of adjacent bits caused by defects on the recording surface. These codes handle damaged bits within a sector; they cannot help when a whole disk stops working. That case calls for redundancy across disks.

**Tip** Check bits: the smallest $r$ with $2^r \geq k + r + 1$ locates one flip among $k$ data bits, one more detects two. $k = 64$ gives 8, a 72-bit word.

## 17.8 Redundant arrays of independent disks

Several disks can be combined into a set that plays the role of a single disk, one that can be larger, faster or more reliable than any of its disks. Such a set is called a *redundant array of independent disks* (RAID). An array has two goals:

- better performance than a single disk, and
- normal read and write service even when a sector goes bad or an entire disk fails.

Not every organization pursues both goals: RAID 0 (Section 17.9) aims only at performance.

Each disk in the array still protects its sectors with its own code (Section 17.7). When a sector cannot be read correctly, or a disk does not respond at all, the array therefore *knows which* disk and which block are affected. Its redundancy does not have to find the error, only to reconstruct the contents of a known block. This is what allows a single parity block to correct errors in an array (Section 17.11), whereas a parity bit in memory can only detect them.

The RAID variants are called levels and numbered; the number identifies an organization, not a ranking. This lesson covers levels 0, 1, 4, 5 and 6.

**Source** Lesson 17, video 15; H&P §D.2; [35].

In [35] the "I" stood for *inexpensive*. Levels 2 and 3 are rarely used and are omitted here.

> **Key point**
> A parity bit only detects an error at an unknown position; a parity block reconstructs lost data at a known position. Disks report their own errors, which is what makes parity-based RAID possible.

## 17.9 RAID 0: striping

The first organization uses no redundancy at all and aims only at performance. The data of one logical disk is distributed over several physical disks so that consecutive blocks are stored on different disks, which is called *striping*. The blocks at the same position on all disks form a *stripe*. An array of $N$ disks that uses striping without any redundancy is *RAID 0*: logical block $i$ is stored on disk $i$ mod $N$ (Figure 17.4a).

**Source** Lesson 17, videos 16–18; H&P §D.2.

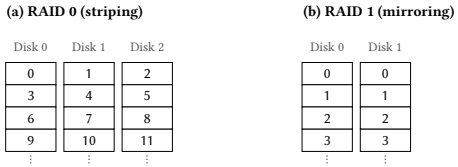



**Figure 17.4:** Placement of logical blocks (a) in RAID 0 with three disks, where each stripe holds three different blocks, and (b) in RAID 1, where both disks hold every block.

### 17.9.1 Performance

The array has $N$ times the capacity of one disk. Requests for blocks on different disks are served simultaneously, and a large transfer is split into $N$ transfers that proceed in parallel, so the throughput is up to $N$ times that of one disk. An access to a single block still needs a seek and a rotational latency on its disk (Lesson 16), so small requests complete sooner only because they spend less time waiting in a queue behind other requests. Large transfers also finish sooner, since their data moves to or from $N$ disks at once.

The unit striped is a *chunk* of tens to hundreds of kilobytes, not a single sector; each block drawn in the figures stands for one chunk.

### 17.9.2 Reliability

Real disks fail more often early and late in their life; the constant rate models the long middle part of their service life.

For reliability calculations we assume that disks fail independently of each other and that each disk has a constant *failure rate* $f_1 = 1/\mathrm{MTTF}_1$, where $\mathrm{MTTF}_1$ is the MTTF of one disk. The failure rate of a set of disks is then the sum of the individual rates. In RAID 0 each disk holds every $N$-th block of the logical disk, so the failure of any disk loses data. The array fails at the rate $N f_1$, and

$$\mathrm{MTTF}_{\mathrm{RAID\,0}} = \frac{\mathrm{MTTF}_1}{N}. \tag{17.4}$$

For an array, a failure means that data is lost, so its MTTF is also called the *mean time to data loss* (MTTDL).

**Example 17.10.** Disks with $\mathrm{MTTF}_1 = 120\,000$ h (about 13.7 years) are combined into a RAID 0 array of five disks. The array is up to five times as fast and as large as one disk, but its MTTF is only $120\,000\,\mathrm{h}/5 = 24\,000$ h, about 2.7 years.

## 17.10 RAID 1: mirroring

**Source** Lesson 17, videos 19–22; H&P §D.2.

The second organization spends disks on reliability. Writing the same data to two disks, so that each holds a complete copy of every block, is called *mirroring*. An array of two mirrored disks (Figure 17.4b) keeps all data available as long as one of its disks works; it is called *RAID 1*.

- **Writes** go to both disks. The two writes proceed in parallel, so writing is as fast as on a single disk.
- **Reads** can be served by either copy, so the read throughput is twice that of one disk.
- **Faults** confined to one disk, including the failure of the whole disk, are tolerated. With only two copies a vote is impossible, but none is needed: the disk's own code reports which copy is bad, and the other copy is used.
- **Cost**: two disks provide the capacity of one.

### 17.10.1 Reliability without repair

While both disks work, the array suffers a disk failure at the rate $2f_1$, so the first failure comes after $\mathrm{MTTF}_1/2$ on average. The array survives it with one disk left. Because the failure rate is constant, the surviving disk is as good as new, and it fails after another $\mathrm{MTTF}_1$ on average. Only then is data lost:

$$\mathrm{MTTF}_{\mathrm{RAID\,1}} = \frac{\mathrm{MTTF}_1}{2} + \mathrm{MTTF}_1 = 1.5\,\mathrm{MTTF}_1. \tag{17.5}$$

### 17.10.2 Reliability with repair

$\mathrm{MTTR}_1$ is a copy time, not a service call: a 16 TB disk refilled at 150 MB/s takes about 30 h, the figure used in the examples below.

In practice a failed disk is replaced and the data is copied onto the new disk from the surviving one. Let $\mathrm{MTTR}_1$ be the average time from the failure until the new disk holds a complete copy. Data is lost only if the second disk fails within this window (Figure 17.5 with $N = 2$). Since $\mathrm{MTTR}_1 \ll \mathrm{MTTF}_1$, the probability of that is about $\mathrm{MTTR}_1/\mathrm{MTTF}_1$, so on average $\mathrm{MTTF}_1/\mathrm{MTTR}_1$ first failures occur before one of them ends in data loss. Each is preceded by $\mathrm{MTTF}_1/2$ of both disks working, hence

$$\mathrm{MTTF}_{\mathrm{RAID\,1}} \approx \frac{\mathrm{MTTF}_1}{2} \times \frac{\mathrm{MTTF}_1}{\mathrm{MTTR}_1} = \frac{\mathrm{MTTF}_1^2}{2\,\mathrm{MTTR}_1}. \tag{17.6}$$

Datasheet MTTFs of 1 000 000–1 500 000 h imply an annual failure rate below 1 %. Replacement rates measured in the field are 2 % to 4 %, an MTTF nearer 300 000 h [36].

**Example 17.11.** With the disks of Example 17.10 ($\mathrm{MTTF}_1 = 120\,000$ h), a RAID 1 array without repair has an MTTF of 180 000 h, about 20.5 years. If a failed disk is replaced and its copy restored within $\mathrm{MTTR}_1 = 30$ h, the MTTF becomes $60\,000\,\mathrm{h} \times 4000 = 2.4 \times 10^8$ h, about 27 000 years.

> **Key point**
> Without repair, redundancy helps little: two mirrored disks last only $1.5\,\mathrm{MTTF}_1$ on average, because the first of them fails after $\mathrm{MTTF}_1/2$. With repair, it multiplies the MTTF by a factor of the order of $\mathrm{MTTF}_1/\mathrm{MTTR}_1$. In a fault-tolerant system, faster repair of a module improves the reliability of the system, not only its availability.

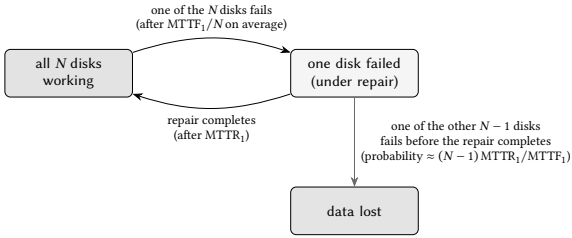



**Figure 17.5:** An array that tolerates one failed disk, with repair. Data is lost only if a second disk fails while the first is being repaired. For RAID 1, $N = 2$.

## 17.11 RAID 4: block-interleaved parity

**Source** Lesson 17, videos 23–27; H&P §D.2.

Parity provides the redundancy of mirroring at a fraction of its cost. An array of $N$ disks can use $N-1$ of them as data disks, striped as in RAID 0, and the remaining one as a parity disk, whose block in each stripe is the exclusive OR of the data blocks of that stripe (Figure 17.6a). This organization is called *RAID 4*. For a stripe with data blocks $D_0, \ldots, D_{N-2}$, applying Equation (17.3) bit by bit gives

$$P = D_0 \oplus D_1 \oplus \cdots \oplus D_{N-2}. \tag{17.7}$$

Because $x \oplus x = 0$, XORing both sides of Equation (17.7) with all surviving blocks shows that a lost block is the XOR of all other blocks of its stripe, parity included. If a disk fails, every block on it can be reconstructed from the other $N-1$ disks. If the failed disk is the parity disk, the parity is simply recomputed. The redundancy costs one disk out of $N$, much less than the second copy of mirroring.

**Caution** Coding theory calls a loss at a known position an *erasure*. One check symbol repairs one erasure, but locating *and* repairing one error of unknown position needs two.

### 17.11.1 Writes

A write must update the parity of its stripe. Recomputing it from all data blocks of the stripe would require reading $N-2$ other disks. Instead, the new parity is obtained from the old parity by removing the old data and adding the new data (Figure 17.6b):

$$P' = P \oplus D \oplus D'. \tag{17.8}$$

A write therefore reads the old data block and the old parity block, and then writes the new data block and the new parity block: two reads and two writes, independently of $N$.

This read–modify–write sequence is known as the *small-write problem*. A write that covers a whole stripe escapes it: the new parity follows from the new data alone, with no read.

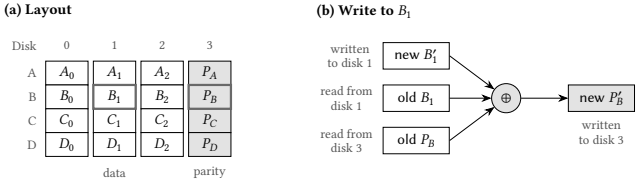



**Figure 17.6:** RAID 4. (a) Three data disks and a parity disk; rows are stripes. (b) A write to block $B_1$ reads $B_1$ and $P_B$, computes the new parity with Equation (17.8), and writes $B_1'$ and $P_B'$.

**Example 17.12.** Consider an array with three data disks and a parity disk, and one stripe with 4-bit blocks (Table 17.2). The parity is $1001 \oplus 0111 \oplus 1010 = 0100$. When disk 1 fails, its block is rebuilt as $1001 \oplus 1010 \oplus 0100 = 0111$. When 0010 is written to disk 1, Equation (17.8) gives the new parity $0100 \oplus 0111 \oplus 0010 = 0001$, which equals the XOR of the new data blocks $1001 \oplus 0010 \oplus 1010$.

### 17.11.2 Performance and reliability

- **Reads** use only the data disks, so the read throughput is $N-1$ times that of one disk.

**Table 17.2:** One stripe of RAID 4 in Example 17.12. The reconstructed and updated blocks are shown in bold.

| | Disk 0 | Disk 1 | Disk 2 | Disk 3 (parity) |
|---|---|---|---|---|
| Initial stripe | 1001 | 0111 | 1010 | 0100 |
| Disk 1 lost, rebuilt | 1001 | **0111** | 1010 | 0100 |
| After writing 0010 to disk 1 | 1001 | **0010** | 1010 | **0001** |

- **Writes** all need the parity disk, once to read and once to write. Writes cannot proceed in parallel even when they go to different data disks, and each write occupies the parity disk for two accesses. The write throughput is therefore half that of a single disk, whatever the value of $N$.
- **Reliability**: the array tolerates the failure of any one disk. Without repair the first failure comes after $\mathrm{MTTF}_1/N$ on average and the second after a further $\mathrm{MTTF}_1/(N-1)$. For $N \geq 3$ the sum is less than $\mathrm{MTTF}_1$: without repair, such an array is less reliable than a single disk. With repair, the argument of Equation (17.6) applies with $N$ disks that can fail first and $N-1$ that can fail during the repair (Figure 17.5):

$$\mathrm{MTTF}_{\mathrm{RAID\ 4}} \approx \frac{\mathrm{MTTF}_1}{N} \times \frac{\mathrm{MTTF}_1}{(N-1)\,\mathrm{MTTR}_1}. \tag{17.9}$$

## 17.12 RAID 5: distributed parity

**Source** Lesson 17, videos 28–29; H&P §D.2.

The parity disk of RAID 4 is a bottleneck for writes. The remedy is to spread the parity over all disks: each stripe still has one parity block, computed exactly as in RAID 4, but the parity blocks of different stripes are stored on different disks, rotating through all $N$ disks (Figure 17.7a). This organization is called *RAID 5*; every disk holds both data and parity.

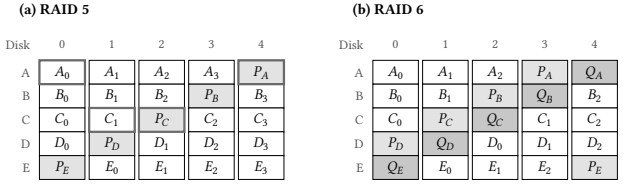



**Figure 17.7:** (a) RAID 5 with five disks. The parity block rotates among the disks; a write to $A_0$ (red) and a write to $C_1$ (green) use disjoint disks and can proceed in parallel. (b) RAID 6 with two check blocks, $P$ and $Q$, per stripe.

- **Reads**: data is spread over all $N$ disks, so the read throughput is $N$ times that of one disk.
- **Writes** still need two reads and two writes (Equation (17.8)), but the disk holding the parity depends on the stripe. Writes to different stripes usually involve different disks and proceed in parallel. The four accesses per write are spread over all $N$ disks, so the write throughput is $N/4$ times that of one disk.
- **Reliability** and **cost** are the same as for RAID 4: one failed disk is tolerated, Equation (17.9) gives the MTTF with repair, and one disk's worth of capacity holds parity.

**Example 17.13.** Six disks, each able to perform 200 accesses per second, with $\mathrm{MTTF}_1 = 120\,000\,\mathrm{h}$ and $\mathrm{MTTR}_1 = 30\,\mathrm{h}$. As RAID 4 they serve 1000 reads or 100 writes per second; as RAID 5, 1200 reads or 300 writes per second. In both cases the MTTF with repair is $(120\,000/6) \times 120\,000/(5 \times 30) = 1.6 \times 10^7\,\mathrm{h}$, about 1800 years.

> **Key point**
> RAID 5 has the reliability and the cost of RAID 4 and better read and write performance. Since it is at least as good in every respect, RAID 4 is rarely used.

## 17.13 RAID 6: two check blocks

**Source** Lesson 17, videos 30–32; H&P §D.2.

RAID 5 tolerates one failed disk; the next organization tolerates two. It adds a second check

block to every stripe (Figure 17.7b): one check block, $P$, is the parity; the other, $Q$, is computed with a different code. This organization is called *RAID 6*. $P$ and $Q$ give two independent equations per stripe, so any two lost blocks of a stripe can be reconstructed: a single lost block is rebuilt from the parity as in RAID 5, and two lost blocks are obtained by solving both equations.

- **Reliability**: the array tolerates the failure of any two disks.
- **Cost**: two disks' worth of capacity hold check blocks.
- **Writes** must update both check blocks: three reads (old data, $P$, $Q$) and three writes, compared with two of each in RAID 5. With six accesses per write spread over $N$ disks, the write throughput is about $N/6$ times that of one disk; reads remain $N$ times as fast.

$Q$ is normally a Reed–Solomon check over GF($2^8$), $Q = \sum_i g^i D_i$ for a fixed $g$. The differing weights are what make the second equation independent; a second XOR would only repeat the first.

**Tip** The write column of Table 17.3 is $N$ divided by the accesses one write costs: four for RAID 4 and 5, six for RAID 6. RAID 4 is 1/2 instead because all four hit the one parity disk.

## 17.13.1 RAID 6 and correlated failures

For independent disk failures, RAID 5 with repair already achieves an MTTF of well over a thousand years (Example 17.13). In RAID 6 data is lost only if a third disk fails while two are being repaired, which multiplies the MTTF by another factor of the order of $\mathrm{MTTF}_1/((N-2)\,\mathrm{MTTR}_1)$, about a thousand with the numbers of the example. Against independent failures, RAID 6 is overkill.

Its value lies in failures that are *not* independent of the first one. The typical case is an operation fault during the repair: the operator replaces a working disk instead of the failed one. A RAID 5 array now misses two disks and has lost data; a RAID 6 array keeps working and can still be repaired.

Repair also reads every surviving disk end to end. Consumer datasheets allow one unrecoverable read error per $10^{14}$ bits; seven 4 TB disks hold twice that, so a RAID 5 rebuild may itself lose a block.

**Table 17.3:** Comparison of the RAID levels for $N$ disks, each with $\mathrm{MTTF}_1$ and $\mathrm{MTTR}_1$. Capacity and throughput are relative to one disk. The MTTF assumes repair for RAID 1, 4, 5 and 6; RAID 0 loses data at the first failure.

| Level | Capacity | Reads | Writes | Failures tolerated | MTTF |
|---|---|---|---|---|---|
| RAID 0 | $N$ | $N$ | $N$ | 0 | $\mathrm{MTTF}_1/N$ |
| RAID 1 ($N=2$) | 1 | 2 | 1 | 1 | $\mathrm{MTTF}_1^2/(2\,\mathrm{MTTR}_1)$ |
| RAID 4 | $N-1$ | $N-1$ | $1/2$ | 1 | $\mathrm{MTTF}_1^2/(N(N-1)\,\mathrm{MTTR}_1)$ |
| RAID 5 | $N-1$ | $N$ | $N/4$ | 1 | $\mathrm{MTTF}_1^2/(N(N-1)\,\mathrm{MTTR}_1)$ |
| RAID 6 | $N-2$ | $N$ | $N/6$ | 2 | much higher than RAID 5 |

**Key point — Lesson summary**

A fault in a module becomes an error when activated and a failure when the error reaches the delivered service. Reliability is measured by the MTTF, availability by MTTF/(MTTF + MTTR). Faults are classified by cause (hardware, design, operation, environment) and by duration (permanent, intermittent, transient), and are dealt with by avoidance, tolerance and faster repair. Checkpointing and $N$-module redundancy tolerate faults in general; parity and ECC protect memory; RAID protects against failed disks. Striping (RAID 0) trades reliability for speed, mirroring (RAID 1) buys reliability with a second copy, and parity (RAID 4, 5) does so at the cost of one disk, with RAID 5 avoiding the parity-disk bottleneck. With repair, the MTTF of a redundant array grows with $\mathrm{MTTF}_1/\mathrm{MTTR}_1$ (Table 17.3); RAID 6 guards mainly against correlated failures such as mistakes during repair.

Lesson 18

# Multi-Processing

| | |
|---|---|
| **Lecture** | Lesson 18, videos 1–22 |
| **Textbook** | H&P [1] §1.1–1.2, §1.5, §5.1, ch. 3, Appendix B |
| **Keywords** | Flynn's taxonomy, MIMD, multicore processor, centralized shared memory, UMA, SMP, distributed shared memory, NUMA, message passing, shared memory, chip multiprocessor, fine-grained multithreading, simultaneous multithreading, working set |

Up to this point the course has made a single processor run a single program as fast as possible—with pipelining, branch prediction, out-of-order execution and a memory hierarchy—and Lesson 17 made such a system dependable. This lesson turns to machines that execute several programs, or several parts of one program, at the same time. It classifies parallel machines, explains why the additional transistors of each technology generation now become additional cores rather than a faster single core, compares the two ways of connecting processors to memory and the two programming models that go with them, and ends with multithreading hardware, in particular simultaneous multithreading, which lets one core execute several threads at once.

## 18.1 Flynn's taxonomy

**Source** Lesson 18, videos 1–2; H&P §1.2; [37].

A machine can do more than one thing at a time in different ways. The classic classification of parallel machines, *Flynn's taxonomy*, looks at two numbers: how many *instruction streams*—each a sequence of instructions with its own program counter—the machine executes at the same time, and how many *data streams* these instructions operate on. Distinguishing "one" from "more than one" for each number gives four classes (Table 18.1).

**Table 18.1:** Flynn's taxonomy of parallel machines.

| Class | Instruction streams | Data streams | Example |
|---|---|---|---|
| SISD | 1 | 1 | uniprocessor |
| SIMD | 1 | $> 1$ | vector processor, SSE/AVX |
| MISD | $> 1$ | 1 | stream processor (rare) |
| MIMD | $> 1$ | $> 1$ | multiprocessor, multicore |

**SISD** A *single instruction, single data* machine executes one instruction stream on one stream of data: the uniprocessor of the previous lessons. A superscalar out-of-order core is still SISD; the instructions it executes in the same cycle all belong to one program.

**SIMD** A *single instruction, multiple data* machine executes one instruction stream, but each instruction operates on several data items. Vector processors are of this kind, and so are the multimedia extensions of current instruction sets (MMX, SSE and AVX in x86), whose instructions apply one operation to every element of a short vector.

**MISD** A *multiple instruction, single data* machine executes several instruction streams on one data stream; a stream processor in which several programs each process the same incoming data would be an example. Such machines are rare.

**MIMD** A *multiple instruction, multiple data* machine executes several instruction streams, each on its own data.

A machine that consists of several processors, each executing its own program (or its own part of a program) on its own data, is a *multiprocessor*; multiprocessors are MIMD machines, and almost every parallel machine in use today belongs to this class.

## 18.2 Why multiprocessors

**Source** Lesson 18, videos 3–4; H&P §1.1, §1.5.

A uniprocessor can be made faster in two ways: by completing more instructions per cycle or by making the cycles shorter. Both have run out of room.

- **Wider.** Cores already handle four or more instructions per cycle. Programs seldom contain enough independent instructions to fill even more slots, so additional width yields diminishing returns (Lesson 6).

- **Faster.** A higher clock frequency requires a higher supply voltage. Since dynamic power is proportional to $V^2 f$ (Equation (1.1)), raising $f$ and $V$ together makes power grow roughly with $f^3$, and processors already dissipate as much power as can be removed at reasonable cost.

Clock rates stopped rising instead: the Pentium 4 reached 3.8 GHz in 2004 (H&P §1.5 shows the plateau), and the fastest desktop parts today, such as Intel's Core i9-13900KS (2023), reach about 6 GHz.

Moore's law, on the other hand, continues: the number of transistors that fit on a chip still doubles about every 18 months. Instead of a more complex single core, the transistors can be used to replicate the core. A *multicore processor* is a single chip that contains two or more complete processors, called *cores*. Doubling the number of cores in every generation can double the performance in every generation—provided that the software uses all the cores.

**Caution** Moore's own figure was one year in 1965 [2] and two years in his 1975 revision; "18 months" is a later popularization.

**Example 18.1.** A core runs at 3 GHz and dissipates 40 W of dynamic power. Running it at 3.6 GHz, 1.2× faster, requires a proportionally higher voltage and raises the power by a factor of about $1.2^3 \approx 1.73$, to 69 W. Two copies of the original core at 3 GHz dissipate 80 W and complete twice the work of one core—if the program keeps both busy. Two copies at 2.4 GHz (0.8×) dissipate $2 \times 40\,\text{W} \times 0.8^3 \approx 41\,\text{W}$, about as much as the single original core, yet together complete 1.6× its work.

## 18.3 Multiprocessors need parallel programs

**Source** Lesson 18, video 5; H&P §5.1.

A faster uniprocessor speeds up every program, including programs written long ago. A multiprocessor speeds up only programs that divide their work among several cores. Compared with sequential (single-threaded) code, such parallel programs are harder in every respect:

- **Development.** Besides solving the problem, the programmer must divide the work among threads and coordinate them.
- **Debugging.** The behavior of a parallel program depends on the relative timing of its threads, which changes from run to run, so an error may appear in one run and not in the next.
- **Performance scaling.** Ideally, performance grows linearly with the number of cores. In practice it grows up to some number of cores and then levels off; improving the program raises the level, but performance still stops growing at some point (Figure 18.1).

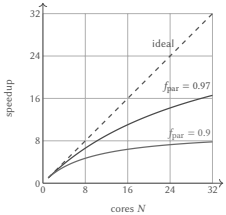



**Figure 18.1:** Speedup on $N$ cores: ideal scaling and Equation (18.1).

Amdahl's law (Equation (2.5)) shows why the levelling off is unavoidable. If a fraction $f_{\text{par}}$ of a program's execution time is divided evenly among $N$ cores and the rest remains sequential, the speedup is at most

$$S(N) \;=\; \frac{1}{(1 - f_{\text{par}}) + \dfrac{f_{\text{par}}}{N}} \;<\; \frac{1}{1 - f_{\text{par}}}. \tag{18.1}$$

Coordinating the threads costs additional time on top of this, so real programs stay below these curves.

**Tip** Half the ideal speedup, $S(N) = N/2$, needs $f_{\text{par}} = (N-2)/(N-1)$: 86 % on 8 cores, 98 % on 64.

**Example 18.2.** In a program, 95 % of the time is spent in a loop whose iterations are independent, so $f_{\text{par}} = 0.95$. On 8 cores the speedup is at most $1/(0.05 + 0.95/8) \approx 5.9$, on 32 cores $1/(0.05 + 0.95/32) \approx 12.5$, and no number of cores takes it beyond $1/0.05 = 20$.

> **Key point**
> Replicating cores turns transistors into throughput far more efficiently than building a larger or faster single core, but only for programs that are written to use many cores, and only to the extent that their sequential part allows.

## 18.4 Centralized shared memory

**Source** Lesson 18, videos 6–8; H&P §5.1.

The simplest multiprocessor connects all processors to one main memory.

**Definition 18.3** (Centralized shared memory). In a multiprocessor with *centralized shared memory* all cores, each with its own caches, are connected by one bus to a single main memory and to the I/O devices (Figure 18.2).

Any core can read and write any memory location and use any I/O device, so cores exchange data simply by writing it to memory where another core reads it. Because the memory is equally far from all cores, every core needs the same time to access a given location: the machine has *uniform memory access* (UMA). And because no core has a special role, the machine is also

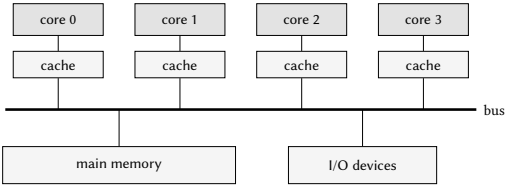



**Figure 18.2:** Centralized shared memory: all cores reach the main memory and the I/O devices through the same bus.

called a *symmetric multiprocessor* (SMP). A multicore chip whose cores share the main memory attached to it is the most common example.

Centralized shared memory does not scale to many cores, for two reasons.

- **Memory size.** One memory holds the data of all cores, so it must grow with the number of cores, and a larger memory is a slower memory (Lessons 12 and 15).
- **Memory bandwidth.** The cache misses of all cores and all I/O transfers go over the same bus. The more cores there are, the more they wait for each other's memory accesses, until memory access is effectively serialized and adding cores brings no further benefit.

Centralized shared memory therefore works well only for small machines, with about 2 to 16 cores.

## 18.5 Distributed shared memory

**Source** Lesson 18, videos 9–10; H&P §5.1.

Larger machines distribute the memory among the processors.

**Definition 18.4** (Distributed shared memory). In a multiprocessor with *distributed shared memory* each core has its own local memory and a network interface, and the cores are connected by a network (Figure 18.3). A core accesses only its local memory directly; data held by another core is obtained by sending that core a request over the network.

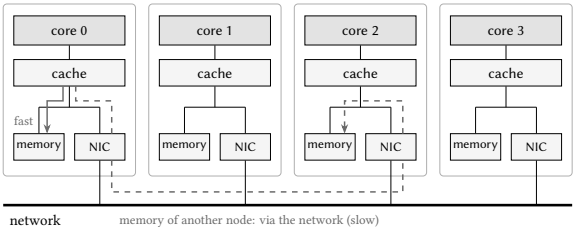



**Figure 18.3:** Distributed shared memory: each node combines a core, its cache, local memory and a network interface card (NIC).

A core with its memory and network interface is almost a computer of its own, a *node*; such a machine is therefore also called a *cluster* or a *multicomputer*. A core reaches its local memory as fast as a uniprocessor does, but data held by another node must travel over the network, which takes far longer. The access time depends on where the data is, so the machine has *non-uniform memory access* (NUMA).

Distributed shared memory scales to very large numbers of cores; clusters and supercomputers are built this way. The reason is not that the network is fast—it is much slower than a memory bus—but that no memory or bus is shared by all cores, and that the programmer, who has to request every remote access explicitly, is forced to think about communication and therefore keeps it to a minimum.

Roughly: 80 ns to local DRAM, 130 ns to the memory of the other socket, 1–2 µs to another node over InfiniBand (typical published measurements).

**Tip** On a NUMA machine, keep each thread on the node that holds the data it uses most. Linux places a page on the node whose thread first touches it; `numactl` overrides the choice.

Since a remote access costs far more than a local one, placement of data decides the performance of a NUMA machine. The operating system should place each page in the memory of the node whose core accesses it most—the private data of a thread, such as its stack, on the node where the thread runs—and keep the thread on that node, so that most memory accesses stay local. A page that several cores use, however, cannot be local to all of them.

**Note 18.5** (Terminology). This chapter uses "distributed shared memory" for every multiprocessor whose memory is divided among its nodes, although with explicit requests over a network no memory is actually shared. H&P reserve the name for machines whose nodes do share one physical address space—any core can issue any address, and the interconnect carries an access to a remote address to the node that holds it—and call machines that communicate only by explicit messages clusters. This chapter calls both kinds NUMA; in H&P's terminology NUMA refers only to the machines with a shared address space, in which the access time depends on where the data is.

## 18.6 Message passing and shared memory

A parallel program can be written in two fundamentally different ways.

**Source** Lesson 18, videos 11–14; H&P §5.1.

**Definition 18.6** (Message passing). In the *message passing* programming model each process has its own memory, which no other process can access. Processes exchange data only by explicitly sending and receiving messages.

**Definition 18.7** (Shared memory). In the *shared memory* programming model all threads share one address space and exchange data simply by writing variables that other threads read.

The two models are compared here on the same task: adding up the $N$ elements of an array with $P$ processes or threads. In both programs every process or thread executes the same code and differs only in its identifier `myPID`, which ranges from 0 to $P-1$.

### 18.6.1 A message-passing program

Each process holds only the $N/P$ elements assigned to it. Distributing the parts of the array is itself explicit work (not shown): the process that initially has the data must send each other process its part. Each process then adds up its own part, and process 0 receives and combines the partial sums of all the others (Example 18.8).

**Example 18.8** (Message passing). Every process executes the following code, with its own $N/P$ elements already in `part`:

```
#define N 4096            /* array size          */
#define P 8               /* number of processes */
double part[N/P];         /* this process's part */
double mySum = 0;
for (int i = 0; i < N/P; i++)
  mySum += part[i];
if (myPID == 0) {
  for (int p = 1; p < P; p++) {
    double pSum;
    recv(p, &pSum);       /* wait for p's sum    */
    mySum += pSum;
  }
  printf("sum: %f\n", mySum);
} else
  send(0, &mySum);        /* to process 0        */
```

### 18.6.2 A shared-memory program

With shared memory the whole array is visible to every thread, so nothing has to be distributed: each thread computes the range of indices it is responsible for and adds its partial sum to a shared total (Example 18.9). Variables declared `shared` exist once for all threads; all other variables are private to each thread.

In practice these are libraries: MPI for message passing—`MPI_Reduce` is exactly this sum—and OpenMP or pthreads for shared memory.

**Example 18.9** (Shared memory). Every thread executes the following code on the shared array:

**Caution** `shared` is notation, not C. With pthreads, globals and the heap are shared and the stack is private.

```
#define N 4096
#define P 8
shared double array[N];
shared double allSum = 0;
shared lock_t sumLock;
double mySum = 0;         /* private to the thread */
```

```
int lo = myPID * N / P;
int hi = (myPID + 1) * N / P;
for (int i = lo; i < hi; i++)
  mySum += array[i];
lock(&sumLock);
allSum += mySum;
unlock(&sumLock);
if (myPID == 0)           /* see text              */
  printf("sum: %f\n", allSum);
```

The lock is essential. The statement `allSum += mySum` reads `allSum`, adds, and writes the result back; if two threads both read the old value before either writes, one of the two partial sums is lost. The lock admits one thread at a time to the update. The program is also still incomplete: thread 0 must wait until all other threads have added their sums before it prints the total. Locks and this kind of waiting are the subject of Lesson 20.

**Tip** Combining $P$ partial sums one after another takes $P - 1$ steps; in a tree it takes $\log_2 P$.

### 18.6.3 Comparison

Table 18.2 summarizes how the two models differ. In message passing, every transfer of data and the decision which process holds which data are written by the programmer, and the hardware only has to connect otherwise independent nodes by a network. With shared memory, communication and data distribution happen implicitly through the shared variables, but the hardware must provide a shared address space, including consistent contents in the caches of all cores (Lesson 19), and support for synchronization (Lesson 20). The two models also require explicit code in different places: message passing needs code to distribute the data, but a receive already waits until the data has arrived, whereas shared memory needs no distribution code but explicit synchronization wherever a thread must wait for the others.

**Caution** That a receive waits is also a trap: if two processes each wait to receive from the other before sending, both block for ever.

**Table 18.2:** Message passing and shared memory compared.

| | Message passing | Shared memory |
|---|---|---|
| Communication | by the programmer | automatic |
| Data distribution | manual | automatic |
| Hardware support | simple | extensive |
| Programming for correctness | difficult | less difficult |
| Programming for performance | difficult | very difficult |

For the programmer the two models distribute the difficulty differently. A shared-memory program is easier to get right: a sequential program can be parallelized one loop at a time, with locks around the updates of shared data. Making it fast is much harder, because the costs of sharing do not appear in the source code—threads waiting for locks, data moving between the caches of different cores, remote accesses on a NUMA machine. A message-passing program is harder to get right, since the data distribution and every send with its matching receive must be written by hand, but once it is correct all communication is visible in the code, and much of the work of making it fast has been done along the way.

> **Key point**
> Message passing makes communication explicit: hard to program correctly, but its costs are visible. Shared memory makes communication implicit: easier to program correctly, but much harder to make fast.

## 18.7 Shared-memory hardware

**Source** Lesson 18, video 15; H&P ch. 3, §5.1.

The threads of a shared-memory program can be run by the hardware in several ways.

1. **Several cores sharing a physical address space.** Every core can issue any physical address. The memory may be centralized (UMA) or distributed among the nodes, which then share one physical address space (NUMA; see the note in Section 18.5). Each thread runs on a core of its own, and the threads truly execute at the same time.
2. **One core, time-sharing.** The operating system runs one thread for a while, then saves its state (PC and registers) and restores the state of another thread. Such *context switches* need no special hardware, and the threads automatically use the same physical memory, but only one thread executes at any moment: time-sharing gains nothing in performance.

3. **One core, hardware multithreading.** The core itself keeps the state of several threads and switches among them without the operating system. By how often the core changes threads, three variants are distinguished, each requiring more hardware support than the one before:
   - *coarse-grained multithreading* changes to another thread only after several cycles, typically when the running thread has to wait for a slow event such as a miss in the last-level cache;
   - *fine-grained multithreading* can change to another thread in every cycle;
   - *simultaneous multithreading* (SMT) executes instructions from several threads in the same cycle. Intel's Hyper-Threading is an implementation of SMT.

## 18.8 Multithreading performance

**Source** Lesson 18, videos 16–17; H&P ch. 3; [38, 39].

Consider two threads and a superscalar core that can handle four instructions per cycle. A single thread rarely fills all four slots of a cycle: dependences among its instructions, mispredicted branches and cache misses leave many slots empty, and in some cycles the thread cannot proceed at all. Figure 18.4 shows schematically how four configurations use the slots. In all four, an instruction for which no slot is free waits for the next cycle, and a thread that waits for an event, such as a cache miss, waits the same number of cycles whether or not it holds the core.

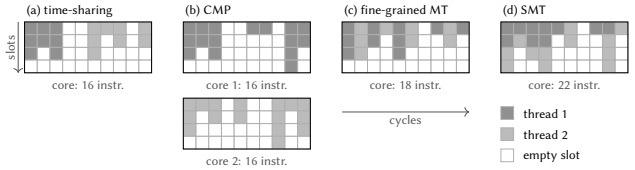



**Figure 18.4:** Schematic use of the four slots per cycle of a core by two threads over ten cycles. Each column is a cycle and each row a slot. Alone, each thread executes 16 instructions in these ten cycles (b); (a) switches threads after five cycles, (c) alternates between the threads that are not waiting.

**Time-sharing** One core runs one thread at a time. Together the threads complete no more work than a single thread, in practice slightly less, because every context switch costs time (not drawn in the figure). The cost is that of one core.

**Chip multiprocessor** Each thread runs on a core of its own. A multicore processor used in this way is a *chip multiprocessor* (CMP). Both threads run at full speed, so the throughput doubles, but so do chip area and power, and each core still leaves as many slots empty as a single thread does.

**Fine-grained multithreading** One core with a separate set of registers for each thread changes threads from cycle to cycle, so a cycle in which one thread must wait is used by the other. The core is busier, and the only additional hardware is the small per-thread state, about 5 % of the cost of a core. The empty slots within a cycle, however, stay empty.

**SMT** One core mixes instructions of both threads in the same cycle and fills many of the slots that a single thread would leave empty.[1] The throughput rises above that of fine-grained multithreading, though it stays below that of two cores, because the threads share a single set of resources. SMT needs more additional hardware than fine-grained multithreading, about 10 % of the cost of a core (Section 18.9).

How much SMT gains depends on the threads. When their instructions compete for the same execution units, the instructions of one thread find few slots left free by the other and the gain is small; SMT approaches the throughput of a CMP only when the resource demands of the two threads complement each other.

[1] SMT was proposed in 1995 [39]. Intel shipped it as Hyper-Threading in the Pentium 4 in 2002, with two threads per core—still the usual x86 number—while IBM's POWER8 and POWER9 run up to eight threads on one core. Intel then dropped SMT again from the Lion Cove cores of its 2024 client chips, judging the area better spent on more cores (IBM and Intel documentation).

**Example 18.10.** Let SMT add 10 % to the cost of a core and raise its throughput on two threads by 35 %. A CMP of two cores without SMT doubles both throughput and cost. Relative to one core that time-shares the threads, the SMT core delivers $1.35/1.10 \approx 1.23$ times the throughput per unit of cost and the CMP $2/2 = 1$, although the absolute throughput of the CMP is higher. If the two threads competed for the same execution units and SMT raised the throughput by only 5 %, the SMT core would deliver $1.05/1.10 \approx 0.95$, and two cores would be the better use of the hardware.

**Key point**
A CMP gives the highest throughput, at twice the cost. SMT gives a smaller gain for about 10 % more cost. When the resource demands of the threads complement each other, SMT is the more efficient use of the hardware; when the threads compete for the same resources, two cores can be the better choice.

## 18.9 Hardware changes for SMT

**Source** Lesson 18, video 18; H&P ch. 3; [39].

SMT can be added to the out-of-order core of Lessons 7 and 8 with few changes (Figure 18.5).

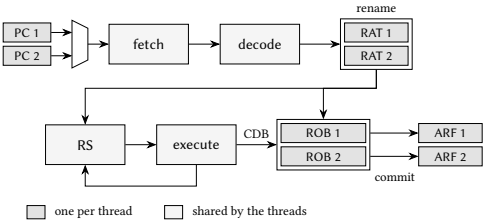



**Figure 18.5:** An out-of-order core with SMT for two threads and separate ROBs. With a shared ROB, only the program counter, the RAT and the architectural registers are kept per thread.

**Fetch** Each thread needs its own PC. The fetch stage takes instructions from both PCs and marks each fetched instruction with the thread it belongs to.

**Decode** Decoding does not depend on the thread and is unchanged.

**Rename** The renaming logic is unchanged, but each thread needs its own RAT: register R1 of thread 1 and register R1 of thread 2 are different architectural registers.

**Dispatch and execution** Unchanged. The reservation stations, the execution units and the CDB are shared; after renaming, operands are identified by tags that are unique across both threads, so instructions of different threads cannot be confused.

**ROB** Each thread can have a separate ROB with its own commit point. The ROB is a large structure, however, so duplicating it is expensive. More often in practice, the instructions of both threads are interleaved in a single shared ROB, each entry recording the thread of its instruction. The small price is that a completed instruction of one thread may have to wait for older instructions of the other thread to commit.

The Pentium 4 partitioned rather than shared: while both threads ran, each got half of the 126 ROB entries [40].

**Commit** Each thread needs its own architectural register file (ARF), into which its instructions commit.

With a shared ROB, the duplicated structures—PC, RAT and ARF—are small; the expensive parts of the core—ROB, reservation stations, execution units, caches and branch predictors—are shared. This is why SMT adds only about 10 % to the cost of a core.

Intel put Hyper-Threading on the Pentium 4/Xeon core for less than 5 % more die area and up to 30 % more throughput [40].

## 18.10 SMT, the data cache and the TLB

**Source** Lesson 18, video 19; H&P Appendix B.

The threads that share an SMT core may belong to different processes, each with its own virtual address space. The same virtual address then refers to different data in each thread, and the caches and the TLB of Lessons 13 and 14 must not confuse the two.

- A *virtually accessed cache* is indexed and tagged with the virtual address alone. A block cached for one thread produces a hit for the other thread's access to the same virtual address and returns the wrong data. Such a cache can be used with SMT only if each entry also records its thread and a hit requires the thread to match as well.
- A *VIPT cache* and a physically accessed cache compare physical tags. Two threads hit the same block only when they access the same physical address, and then they really do access the same data. These caches work correctly with SMT as long as the TLB delivers the right translation for each thread.
- The *TLB* must therefore be thread-aware: each entry records the thread it belongs to, the thread identifier of the access is passed into the lookup, and a hit requires both the thread and the virtual page number to match (Figure 18.6).

**Caution** What must differ is the address space, not the thread: two threads of one process share their translations. Real TLBs therefore tag entries with an address-space identifier—PCID on x86-64, ASID on Arm.

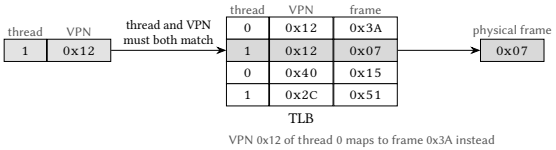



**Figure 18.6:** A thread-aware TLB. The same virtual page number is translated to different frames for the two threads.

## 18.11 SMT and cache performance

**Source** Lesson 18, videos 20–22; H&P ch. 3.

The caches of a core are shared by all its SMT threads, which has one advantage and one serious drawback.

- **Fast data sharing.** When one thread stores a value that the other thread then loads, the load finds the data already in the cache.
- **Shared capacity.** The threads also share the capacity and the associativity of the cache, and so they compete for it.

The competition is described in terms of the *working set* of a thread: the set of blocks it is actively using at a given time. Let $WS_0$ and $WS_1$ be the sizes of the working sets of two SMT threads and $WS_{01}$ the size of the part both use. As long as

$$WS_0 + WS_1 - WS_{01} \;\leq\; \text{cache size}, \tag{18.2}$$

both threads find their blocks in the cache almost as often as when running alone. When the combined working set exceeds the cache, the threads keep evicting each other's blocks and the cache *thrashes*. If each working set would fit on its own, SMT can then be slower than running the threads one at a time: a time-shared thread has the whole cache to itself for millions of cycles and refills it only after each context switch, whereas SMT threads evict each other's blocks all the time. A scheduler should therefore run together threads whose combined working set fits the cache, and run a thread whose working set alone exceeds the cache without SMT partners: it would evict their blocks without gaining anything itself.

**Caution** SMT raises throughput only if the cache holds the working sets of all threads at the same time.

**Example 18.11.** A core has a 64 KB L1 data cache. Threads A, B and C have working sets of 40 KB, 36 KB and 28 KB; each fits into the cache alone. A and B work on common data, 20 KB of which appears in both working sets, so running them together with SMT needs $40 + 36 - 20 = 56$ KB, which fits. A and C share no data and together need 68 KB; with SMT they thrash, and time-sharing them may well be faster.

Real L1 data caches are smaller: 32 KB on AMD's Zen 4, 48 KB on Intel's Golden Cove, with 1–2 MB of L2 per core behind them (AMD and Intel optimization manuals).

**Key point — Lesson summary**

Multiprocessors are MIMD machines. They have replaced ever-faster uniprocessors because wider pipelines give diminishing returns and higher frequencies cost power roughly as $f^3$, whereas replicated cores turn Moore's-law transistors into throughput—but only for parallel programs, whose speedup levels off (Equation (18.1)). Centralized shared memory (UMA, SMP) is simple but limited to a few cores by memory size and bandwidth; distributed shared memory (NUMA) scales to large machines at the price of slow remote accesses. Message passing is hard to program correctly, shared memory hard to make fast. A core can run several threads by time-sharing, by coarse- or fine-grained multithreading, or by SMT, which duplicates little more than the PC, the RAT and the ARF, needs a thread-aware TLB, and pays off only if the shared cache holds the combined working set (Equation (18.2)). Cores that share memory must also keep their caches consistent, the topic of Lesson 19.

# Cache Coherence

| | |
|---|---|
| **Lecture** | Lesson 19, videos 1–33 |
| **Textbook** | H&P [1] §5.2–5.4 |
| **Keywords** | cache coherence, bus snooping, write-update coherence, write-invalidate coherence, MSI protocol, cache-to-cache transfer, MOSI protocol, MESI protocol, MOESI protocol, directory-based coherence, coherence miss, true sharing, false sharing |

In the shared-memory multiprocessors of Lesson 18 every core has caches of its own, while all cores share one main memory. A cache holds a private copy of the data it contains: when one core writes to its copy, the copies in the other caches—and, under the write-back policy, main memory—still hold the old value. This lesson shows how such stale copies make a multiprocessor return wrong results, defines the property that rules them out, cache coherence, and develops the mechanisms that provide it: write-update and write-invalidate coherence on a shared bus, the MSI protocol and its refinements MOSI, MESI and MOESI, and directory-based coherence for larger machines. It ends with the additional cache misses that coherence causes.

**Caution** Coherence is about one location at a time; the order of accesses to *different* locations is memory consistency (Lesson 21).

## 19.1 The coherence problem

**Source** Lesson 19, videos 1–3; H&P §5.2.

Consider a multiprocessor with centralized shared memory (Section 18.4) in which each core has a private cache that follows the write-back policy of Lesson 12. In a uniprocessor this policy is harmless: main memory may hold an out-of-date value for a while, but the only processor that could read it finds the newer copy in its own cache first. With several caches this is no longer true. Figure 19.1 follows a memory location X that initially holds 5.

X is one word; caches act on its whole block—64 bytes on x86-64 and most Arm cores, 128 on Apple's M-series (Lesson 12).

1. Core 0 reads X. The read misses, and core 0's cache loads the block that contains X from memory.
2. Core 1 reads X and likewise loads a copy of the block.
3. Core 0 writes 8 to X. The write hits and, under write-back, changes only the copy in core 0's cache, which becomes dirty.
4. Core 1 reads X. The read hits in core 1's cache and returns 5, although 8 has been written.
5. Core 2 reads X. The read misses and is served by main memory, which also still holds 5.

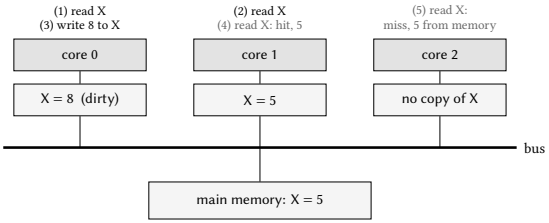



**Figure 19.1:** Incoherent caches. Cores 0 and 1 read X, which holds 5 (steps 1 and 2), and core 0 writes 8 into its cache (step 3). The later reads of core 1 (step 4) and core 2 (step 5) both return the stale value 5.

The two wrong reads have different causes. Step 5 reads a stale value from memory; the write-through policy, which updates memory on every write, would avoid it. Step 4 reads a stale copy in another cache, and no write policy of an individual cache can avoid that: however core 0's cache handles the write, the copy in core 1's cache does not change.

**Caution** Write-through does not make private caches coherent: the copies in the other caches still go stale.

## 19.2 Coherence

**Source** Lesson 19, videos 4–5; H&P §5.2.

A shared-memory program expects the behavior of a single memory that all cores share, without caches.

**Definition 19.1** (Cache coherence). A multiprocessor whose cores have private caches provides *cache coherence* if the following hold for every memory location X.

1. A read of X on a core returns the value most recently written to X by that core, if no other core has written X in between.
2. If one core writes X and another core reads X sufficiently later, with no write to X in between, the read returns the value written by the first core.
3. Writes to X are serialized: all cores observe any two writes to X in the same order.

The first condition is the behavior of memory in a uniprocessor, which every cache already preserves. The second extends it to other cores but allows a delay: a read issued while another core's write is still in progress may return the old value. The third lets all cores agree which of two nearly simultaneous writes came last. The reads of steps 4 and 5 in Figure 19.1 violate the second condition: the caches are *incoherent*.

**Note 19.2.** Coherence is a requirement on what reads return, not on what the copies contain. Copies may disagree without violating it: main memory may hold an older value than a dirty block in a cache, and a cache may still contain an old value in a line that is marked invalid. The system is coherent as long as every read is served from a copy that holds the latest value.

## 19.3 Achieving coherence

**Source** Lesson 19, video 6; H&P §5.2.

The definition and the example of Figure 19.1 show what a mechanism for coherence has to do.

1. **Make writes visible.** Under write-back a write that hits causes no activity outside the core, so the other caches cannot know that their copies have become stale. Writes to blocks that other caches may hold must therefore be announced to them.
2. **Act on the writes of other cores.** A cache that holds a copy of a block written by another core must either *update* its copy with the new value or *invalidate* it, so that its next access to the block misses and fetches the new value.
3. **Order the writes.** Writes to the same block by different cores must be applied to all copies in one order that all caches observe.
4. **Serve reads from the latest copy.** A read that misses must obtain the latest value, which may be in another cache rather than in memory.

With centralized shared memory, all caches and main memory are connected to one bus, and every operation on the bus reaches every device connected to it. Coherence can therefore be built on *bus snooping*: every cache monitors all operations on the bus, including those addressed to memory or to other caches, and reacts to those that concern blocks it holds. Snooping also orders the operations, because the bus carries one operation at a time and every cache observes them in that order. Machines without a shared bus need another mechanism for both tasks, the directory of Section 19.9. A coherence mechanism thus combines two choices: other copies are either updated or invalidated, and writes are observed and ordered either by snooping or by a directory.

## 19.4 Write-update coherence

**Source** Lesson 19, videos 7–11; H&P §5.2.

The first way of acting on a write is to propagate the new value to every copy.

**Definition 19.3** (Write-update coherence). Under *write-update coherence* every write is broadcast with its address and its new value, and every other cache that holds a copy of the written block updates its copy.

With snooping, each core simply places every write on the bus. Main memory performs the write like any other, and the other caches snoop it and update their copies if they hold the block. In Figure 19.1 the write of step 3 would then change X to 8 in memory and in core 1's cache, and both later reads would return 8. Because the bus carries one operation at a time, writes by different cores are applied to all copies in the same order, even when two cores write the same location at the same moment, as the third condition of coherence requires.

**Caution** Snooping is not a search of the other caches: each cache watches the bus and looks up only its own tags.

This basic scheme is correct but expensive: every write occupies the shared bus and is also performed in main memory—exactly the cost that the write-back policy is meant to avoid. Two optimizations remove most of it.

### 19.4.1 Avoiding memory writes

The dirty bit of Lesson 12 lets a cache defer the memory write. A write is still placed on the bus so that the other caches can update their copies, but memory ignores it; instead, the writing

cache sets the dirty bit of the block and writes the block back to memory when it evicts it. A cache that snoops another core's write to a block it holds dirty updates its copy and clears its dirty bit, so only the last writer is responsible for the block. While a dirty copy exists, memory is stale. The cache with the dirty copy therefore also snoops the bus for *reads* of that block: when another core misses on the block and places the read on the bus, this cache supplies the data, and memory does not respond.

### 19.4.2 Avoiding bus writes

Most blocks are used by only one core, and broadcasting the writes to such blocks occupies the bus for nothing. A *shared bit* in each cache line records whether other caches may hold the block. When a cache snoops a read of a block that it holds, it sets its shared bit and signals on a *shared line* of the bus that a copy exists; the reading cache sets its own shared bit if this signal is present. A write to a block whose shared bit is clear is performed locally, as in a uniprocessor; only writes to blocks whose shared bit is set are placed on the bus.

One wired-OR wire carries the shared signal: any cache that holds the block pulls it, so all caches answer at once.

The shared bit is conservative: it stays set after the other copies have been evicted.

**Example 19.4.** Two cores access a block X that no cache holds initially. Table 19.1 counts the bus operations and memory writes of six accesses under the basic scheme and with the optimizations added. With the dirty bit, memory is not written during the sequence (core 0 writes the block back once, when it evicts it), and the read of step 4 is served by core 0. With the shared bit in addition, the two writes that occur before core 1 has a copy stay local, and only the write of step 5 is broadcast.

**Table 19.1:** Write-update coherence (Example 19.4). rd: read on the bus; wr: write on the bus or into memory; rd*: read served by core 0 instead of memory.

| | | Basic | | Dirty bit | | Dirty and shared bits | |
|---|---|---|---|---|---|---|---|
| Step | Access | Bus | Memory | Bus | Memory | Bus | Memory |
| 1 | core 0 reads X | rd | — | rd | — | rd | — |
| 2 | core 0 writes X | wr | wr | wr | — | — | — |
| 3 | core 0 writes X | wr | wr | wr | — | — | — |
| 4 | core 1 reads X | rd | — | rd* | — | rd* | — |
| 5 | core 0 writes X | wr | wr | wr | — | wr | — |
| 6 | core 1 reads X | — | — | — | — | — | — |
| Total | | 5 | 3 | 5 | 0 | 3 | 0 |

## 19.5 Write-invalidate coherence

**Source** Lesson 19, videos 12–15; H&P §5.2.

The second way of acting on a write is to remove the other copies instead of updating them.

**Definition 19.5** (Write-invalidate coherence). Under *write-invalidate coherence* a core that writes to a block which other caches may hold first broadcasts an invalidation of the block, and every other cache that holds a copy of the block discards it by clearing its valid bit.

An invalidation carries only the address of the block. Once it has been broadcast, the writing cache holds the only valid copy, so further writes to the block—to any address within it—are local and need no bus operation until another core accesses the block. That access misses, because its copy has been invalidated, and obtains the latest value either from memory, if the block has been written back, or from the cache that wrote it.

### 19.5.1 Update versus invalidate

Which scheme needs fewer bus operations depends on the pattern of accesses.

- **Several writes to a block before another core reads it.** Write-update broadcasts every write; write-invalidate broadcasts a single invalidation. This holds for repeated writes to one address and equally for writes to different addresses in the same block, since invalidation works per block.
- **A thread moves to another core.** The blocks that the thread used remain in the cache of its old core, and when the thread reads them on its new core, both caches mark them shared. Under write-update, every later write of the thread to such a block is broadcast and updates a copy that the old core no longer uses, until that copy happens to be evicted.

Under write-invalidate, the first write removes the old copy, and all further writes are local.

- **Writes and reads by different cores alternate.** If one core writes a block and another core reads the new value after every write, write-update needs one bus write per write, and the reads all hit. Write-invalidate needs an invalidation per write and a miss per read.

Only the last pattern favors write-update, and the first two are the common case. Multiprocessors therefore use write-invalidate coherence, and all protocols in the rest of this lesson are based on it.

Write-update reached hardware in a few 1980s machines—Xerox PARC's Dragon, DEC SRC's Firefly—and mainstream processors since then invalidate (H&P §5.2).

**Example 19.6.** Cores 0 and 1 both hold a copy of block X. If core 0 writes X ten times and core 1 then reads X once, write-update needs 10 bus writes, and core 1's read hits; write-invalidate needs one invalidation and one bus read for core 1's miss, 2 bus operations in total. If instead core 1 reads X after each of the ten writes, write-update still needs 10 bus operations, whereas write-invalidate needs 10 invalidations and 10 misses, 20 in total.

> **Key point**
> Write-update keeps every copy current at the price of a bus operation per write; write-invalidate pays one invalidation per block and then lets the writing core proceed locally. Because writes usually come in bursts from one core, practical coherence protocols invalidate.

## 19.6 The MSI protocol

Write-invalidate coherence with snooping is described by a small state machine for each cache line. The state of the block in a line changes with the accesses of the core itself and with the accesses of other cores that the cache observes on the bus. The simplest such state machine, the *MSI protocol*, uses the valid and dirty bits of a line to distinguish three states.

**Source** Lesson 19, videos 16–19; H&P §5.2.

**M (modified)** Valid and dirty. The block has been written in this cache, and all other copies have been invalidated: this cache holds the only valid copy, which is newer than the block in memory. Reads and writes are local.

**S (shared)** Valid and not dirty. Other caches may hold valid copies as well. Reads are local; a write must first broadcast an invalidation.

**I (invalid)** The valid bit is clear. Every access misses.

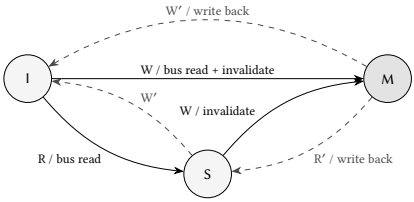



**Figure 19.2:** The MSI protocol for one block in one cache. R, W: read, write by this core; R′, W′ (dashed): read, write by another core, observed on the bus. Accesses without an edge leave the state unchanged and need no bus operation. The shaded state holds a block newer than memory.

Figure 19.2 shows the transitions. Accesses by the core itself (solid edges) are handled as follows.

- A read in I misses: the cache places the read on the bus, loads the block and enters S.
- A write in I misses too: in one bus operation, the cache reads the block and invalidates all other copies, and it enters M.
- A write in S broadcasts an invalidation and enters M.
- Reads in S, and reads and writes in M, need no bus operation.

Accesses by other cores (dashed edges) are observed on the bus.

- A write by another core, announced by its invalidation, takes S to I. The copy is discarded without being written back, since memory holds the same value.
- In M the cache holds the only up-to-date value of the block. When another core reads the block, the cache writes the block back to memory and enters S, because the other

core now holds a copy too. When another core writes the block, the cache writes the block back to memory, from which the writer obtains it, and enters I.

When a cache evicts a block, it writes the block back to memory only in M; a block in S is simply discarded.

**Tip** In MSI every edge follows one of three rules: a write by this core ends in M, a write by another core in I, a read in S.

### 19.6.1 Cache-to-cache transfers

When a core misses on a block that another cache holds in M, whether on a read or on a write, memory would answer the bus read with a stale value. The cache in M, which snoops the read, can prevent this in two ways.

1. *Abort and retry*: it aborts the read and writes the block back to memory; the reading core then issues the read again, and memory serves it with the current value. This costs a memory write and a memory read.
2. *Intervention*: it signals that memory must not answer and places the block on the bus itself. On a read miss, both copies end in S, which is not dirty, so memory takes the block from the bus as well.

Intervention, a *cache-to-cache transfer*, needs more complex bus hardware, but a single transfer serves the miss and, on a read miss, also performs the write-back that MSI requires on the transition from M to S. The dirty-bit optimization of write-update coherence (Section 19.4) answers reads in the same way, but there the cache stays dirty and memory does not take the block.

A transfer between caches costs at least an L3 hit, about 44 cycles on Skylake; a DRAM access costs 60–100 ns (Intel optimization manual; Lesson 14).

**Example 19.7.** Three cores access a block X that no cache holds initially (Table 19.2). The reads of steps 1 and 2 leave two copies in S. Core 1's write in step 3 invalidates core 0's copy, and its second write (step 4) is local. When core 2's read appears on the bus in step 5, core 1 holds the only current value; it writes the block back to memory, and both copies end in S. Core 0's write in step 6 misses and invalidates both of them. Five of the seven accesses need a bus operation, and one causes a memory write.

**Table 19.2:** Block X under the MSI protocol (Example 19.7): state of the block in each cache after the access.

| Step | Access | Core 0 | Core 1 | Core 2 | Bus and memory |
|---|---|---|---|---|---|
| 1 | core 0 reads X | S | I | I | read, served by memory |
| 2 | core 1 reads X | S | S | I | read, served by memory |
| 3 | core 1 writes X | I | M | I | invalidation |
| 4 | core 1 writes X | I | M | I | — |
| 5 | core 2 reads X | I | S | S | read; core 1 writes X back |
| 6 | core 0 writes X | M | I | I | read and invalidation |
| 7 | core 0 reads X | M | I | I | — |

## 19.7 The MOSI protocol

**Source** Lesson 19, videos 20–21.

A cache-to-cache transfer does not remove the memory write of the transition from M to S. After the transfer both copies are in S, which is not dirty, so both caches would discard their copies on eviction; unless memory takes the block during the transfer, the latest value is lost once both copies are gone. A block that one core writes and another core reads in alternation therefore costs a memory write for every write, although neither cache evicts it.

The *MOSI protocol* avoids these writes by making one of the copies of a shared block its owner.

**O (owned)** Valid; other caches may hold copies in S. The block is newer than the block in memory, and this cache is responsible for it: it answers reads of the block by other cores and writes the block back to memory when it evicts it. Reads are local; a write broadcasts an invalidation and enters M.

**Caution** "Owned" is not "exclusive": other caches keep valid copies in S. The owner is only the cache responsible for the block.
AMD processors use the O state: the AMD64 Architecture Programmer's Manual specifies MOESI, so a dirty line can pass from core to core without ever reaching memory.

When another core reads a block that the cache holds in M, the cache supplies the block by a cache-to-cache transfer and enters O instead of S, without writing memory (Figure 19.3). In O it supplies the block to every further reader. A write by the owner broadcasts an invalidation and returns to M; a write by another core takes O to I, and only if the writer missed does the owner first supply the block, since a writer in S already holds the current value. The block reaches memory only when a cache evicts it in M or O. In Example 19.7, MOSI would take core

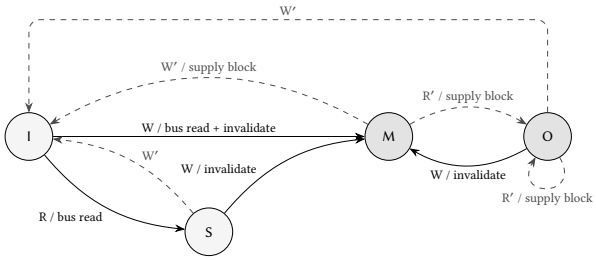



**Figure 19.3:** The MOSI protocol. Notation as in Figure 19.2; the shaded states are responsible for a block newer than memory. On W′ in O, the block is supplied only if the writer misses.

1 to O instead of S in step 5, without the memory write, and core 1 would supply the block for core 0's write in step 6.

## 19.8 The exclusive state

**Source** Lesson 19, videos 22–25; H&P §5.2; [41], the Illinois protocol.

MSI and MOSI handle a very common access pattern inefficiently. Programs often read a variable and then write it, as in an increment, and most variables are used by only one thread. For such a block the read misses and enters S, and the write then broadcasts an invalidation—although no other cache holds a copy. Every such read-write pair needs two bus operations, where a uniprocessor has only the miss.

S covers two cases that the cache cannot tell apart: other caches hold copies, or the block is unshared but not yet written. A fourth state separates them.

**E (exclusive)** Valid and not dirty; no other cache holds a copy. Reads are local, and so is a write, which enters M without any bus operation.

**Tip** Two questions fix the state: may another cache hold a copy, and must this cache write the block back? M: no, yes. E: no, no. S: yes, no. O: yes, yes.

A read in I enters E if no other cache holds the block, and S otherwise. The cache learns which case applies during the bus read itself: caches that hold the block snoop the read and signal on the shared line of the bus, as in Section 19.4. When another core reads a block held in E, the cache enters S without a write, since memory is current. A write by another core takes E to I.

Adding E to MSI gives the *MESI protocol*, and adding it to MOSI gives the *MOESI protocol* (Figure 19.4). Table 19.3 summarizes the five states.

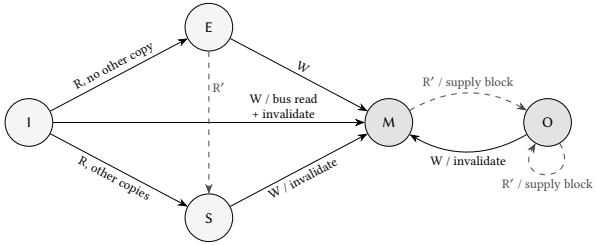



**Figure 19.4:** The MOESI protocol. Notation as in Figure 19.2. Every transition out of I performs a bus read. A write by another core takes every valid state to I; if the writer misses, a cache in M or O supplies the block (these edges are not drawn).

**Table 19.3:** The states of MOESI. MSI uses M, S and I; MOSI adds O and MESI adds E. A responsible cache answers reads by other cores and writes the block back on eviction.

| State | Valid | Copies elsewhere | Responsible | Write without bus |
|---|---|---|---|---|
| M (modified) | yes | no | yes | yes |
| O (owned) | yes | possible | yes | no |
| E (exclusive) | yes | no | no | yes |
| S (shared) | yes | possible | no | no |
| I (invalid) | no | possible | no | no |

Intel's multi-socket protocol adds a fifth state, F (forward): like O it marks the one sharer that answers reads, but F is clean (Intel QPI/UPI documentation).

**Example 19.8.** Two cores access a block X that no cache holds initially: core 0 reads and then writes X, core 1 reads it, core 0 writes it again, and core 1 reads it again. Table 19.4 gives the states and operations under the four protocols. E saves the invalidation of step 2, because core 0's read found no other copy; O saves the memory writes of steps 3 and 5, at the price of one write when core 0 eventually evicts X.

**Table 19.4:** Block X under four protocols (Example 19.8): states in core 0 and core 1 after each access. b: bus operation; m: memory write.

| | | MSI | | MOSI | | MESI | | MOESI | |
|---|---|---|---|---|---|---|---|---|---|
| Step | Access | States | Ops | States | Ops | States | Ops | States | Ops |
| 1 | core 0 reads X | S I | b | S I | b | E I | b | E I | b |
| 2 | core 0 writes X | M I | b | M I | b | M I | — | M I | — |
| 3 | core 1 reads X | S S | b, m | O S | b | S S | b, m | O S | b |
| 4 | core 0 writes X | M I | b | M I | b | M I | b | M I | b |
| 5 | core 1 reads X | S S | b, m | O S | b | S S | b, m | O S | b |
| Bus operations | | | 5 | | 5 | | 4 | | 4 |
| Memory writes | | | 2 | | 0 | | 2 | | 0 |

> **Key point**
> In all four protocols only a cache in M or O is responsible for a block newer than memory, and only a cache in M or E may write without a bus operation. The two additions are independent: the E state saves the invalidation after reading unshared data, and the O state saves the memory write when a modified block is read by another core.

## 19.9 Directory-based coherence

**Source** Lesson 19, videos 26–30; H&P §5.4; [42].

Snooping requires every cache to observe every bus operation, which ties it to a shared bus. A shared bus limits a machine to a few cores (Lesson 18), and a larger machine whose cores share one address space over a network (distributed shared memory in the sense of H&P, see the terminology note in Lesson 18) has no bus at all. Sending every read and invalidation to all caches over the network would cost far more than the operations themselves, and without a bus nothing orders the operations of different cores on a block. Larger machines therefore keep explicit track of where each block is cached.

A bus serves a handful of cores; a server socket today holds over a hundred, up to 192 on AMD EPYC 9005, connected by an on-chip network (AMD product data).

**Definition 19.9** (Directory-based coherence). Under *directory-based coherence* a *directory* records, for every block, which caches hold a copy of it. A cache sends each operation that a snooping protocol would broadcast to the directory, which forwards it only to the caches that hold the block. All such requests for a block pass through its directory entry, which thereby serializes them.

A directory entry needs little information (Figure 19.5):

- an *exclusive* bit, which indicates that a single cache holds the block, in M or E, and can write it without notifying anyone;
- one *presence* bit per cache, which indicates whether that cache holds a valid copy.

One presence bit per core per block: for 64 cores and 64-byte blocks a full directory would take some 13 % of memory (Lesson 22).

The states themselves are still kept in the caches, so the protocols of the previous sections can be used unchanged. Accesses that a protocol handles locally—reads in every valid state, writes in M and E—do not involve the directory; the others are sent to it.

- **Read miss.** If no cache holds the block, memory supplies it, and the reader becomes its only holder (in E under MESI and MOESI, with the exclusive bit set). Otherwise the directory forwards the read to a cache that holds the block, which supplies it by a cache-to-cache transfer; if the entry is exclusive, the read goes to the exclusive holder, which changes its state as on a snooped read, and the exclusive bit is cleared. In both cases the reader is added to the entry.[1]
- **Write in S or O, or write miss.** The directory forwards the invalidation only to the caches whose presence bits are set and records the writer as the exclusive holder. On a write miss, a cache that holds the block supplies it before entering I, or memory does if no cache holds it.

When a cache evicts a block, writing it back in M or O, it notifies the directory, which clears the presence bit of that cache and the exclusive bit, so that the entry remains exact.

[1] Directories are older than multicore: Stanford's DASH of the early 1990s made a coherent machine of up to 64 processors out of commercial workstations plus directory boards, and led to the SGI Origin 2000.

The directory need not reside in one place. It is divided into slices kept on different nodes, each slice serving a set of blocks, so that no single node has to handle all requests. Every request for a block goes to the slice that serves it, the home of the block, and this slice determines the order of all accesses to the block.

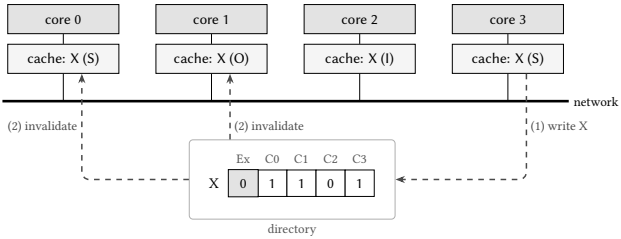



**Figure 19.5:** Directory-based coherence in step 5 of Example 19.10. The entry of X shows that cores 0, 1 and 3 hold copies and that none holds X exclusively. Core 3's write (1) is forwarded as an invalidation to cores 0 and 1 only (2).

**Example 19.10.** Four cores use MOESI with a directory (Table 19.5). Core 1 reads and then writes X; its read found no other copy, so X is in E in core 1's cache, and the write needs no message. The read of core 3 goes to the directory, which forwards it to the exclusive holder, core 1; core 1 supplies the block and becomes its owner. The directory forwards the read of core 0 to one of the holders, again core 1. When core 3 writes X in step 5, the directory sends invalidations to cores 0 and 1 only; core 2, which holds no copy, receives no message, whereas a broadcast would have reached it as well.

**Table 19.5:** Block X with directory-based MOESI (Example 19.10): cache states and directory entry (exclusive bit; presence bits of cores 0 to 3) after each access, and the messages sent (d: directory).

| | | Cache of core | | | | Entry | | |
|---|---|---|---|---|---|---|---|---|
| Step | Access | 0 | 1 | 2 | 3 | Ex | Presence | Messages |
| 1 | core 1 reads X | I | E | I | I | 1 | 0100 | 1→d, memory→1 |
| 2 | core 1 writes X | I | M | I | I | 1 | 0100 | — |
| 3 | core 3 reads X | I | O | I | S | 0 | 0101 | 3→d→1→3 |
| 4 | core 0 reads X | S | O | I | S | 0 | 1101 | 0→d→1→0 |
| 5 | core 3 writes X | I | I | I | M | 1 | 0001 | 3→d→0, 1 |
| 6 | core 3 writes X | I | I | I | M | 1 | 0001 | — |

## 19.10 Cache misses and coherence

Lesson 14 divided cache misses into compulsory, capacity and conflict misses. Invalidation adds a fourth kind.

**Source** Lesson 19, videos 31–33; H&P §5.3.

**Definition 19.11** (Coherence miss). A *coherence miss* is a miss on a block whose copy in the cache has been invalidated because another core wrote the block.

Coherence misses are of two kinds.

**True sharing** The core accesses data that the other core actually wrote. Such *true sharing* makes the miss necessary: without the miss, the core would read a stale value.

**False sharing** The other core wrote different data that merely lies in the same block. The data that the core accesses has not changed, and the miss is caused only by coherence being maintained per block rather than per address. Such *false sharing* increases with the block size, since larger blocks contain more unrelated data.

**Example 19.12.** Threads on cores 0 and 1 each increment a counter of their own, c0 and c1, 1000 times, and the increments of the two threads alternate. The counters are adjacent and lie in the same cache block (Figure 19.6). Under MOESI, core 0's first increment finds no other copy, and its write needs no invalidation. From core 1's first increment on, however,

every increment writes the block while the other core holds a copy, and invalidates it, so every increment from core 0's second one on starts with a coherence miss: 1998 misses, all due to false sharing, because neither thread ever reads the other's counter. With the counters in different blocks, only the two compulsory misses remain.

**Tip** Separate what different threads write: `alignas(64)` on x86-64; Linux's `perf c2c` finds the lines where that is missing.

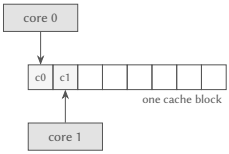



**Figure 19.6:** False sharing: counters used by different cores lie in the same cache block.

**Key point — Lesson summary**

The private caches of a shared-memory multiprocessor must be coherent: a core sees its own writes, the writes of other cores after a short time, and all writes to a location in the same order as every other core. On a shared bus, caches snoop each other's operations, and the bus orders them. Write-update coherence broadcasts every write to all copies; write-invalidate coherence broadcasts one invalidation per block and wins for the common access patterns. MSI implements write-invalidate with three states; cache-to-cache transfers serve reads from the cache that holds the current value, the O state of MOSI avoids writing that value to memory at every transfer, and the E state of MESI avoids invalidations for data used by a single core; MOESI combines both. Machines without a shared bus replace broadcasts by a directory that records which caches hold each block and orders the requests for it. Invalidation causes coherence misses, the fourth kind of miss, which stem from true or false sharing.

# Synchronization

| | |
|---|---|
| **Lecture** | Lesson 20, videos 1–23 |
| **Textbook** | H&P [1] §5.5, Appendix I.4 |
| **Keywords** | lock, mutual exclusion, critical section, spin lock, Lamport's bakery algorithm, atomic instruction, atomic exchange, test-and-set, load-linked/store-conditional, link register, cache coherence, barrier synchronization |

Lesson 19 made the caches of all cores present one coherent view of the shared memory. Coherent memory alone does not make a shared-memory program correct: when several threads read and update the same variable, the result depends on how their instructions interleave. This lesson introduces locks, which admit only one thread at a time to such an update, shows why the obvious lock built from ordinary loads and stores is incorrect and which atomic instructions processors provide instead, examines how the lock variable interacts with cache coherence, and ends with barriers, which make a group of threads wait for each other.

## 20.1 The need for synchronization

**Source** Lesson 20, videos 1–2; H&P §5.5.

Threads that share memory must coordinate their updates of shared data. As an example, consider a program that counts how often each letter occurs in a long document. Two threads each process one half of the document and increment a shared array of 26 counters (Example 20.1).

`count` is 26 ints, 104 bytes, and occupies two or three 64-byte blocks: the threads invalidate each other's copy even when they count different letters (false sharing, Lesson 19).

**Example 20.1** (Counting letters). Each thread executes the following loop over its half of the document, which contains only lowercase letters:

```
shared char doc[N];
shared int  count[26];
for (int i = lo; i < hi; i++) {  /* my half */
  int c = doc[i] - 'a';
  count[c]++;
}
```

The statement `count[c]++` is one line of source code but three instructions for the processor: a load of the counter, an addition and a store.

```
LW   R1, 0(R2)       ; R2 = address of count[c]
ADDI R1, R1, 1
SW   R1, 0(R2)
```

Each thread has its own registers, but when both threads encounter the same letter, their R2 hold the same address. If both threads load the counter before either stores the incremented value, both compute the same new value and one increment is lost (Table 20.1).

Unsynchronized accesses to one location from two threads, at least one of them a write, are a *data race* (Lesson 21): C11 and C++11 give such a program no defined behaviour.

**Table 20.1:** Both threads increment the counter of the letter e, which holds 41. After two increments it holds 42 instead of 43.

| Step | Thread A | Thread B | R1 of A | R1 of B | Counter |
|---|---|---|---|---|---|
| 1 | `LW R1, 0(R2)` | | 41 | – | 41 |
| 2 | | `LW R1, 0(R2)` | 41 | 41 | 41 |
| 3 | `ADDI R1, R1, 1` | | 42 | 41 | 41 |
| 4 | | `ADDI R1, R1, 1` | 42 | 42 | 41 |
| 5 | `SW R1, 0(R2)` | | 42 | 42 | 42 |
| 6 | | `SW R1, 0(R2)` | 42 | 42 | 42 |

Whether an increment is lost depends on the relative timing of the threads. Most of the time the two threads process different letters, or one thread finishes its three instructions before the other starts, and the counts are correct; occasionally a count comes out too low, and the error need not reappear when the program is run again. The load, the addition and the store must therefore execute as one indivisible unit with respect to every other thread that updates the same counter. Providing this is the task of *synchronization*.

The same loss occurs on a single time-shared core if a context switch falls between the load and the store.

## 20.2 Locks

**Source** Lesson 20, videos 3–4; H&P §5.5.

The basic synchronization mechanism is the lock.

**Definition 20.2** (Lock). A *lock* is a shared object with two operations. `lock` waits until the lock is free and then marks it as held by the calling thread; `unlock` marks it free again. At any time at most one thread holds a given lock.

With one lock per counter, the loop of Example 20.1 becomes

```
shared lock_t countLock[26];
  ...
  lock(&countLock[c]);
  count[c]++;
  unlock(&countLock[c]);
```

If thread B calls `lock` while thread A holds the lock, B waits until A calls `unlock`, and only then loads the counter (Figure 20.1). Thread B therefore sees the value that thread A has stored, and no increment is lost.

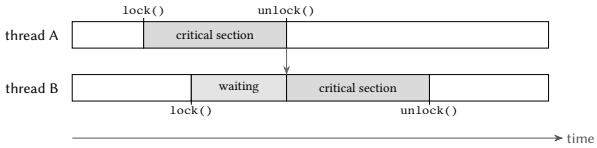



**Figure 20.1:** Two threads using the same lock: thread B waits in `lock` until thread A leaves its critical section.

**Definition 20.3** (Mutual exclusion). The guarantee that at most one thread at a time executes code protected by a given lock is called *mutual exclusion*.

For this reason a lock is also called a *mutex*.

**Definition 20.4** (Critical section). A *critical section* is a sequence of instructions that must execute under mutual exclusion; it begins with `lock` and ends with `unlock`.

A lock protects *data*, not code. Mutual exclusion holds only among threads that use the same lock, so every access to a given shared variable must be made under one and the same lock; threads that update the same variable while holding different locks do not exclude each other at all. How many variables one lock protects is a choice with performance consequences. A single lock for all 26 counters is correct, but every increment then excludes every other one, even of a different letter, and the threads spend much of their time waiting for each other. With one lock per counter, threads that increment different counters proceed in parallel, and a thread waits only while another thread updates the same counter.

**Tip** Use the same lock for every access to a given shared variable and different locks for independent variables.

## 20.3 Implementing a lock

**Source** Lesson 20, videos 5–7; H&P §5.5; [43].

A lock can be represented by a shared *lock variable* that holds 0 when the lock is free and 1 when it is held. The obvious implementation waits until the variable is 0 and then sets it to 1:

```
void lock(lock_t *l) {
  while (*l == 1)
    ;               /* wait until the lock is free */
  *l = 1;           /* take it                     */
}
void unlock(lock_t *l) {
  *l = 0;
}
```

A waiting thread keeps executing the loop, reading the lock variable over and over; a lock that waits in this way is a *spin lock*.

This implementation does not work. In machine code, testing the lock variable and setting it are separate instructions:

```
lock:  LW   R1, 0(R2)     ; R2 = address of lock variable
       BNEZ R1, lock      ; held: read it again
       ADDI R1, R0, 1
       SW   R1, 0(R2)     ; lock variable <- 1
```

If two threads both execute the LW while the lock is free, both read 0, both leave the loop, both store 1, and both enter the critical section. This is the lost update of Section 20.1 once more: the test and the set are a read and an update of a shared variable, so `lock` would itself need a critical section—the very thing it is supposed to provide.

There are two ways out.

- **Software.** A software solution is *Lamport's bakery algorithm* [43], which achieves mutual exclusion with ordinary loads and stores. It is complicated, and every acquisition executes many instructions, which makes the lock slow—too slow for locks that are acquired very frequently.
- **Hardware.** The processor provides an instruction that reads and writes the lock variable in one indivisible step.

Dekker's, Peterson's and Lamport's algorithms are correct only if loads and stores become visible in program order; otherwise they need memory fences (Lesson 21).

**Definition 20.5** (Atomic instruction). An *atomic instruction* performs several steps—here, reading a memory location and writing it—as one indivisible operation: no other instruction, on this or any other core, can access the location between the steps.

To implement a lock, the atomic step must contain both the read and the write of the lock variable. The test of the value that was read can follow in ordinary instructions, because once the variable has been written, no other thread can still read the old value.

## 20.4 Atomic read/write instructions

Instruction sets provide more than one instruction that reads and writes a memory location as one atomic step. The simplest of them swaps a register with memory.

**Source** Lesson 20, videos 8–10; H&P §5.5.

x86 has no LL/SC: it has XCHG, atomic even without the `lock` prefix, and `lock cmpxchg`, a compare-and-swap. Intel SDM vol. 2.

**Definition 20.6** (Atomic exchange). An *atomic exchange* `EXCH R1, 0(R2)` atomically writes the content of R1 into the memory location at the address in R2 and leaves the previous content of that location in R1.

A thread acquires the lock by exchanging a 1 into the lock variable until it gets a 0 back:

```
       ADDI R1, R0, 1
lock:  EXCH R1, 0(R2)    ; swap R1 and the lock variable
       BNEZ R1, lock     ; was held: try again
       ...               ; critical section
unlock: SW  R0, 0(R2)    ; lock variable <- 0
```

If the lock is held, the exchange writes 1 over 1 and returns 1, so R1 is again 1 for the next attempt. If the lock is free, the first exchange to execute returns 0 and leaves 1 behind, and every later one returns 1. Because nothing can intervene between the read and the write, two threads can never both read 0.

The exchange writes the lock variable on every attempt, also while another thread holds the lock, where the write changes nothing; Section 20.6 shows what this costs. The second instruction writes only when the value it reads passes a test.

**Definition 20.7** (Test-and-set). A *test-and-set* instruction tests a memory location and writes it only if the value passes the test. Here the test is a comparison with 0: `TSTSW R1, 0(R2)` sets Mem[R2] ← R1 and R1 ← 1 if Mem[R2] = 0, and otherwise leaves memory unchanged and sets R1 ← 0.

**Caution** Conventions differ: TSTSW returns 1 on success, while the classic test-and-set and x86's BTS return the *old* value, where 0 means success.

The instruction reports in R1 whether it has written, so the lock is taken when R1 comes back as 1. A thread whose attempt failed must put the value to be stored back into R1 before it retries:

```
lock:   ADDI R1, R0, 1   ; value to store into the lock var.
        TSTSW R1, 0(R2)  ; store if the lock variable is 0
        BEQZ R1, lock    ; did not store: try again
        ...              ; critical section
unlock: SW   R0, 0(R2)   ; lock variable <- 0
```

While another thread holds the lock, the test fails and the instruction writes nothing.

Both instructions are awkward for a load/store instruction set. Every other instruction accesses data memory at most once: a load reads and a store writes, both in the single MEM stage of the pipeline (Lesson 3). An exchange or a test-and-set reads *and* writes the same location within one instruction, so the pipeline needs a datapath with two memory accesses—in effect a second memory stage—and the processor must ensure that nothing, neither an access by another core nor an exception, comes between them.

## 20.5 Load-linked/store-conditional

**Source** Lesson 20, videos 11–13; H&P §5.5.

The alternative splits the atomic read and write into two instructions, each of which accesses memory only once, like an ordinary load or store.

**Definition 20.8** (Load-linked/store-conditional). The instruction pair *load-linked/store-conditional* (LL/SC) consists of two instructions:

`LL R1, 0(R2)` loads like LW, R1 ← Mem[R2], and records the address in the *link register* of the core;

`SC R1, 0(R2)` stores like SW, Mem[R2] ← R1, but only if the link register still holds this address; it then sets R1 to 1 to report success. Otherwise it stores nothing and sets R1 to 0.

What makes the pair atomic is the rule for clearing the link register: every write to a memory location—an SC or an ordinary store, by this core or by any other—clears every link register that holds the address of that location (Figure 20.2). A core learns of the writes of other cores through the coherence protocol of Lesson 19: the invalidation of the block that holds the linked address clears the link register. Detection therefore works per block, so a write to another address in the same block clears the link as well; this costs a retry, but never correctness.

Per H&P, a context switch between LL and SC also makes the SC fail.

An SC succeeds only if nobody has written the location since the matching LL; in particular, the value that LL read is then still in memory. The converse does not hold: writing the same value again also makes the SC fail, because LL/SC detects writes, not changes of value. Other instructions may run between LL and SC, including instructions of other threads that access the same location; if they interfere, the SC detects it and fails, and the program simply tries again. The effect is that of an atomic read and write, although neither instruction by itself does more than a normal load or store.

**Caution** In Arm, "link register" (X30) is the return-address register; the LL/SC state there is called the *exclusive monitor*.

**Tip** Only register-to-register instructions belong between LL and SC; another memory access can clear the link.

A lock built with LL/SC loads the lock variable, tests it, and tries to store 1 only if it was 0:

```
lock:   LL   R1, 0(R2)   ; R1 <- lock variable, link
        BNEZ R1, lock    ; held: try again
        ADDI R1, R0, 1   ; R1 <- 1
        SC   R1, 0(R2)   ; store 1 if still linked
        BEQZ R1, lock    ; SC failed: start over
        ...              ; critical section
unlock: SW   R0, 0(R2)   ; lock variable <- 0
```

**Example 20.9** (Two cores competing for a lock). Cores 0 and 1 execute the `lock` code above at the same time; the lock variable, at address A, is 0. Table 20.2 traces the instructions in the order in which they execute. Both cores find the lock free, but only one SC can succeed: its store clears the other core's link register. Core 1 goes back to LL, now reads 1, and waits.

RISC-V guarantees progress only for a *constrained* LR/SC loop: at most 16 instructions, with no loads, stores or backward branches between them. RISC-V ISA manual.

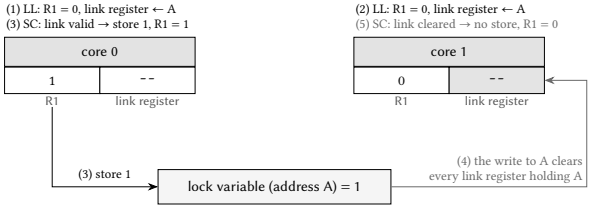



**Figure 20.2:** LL/SC on two cores. The successful store of core 0 clears the link register of core 1, whose store-conditional then fails.

**Table 20.2:** Trace of Example 20.9. A dash in a link-register column means that the register holds no address.

| | | | | Link register | | Lock |
|---|---|---|---|---|---|---|
| Step | Core | Instruction | Effect | Core 0 | Core 1 | variable |
| 1 | 0 | `LL R1, 0(R2)` | R1 = 0 | A | – | 0 |
| 2 | 1 | `LL R1, 0(R2)` | R1 = 0 | A | A | 0 |
| 3 | 0 | BNEZ, ADDI | R1 = 1 | A | A | 0 |
| 4 | 1 | BNEZ, ADDI | R1 = 1 | A | A | 0 |
| 5 | 0 | `SC R1, 0(R2)` | stores 1, R1 = 1 | – | – | 1 |
| 6 | 1 | `SC R1, 0(R2)` | no store, R1 = 0 | – | – | 1 |
| 7 | 0 | `BEQZ R1, lock` | not taken: holds lock | – | – | 1 |
| 8 | 1 | `BEQZ R1, lock` | taken: start over | – | – | 1 |
| 9 | 1 | `LL R1, 0(R2)` | R1 = 1 | – | A | 1 |
| 10 | 1 | `BNEZ R1, lock` | taken: lock is held | – | A | 1 |

A critical section that consists of a single read-modify-write needs no lock at all when LL/SC is available: the pair is applied to the shared variable itself. For the counter of Example 20.1,

```
retry:  LL   R1, 0(R2)   ; R2 = address of count[c]
        ADDI R1, R1, 1
        SC   R1, 0(R2)   ; store if nobody wrote the counter
        BEQZ R1, retry   ; someone did: read and add again
```

increments the counter atomically. If another thread writes the counter between the LL and the SC, the SC fails and the increment is repeated with the new value, so no update is lost.

The common read–modify–writes have an atomic instruction of their own, which needs no retry: `amoadd.w` in RISC-V, LDADD in Armv8.1, `lock xadd` on x86.

**Key point**
A lock needs the read and the write of the lock variable to form one atomic step. Atomic exchange and test-and-set do both in one unusual instruction; LL/SC uses two ordinary memory accesses and lets the store fail if anyone wrote the location in between, which also makes a short read-modify-write atomic without any lock.

## 20.6 Locks and performance

The lock variable is a shared variable like any other: every core that uses the lock has the block containing it in its cache, and the cache coherence protocol of Lesson 19 keeps these copies coherent. A spin lock built on atomic exchange interacts badly with coherence.

**Source** Lesson 20, videos 14–17; H&P §5.5.

EXCH writes the lock variable even when the lock is held; writing 1 over 1 changes no value, but for the coherence protocol it is a write like any other. Under the MSI protocol the writing core must have the block in the modified state, and all other copies are invalidated. While one core holds the lock, each waiting core that executes EXCH takes the block away from the core that tried before it. Almost every attempt thus causes a coherence miss, because another waiting core has written the block since this core's last attempt, and the block is carried over the bus once more (Table 20.3, top). The waiting cores accomplish nothing, yet their bus traffic costs energy and slows down the memory accesses of all other cores, including the lock holder executing its critical section, which also has to fetch the block back when it unlocks.

Taking the block out of another core's cache costs at least an L3 hit—about 44 cycles on Skylake. Intel optimization manual.

**Tip** Spin only while the expected wait is shorter than a context switch; for longer waits block instead. User-level locks spin briefly, then sleep in the kernel (on Linux through `futex`).

The remedy is to test the lock variable with an ordinary load first and to use the atomic instruction only when the lock appears free:

```
lock:   LW   R1, 0(R2)    ; ordinary read of lock variable
        BNEZ R1, lock     ; held: keep reading
        ADDI R1, R0, 1
        EXCH R1, 0(R2)    ; looks free: try to take it
        BNEZ R1, lock     ; another thread won: start over
```

Waiting cores now only read the lock variable, so all of them keep the block in the shared state and their loads are cache hits: after one miss each, the waiting cores cause no bus traffic while the lock is held (Table 20.3, bottom). Writes, and with them invalidations, happen only around a release: the store of `unlock`, and the atomic attempts of the cores that find the lock free, of which only one succeeds.

The resulting lock is known as *test-and-test-and-set*, after the plain test that now precedes the atomic one.

**Table 20.3:** MSI state of the block holding the lock variable in the caches of four cores (M, S, I: modified, shared, invalid) while core 0 holds the lock. Top: cores 1–3 spin on EXCH. Bottom: they test with LW first.

| Access | C0 | C1 | C2 | C3 | Bus traffic |
|---|---|---|---|---|---|
| *Spinning on atomic exchange* | | | | | |
| start: C0 holds the lock | M | I | I | I | |
| C1: EXCH (reads 1) | I | M | I | I | miss, block moves |
| C2: EXCH (reads 1) | I | I | M | I | miss, block moves |
| C3: EXCH (reads 1) | I | I | I | M | miss, block moves |
| C1: EXCH (reads 1) | I | M | I | I | miss, block moves |
| … one miss per attempt, until | | | | | |
| C0: SW R0 (unlock) | M | I | I | I | miss, block moves |
| *Testing first* | | | | | |
| start: C0 holds the lock | M | I | I | I | |
| C1: LW (reads 1) | S | S | I | I | miss |
| C2: LW (reads 1) | S | S | S | I | miss |
| C3: LW (reads 1) | S | S | S | S | miss |
| C1–C3: LW, repeatedly | S | S | S | S | none (hits) |
| C0: SW R0 (unlock) | M | I | I | I | invalidation |
| C2: LW (reads 0) | S | I | S | I | miss |
| C2: EXCH (reads 0, acquires) | I | I | M | I | invalidation |

Two points about the test-first lock are easy to get wrong.

- **The load is only a filter.** Mutual exclusion still comes from the atomic instruction. Between the LW that finds the lock free and the EXCH, another thread may take the lock, so the value returned by EXCH must still be tested. A lock that branches into the critical section on the result of the LW alone is the broken lock of Section 20.3.
- **The other instructions test first by themselves.** LL is a read, and the lock of Section 20.5 attempts the SC only after LL has found the lock free; a test-and-set writes only when it finds the lock free. A waiting core that keeps finding 1 never writes, so no extra load is needed.

**Caution** Hardware usually obtains the block in M for any atomic read-modify-write, even one whose test fails; there a plain load first still pays off.

`unlock`, finally, needs no atomic instruction: an ordinary store of 0 suffices. Only the thread that holds the lock executes `unlock`, so two threads never release the same lock at the same time. A thread trying to acquire the lock at that moment reads either 1, before the store, and keeps waiting, or 0, after it, and competes for the lock—both are correct outcomes. With LL/SC the store also clears the link registers of cores that have just load-linked the variable; these cores read 1 and do not attempt an SC, so they are not affected.

> **Key point**
> Correctness requires the test and the write of the lock variable to be one atomic step; performance requires the write to be rare. Waiting threads should only read the lock variable, so that its block stays shared in all caches.

## 20.7 Barrier synchronization

**Source** Lesson 20, videos 18–23; H&P Appendix I.4.

Locks make threads take turns. A different form of coordination makes threads wait for each other.

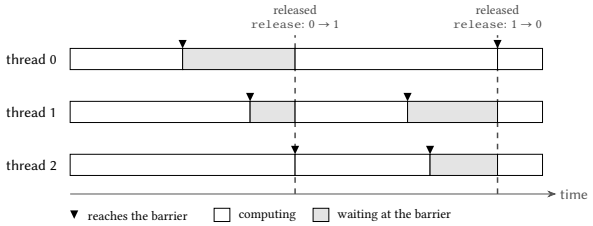



**Figure 20.3:** A barrier used in two successive rounds by three threads. Each round is released when its last thread arrives.

**Definition 20.10** (Barrier synchronization). In *barrier synchronization* a group of $N$ threads calls a *barrier* at a common point in the program. A thread that reaches the barrier waits there; when the last of the $N$ threads arrives, all of them continue (Figure 20.3).

Barriers suit parallel programs that proceed in steps, where every thread computes its part of one step and the next step needs the results of all threads. The shared-memory sum of Lesson 18 is an example: thread 0 may print the total only after all threads have added their partial sums, that is, after a barrier.

The same primitive is `pthread_barrier_wait`, the implicit barrier at the end of an OpenMP parallel region, and `MPI_Barrier`.

### 20.7.1 A simple implementation

A barrier uses two shared variables: a counter of the threads that have arrived, and a *release flag* on which the waiting threads spin.[1]

```
shared int    count = 0, release = 0;
shared lock_t countLock;
void barrier(void) {
  lock(&countLock);
  if (count == 0) release = 0;  /* first: close it */
  int arrived = ++count;        /* count myself    */
  unlock(&countLock);
  if (arrived == N) {           /* last: open it   */
    count = 0;
    release = 1;
  } else
    while (release == 0)
      ;                         /* wait            */
}
```

[1] Both variables are shared by all $N$ threads, so every barrier serializes $N$ lock acquisitions and $N$ coherence misses on one block. Scalable barriers combine the arrivals in a tree and let each thread spin on a variable of its own [44].

The counter needs the lock: a lost increment (Table 20.1) would keep it below $N$ forever. The first thread of a round clears the release flag, so that the threads of this round wait; the last one resets the counter for the next round and sets the flag, which ends the waiting loops of all the others.

A single atomic increment cannot replace the lock here, because clearing the flag must happen in the same critical section. In the reusable barrier below, one atomic increment that returns the new value (an LL/SC loop that keeps the sum in a second register) does suffice.

### 20.7.2 Why the simple implementation fails

A barrier is normally called in every step of a loop, and reused in this way the implementation above deadlocks. The flag is set while the other threads are still spinning on it, and nothing forces a thread to read it before it changes again. Suppose one thread is slow to notice the release because it is handling an interrupt. The other threads leave the barrier, compute the next step of the program and call the barrier again. The first of them to arrive finds the counter at 0 and clears the release flag. The slow thread then reads 0 and goes on waiting at a barrier that has long been released, while the other $N-1$ threads wait at the next barrier for a thread that never arrives. All $N$ threads wait forever.

A waiting thread must therefore be able to recognize that *its own* round has been released, however late it looks at the flag.

### 20.7.3 A reusable barrier

The reusable barrier never clears the release flag; it only toggles it, once per round. Each thread keeps a private variable `localSense` that it flips on entry, so that `localSense` is the

Numbering the rounds is an equivalent generalization: the sense is the parity of the round number.

value the flag will take when the current round is released; the last thread to arrive writes that value into the flag. The counter, the flag and the lock are the shared variables of the previous implementation.

```
private int localSense = 0;     /* one per thread  */
void barrier(void) {
  localSense = !localSense;     /* flip my sense   */
  lock(&countLock);
  int arrived = ++count;
  unlock(&countLock);
  if (arrived == N) {           /* last: open it   */
    count = 0;
    release = localSense;
  } else
    while (release != localSense)
      ;                         /* wait            */
}
```

**Caution** The last thread must reset the counter *before* its release becomes visible to the others; on a processor that reorders stores (Lesson 21) the two writes need a fence between them.

Because the flag is never reinitialized, the thread that was slow above still finds the value that released its round, however late it reads the flag, and leaves the barrier. A thread that reaches the next barrier while others are still waiting in this one does no harm: it only increments the counter, and its own `localSense` now holds the opposite value, so it waits for the next toggle. Successive rounds alternate between the two values, so a thread can never mistake the release of the next round for its own. In Figure 20.3, `release` becomes 1 when thread 2 completes the first round and 0 again when thread 0 completes the second.

The shared state does not grow with $N$: one counter, one flag and one lock, plus one private bit in each thread.

**Key point — Lesson summary**

Threads that update shared data must synchronize, or updates are lost depending on timing. A lock provides mutual exclusion for a critical section, and all accesses to a variable must use the same lock. The obvious lock built from ordinary loads and stores is incorrect, because its own test and set would need a critical section; software solutions such as Lamport's bakery algorithm are correct but too slow, so processors provide atomic instructions. Atomic exchange and test-and-set read and write memory in one instruction, which is awkward for a load/store pipeline; LL/SC uses two ordinary memory accesses and a link register that makes the store fail if the location was written in between, which also makes a short read-modify-write atomic without a lock. A spin lock should test the lock variable with a plain read before its atomic write, so that waiting cores keep the block shared instead of moving it between caches; unlock is a plain store. Barrier synchronization makes a group of threads wait for each other; a barrier that is reused must release each round with a flag value of its own, toggled from round to round, rather than reset a flag that a late thread has not yet seen.

# Memory Consistency

| | |
|---|---|
| **Lecture** | Lesson 21, videos 1–14 |
| **Textbook** | H&P [1] §5.6; [45] |
| **Keywords** | memory consistency, cache coherence, program order, consistency model, sequential consistency, strong consistency, load replay, relaxed consistency, MSYNC, data race, weak consistency, release consistency |

Lesson 19 made every read of a memory location return the value most recently written to that location, and Lesson 20 used locks and barriers to coordinate the threads that update shared data. Neither addresses the order in which accesses to *different* locations take effect, and an out-of-order processor changes that order as a matter of course. This lesson shows that such reordering can produce results that no execution of the program in its own order could produce, defines sequential consistency, the guarantee that rules these results out, and explains how an out-of-order processor provides it. It then turns to relaxed consistency, in which the program marks the places where order matters, and to data races, which decide whether a program can tell the difference.

## 21.1 Coherence and consistency

**Source** Lesson 21, videos 1–2; H&P §5.6.

Cache coherence (Section 19.2) constrains the accesses to one memory location: a read of the location, by any core, returns the value of the most recent write to that same location, by any core. This is what allows threads to share data at all. Coherence, however, treats each location on its own. It does not say whether a core that writes location A and then reads location B performs the two accesses in this order as far as the other cores can tell, or whether two writes of one core, to A and to B, become visible to the other cores in the order in which the core made them.

**Definition 21.1** (Memory consistency). The *memory consistency* of a shared-memory system is its guarantee about the order in which the memory accesses of all cores, including accesses to different locations, take effect. It determines which combinations of values the reads of a parallel program can return.

A write takes effect when it becomes visible to reads; a read takes effect at the moment whose memory contents it returns. A precise statement of the orders that a system allows is a *consistency model*. This lesson develops the most important model, sequential consistency, and its relaxed alternatives; Section 21.7 surveys further models.

The two properties have different origins. Coherence is needed because caches keep copies of memory: a system without caches has a single copy of every location and is coherent by construction. Consistency does not depend on caches. Even with a single copy of every location, a processor that performs its memory accesses in a different order than the program specifies changes which values its reads return—and, as the next section shows, current processors do exactly that. A coherent system is therefore not necessarily consistent.

## 21.2 Why consistency matters

**Source** Lesson 21, videos 3–5; H&P §5.6.

Whether the order of accesses to different locations matters is best seen in a small program (Example 21.2).

**Example 21.2** (Two flags). Two cores execute the same code. On core 0, R4 holds the address of a flag X and R5 the address of a flag Y; on core 1 the roles of X and Y are exchanged. Both flags are initially 0.

```
      ADDI R1, R0, 1     ; R1 <- 1
      SW   R1, 0(R4)     ; own flag <- 1
      LW   R2, 0(R5)     ; R2 <- other core's flag
      BNEZ R2, skip      ; other core has announced
      ...                ; work for one core only
skip: ...
```

Each core first announces with its own flag that it is about to do the work, and does the work only if the other core has not announced.

If each core performs its two memory accesses in program order, at most one core does the work. Suppose core 0 reads Y as 0. Then its read of Y took effect before core 1's store to Y, and therefore also before core 1's read of X, which follows that store. Core 0's store to X precedes its own read of Y, so it precedes core 1's read of X as well, and that read returns 1. (If both stores take effect before both reads, neither core does the work.)

The two-flag program is the core of Dekker's and Peterson's mutual-exclusion algorithms (Lesson 20), which use ordinary loads and stores.

An out-of-order processor does not perform the accesses in program order. A store writes memory only when it commits, whereas a load reads memory as soon as it has computed its address and found no earlier store to the same address in the load-store queue (Section 9.3). The store and the load of Example 21.2 access different addresses, so the load can read memory long before the store commits. If this happens on both cores (Figure 21.1), both loads read 0 and both cores do the work.

The distance is bounded by the reorder buffer: 224 entries on a Skylake core (Lesson 8). Agner Fog, microarchitecture manual.

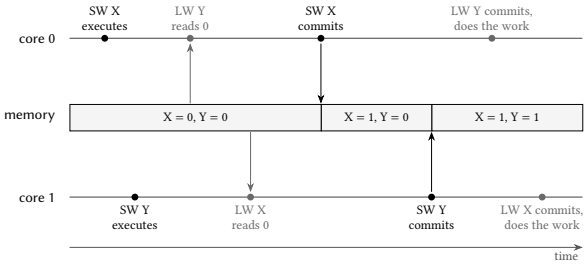



**Figure 21.1:** Example 21.2 on two out-of-order cores. Each core's load reads memory before its own store commits; both loads read 0, and both cores do the work.

**Caution** No cache holds a stale value here, so no coherence protocol can prevent this outcome: the cause is reordering, not stale data.

Every read in this execution is coherent: when each load reads its flag, the flag really holds 0, and the stores take effect at commit, in program order. The result is nevertheless impossible in any execution in program order. What the out-of-order cores have changed is the order of accesses to different locations: each core's read of the other flag took effect before its own store.

### 21.2.1 Reordered loads in correct programs

The two-flag program may look contrived, but the same kind of reordering breaks common and otherwise correct patterns. One is a *data-ready flag*: a writer thread stores new data and then sets a flag, and a reader thread waits until the flag is set and then uses the data. With R4 holding the address of the flag and R5 that of the data, the writer executes

```
      SW   R6, 0(R5)     ; data  <- new value
      SW   R7, 0(R4)     ; ready <- R7 (= 1)
```

and the reader executes

```
poll: LW   R1, 0(R4)     ; R1 <- ready
      BEQZ R1, poll      ; not set: poll again
      LW   R2, 0(R5)     ; R2 <- data
```

In program order, the reader's load of the data follows the load of the flag that found it set, which follows the writer's store to the flag, which in turn follows the writer's store of the data: the reader always obtains the new data. Branch prediction (Lesson 4) removes this guarantee. The reader's core predicts that the loop exits and executes the load of the data speculatively, possibly before the load of the flag in the same iteration. Table 21.1 traces such an execution in which the data holds 3 and the writer stores 8. The load of the flag executes after the writer has set it, so the prediction turns out to be correct, and the load of the data commits with the old value.

The same happens when a thread creates another thread, waits until that thread has exited, and then uses the results it produced. The wait ends when a load of the thread's status, which the operating system writes when the thread exits, finds the thread finished; the load of the results, which follows in program order, can execute earlier, before the other thread has written them.

Thread libraries hide this: POSIX requires `pthread_join` to order the writes of the exiting thread before the accesses of the joining one.

**Table 21.1:** The data-ready flag on an out-of-order reader core (core 0) and a writer core (core 1). Memory contents after each step.

| Step | Core | Event | ready | data |
|---|---|---|---|---|
| 1 | 0 | `LW data` executes on the predicted path: R2 = 3 | 0 | 3 |
| 2 | 1 | `SW data` commits | 0 | 8 |
| 3 | 1 | `SW ready` commits | 1 | 8 |
| 4 | 0 | `LW ready` executes: R1 = 1 | 1 | 8 |
| 5 | 0 | `BEQZ` resolves: not taken, as predicted | 1 | 8 |
| 6 | 0 | the three instructions commit with R2 = 3 | 1 | 8 |

In both patterns all caches are coherent, and the program is correct whenever its accesses take effect in program order. To be able to write such programs at all, programmers need a guarantee about the order of accesses to different locations—a guarantee beyond coherence.

> **Key point**
> Coherence orders the accesses to each location separately. Out-of-order execution and branch prediction let loads take effect before earlier accesses to other locations, which can produce results that no execution in program order could produce.

## 21.3 Sequential consistency

The most intuitive guarantee requires memory to behave as if the cores took turns.

**Source** Lesson 21, video 6; H&P §5.6; [46].

**Definition 21.3** (Sequential consistency). A shared-memory system provides *sequential consistency* if the result of every execution is the same as if the memory accesses of each core were performed in program order and the accesses of different cores were interleaved in some arbitrary order.

The definition, which goes back to Lamport [46], has two parts. Within one core the order is fixed: the accesses appear in the order that the program specifies. Across cores any interleaving is allowed; which one an execution corresponds to depends on the timing of the cores, so a correct program must produce a correct result for every interleaving. The words "as if" matter: the hardware need not actually perform the accesses in such an order, as long as no program can tell the difference.

**Example 21.4** (Interleavings of the two-flag program). In Example 21.2 core 0 stores X ($w_0$) and then reads Y ($r_0$); core 1 stores Y ($w_1$) and then reads X ($r_1$). Keeping the order of each core leaves six interleavings (Table 21.2). In every one of them at least one read returns 1, so a sequentially consistent system never lets both cores do the work: the execution of Figure 21.1 is not sequentially consistent.

**Tip** Interleavings that keep both program orders: $\binom{m+n}{m}$ for $m$ and $n$ accesses, here $\binom{4}{2} = 6$.

**Table 21.2:** The six interleavings of the accesses in Example 21.4 that keep each core's program order.

| | Order | $r_0$ (Y) | $r_1$ (X) | Does the work |
|---|---|---|---|---|
| 1 | $w_0\ r_0\ w_1\ r_1$ | 0 | 1 | core 0 |
| 2 | $w_0\ w_1\ r_0\ r_1$ | 1 | 1 | neither |
| 3 | $w_0\ w_1\ r_1\ r_0$ | 1 | 1 | neither |
| 4 | $w_1\ w_0\ r_0\ r_1$ | 1 | 1 | neither |
| 5 | $w_1\ w_0\ r_1\ r_0$ | 1 | 1 | neither |
| 6 | $w_1\ r_1\ w_0\ r_0$ | 1 | 0 | core 1 |

**Note 21.5** (Strong consistency). Sequential consistency requires only that *some* interleaving explain the results; the interleaving need not agree with the real time at which the accesses occur, so a read may return an old value although another core had already completed a write to that location. A system in which the order must also respect real time—an access that completes before another begins comes first, so that every read returns the value most recently written—provides *strong consistency*, called atomic consistency by Mosberger [45].

## 21.4 Implementing sequential consistency

### 21.4.1 The simple implementation

**Source** Lesson 21, videos 7–8; H&P §5.6.

The simplest way to provide sequential consistency is to remove the "as if" from its definition. Each core performs its memory accesses one at a time and in program order: it starts an access only when all its earlier accesses have been completed. Coherence makes each access visible to all cores as it takes effect, so the order in which the accesses of all cores take effect is itself an interleaving of the kind the definition requires.

This implementation is correct but very slow. An out-of-order core can still reorder the instructions that do not access memory, but it can have only one memory access in progress at a time: a load cannot execute until every earlier store has committed, and every instruction that uses the loaded value waits with it. Memory accesses are, however, exactly the slow operations that out-of-order execution most needs to overlap. A cache miss takes many cycles, and in a multiprocessor an access may in addition have to wait for the bus and the coherence protocol (Lesson 19). Limited to one memory access at a time, a core loses most of the benefit of out-of-order execution.

Serialized, each latency is paid in full: L1 4 cycles, L2 12, L3 44, memory 60–100 ns on a Skylake-class core (Lesson 14).

### 21.4.2 Reordering loads safely

A better implementation lets the core execute loads out of order, detects the cases in which this could violate sequential consistency, and repairs them. Two observations make this possible.

- **Only loads need attention.** Stores write memory at commit, in program order (Section 9.2), so they already take effect in program order. A load that reads the location of an earlier store of the same core obtains the store's value from the load-store queue (Lesson 9).
- **The ROB records the program order.** Whatever the order of execution, the ROB holds the instructions in program order, and a load that has executed early can be regarded as taking effect when it commits. The value it read is correct for that point unless another core has written the location in between.

Intel's counters call these memory-ordering machine clears (`MACHINE_CLEARS.MEMORY_ORDERING`); each flushes the pipeline and costs at least a branch misprediction, 16–20 cycles on Skylake (Lesson 4).

The core therefore watches the coherence traffic of the other cores for the locations of all loads that have executed but not yet committed. Under write-invalidate coherence, a write by another core arrives as an invalidation of the block that contains the location (Section 19.5). If such a write is observed, the load may have read a stale value: the load and all instructions after it are cancelled and executed again, exactly as after a branch misprediction (Section 8.5). This is always possible because the load has not committed. Once the load commits, it has taken effect in program order, and its location no longer needs to be watched.

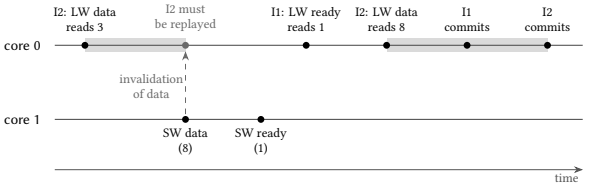



**Figure 21.2:** Replaying a reordered load. In program order core 0 executes I1: `LW ready` and then I2: `LW data`. While I2 has executed but not committed (shaded), core 1 writes the data; its invalidation forces I2 to be executed again.

Figure 21.2 applies this to the data-ready flag of Table 21.1. Core 0 executes the load of the data, I2, before the load of the flag, I1, and reads the old value 3. Core 1 then stores 8 into the data. The invalidation reaches core 0 while I2 is uncommitted, so I2 is executed again and now reads 8. I1, executed after core 1 has set the flag, reads 1, and both loads commit with the values they would have returned in program order.

A replay is needed only when another core writes a location between the execution and the commit of a load that reads it. This is uncommon in most code, but frequent in loops that poll a location another core writes, such as the reader of the data-ready flag. Because invalidations refer to whole blocks, a write to another location in the same block also causes a replay; this costs time but never produces a wrong result.

Blocks are 64 bytes on x86 and on most Arm cores, so unrelated variables share one: false sharing (Lesson 19) causes replays as well as coherence misses.

> **Key point**
> Sequential consistency does not forbid reordering memory accesses; it forbids reordering that a program could observe. An out-of-order core may execute loads early if it replays every load whose location another core writes before the load commits.

## 21.5 Relaxed consistency

Sequential consistency constrains every pair of accesses that a core makes, whether or not the program depends on their order. For two accesses that a core makes in program order, the first to location A and the second to location B, there are four kinds of ordering: WR A → WR B, WR A → RD B, RD A → WR B and RD A → RD B. Each requires the first access to take effect before the second, as observed by all cores (Table 21.3). Sequential consistency obeys all four, always.

**Source** Lesson 21, videos 9–11; H&P §5.6.

x86 relaxes only WR → RD; Arm, POWER and RISC-V relax all four. Intel SDM vol. 3A, "Memory Ordering".

**Table 21.3:** The four orderings between two accesses of one core, first to A and then to B, and what a core that violates them may do.

| Ordering | Violation | Example |
|---|---|---|
| WR A → WR B | new value of B visible before new value of A | flag set before the data is stored |
| WR A → RD B | B read before the write to A is visible | two flags (Example 21.2) |
| RD A → WR B | new value of B visible before A is read | a reply visible before the request has been read |
| RD A → RD B | B read before A is read | data-ready flag (Table 21.1) |

**Definition 21.6** (Relaxed consistency). A system with *relaxed consistency* does not always obey some of these orderings for ordinary memory accesses, and provides a way for programs to enforce the order where their correctness depends on it.

The two orderings that end in a read, WR → RD and RD → RD, are the natural candidates for relaxation. If a load may take effect before every earlier access of its core to other locations, the core of Section 21.4 needs neither to watch coherence traffic nor to replay loads because of other cores' writes: every load simply executes when it is ready, as in Figure 21.1 and Table 21.1. Relaxing RD → RD alone only shortens the window in which a load must be watched, because a load must still take effect after every earlier store of its core.

### 21.5.1 Synchronizing accesses

If even two reads may be reordered, a program needs some way to obtain a particular order where it matters. A system with relaxed consistency therefore distinguishes two kinds of operations. Ordinary loads and stores may be reordered as the model allows. Special operations, which the program uses where order matters, are never reordered. This chapter calls such an instruction *MSYNC*: every memory access that precedes an `MSYNC` in program order takes effect before any access that follows it (Figure 21.3). Between two `MSYNC` instructions, accesses may still be reordered.

x86 has such an instruction, *MFENCE*: the accesses before it become globally visible before any that follow.

**Caution** `MSYNC` is a *full* fence. Real ISAs also have partial ones—x86 `SFENCE`, Arm `DMB ISHST`—that order only some of the four pairs.

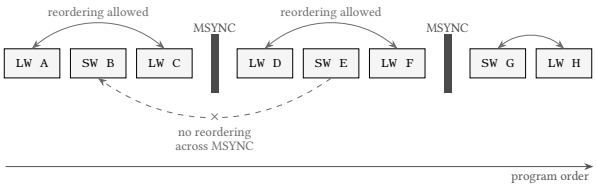



**Figure 21.3:** `MSYNC` divides the accesses of a core, in program order, into groups. Accesses to different locations within a group may take effect in a different order, as far as the model allows; no access takes effect before an access in an earlier group.

With one MSYNC on each side, the data-ready flag is correct under relaxed consistency. The writer and the reader execute

```
; writer
      SW   R6, 0(R5)    ; data  <- new value
      MSYNC             ; the data is visible
      SW   R7, 0(R4)    ; ready <- 1
; reader
poll: LW   R1, 0(R4)    ; R1 <- ready
      BEQZ R1, poll     ; not set: poll again
      MSYNC             ; the load of ready is done
      LW   R2, 0(R5)    ; R2 <- data
```

On the reader, MSYNC makes the load of the flag that found the flag set take effect before the load of the data. On the writer, it makes the store of the data take effect before the store to the flag, which a relaxation of WR → WR could otherwise make visible first. Because the reader's load of the flag returned 1, the flag, and before it the data, had been stored when that load took effect, and the load of the data comes later still. If only the orderings that end in a read are relaxed, the writer's two stores already take effect in program order and the reader's MSYNC alone suffices; the writer's MSYNC is needed when WR → WR may be relaxed as well.

The flag is a simple form of synchronization, and the placement of the two MSYNCs follows a general rule: place MSYNC *after* an operation that acquires the right to proceed and *before* an operation that releases it. A thread that uses a lock (Lesson 20) executes MSYNC right after `lock`, so that the accesses of its critical section take effect after the acquisition and see all writes of earlier lock holders, and right before `unlock`, so that all writes of its critical section have taken effect when the next thread acquires the lock. Such instructions normally belong inside the lock and barrier routines of a synchronization library rather than in every program.

**Tip** Remember the placement as two fences facing *into* the critical section: nothing may leave it at either end.

Arm folds the fence into the access itself: LDAR (acquire) and STLR (release) order one direction only; RISC-V has `.aq`/`.rl` bits.

## 21.6 Data races and consistency

Whether relaxed consistency can change the result of a program depends on how the program accesses shared data.

**Source** Lesson 21, video 12; H&P §5.6; [47].

**Definition 21.7** (Data race). Two accesses to the same memory location made by different cores form a *data race* if at least one of them is a write—RD → WR, WR → RD or WR → WR—and they are not ordered by synchronization, that is, no synchronization operation determines which of them takes effect first.

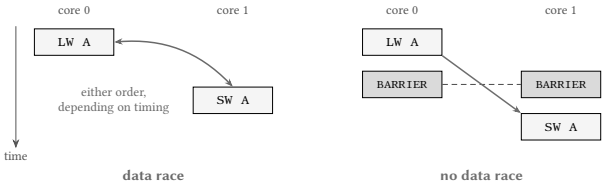



**Figure 21.4:** Left: nothing orders the load and the store of A, a data race. Right: a barrier after the load and before the store makes the load take effect first.

On the left of Figure 21.4, a load of A on core 0 and a store to A on core 1 have nothing between them; which of them takes effect first depends on the timing of the cores, and the load returns the old or the new value accordingly. On the right, core 0 reaches a barrier (Section 20.7) after its load, and core 1 reaches the same barrier before its store. No thread leaves a barrier before all threads have arrived, so the load always takes effect before the store, and there is no data race. Accesses protected by the same lock are ordered as well: one critical section ends with `unlock` before the other begins with `lock`.

Race detectors (ThreadSanitizer, Helgrind) search for exactly this: two conflicting accesses that no lock, barrier or fence orders.

The importance of data races lies in the following property [47].

- A **data-race-free program** produces the same results as under sequential consistency under every consistency model that recognizes its synchronization: one in which the synchronization operations are themselves sequentially consistent and each of them separates the earlier accesses of its core from the later ones, as MSYNC does.[1] Between two synchronization operations no other core makes a conflicting access to the locations a core uses, so reordering that core's accesses to different locations, or its reads of one

[1] Programming languages turned this into a contract. C++11 and Java guarantee that a data-race-free program behaves as under sequential consistency, and in C++ a data race makes the whole program undefined. The synchronization these models recognize is `std::atomic`—its default is `memory_order_seq_cst`, with release/acquire as the cheaper option—and the locks built on it [48].

location, does not change any value that is read; every pair of accesses whose order could change a value is ordered by synchronization, which such a model keeps in order.
- A program **with data races** has no such guarantee. On a system with relaxed consistency anything can happen, including results that no interleaving explains.

A program can therefore be developed and debugged under sequential consistency, where even a program with data races produces results that some interleaving explains, and then run under relaxed consistency for speed once synchronization has removed the races.

The examples of this lesson all contain data races. In Example 21.2 each core reads a flag that the other core writes without synchronization. The data-ready flag of Table 21.1 is a form of synchronization that the memory system does not recognize: under sequential consistency the flag orders the reader's load of the data after the writer's store, but a system with relaxed consistency does not know this. The MSYNCs of Section 21.5 make that ordering visible to it, so the load of the data no longer races with the store; the polling loads of the flag themselves remain unsynchronized, but they can only delay the reader, because the flag changes only from 0 to 1. The lost increment of Table 20.1 is also a data race. Under sequential consistency its possible results are at least limited to those of the interleavings of the two threads; under relaxed consistency even that limit disappears.

**Caution** A program that works on a sequentially consistent system can still fail under relaxed consistency if it contains a data race.

> **Key point**
> A data-race-free program whose synchronization the memory system recognizes cannot tell relaxed consistency from sequential consistency, so relaxing consistency costs a properly synchronized program nothing. A program with data races can observe anything.

## 21.7 Consistency models

Sequential consistency and the relaxed consistency of Section 21.5 are two points in a wide range of consistency models. The models differ in which orderings they relax, and in whether and how they distinguish synchronization from other accesses. Some models treat all accesses alike; from stronger to weaker:

**Source** Lesson 21, videos 13–14; [45].

**Strong consistency** Every read returns the value most recently written in real time (Section 21.3).

**Sequential consistency** Some interleaving that keeps the program order of each core explains all results.

**Causal consistency** Under *causal consistency* all cores must see causally related writes—two writes of the same core, or a write made by a core that had read, directly or indirectly, the result of the earlier one—in the same order; concurrent writes may be seen in different orders by different cores.

**Processor consistency** Under *processor consistency* the writes of one core are seen by all cores in the order in which that core made them, but different cores may see the writes of different cores in different orders, and a core's read may take effect before its own earlier write to another location.

Neither causal nor processor consistency forbids the execution of Figure 21.1: the two writes are concurrent, so each core may see the other core's write only after its own read.

Other models distinguish synchronization, and a data-race-free program that uses the synchronization they recognize behaves under them as under sequential consistency:

**Weak consistency** Under *weak consistency* accesses to synchronization variables, such as locks, are sequentially consistent and act like MSYNC: every earlier access takes effect before them and no later access does. All other accesses may be reordered freely between them.

**Release consistency** Under *release consistency* synchronization is split into acquiring and releasing, as in `lock` and `unlock`. An acquire keeps later accesses from taking effect before it, and a release waits until all earlier accesses have taken effect, so ordinary accesses are never reordered across the boundaries of a critical section—the placement of MSYNC in Section 21.5.[2]

Release consistency was defined in 1990 for the Stanford DASH multiprocessor, the machine that made the acquire/release split a hardware notion. Gharachorloo et al., ISCA 1990.

**Lazy release consistency** Under *lazy release consistency*, release consistency implemented lazily, originally for distributed shared memory built in software, the updates made in a critical section are propagated to another node only when that node next acquires the lock, not at every release.

[2] Lazy release consistency [49] was designed for software distributed shared memory on a network of workstations, where propagating an update means sending a message: the TreadMarks system fetches the changes (diffs) to a page only when a node that has acquired the lock touches it.

**Scope consistency** Under *scope consistency* each lock defines a scope: a thread that enters a critical section sees the updates made in earlier critical sections of the same lock, but

not necessarily those made under other locks [50].

The list is not complete, and new models continue to be proposed. What they share is the arrangement of this lesson: the system may reorder the accesses that its model leaves unordered, and a program that needs a particular order obtains it through synchronization.

> **Key point — Lesson summary**
> Coherence orders the accesses to one location; memory consistency concerns the order of accesses to different locations, which out-of-order execution and branch prediction change. Sequential consistency requires every execution to look as if each core's accesses were performed in program order and those of different cores interleaved. Performing accesses one at a time provides it but ruins out-of-order execution; instead, a core can execute loads early and replay any load whose location another core writes before the load commits. Relaxed consistency does not always keep the orderings between accesses and gives programs synchronizing operations, such as `MSYNC`, placed after acquiring and before releasing. A program without data races—conflicting accesses on different cores not ordered by synchronization—behaves as under sequential consistency in any model that recognizes its synchronization; a program with data races can observe anything. Weak, release, lazy release and scope consistency differ in how they recognize synchronization, and causal and processor consistency in which orders of writes all cores must agree on.

# Many Cores

| | |
|---|---|
| **Lecture** | Lesson 22, videos 1–19 |
| **Textbook** | H&P [1] §1.5, §5.4, Appendix F |
| **Keywords** | many-core processor, coherence traffic, network on chip, mesh, torus, tile, off-chip traffic, distributed last-level cache, on-chip directory, partial directory, power budget, turbo mode, logical processor |

Lessons 18–21 developed multiprocessors whose cores typically share a bus and a last-level cache, keep their caches coherent by snooping on the bus (or, in a machine without a common bus, through a directory), coordinate with locks, and rely on a consistency model for shared data. Every technology generation fits more cores on a chip, however, and mechanisms that serve four or eight cores well break down when a chip has dozens or hundreds of them. This lesson examines the challenges that such chips face—coherence traffic, off-chip traffic, the size of the coherence directory, the shared power budget and the placement of threads by the operating system—and the solution to each.

## 22.1 Coherence traffic

**Source** Lesson 22, videos 1–2; H&P ch. 5.

A *many-core processor* is a multicore processor with many cores on one chip: dozens to hundreds rather than two to eight. The first challenge of such a chip concerns cache coherence (Lesson 19). Every write to a shared location that other cores have cached causes *coherence traffic*: invalidations sent to the other caches, and coherence misses when those cores access the block again. With more cores the chip performs more writes per second in total, so the rate of coherence messages grows with the number of cores. On a shared bus this traffic soon becomes the bottleneck:

- a bus carries one transfer at a time, so the cores wait for each other's coherence messages and cache misses;
- a bus that connects more cores is longer and has more devices attached, so each transfer is slower as well.

"Hundreds" of cores have arrived: AMD's EPYC 9965 puts 192 cores in one socket within a 500 W budget, about 2.6 W per core. AMD product page.

A many-core chip therefore needs two things: an on-chip interconnect whose capacity grows with the number of cores, and a coherence mechanism that does not rely on every cache observing every operation on a single bus. The second is directory-based coherence (Section 19.9), which delivers each message only to the caches that hold the block, instead of broadcasting it to all cores.

Intel's last front-side buses ran at 1600 MT/s, 12.8 GB/s each; server chipsets had already given each socket a bus of its own before QuickPath replaced the buses with point-to-point links in 2008 (Intel chipset datasheets).

## 22.2 Network on chip

**Source** Lesson 22, videos 3–4; H&P Appendix F; [51].

The shared bus is replaced by a network.

**Definition 22.1** (Network on chip). A *network on chip* connects the cores of a chip by point-to-point links between routers, one router per core. A message travels from router to router until it reaches its destination, and messages on different links travel at the same time.

In the *mesh* topology the routers form a two-dimensional grid, and each router is linked to its neighbours to the north, south, east and west (Figure 22.1b). Any core reaches any other core over a path of several links, or *hops*, and transfers that use different links proceed at the same time, so the network as a whole carries many times the traffic of a single link. Two further properties suit the mesh to a chip.

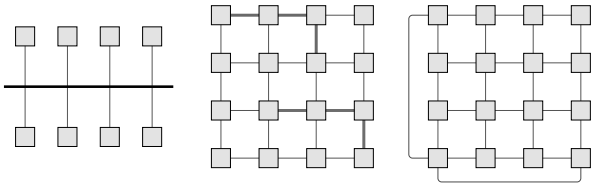
**Scaling** Adding cores enlarges the grid, and every added core adds links, so the capacity of the network grows with the number of cores, unlike that of a bus. Every link connects two neighbouring cores and stays short however large the chip, whereas a bus must span all cores.

**Planar layout** The links connect only neighbours and do not cross, so the mesh is easily laid out on the flat surface of a chip.

Such a chip is built by replicating a *tile*: a core with its private caches and its router, to which later sections add a part of the structures that all cores share.[1]

Other topologies shorten the paths at the cost of a more difficult layout. A *torus* is a mesh

[1] Intel's server chips made the same move: the Xeon Scalable family of 2017 replaced the ring of earlier Xeons by a mesh, to escape the "latencies and bandwidth constraints associated with previous ring-based architecture" (Intel, *Xeon Processor Scalable Family Technical Overview*), and every Xeon Scalable generation since has used one. Tilera's TILE64 had already put 64 tiles of processor, cache and router on a mesh in 2007.



(a) bus (b) mesh (c) torus

**Figure 22.1:** Interconnects for many cores. A bus carries one transfer at a time; the mesh carries the two highlighted transfers at the same time on different links; the torus adds a wraparound link between the first and the last router of every row and every column—only the link of the left column and the one of the bottom row are drawn.

with additional wraparound links between the first and the last router of every row and every column (Figure 22.1c), which roughly halves the longest path; the wraparound links, however, are long or cross other links. The *flattened butterfly* and the *hypercube* link each router to more routers—to all routers of its own row and column in a flattened butterfly, to $\log_2 N$ routers in a hypercube—which shortens the paths further but requires long, crossing links that are hard to build on a chip.

Off the chip they are practical: IBM's Blue Gene/L used a 3-D torus and Blue Gene/Q a 5-D one, and Cray's XC supercomputers a dragonfly network. IBM/Cray system descriptions.

If the traffic were spread evenly over the links, a network of $L$ links, each carrying up to $C$ messages per second, would carry

$$\text{throughput} \approx \frac{L \cdot C}{h} \tag{22.1}$$

messages per second, where $h$ is the average number of hops per message, because a message occupies a link on every hop. A $k \times k$ mesh has $L = 2k(k-1)$ links, and for a destination drawn at random among the other tiles the average distance is $h = 2k/3$ hops, which grows only with the square root of the number of cores $N = k^2$.

**Tip** Along one dimension of a $k$-node line the mean distance is $(k^2-1)/3k$; two dimensions, with the source tile itself excluded, give exactly $2k/3$.

**Caution** A mesh raises throughput but does not shorten latency: a message to a distant tile passes many routers, each adding delay.

Equation (22.1) is an average-load estimate that no real routing reaches. The load on a link is the sum of the rates of all messages routed over it, and the network carries the offered traffic only while every link stays below $C$, so the busiest link, not the average, sets the limit. When messages are addressed all over the chip and the routers send each message along one dimension and then the other, the links that cross the middle of the mesh are the busiest: about half of all messages cross these $k$ links, so the network saturates when it carries about $2kC$ messages per second. The capacity of a mesh therefore grows with $\sqrt{N}$ for such traffic, and in proportion to $N$ only while messages stay among neighbouring tiles; a bus, by contrast, never carries more than one link's worth of traffic however many cores it connects. Since the capacity per core falls as $1/\sqrt{N}$ once traffic crosses the chip, keeping a core's data in its own tile matters (Section 22.3).

**Example 22.2.** Let each link, and the bus, carry $C$ messages per second. With 16 cores the bus carries $C$; a $4 \times 4$ mesh has 24 links and an average distance of $8/3 \approx 2.67$ hops, for which Equation (22.1) estimates $24C/2.67 \approx 9C$, while its middle links limit it to about $8C$. With 64 cores the bus still carries at most $C$ (in practice less, since it is longer), while an $8 \times 8$ mesh has 112 links and an average distance of $16/3 \approx 5.33$ hops, which the estimate turns into $21C$ and the middle links into about $16C$: four times the cores, twice the capacity. If all traffic went to neighbouring tiles ($h = 1$), the two meshes would carry $24C$ and $112C$.

## 22.3 Off-chip traffic and the distributed LLC

**Source** Lesson 22, videos 5–7.

The second challenge is the traffic that leaves the chip. Each core has its own caches, and the miss rate per core does not change with the number of cores, so a chip with more cores sends proportionally more requests to main memory. These requests pass through the pins of the chip; the number of pins also grows from generation to generation, but far more slowly than the number of cores, so the memory interface becomes a bottleneck. The number of memory requests *per core* must therefore go down: more accesses must be served on the chip, by a last-level cache that all cores share and whose size grows roughly in proportion to the number of cores. A single large LLC, however, is slow, since a larger cache has a longer hit time (Lesson 12), and it has a single entry point through which the L2 misses of all cores must pass, which is a bottleneck of its own.

**Definition 22.3** (Distributed last-level cache). A *distributed last-level cache* is logically a single cache shared by all cores—each block is stored in it at most once, not once per core—but is physically divided into *slices*, one slice in each tile (Figure 22.2).

The size of the LLC is the number of cores times the size of a slice, so it grows automatically with the number of cores, while each slice remains small and fast and the requests of the cores are spread over many slices.

Intel's server LLC is sliced exactly so: 1.375 MB per core on the Xeon Scalable family, 38.5 MB over the 28 cores of a Platinum 8180. Intel ARK.

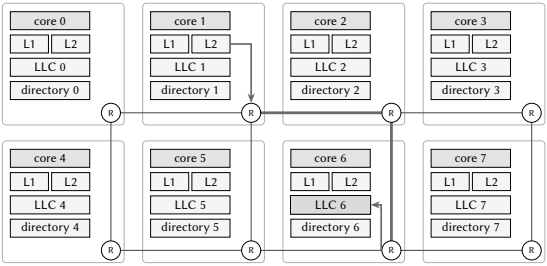



**Figure 22.2:** Eight tiles connected by a mesh. Each tile holds a core, its L1 and L2 caches, one slice of the distributed LLC, a part of the on-chip directory (Section 22.4) and a router (R). An L2 miss of core 1 on a block whose set number ends in binary 110 is sent to LLC slice 6.

On an L2 miss the core must send its request to the one slice that may hold the block, and it must be able to compute that slice from the address alone. The blocks are distributed among the tiles in round-robin fashion, in one of two ways.

**By cache index** The lowest bits of the set number select the tile, so consecutive sets go to consecutive tiles. The blocks are spread evenly, but without regard to which core uses them: the blocks of a thread are scattered over all tiles, and most of its LLC accesses travel to a distant tile.

**By page number** Bits of the physical page number select the tile, so all blocks of a page are in the same slice. The operating system decides which physical frame each page receives; it can give a thread frames whose slices lie in or near the thread's own tile, and most LLC accesses stay local—the same idea as keeping a thread's memory on its own node in a NUMA machine (Section 18.5).

**Caution** Real chips do not take a field of the address: Intel selects the slice with an undocumented hash that XORs many physical address bits, so that even strided accesses spread over the slices [52].

**Example 22.4.** A chip has 32 tiles with a 1 MB LLC slice each, so its LLC holds 32 MB. With 64 B blocks and 8-way set associativity, the LLC has $2^{25}/(2^6 \cdot 2^3) = 2^{16}$ sets, of which each slice holds $2^{11} = 2048$. If the tile is selected by cache index, the 16-bit set number consists of address bits 6–21: its lowest 5 bits (address bits 6–10) select one of the 32 tiles, the remaining 11 bits (address bits 11–21) select the set within that slice, and the tag consists of the address bits from bit 22 up, as in any cache with $2^{16}$ sets.

## 22.4 The on-chip directory

**Source** Lesson 22, videos 8–11; H&P §5.4; [42].

Directory-based coherence brings a challenge of its own. The directory of Section 19.9 has an entry for every memory block, and each entry has one presence bit per core plus an exclusive bit. Main memory of many gigabytes consists of hundreds of millions of blocks, and a directory with an entry for each of them does not fit on a chip.

**Example 22.5.** A 64-core system has 32 GB of memory divided into 64 B blocks, that is, $2^{35}/2^6 = 2^{29} \approx 537$ million blocks. At 65 bits per entry the full directory needs about $3.5 \times 10^{10}$ bits, or 4.1 GB—about 13 % of main memory itself.

Two decisions adapt the directory to the chip. The first is where an entry is kept. The directory is divided among the tiles, and the entry of a block resides in the tile that holds the block's LLC slice, its *home* tile. A core whose private caches miss must contact that tile anyway to search the LLC, so it reaches the directory entry without an additional message; no tile becomes a central bottleneck either. The second decision, the one that makes the directory small enough, is which blocks have an entry at all.

**Definition 22.6** (On-chip directory). An *on-chip directory* is a coherence directory kept on the chip: it is distributed over the tiles, the entry of each block residing in the tile of the block's

LLC slice, and it is a *partial directory*, which has a fixed number of entries in each tile and holds entries only for some of the blocks.

An entry is needed only for a block that is held by at least one private cache (L1 or L2), that is, whose entry would have at least one presence bit set.[2] A block that is only in the LLC or in main memory would have an entry of zeros, and a missing entry says the same thing. The directory therefore allocates an entry when a private cache obtains a block that has none, and the entry is no longer needed once no private cache holds the block. When a new entry is needed but all entries of the tile are in use, the directory replaces one:

1. it selects a victim entry, for example the least recently used one;
2. it sends an invalidation to every tile whose presence bit is set in the victim entry; these caches discard their copies of the block (a modified copy is first written back);
3. it uses the entry for the new block.

The invalidated caches miss the next time they access the block, although no other core has written to it. Directory replacement thus causes a new kind of cache miss, in addition to the misses of the 3C model (Lesson 14) and the coherence misses of Lesson 19. The more entries the directory has, the rarer such misses become.

[2] Intel's Xeon Scalable family needed exactly this structure. Its earlier LLC was inclusive, so the LLC tags themselves recorded which private caches held a block; the 2017 redesign made the LLC non-inclusive (1.375 MB per core against 1 MB of L2) and added a separate *snoop filter* to track "cache lines in the L1 or MLC of cores when it is not allocated in the LLC" (Intel, *Xeon Processor Scalable Family Technical Overview*).

The partial directory behaves like a cache of directory entries, with its own replacement policy.

**Example 22.7.** Table 22.1 traces the directory part of one tile with room for two entries in a chip with four cores, C0–C3. Blocks A, B and C all have this tile as their home tile, and initially no private cache holds them.

**Table 22.1:** A partial directory with two entries. Each entry shows the block, the exclusive bit and the presence bits of C0–C3.

| Step | Event | Entry 1 | Entry 2 | Directory action |
|---|---|---|---|---|
| 1 | C0 reads A | A 1 1000 | — | allocate entry 1 |
| 2 | C2 reads A | A 0 1010 | — | forward read to C0 |
| 3 | C1 writes B | A 0 1010 | B 1 0100 | allocate entry 2 |
| 4 | C3 reads C | C 1 0001 | B 1 0100 | replace A: invalidate C0, C2 |
| 5 | C0 reads A | C 1 0001 | A 1 1000 | replace B: invalidate C1 |

In step 4 the directory is full, and A, last used in step 2, is the least recently used entry; C0 and C2 lose their copies of A. The read of A by C0 in step 5 is a miss caused only by that replacement, and it in turn evicts B from the directory and from C1, whose modified copy is written back to the LLC.

> **Key point**
> The distributed LLC and the on-chip directory both divide a shared structure among the tiles, so that it grows with the number of cores without a central bottleneck. A block's LLC slice and its directory entry reside in the same tile, and the directory keeps entries only for blocks in private caches, at the price of occasional misses caused by directory replacement.

## 22.5 Sharing the power budget

**Source** Lesson 22, videos 12–14; H&P §1.5.

The third challenge is power. A chip can dissipate only as much power as its package and cooling remove, and this power budget is shared by all its cores. The more cores a chip has, the less power each core receives, and the lower its supply voltage and clock frequency must be. The dynamic power of a core is proportional to $V^2 f$ (Equation (1.1)), and the frequency a core reaches is roughly proportional to its voltage, so the power grows roughly with $f^3$ (Example 18.1). If $N$ cores share the budget of a single core, each receives $1/N$ of it and runs at

$$f_N = \frac{f_1}{\sqrt[3]{N}}, \tag{22.2}$$

where $f_1$ is the frequency of the single core. This relation assumes that the voltage can be lowered in proportion to the frequency; near its lower limit, power falls only linearly with $f$, and dividing the budget costs even more frequency.

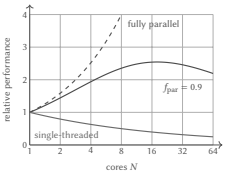



**Figure 22.3:** Performance of $N$ cores sharing the power budget of one core, by Equation (22.3).

Since the CPI does not change, a single-threaded program runs $\sqrt[3]{N}$ times slower on a chip with $N$ cores than on a single core with the same budget: more cores make a sequential program

slower. A program whose parallel fraction is $f_{\text{par}}$ gains from the additional cores, as Amdahl's law (Equation (18.1)) describes, but loses from the lower frequency:

$$\text{performance}(N) \;=\; \frac{1}{\sqrt[3]{N}} \cdot \frac{1}{(1-f_{\text{par}}) + \dfrac{f_{\text{par}}}{N}}, \tag{22.3}$$

relative to the single core. More generally, if a fraction $t_p$ of the single-core execution time can use $p$ threads, the program needs $\sqrt[3]{N}\sum_p t_p/\min(p,N)$ of that time on $N$ cores. Beyond some number of cores the loss in frequency outweighs the gain in parallelism (Figure 22.3).

**Tip** Equation (22.3) is largest at $N = 2f_{\text{par}}/(1-f_{\text{par}})$: 18 cores for $f_{\text{par}} = 0.9$, 198 for 0.99.

**Example 22.8.** A chip has a 120 W budget; a single core using all of it runs at 4.2 GHz. Eight cores receive 15 W each and run at $4.2\,\text{GHz}/\sqrt[3]{8} = 2.1\,\text{GHz}$, so a single-threaded program runs at half its speed on the single core. A program with $f_{\text{par}} = 0.9$ performs $0.5/(0.1+0.9/8) \approx 2.35$ times as well as on the single core. With 27 cores at 1.4 GHz it reaches $(1/3)/(0.1+0.9/27) = 2.5$, but with 64 cores at 1.05 GHz only $0.25/(0.1+0.9/64) \approx 2.19$.

## 22.6 Turbo mode

When a program has no parallelism, the power budget of the idle cores can be given to the core that runs it. In *turbo mode* the chip keeps only one core (or a few) active, turns the others off, and raises the voltage and the clock frequency of the active core above their normal values. With the entire budget of $N$ cores, the frequency could rise by a factor of $\sqrt[3]{N}$—1.59 for four cores—which would recover exactly the frequency lost by dividing the budget (Equation (22.2)). Table 22.2 shows two quad-core processors of 2013; their design power is that of the whole package, which also contains the integrated graphics and the uncore, and is therefore not a budget for the cores alone.

**Source** Lesson 22, video 15; H&P §1.5.

**Table 22.2:** Normal and turbo clock frequencies of two quad-core Intel Core processors. The power factor is the cube of the ratio of the turbo frequency to the normal one.

| Processor | Design power | Normal | Turbo | Frequency | Power |
|---|---|---|---|---|---|
| i7-4702MQ (laptop) | 37 W | 2.2 GHz | 3.2 GHz | 1.45× | 3.1× |
| i7-4771 (desktop) | 84 W | 3.5 GHz | 3.9 GHz | 1.11× | 1.38× |

Neither processor uses the whole budget of four cores for one core. What limits a chip is its temperature, not only its total power: in turbo mode the power is concentrated in the area of one core, only part of its heat spreads into the idle cores around it, and the active core reaches its maximum temperature before it consumes four times its normal power. The laptop processor normally runs its cores at low power, so one core can still raise its power about threefold. The desktop processor, designed for better cooling, runs its cores at high power already in normal operation, close to what the area of one core can dissipate, and gains little from turbo mode.

A current part shows the same gap: the Core i9-14900K runs its performance cores at 3.2 GHz within 125 W and reaches 6 GHz within a 253 W turbo power. Intel ARK.

## 22.7 SMT, cores and chips

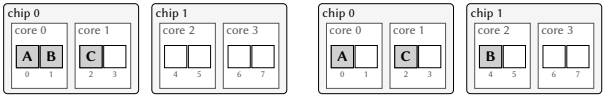
The last challenge concerns the operating system. A many-core system uses all the kinds of parallelism of Lesson 18 at once: a board carries several chips, each chip several cores, and each core runs several threads with SMT. The operating system sees every hardware thread as a *logical processor* on which it can run one software thread; two chips with eight cores each and two SMT threads per core, for example, appear as 32 logical processors.

**Source** Lesson 22, videos 16–17.

The logical processors are not equivalent, however. Two threads on the same core share its pipeline and caches, and each runs much slower than on a core of its own (Section 18.8). Threads on the same chip share its last-level cache, its power budget and its memory interface. Threads on different chips share none of these, but communicate over the slower links between the chips. An operating system that treats all logical processors as identical may start three threads on logical processors 0, 1 and 2, which places two of the threads on one core and leaves the other chip idle (Figure 22.4a). A topology-aware operating system spreads the threads first over the chips, then over the cores of each chip, and uses the second SMT thread of a core only when every core is busy (Figure 22.4b).

**Caution** `nproc` and `hardware_concurrency` count logical processors, not cores: on a 2-way SMT machine they return twice the number of cores, and that many compute-bound threads run two per core.

Linux publishes the topology in sysfs (`core_cpus` for the SMT siblings, `package_cpus` for the chip) and builds its scheduling domains from it; `lscpu` and `lstopo` print it.




(a) topology-unaware (b) topology-aware

**Figure 22.4:** Three threads A, B and C on two chips with two cores and two SMT threads per core; small numbers are logical processors. (a) Threads A and B share core 0. (b) Every thread has a core of its own.

**Note 22.9.** Spreading is best for independent threads; threads that share much data communicate faster on one chip, where the data stays in one LLC. Either choice requires that the operating system know the topology.

## 22.8 Challenges and solutions

**Source** Lesson 22, videos 18–19.

Table 22.3 collects the challenges of this lesson and their solutions. Each arises because a mechanism that serves a few cores does not grow with the number of cores.

**Table 22.3:** Many-core challenges and their solutions.

| Challenge | Cause | Solution |
|---|---|---|
| Coherence traffic | more writes to shared data; a bus carries one transfer at a time | network on chip (mesh), directory-based coherence |
| Off-chip traffic | more cache misses; pins grow more slowly than cores | distributed LLC |
| Directory size | an entry for every memory block | on-chip partial directory |
| Power budget | one budget shared by more cores | turbo mode when few cores are active |
| Thread placement | SMT threads, cores and chips differ | topology-aware operating system |

**Key point — Lesson summary**
A many-core processor cannot simply scale up the designs of Lessons 18–21. Coherence traffic outgrows a bus, so the cores are connected by a network on chip, usually a mesh of tiles whose capacity, unlike that of a bus, grows with the number of cores—with $\sqrt{N}$ for traffic across the chip and with $N$ for traffic between neighbours—and use directory-based coherence. A distributed LLC gives each tile a slice, selected by cache index or, for locality, by page number; the coherence directory becomes an on-chip partial directory in the tile of each block's LLC slice. A shared power budget lowers the frequency of every core by a factor of $\sqrt[3]{N}$ (Equation (22.2)), which turbo mode partly recovers for sequential programs, and the operating system must know which logical processors share a core or a chip.

# Bibliography


[1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2017.

[2] Gordon E. Moore. "Cramming More Components onto Integrated Circuits". In: *Electronics* 38.8 (1965), pp. 114–117.

[3] William A. Wulf and Sally A. McKee. "Hitting the Memory Wall: Implications of the Obvious". In: *ACM SIGARCH Computer Architecture News* 23.1 (1995), pp. 20–24.

[4] Standard Performance Evaluation Corporation. *SPEC Benchmarks*. URL: https://www.spec.org/ (visited on 09/14/2026).

[5] Gene M. Amdahl. "Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities". In: *Proceedings of the AFIPS Spring Joint Computer Conference*. Vol. 30. 1967, pp. 483–485.

[6] Johnny K. F. Lee and Alan Jay Smith. "Branch Prediction Strategies and Branch Target Buffer Design". In: *Computer* 17.1 (1984), pp. 6–22.

[7] James E. Smith. "A Study of Branch Prediction Strategies". In: *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*. 1981, pp. 135–148.

[8] Tse-Yu Yeh and Yale N. Patt. "Two-Level Adaptive Training Branch Prediction". In: *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. 1991, pp. 51–61.

[9] Scott McFarling. *Combining Branch Predictors*. WRL Technical Note TN-36. Digital Equipment Corporation, Western Research Laboratory, 1993.

[10] David R. Kaeli and Philip G. Emma. "Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns". In: *Proceedings of the 18th Annual International Symposium on Computer Architecture (ISCA)*. 1991, pp. 34–42.

[11] J. R. Allen et al. "Conversion of Control Dependence to Data Dependence". In: *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 1983, pp. 177–189.

[12] Scott A. Mahlke et al. "A Comparison of Full and Partial Predicated Execution Support for ILP Processors". In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 138–150.

[13] Robert M. Keller. "Look-Ahead Processors". In: *ACM Computing Surveys* 7.4 (1975), pp. 177–195.

[14] David W. Wall. "Limits of Instruction-Level Parallelism". In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1991, pp. 176–188.

[15] Robert M. Tomasulo. "An Efficient Algorithm for Exploiting Multiple Arithmetic Units". In: *IBM Journal of Research and Development* 11.1 (1967), pp. 25–33.

[16] Subbarao Palacharla, Norman P. Jouppi, and James E. Smith. "Complexity-Effective Superscalar Processors". In: *Proceedings of the 24th Annual International Symposium on Computer Architecture (ISCA)*. 1997, pp. 206–218.

[17] James E. Smith and Andrew R. Pleszkun. "Implementing Precise Interrupts in Pipelined Processors". In: *IEEE Transactions on Computers* 37.5 (1988), pp. 562–573.

[18] George Z. Chrysos and Joel S. Emer. "Memory Dependence Prediction using Store Sets". In: *Proceedings of the 25th Annual International Symposium on Computer Architecture (ISCA)*. 1998, pp. 142–153.

[19] Monica Lam. "Software Pipelining: An Effective Scheduling Technique for VLIW Machines". In: *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI)*. 1988, pp. 318–328.

[20] Joseph A. Fisher. "Trace Scheduling: A Technique for Global Microcode Compaction". In: *IEEE Transactions on Computers* C-30.7 (1981), pp. 478–490.

[21] Joseph A. Fisher. "Very Long Instruction Word Architectures and the ELI-512". In: *Proceedings of the 10th Annual International Symposium on Computer Architecture (ISCA)*. 1983, pp. 140–150.

[22] James C. Dehnert et al. "The Transmeta Code Morphing Software: Using Speculation, Recovery, and Adaptive Retranslation to Address Real-Life Challenges". In: *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. 2003, pp. 15–24.

[23] Michael S. Schlansker and B. Ramakrishna Rau. "EPIC: Explicitly Parallel Instruction Computing". In: *Computer* 33.2 (2000), pp. 37–45.

[24] Peter J. Denning. "The Locality Principle". In: *Communications of the ACM* 48.7 (2005), pp. 19–24.

[25] Maurice V. Wilkes. "Slave Memories and Dynamic Storage Allocation". In: *IEEE Transactions on Electronic Computers* EC-14.2 (1965), pp. 270–271.

[26] John S. Liptay. "Structural Aspects of the System/360 Model 85, II: The Cache". In: *IBM Systems Journal* 7.1 (1968), pp. 15–21.

[27] T. Kilburn et al. "One-Level Storage System". In: *IRE Transactions on Electronic Computers* EC-11.2 (1962), pp. 223–235.

[28] Mark D. Hill and Alan Jay Smith. “Evaluating Associativity in CPU Caches”. In: *IEEE Transactions on Computers* 38.12 (1989), pp. 1612–1630.

[29] David Callahan, Ken Kennedy, and Allan Porterfield. “Software Prefetching”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. 1991, pp. 40–52.

[30] Norman P. Jouppi. “Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers”. In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 364–373.

[31] Jean-Loup Baer and Wen-Hann Wang. “On the Inclusion Properties for Multi-Level Cache Hierarchies”. In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988, pp. 73–80.

[32] Robert H. Dennard. “Field-Effect Transistor Memory”. U.S. pat. 3387286. International Business Machines Corporation. June 4, 1968.

[33] Algirdas Avižienis et al. “Basic Concepts and Taxonomy of Dependable and Secure Computing”. In: *IEEE Transactions on Dependable and Secure Computing* 1.1 (2004), pp. 11–33.

[34] Bianca Schroeder, Eduardo Pinheiro, and Wolf-Dietrich Weber. “DRAM Errors in the Wild: A Large-Scale Field Study”. In: *Proceedings of the Eleventh International Joint Conference on Measurement and Modeling of Computer Systems (SIGMETRICS/Performance ’09)*. 2009.

[35] David A. Patterson, Garth Gibson, and Randy H. Katz. “A Case for Redundant Arrays of Inexpensive Disks (RAID)”. In: *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. 1988, pp. 109–116.

[36] Bianca Schroeder and Garth A. Gibson. “Disk Failures in the Real World: What Does an MTTF of 1,000,000 Hours Mean to You?” In: *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST ’07)*. 2007.

[37] Michael J. Flynn. “Very High-Speed Computing Systems”. In: *Proceedings of the IEEE* 54.12 (1966), pp. 1901–1909.

[38] Kunle Olukotun et al. “The Case for a Single-Chip Multiprocessor”. In: *Proceedings of the Seventh International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1996.

[39] Dean M. Tullsen, Susan J. Eggers, and Henry M. Levy. “Simultaneous Multithreading: Maximizing On-Chip Parallelism”. In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 392–403.

[40] Deborah T. Marr et al. “Hyper-Threading Technology Architecture and Microarchitecture”. In: *Intel Technology Journal* 6.1 (2002), pp. 4–15.

[41] Mark S. Papamarcos and Janak H. Patel. “A Low-Overhead Coherence Solution for Multiprocessors with Private Cache Memories”. In: *Proceedings of the 11th Annual International Symposium on Computer Architecture (ISCA)*. 1984, pp. 348–354.

[42] Anant Agarwal et al. “An Evaluation of Directory Schemes for Cache Coherence”. In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988.

[43] Leslie Lamport. “A New Solution of Dijkstra’s Concurrent Programming Problem”. In: *Communications of the ACM* 17.8 (1974), pp. 453–455.

[44] John M. Mellor-Crummey and Michael L. Scott. “Algorithms for Scalable Synchronization on Shared-Memory Multiprocessors”. In: *ACM Transactions on Computer Systems* 9.1 (1991), pp. 21–65.

[45] David Mosberger. “Memory Consistency Models”. In: *ACM SIGOPS Operating Systems Review* 27.1 (1993), pp. 18–26.

[46] Leslie Lamport. “How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs”. In: *IEEE Transactions on Computers* C-28.9 (1979), pp. 690–691.

[47] Sarita V. Adve and Mark D. Hill. “Weak Ordering—A New Definition”. In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 2–14.

[48] Hans-J. Boehm and Sarita V. Adve. “Foundations of the C++ Concurrency Memory Model”. In: *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2008.

[49] Pete Keleher, Alan L. Cox, and Willy Zwaenepoel. “Lazy Release Consistency for Software Distributed Shared Memory”. In: *Proceedings of the 19th Annual International Symposium on Computer Architecture (ISCA)*. 1992.

[50] Liviu Iftode, Jaswinder Pal Singh, and Kai Li. “Scope Consistency: A Bridge between Release Consistency and Entry Consistency”. In: *Proceedings of the 8th Annual ACM Symposium on Parallel Algorithms and Architectures (SPAA)*. 1996, pp. 277–287.

[51] William J. Dally and Brian Towles. “Route Packets, Not Wires: On-Chip Interconnection Networks”. In: *Proceedings of the 38th Design Automation Conference (DAC)*. 2001, pp. 684–689.

[52] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. “Systematic Reverse Engineering of Cache Slice Selection in Intel Processors”. In: *Proceedings of the 18th Euromicro Conference on Digital System Design (DSD)*. 2015.

# Index


**Symbols**
1BP, *see* one-bit predictor
2BC, *see* two-bit predictor
2BP, *see* two-bit predictor

**A**
activity factor, 2
aliasing, 89
AMAT, *see* average memory access time
Amdahl's law, 6
architectural register, 28
associativity, 72
atomic exchange, 139
atomic instruction, 139
availability, 112
average memory access time, 70

**B**
backward compatibility, 67
barrier synchronization, 143
benchmark, 4
benchmark suite, 4
BHT, *see* branch history table
bit line, 98
block number, 72
block offset, 72
block size, 71
blocking cache, 94
branch hint, 67
branch history table, 14
branch misprediction, 11
branch misprediction recovery, 45
branch prediction, 11
branch prediction accuracy, 12
branch target address, 11
branch target buffer, 13
broadcast, *see* write result
BTB, *see* branch target buffer
bubble, 8
bus snooping, 129

**C**
cache, 70
cache block, 71
cache coherence, 128, 145
cache hierarchy, 95
cache hit, 70
cache line, 71
cache miss, 70
cache-to-cache transfer, 132
capacity miss, 91
CAS, *see* column address strobe
causal consistency, 151
CDB, *see* common data bus
centralized shared memory, 121
checkpointing, 113
chip multiprocessor, 125
CMOVcc, 23
CMP, *see* chip multiprocessor
coarse-grained multithreading, 125
code bloat, 62
coherence miss, 135
column address strobe, 102
column multiplexer, 100
commit, 44, 55
commit pointer, 44
common data bus, 34
compulsory miss, 91
computer architecture, 1
condition code, 23
conditional move, 22
conflict miss, 92
consistency model, 145
control dependence, 8
control hazard, 9
correlating prefetcher, 93
critical section, 138
cylinder, 106

**D**
data dependence, 8
data hazard, 9
data race, 150
dependability, 111
design fault, 113
destructive read, 99
die, 3
digital signal processor, 68
diminishing returns, 6
direct-mapped cache, 72
direction predictor, 14
directory-based coherence, 134
dirty bit, 76
disk head, 106
dispatch, 35
distributed last-level cache, 155
distributed shared memory, 122
DRAM, *see* dynamic random-access memory
DSP, *see* digital signal processor
dynamic power, 2
dynamic random-access memory, 98

**E**
ECC, *see* error-correcting code
environmental fault, 113
error, 111
error-correcting code, 115

**F**
failure, 111
false dependence, 9, 28
false sharing, 135
fast page mode, 102
fault, 111

fault avoidance, 113
fault tolerance, 113
fine-grained multithreading, 125
five-stage pipeline, 7
flash memory, 108
flat page table, 81
flush, 8
Flynn's taxonomy, 120
forwarding, 9
front-side bus, 103
FSB, *see* front-side bus
full predication, 24, 67
fully associative cache, 74
function call inlining, 63, 67

**G**
geometric mean, 5
global branch history, 17
global hit rate, 96
global miss rate, 96
GShare predictor, 17

**H**
half-miss, 95
hard disk, *see* magnetic disk
hardware fault, 113
hardware prefetching, 93
hazard, 9
hierarchical predictor, 18
history-based predictor, 16
hit rate, 70
hit time, 70
hybrid disk, 108

**I**
I/O bus, 109
ideal processor, 30
if-conversion, 22, 60, 67
ILP, *see* instruction-level parallelism
in-order execution, 31
in-order load-store execution, 53
inclusion property, 97
index, 72
instruction queue, 34
instruction scheduling, 59
instruction-level parallelism, 30
instructions per cycle, 31
intermittent fault, 113
internal fragmentation, 83
IPC, *see* instructions per cycle
iron law, 5
issue, 35
issue pointer, 44
Itanium, 24, 68

**L**
L1 cache, 71
L2 cache, 95
Lamport's bakery algorithm, 139
last-level cache, 95
latch, 7
latency, 4
lazy release consistency, 151
least recently used, 75
Lhadma's law, 6
link register, 140
LL/SC, *see* load-linked/store-conditional
LLC, *see* last-level cache
load-linked/store-conditional, 140
load-store queue, 52
local hit rate, 96
local miss rate, 96
locality principle, 69
lock, 138
logical processor, 157
loop interchange, 94
loop unrolling, 61, 66
LRU, *see* least recently used
LSQ, *see* load-store queue

**M**
magnetic disk, 105
magnetic tape, 107
many-core processor, 153
mean time between failures, 112
mean time to failure, 112
mean time to repair, 112
memory cell, 98
memory consistency, 145
memory controller, 103
memory wall, 2
mesh, 153
MESI protocol, 133
message passing, 123
meta-predictor, 18
mezzanine bus, 109
MFENCE, 149
MIMD, *see* multiple instruction, multiple data
mirroring, 116
MISD, *see* multiple instruction, single data
misprediction penalty, 12
miss penalty, 70
miss rate, 70
miss status handling register, 94
miss-under-miss support, 94
misses per 1000 instructions, 96
MOESI protocol, 133
Moore's law, 1
MOSI protocol, 132
MOVN, 22
MOVZ, 22
MPKI, *see* misses per 1000 instructions
MSHR, *see* miss status handling register
MSI protocol, 131
MSYNC, 149
MTBF, *see* mean time between failures
MTTF, *see* mean time to failure
MTTR, *see* mean time to repair
multi-level page table, 81
multicore processor, 121
multiple instruction, multiple data, 120
multiple instruction, single data, 120
multiprocessor, 120

mutex, *see* lock
mutual exclusion, 138

**N**
*N*-bit history predictor, 16
*N*-module redundancy, 114
network on chip, 153
NMRU, *see* not most recently used
non-blocking cache, 94
non-uniform memory access, 122
northbridge, 103
not most recently used, 91
not-taken branch, 11
NUMA, *see* non-uniform memory access

**O**
oldest first, 36
on-chip directory, 155
one-bit predictor, 14
operation fault, 113
optical disk, 107
out-of-order execution, 31, 33, 50
out-of-order load-store execution, 53
overlap miss, 94

**P**
page, 79
page fault, 80
page frame, 79
page offset, 80
page size, 83
page table, 79
page table entry, 81
page table walk, 84
parity, 114
partial directory, 156
pattern history table, 17
permanent fault, 113
PFN, *see* physical frame number
phantom exception, 43
PHT, *see* pattern history table
physical address, 78
physical frame number, 80
physical memory, 78
physical register, 28
physically accessed cache, 87
pipeline depth, 10
pipelined cache, 86
pipelining, 7
platter, 105
PLRU, *see* pseudo-LRU
precise exception, 42
predecoding, 20
predicate register, 25
predication, 21
predict not taken, 12
prefetch instruction, 93
prefetching, 92
private branch history, 17
processor consistency, 151
program counter, 7
pseudo-LRU, 91
PShare predictor, 17
PTE, *see* page table entry

**R**
RAID, *see* redundant array of independent disks
RAID 0, 115
RAID 1, 116
RAID 4, 117
RAID 5, 118
RAID 6, 119
random access, 98
RAS, *see* return address stack
RAT, *see* register alias table
read-after-write dependence, 8, 28
redundant array of independent disks, 115
refresh, 99
register alias table, 28, 46
register renaming, 28
relaxed consistency, 149
release consistency, 151
reliability, 112
reorder buffer, 43
replacement policy, 75
reservation station, 34, 44
retirement, *see* commit
return address stack, 20
ROB, *see* reorder buffer
rotational latency, 106
row address strobe, 102
row decoder, 100
RS, *see* reservation station

**S**
saturating counter, 15
scope consistency, 151
sector, 106
seek time, 106
sense amplifier, 99
sequential access, 108
sequential consistency, 147
set-associative cache, 74
shared memory, 123
SIMD, *see* single instruction, multiple data
simultaneous multithreading, 125
single instruction, multiple data, 120
single instruction, single data, 120
SISD, *see* single instruction, single data
SMP, *see* symmetric multiprocessor
SMT, *see* simultaneous multithreading
software pipelining, 64
solid-state disk, 108
southbridge, 110
spatial locality, 69
speedup, 4
spin lock, 138
spindle, 105
SRAM, *see* static random-access memory
SSD, *see* solid-state disk
stage, 7
stale result, 37

stall, 8
static power, 3
static random-access memory, 98
stop bit, 67
store-to-load forwarding, 52
stream buffer, 93
stride prefetcher, 93
striping, 115
strong consistency, 148
superscalar, 49, 65
symmetric multiprocessor, 122

**T**
tag, 72
taken branch, 11
temporal locality, 69
test-and-set, 139
throughput, 4
tile, 153
TLB, *see* translation lookaside buffer
TLB miss, 84
Tomasulo's algorithm, 33, 44
torus, 153
tournament predictor, 18
trace scheduling, 64
track, 106
transient fault, 113
translation lookaside buffer, 84
tree height reduction, 58, 67
trench cell, 100
true sharing, 135
turbo mode, 157
two-bit predictor, 15

**U**
UMA, *see* uniform memory access
unified reservation station, 49
uniform memory access, 121

**V**
valid bit, 72
very long instruction word, 65
VIPT, *see* virtually indexed physically tagged cache
virtual address, 78
virtual address space, 78
virtual memory, 78
virtual page number, 80
virtually accessed cache, 88
virtually indexed physically tagged cache, 88
VLIW, *see* very long instruction word
VPN, *see* virtual page number

**W**
wafer, 3
way prediction, 90
weak consistency, 151
word line, 98
working set, 127
write result, 35
write-after-read dependence, 9, 28
write-after-write dependence, 9, 28
write-allocate policy, 75
write-back policy, 76
write-invalidate coherence, 130
write-through policy, 75
write-update coherence, 129

**Y**
yield, 3